Raccolta di Materiale ed Esercizi Per la Programmazione in linguaggio Python

INDICE

Ambiente di sviluppo 4

Indicazioni e materiale 7

Esercizi e Spiegazioni divisi per argomento 8

1 Variabili e operazioni elementari 8

1.1 Esercizi 8

2 Algoritmi e strutture di controllo 10

2.1 Esercizi 10

3 Funzioni 28

3.1 Funzioni semplici 28

3.2 Funzioni ricorsive 35

3.3 Algoritmi di ordinamento avanzati 46

4 Strutture dati fondamentali: Liste, Stringhe, Matrici e Dizionari 47

4.1 Liste di soli numeri 47

4.2 Stringhe 58

4.3 Espressioni regolari 73

4.4 Liste miste 74

4.5 Matrici di soli numeri 79

4.6 Matrici miste 98

4.7 Dizionari 108

4.8 Dizionari con stringhe 124

5 Algoritmi di ordinamento base 129

6 Files 132

6.1 Spiegazioni ed esempi 132

6.2 Esercizi 133

7 Classi – Spiegazioni ed esempi 167

7.1 Le classi 167

7.2 Metodi statici e metodi di classe 169

7.3 Classe PuntoB completa 171

7.4 Ereditarietà 172

7.5 Polimorfismo 174

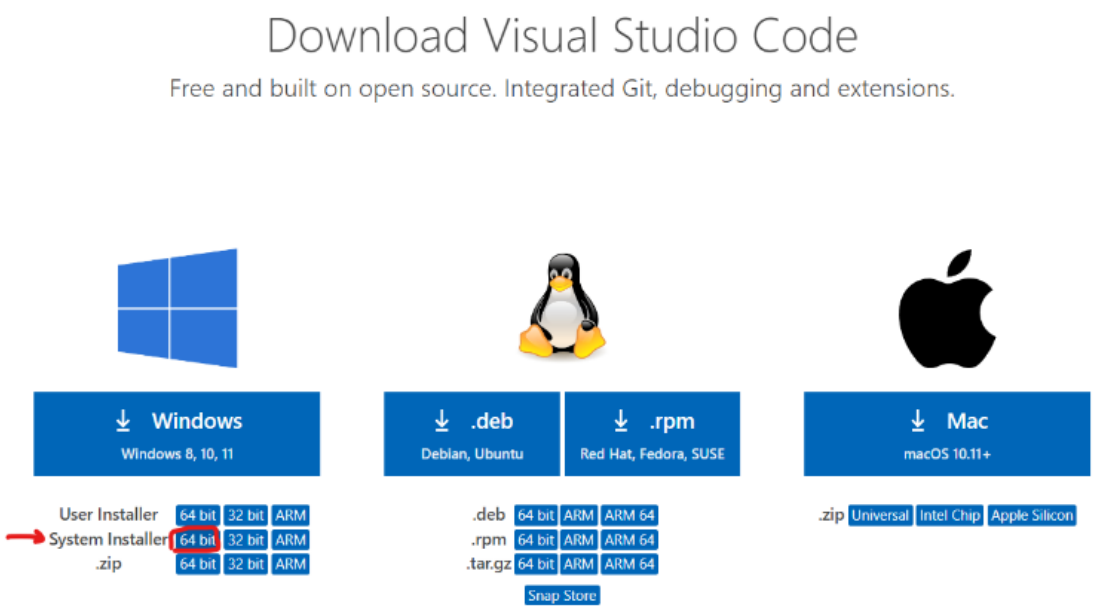
7.6 Elenco dei Dunder Methods 175

8 Classi – Esercizi 177
8.1 Esercizi senza ereditarietà 177
8.2 Esercizi con Ereditarietà e Polimorfismo 204
9 Strutture dati avanzate 208
9.1 Pile / Stack 208
9.2 Code / Queue 212
9.3 Grafi 213
9.4 Alberi 245
10 Pygame 266
11 Altro 268
11.1 Sistema 268

Ambiente di sviluppo

Per programmare in linguaggio Python è possibile usare qualsiasi IDE ma comunque è necessario scaricare e installare l'interprete. Al seguente link si può trovare il pacchetto di installazione:

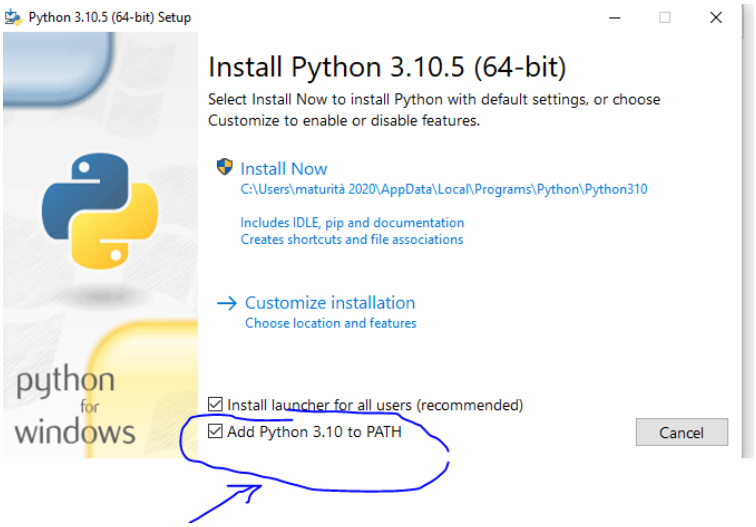
https://www.python.org/downloads/



Consiglio, a meno di particolari necessità, di scaricare il system installer a 64bit.

Durante l'installazione:

- Flaggare l'opzione "Add Python x.xx to PATH" come mostrato nella seguente immagine;

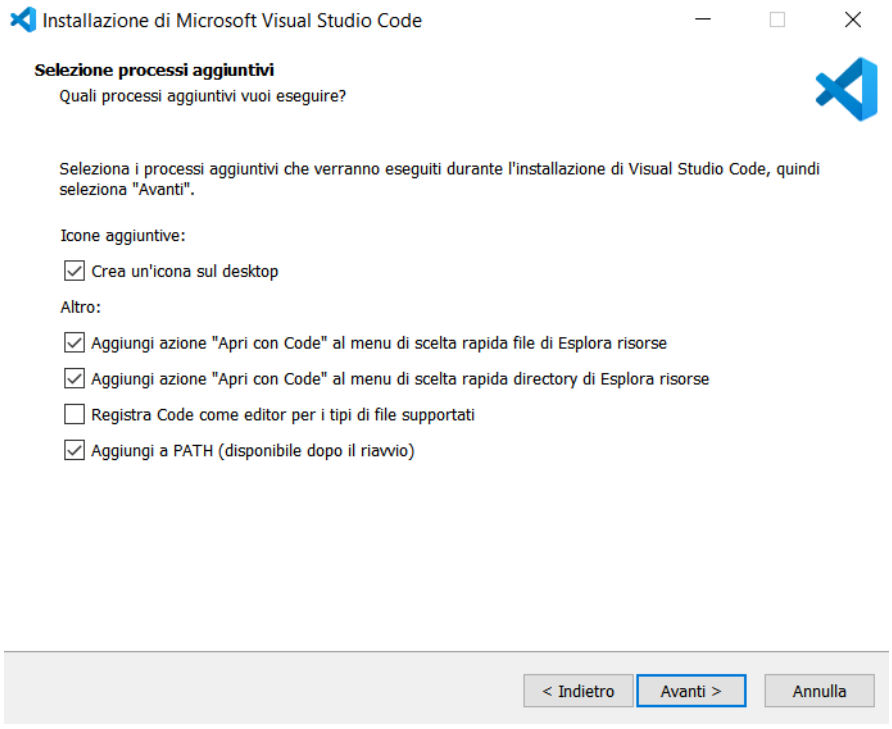


- Scegliere customize installation.
- Alla fase "optional features" lasciare tutto flaggato
- Alla fase "Advanced options" consiglierei, a meno di casi particolari, di aggiungere il flag alla prima voce, lasciare il flag agli altri 3 già flaggati e se si vuole flaggare anche la quinta voce.

Per verificare la corretta installazione basta eseguire in una qualsiasi console (cerca cmd su start) il comando “python” (python3 se su mac)

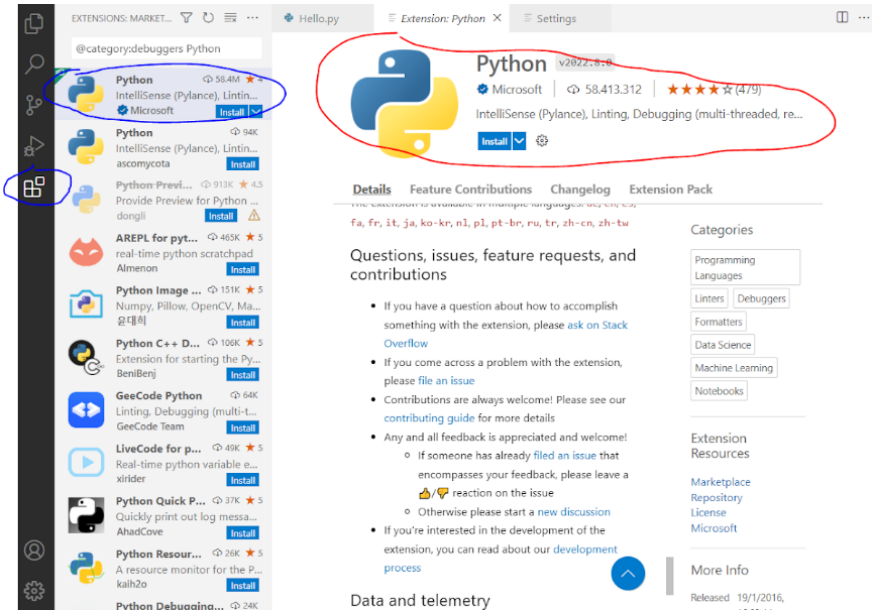
Per chi non lo avesse ancora fatto, installare visual studio code, consiglierei di usare, a meno di esigenze particolari, il system installer da 64bit che puoi trovare sul sito https://code.visualstudio.com/download .

Ricordarsi durante l’installazione di flaggare le voci corrette durante tutti i passaggi come mostrato dalla seguente immagine:

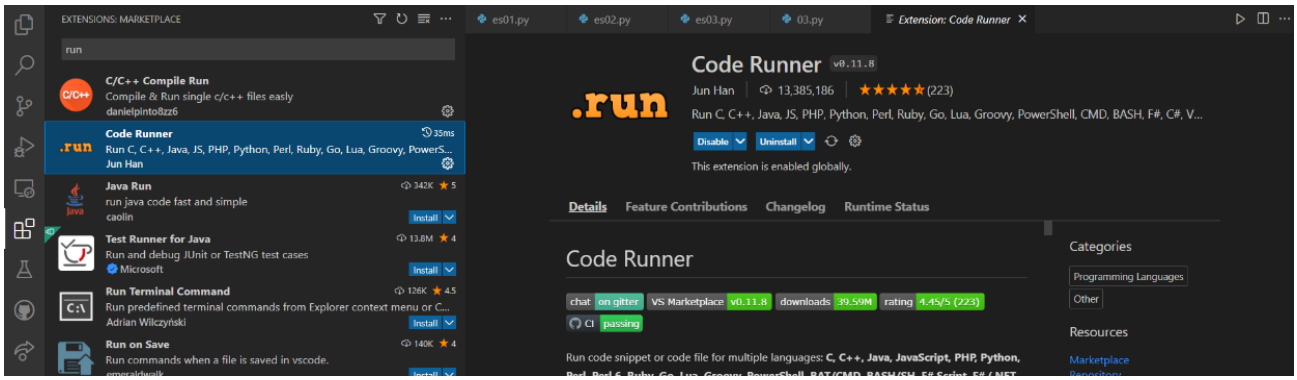


Per programmare comodamente in python con Visual studio code consiglio di installare le seguenti estensioni:

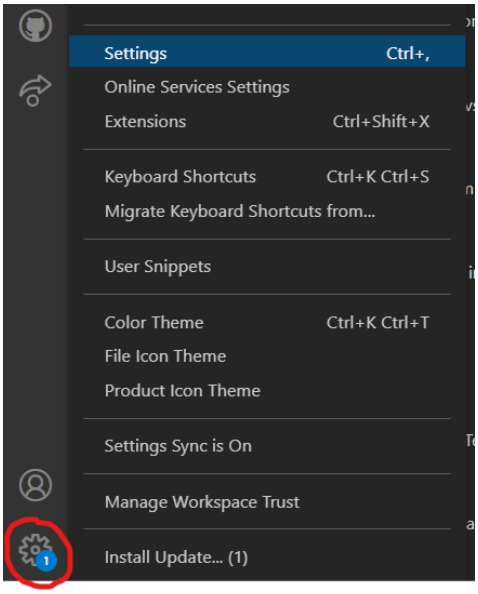
- Python.



- Code Runner

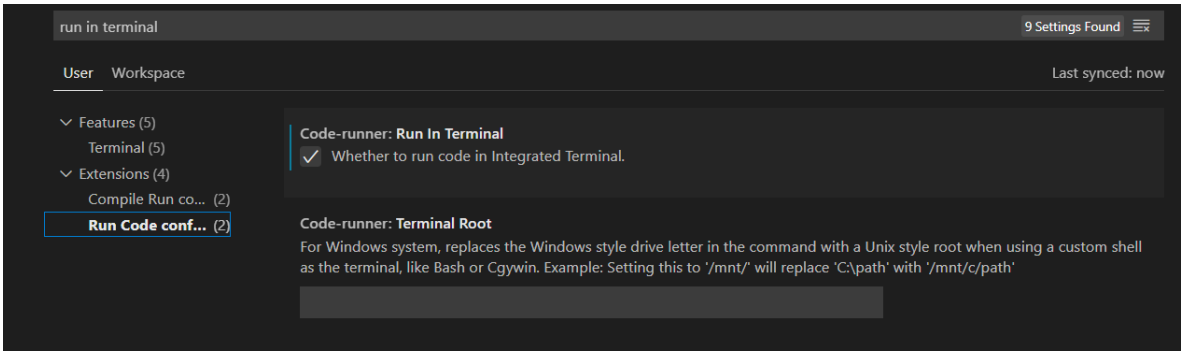


Una volta installate le estensioni, vanno modificate alcune impostazioni. Il menù impostazioni è velocemente accessibile dal tasto in basso a sinistra:

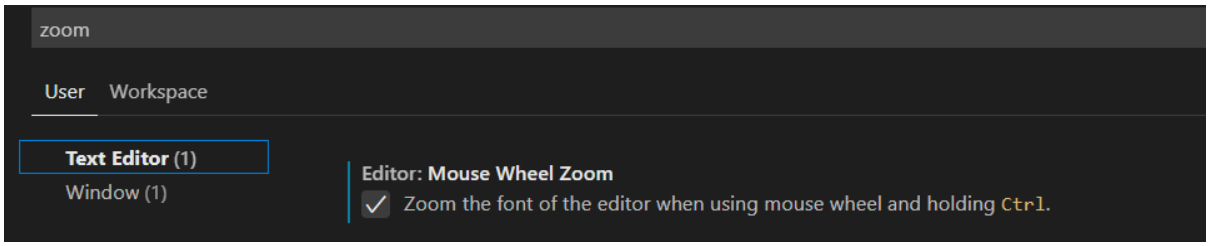


Le impostazioni da modificare sono:

- “run in terminal” da spuntare nelle impostazioni e trovabile velocemente inserendo la voce nella barra di ricerca e cliccando sull’estensione interessata come nella seguente immagine:



- “Mouse Wheel Zoom” da spuntare come fatto per l’impostazione precedente



È possibile modificare a piacere altre impostazioni come:

- Save file before run
- Save all files before run
- Color Theme
- Clear Previous Output

Indicazioni e materiale

Come manuale contenente esempi e spiegazioni di base su ogni aspetto del linguaggio consiglio di guardare il sito https://www.w3schools.com/python/ .

Online sono presenti molti altri tutorial anche in lingua italiana come ad esempio

1. https://www.programmareinpython.it/corsi-e-lezioni-python-dal-nostro-canale-youtube/
2. https://www.html.it/guide/guida-python/

Esercizi e Spiegazioni divisi per argomento

La prima parte di eserciziario è stata inizialmente adattata dall'eserciziario per il linguaggio C, possono quindi essere presenti esercizi che fanno riferimento a elementi di tale linguaggio che devono ancora essere modificati.

Di seguito sono riportati gli esercizi normalmente svolti durante le lezioni del mio corso, voglio comunque riportare questo link (https://q2a.di.uniroma1.it/assets/eserciziario-python/it/script/) dove potete trovare un gran numero di esercizi, alcuni semplici altri anche piuttosto difficili, proposti per un corso universitario di informatica.

1 Variabili e operazioni elementari

1.1 Esercizi

1.1.1 Operazioni matematiche elementari

Dati in input due numeri, scrivi il risultato di tutte le operazioni che è possibile fare con i due numeri (somma, sottrazione, moltiplicazione, divisione intera, resto della divisione, divisione con decimali, potenza)

Soluzione

```
a = int(input("Scrivi il primo numero: "))
b = int(input("Scrivi il secondo numero: "))

print(f"{a} + {b} = {a+b}")
print(f"{a} - {b} = {a-b}")
print(f"{a} * {b} = {a*b}")
print(f"{a} ** {b} = {a**b}")       # elevato a
print(f"{a} / {b} = {a/b:.2f}")     # produce un float
print(f"{a} // {b} = {a//b}")       # produce un intero troncato
print(f"{a} % {b} = {a%b}")         # resto
```

1.1.2 Arrotondamenti e valore assoluto

Dato un numero decimale, ottieni e stampa:

1. Valore intero approssimato per difetto
2. Valore intero approssimato per eccesso
3. Valore intero approssimato in maniera intelligente
4. approssimazione del numero al secondo decimale
5. Valore assoluto del numero

Soluzione

```
from math import ceil, floor, pi

n = -pi

print(f"a. {n} intero troncato: {int(n)}")
print(f"a. {n} intero approssimato per difetto: {floor(n)}") # devo importare math
print(f"b. {n} intero approssimato per eccesso: {ceil(n)}") # devo importare math
print(f"c. {n} intero approssimato in maniera intelligente: {round(n)}")
print(f"d. {n} intero approssimato al secondo decimale: {round(n, 2)}")
print(f"e. {n} valore assoluto del numero: {abs(n)}")
```

1.1.3 Cerchio: da area a circonferenza

Data l'area del cerchio, stampare la circonferenza

Soluzione

```
import math

area = float(input("inserisci l'area del cerchio: "))
circonferenza = 2*math.pi*(area/math.pi)**(1/2)
print("circonferenza: ", circonferenza)
```

1.1.4 Area del triangolo

Data la base e l'altezza di un triangolo, scrivere l'area

Soluzione

```
base = float(input("scrivi il valore della base: "))
altezza = float(input("scrivi il valore dell'altezza': "))
area = base*altezza/2
print("L'area è: ", area)
```

2 Algoritmi e strutture di controllo

2.1 Esercizi

2.1.1 Divisione con controllo

Dati due numeri calcolare il loro quoziente se il divisore è != 0, restituire “impossibile” se il divisore = 0

Soluzione

```
num = float(input("scrivi il valore del numeratore: "))
den = float(input("scrivi il valore del denominatore': "))
if den == 0:
    print("Non è possibile dividere per 0")
else:
    print(num, "/", den, " = ", num/den)
```

2.1.2 Confronto due numeri

Dati in input due numeri, stampa il più grande tra i due

Soluzione

```
n1 = float(input("Scrivi il primo numero: "))
n2 = float(input("Scrivi il secondo numero: "))

if n1 > n2:
    print(f"{n1} > {n2}")
elif n2 > n1:
    print(f"{n2} > {n1}")
else:
    print(f"{n1} = {n2}")
```

2.1.3 Confronto 3 numeri

Dati in input 3 numeri, stamparli in ordine crescente

2.1.4 Ricerca potenza di 2

Dato in input un numero n positivo, stampa la prima potenza di 2 maggiore o uguale a n

Soluzione

```
# 2.1.3 Dato in input un numero n non negativo,
# stampa la prima potenza di 2 maggiore o uguale a n

n = int(input("Scrivi un numero positivo: "))
while n <= 0:
    n = int(input("il numero deve essere positivo: "))

pot2 = 1
while pot2 < n:
```

```
    pot2 *= 2 # equivale a pot2 = pot2 * 2

print(f"La prima potenza di 2 maggiore o uguale a {n} è {pot2}")
```

2.1.5 Moltiplicazione con le somme

Moltiplicazione di due numeri senza usare l'operatore *

Soluzione col while

```
# 2.1.4 Moltiplicazione di due numeri senza usare l'operatore *

n1 = int(input("Scrivi il primo numero: "))
n2 = int(input("Scrivi il secondo numero: "))

risultato = 0
contatore = 0
while contatore < n2:
    risultato += n1
    contatore += 1

print(f"{n1} * {n2} = {risultato}")
```

Soluzione col for

```
# 2.1.4 Moltiplicazione di due numeri senza usare l'operatore *

n1 = int(input("Scrivi il primo numero: "))
n2 = int(input("Scrivi il secondo numero: "))

risultato = 0
for contatore in range(n2):
    risultato += n1

print(f"{n1} * {n2} = {risultato}")
```

2.1.6 Divisione con somme o sottrazioni

Divisione di due numeri interi senza usare l'operatore di divisione

Variante: Stampa anche il valore del resto

Soluzione

```
num = int(input("scrivi il valore del numeratore: "))
den = int(input("scrivi il valore del denominatore': "))

ris = 0
resto = num
while resto > den:
    resto -= den
    ris += 1

# print(num, "/", den, " = ", ris, " resto ", resto)
print(f"{num} / {den} = {ris}  resto {resto}")
```

2.1.7 Cicli crescenti e decrescenti

dati dall'utente due numeri interi n1 e n2 il programma deve stampare:

1. tutti i numeri dal più piccolo dei due al più grande dei due
2. tutti i numeri dal più grande dei due al più piccolo dei due
3. tutti i numeri da n1 a n2 (comunque siano n1 e n2 quindi potresti dover andare in ordine crescente o decrescente)
4. tutti i numeri da n2 a n1 (comunque siano n1 e n2 quindi potresti dover andare in ordine crescente o decrescente)

Soluzione

```
n1 = int(input("Scrivi il primo numero: "))
n2 = int(input("Scrivi il secondo numero: "))

# 1.    tutti i numeri dal più piccolo dei due al più grande dei due
if n1 > n2:
    max = n1
    min = n2
else:
    max = n2
    min = n1

for i in range(min, max+1):
    print(i, end=" ")

print("\n")

# 2.    tutti i numeri dal più grande dei due al più piccolo dei due
for i in range(max, min-1, -1):
    print(i, end=" ")

print("\n")

# 3.    tutti i numeri da n1 a n2 (comunque siano n1 e n2 quindi potresti dover andare in ordine crescente o decrescente)
if n1 < n2:
    for i in range(n1, n2+1):
        print(i, end=" ")
else:
    for i in range(n1, n2-1, -1):
        print(i, end=" ")

print("\n")

# 4.    tutti i numeri da n2 a n1 (comunque siano n1 e n2 quindi potresti dover andare in ordine crescente o decrescente)
if n2 < n1:
    for i in range(n2, n1+1):
        print(i, end=" ")
else:
    for i in range(n2, n1-1, -1):
```

```
        print(i, end=" ")

print("\n")
```

2.1.8 Stampa divisori di un numero

Dato in input un numero, stampa tutti i suoi divisori

Soluzione

```
# 2.1.7 Dato in input un numero, stampa tutti i suoi divisori
# Variante: stampa la sua scomposizione in fattori primi

n = int(input("Scrivi un numero: "))

for div in range(2,n):
    if n % div == 0:
        print(div, end=" ")
```

2.1.9 Conta numeri di sequenza in input terminata da 0

Inserire in input una sequenza di n numeri diversi da zero 0, quando si inserisce 0 si termina l'input, poi comunicare quanti sono i numeri inseriti

Variante: i numeri inseriti sono voti che per esser validi devono essere compresi tra 1 a 10 (richiedi se il valore inserito è errato). Stampa la media dei voti.

Soluzione

```
# VERSIONE 1
c = 0
num = -1
while num!=0:
  num=int(input("scrivi un numero, 0 per uscire: "))
  if num!=0:
    c+=1
print(c)

# VERSIONE 2
c = 0
num = -1
while True:
  num=int(input("scrivi un numero, 0 per uscire: "))
  if num == 0:
    break
  c+=1
print(c)
```

2.1.10 Moltiplicazione con le somme

Moltiplicazione di due numeri senza usare l'operatore di moltiplicazione

Soluzione

```
a = int(input("scrivi il primo valore: "))
b = int(input("scrivi il secondo valore: "))
```

```
ris=0
for i in range(b):
  ris += a

print(a, " x ", b, " = ", ris)
```

2.1.11 Classificazione dei triangoli in base ai lati

Dati in input tre numeri reali che rappresentano le lunghezze dei lati di un triangolo, dire che tipo di triangolo essi formano (classificazione dei triangoli in base ai lati).

Soluzione

```
a = float(input("scrivi il primo valore: "))
b = float(input("scrivi il secondo valore: "))
c = float(input("scrivi il terzo valore: "))

if a==b and b==c:
  ris = "equilatero"
elif a==b or b==c or a==c:
  ris = "isoscele"
else:
  ris = "scaleno"

print(a, ", ", b, ", ", c, " formano un triangolo ", ris)
```

2.1.12 Sequenza in input lunga n, somma pari, prodotto dispari

Data una sequenza di n numeri interi (valore di n dato dall'utente), calcolare la somma dei pari ed il prodotto dei dispari

Soluzione

```
# VERSIONE 1
n = int(input("Scrivi quanti numeri vuoi inserire: "))
somma = 0
prodotto = 1

for i in range(n):
    num = float(input("Scrivi il "+str(i+1)+"° numero: "))
    if num%2 == 0: #cioè se è pari
        somma+=num
    else:
        prodotto*=num

print("somma dei pari: ", somma)
print("prodotto dei dispari: ", prodotto)

# VERSIONE 2
n = int(input("Scrivi quanti numeri vuoi inserire: "))
somma = 0
prodotto = 1

l = []
```

```
for i in range(n):
    num = int(input("Scrivi il "+str(i+1)+"° numero: "))
    l.append(num)

print(l)

for num in l:
    if num%2 == 0: #cioè se è pari
        somma+=num
    else:
        prodotto*=num

print("somma dei pari: ", somma)
print("prodotto dei dispari: ", prodotto)
```

2.1.13 Conta numeri compresi tra due valori

Con un opportuno ciclo chiedere all'utente di inserire due numeri n1 e n2 compresi tra 1 e 100. Se i numeri non fossero corretti, rimanere nel ciclo e ripetere la richiesta. Stampare quanti sono i numeri pari compresi tra i due numeri

2.1.14 Maggiore e minore di sequenza terminata da 0

Data una sequenza di numeri terminata dal numero 0 (leggo numeri finchè non mi viene dato il numero 0), stampo il maggiore e il minore

2.1.15 Fattoriale

Letto un numero intero positivo n stampare il fattoriale: $n! = 1 \cdot 2 \cdot 3 \cdot \ldots \cdot n$

Variante: usa una funzione che restituisca il risultato

Soluzione

```
def fattorialeIterativoCrescente(n):
    if n < 0:
        return None
    # if n == 0:
    #     return 1
    ris = 1

    # da 1 a n
    for i in range(1,n+1):
        ris *= i

    return ris

def fattorialeIterativoDecrescente(n):
    if n < 0:
        return None
    # if n == 0:
    #     return 1
    ris = 1
    # da n a 1
    for i in range(n, 0, -1):
        ris *= i

    return ris
```

```python
def fattorialeRicorsivo(n):
    if n < 0:
        return None
    elif n == 0:
        return 1
    else:
        return n * fattorialeRicorsivo(n-1)


n = int(input("Scrivi un numero: "))
print(f"{n}! = {fattorialeIterativoCrescente(n)}")
print(f"{n}! = {fattorialeIterativoDecrescente(n)}")
print(f"{n}! = {fattorialeRicorsivo(n)}")
```

2.1.16 Calcolatrice semplice

Realizzare un programma che, presi in input 2 operandi reali e un operatore (+, -, *, /), esegue l'operazione stampandone il risultato

2.1.17 Prodotto positivi e somma negativi di sequenza terminata da 0

Progettare un algoritmo che legga da terminale una sequenza di interi positivi e negativi terminati dal valore 0 (uno su ogni linea) e stampi il prodotto degli interi positivi e la somma dei negativi.

2.1.18 Prodotto positivi e somma negativi di sequenza terminata con domanda

Progettare un algoritmo che legga da terminale una sequenza di interi positivi e negativi fintanto che l'utente dice di volerne inserire ancora, e stampi il prodotto degli interi positivi e la somma dei negativi.

Soluzione

```python
somma = 0
prod = 1

num = int(input("Scrivi un numero: "))
if num > 0:
    prod *= num
else:
    somma += num

risposta = int(input("Vuoi inserire un altro numero? (1 sì, 0 no) "))
# while risposta != 1 and risposta != 0:
while risposta not in [1,0]:
    risposta = int(input("Non ho capito la risposta, vuoi inserire un altro numero? (1 sì, 0 no)
"))

while risposta == 1:
    num = int(input("Scrivi un numero: "))
    if num > 0:
        prod *= num
    else:
        somma += num

    risposta = int(input("Vuoi inserire un altro numero? (1 sì, 0 no) "))
    while risposta != 1 and risposta != 0:
        risposta = int(input("Non ho capito la risposta, vuoi inserire un altro numero? (1 sì, 0
no) "))
```

```
print("somma dei negativi:", somma)
print("prodotto dei positivi:", prod)
```

Soluzione con uso di una lista

```
num = int(input("Scrivi un numero: "))
numeri = [num]

risposta = int(input("Vuoi inserire un altro numero? (1 sì, 0 no) "))
# while risposta != 1 and risposta != 0:
while risposta not in [1,0]:
    risposta = int(input("Non ho capito la risposta, vuoi inserire un altro numero? (1 sì, 0 no)
"))

while risposta == 1:
    num = int(input("Scrivi un numero: "))
    # numeri.append(num)
    numeri += [num]

    risposta = int(input("Vuoi inserire un altro numero? (1 sì, 0 no) "))
    while risposta != 1 and risposta != 0:
        risposta = int(input("Non ho capito la risposta, vuoi inserire un altro numero? (1 sì, 0
no) "))

somma = 0
prod = 1
for numero in numeri:
    if numero < 0:
        somma += numero
    else:
        prod *= numero

print("somma dei negativi:", somma)
print("prodotto dei positivi:", prod)
```

2.1.19 Elevamento a potenza con ** e con solo *

Dati due numeri in input b ed e, calcola e scrivi b^e in due modi diversi, usando l'operatore ** e senza poterlo usare.

Soluzione

```
b = 2
e = 5

print(f"{b}^{e} = {b**e}")

ris = 1
for i in range(e):
    ris = ris*b

print(f"{b}^{e} = {ris}")
```

2.1.20 Confronto tra orari in input formati da min e sec

Dati in ingresso 4 numeri, che rappresentano gli orari in cui avvengono due diversi eventi della giornata, in modo che i primi 2 numeri siano ore e minuti del primo orario e gli altri 2 numeri siano ore e minuti del secondo orario, stabilire quale evento è avvenuto prima. Inserisci dei controlli durante l'input in modo che le ore possano andare da 0 a 23 e i minuti da 0 a 59.

2.1.21 Conto alla rovescia con min e sec

Dato un numero di minuti e un numero di secondi rappresentare il conto alla rovescia.

2.1.22 Fibonacci

Dato in input un numero intero n, stampa l'ennesimo numero della successione di Fibonacci. La successione di Fibonacci è quella successione di numeri in cui ogni numero è la somma dei due numeri precedenti. I primi due numeri sono 1 1. La successione inizia così: 1 1 2 3 5 8 13 21 …

Variante: scrivi e usa una funzione che restituisca il risultato

Soluzione

```
def fibonacciIterativo(n):
    if n < 0:
        return None
    if n == 0:
        return 0
    if n == 1:
        return 1

    fm2 = 0
    fm1 = 1
    for i in range(2, n+1):
        f = fm1 + fm2

        fm2 = fm1
        fm1 = f
    return f




def fibonacciRicorsivo(n):
    if n < 0:
        return None
    if n == 0:
        return 0
    if n == 1:
        return 1

    return (fibonacciRicorsivo(n-1) + fibonacciRicorsivo(n-2))

n = int(input("Scrivi un numero: "))
print(f"F({n}) = {fibonacciIterativo(n)}")
print(f"F({n}) = {fibonacciRicorsivo(n)}")
```

2.1.23 Conteggio cifre di un numero

Dato in input un numero intero, conta da quante cifre è composto.

Soluzione

```
# Dato in input un numero intero, conta da quante cifre è composto.

num = int(input("Scrivi un numero: "))

tmp = num
ncifre = 1
while tmp >= 10:
    tmp /= 10
    ncifre += 1

print(f"{num} è composto da {ncifre} cifre")
```

2.1.24 Stampa cifre di numero di 3 cifre

Dato in input un numero intero n di 3 cifre (in questo caso sarebbe utile mettere un controllo sull'input, cioè continuare a richiedere il numero fintanto che il numero dato non è di 3 cifre), stampa separatamente le sue cifre. Consiglio: usa divisioni per 10 e resti della divisione per 10. Puoi provare anche a generalizzare il programma in modo che funzioni anche con numeri di dimensione diversa da 3 cifre.

2.1.25 Stampa cifre generico

Dato in input un numero n intero, stampa separatamente le sue cifre.

Soluzione

```
# 2.1.25    Stampa cifre generico
# Dato in input un numero n intero, stampa separatamente le sue cifre.

num = int(input("Scrivi un numero: "))

pot10 = 1
tmp = num
while tmp > 10:
    tmp /= 10
    pot10 *= 10

tmp = num
while pot10 > 0:
    cifra = tmp // pot10
    print(cifra, end = " ")

    tmp %= pot10
    pot10 //= 10
```

2.1.26 Primalità di un numero

Dato in input un numero intero n, stabilisci se è primo

Soluzione

```
n = int(input("Scrivi un numero maggiore di 1: "))
while n <= 1:
    n = int(input("Scrivi un numero maggiore di 1: "))

primo = True
for i in range(2, (n//2)+1):
    if n % i == 0:
        primo = False
        break

if primo:
    print(f"{n} è primo")
else:
    print(f"{n} non è primo")
```

2.1.27 Scomposizione in fattori primi

Dato in input un numero intero n, stampa la sua scomposizione in fattori primi

Variante: scrivi una funzione che restituisca la lista dei fattori

Soluzione

Versione base

```
num = int(input("Scrivi un numero: "))

tmp = num
for fattore in range(2, num+1):
    while tmp % fattore == 0:
        tmp /= fattore
        print(fattore, end = ' ')
```

Variante

```
def fattoriPrimi(n):
    tmp = n
    primi = []
    for fattore in range(2, num+1):
        while tmp % fattore == 0:
            tmp /= fattore
            # primi.append(fattore)
            primi += [fattore]
    return primi

num = int(input("Scrivi un numero: "))
print(fattoriPrimi(num))
```

2.1.28 Maggiore, minore e media di sequenza in input lunga n

Data una sequenza di n numeri (n dato dall'utente) stabilire qual è il numero più grande, qual è il più piccolo e calcolare la media.

2.1.29 Secondo più grande e secondo più piccolo di sequenza in input lunga n

Data una sequenza di n numeri (n dato dall'utente) stabilire qual è il secondo numero più grande, qual è il secondo più piccolo.

2.1.30 Maggior differenza tra numeri consecutivi

Data una sequenza di numeri positivi (fai il controllo sull'input) terminati da 0, scrivi qual è la maggior differenza tra due numeri dati consecutivamente

2.1.31 Somma dei numeri divisibili per 3

Data una sequenza di numeri positivi (fai il controllo sull'input) terminati da 0, scrivi la somma dei numeri divisibili per 3

2.1.32 Somma e prodotto dei numeri minori di n

Dato un numero n scrivi somma e prodotto di tutti i numeri minori o uguali a n.

2.1.33 Quanto si può dividere per 2

Dato un numero n stabilire quante volte è possibile dividerlo per 2. (esempio 20 è divisibile per 2, 2 volte)

2.1.34 Calcolo spesa con sconti

Data una sequenza di prezzi di prodotti, calcolare la spesa totale sapendo che se un prodotto costa meno di 100€ lo devo scontare del 10% altrimenti del 5%. Decidi tu il metodo per capire quando terminare la lettura della sequenza di prezzi.

1.1.1 Promosso, rimandato o bocciato

Data una sequenza di 5 numeri che rappresentano i voti presi nelle diverse materie, stabilire se lo studente sarà promosso, bocciato o rimandato a settembre. Lo studente è promosso se non ha insufficienze, è bocciato se ha almeno 3 insufficienze, altrimenti è rimandato. Ricordati di controllare i valori dei voti in input che devono essere voti validi.

2.1.35 Lettura e stampa di stringhe

Leggi 3 parole e stampale tutte insieme in un'unica volta (solo un parallelogramma di scrittura).

2.1.36 Stampa sequenze e multipli

Dato un numero n positivo stampa tutti i numeri da 1 a n, i primi n multipli di 2 (2 compreso che consideri il primo multiplo) e i primi n multipli di 3

Soluzione

```
n = 10

# numeri da 1 a n
for i in range(1, n+1):
    print(f"{i:2}", end = ' ')
print()

# primi n multipli di 2
for i in range(2, n*2 + 1, 2):
    print(f"{i:2}", end = ' ')
print()

# primi n multipli di 3
for i in range(3, n*3 + 1, 3):
    print(f"{i:2}", end = ' ')
print()

# primi n multipli di 3 col while
i = 3
stampati = 0
while stampati < n:
    print(f"{i:2}", end = ' ')
    i += 3
    stampati += 1
print()
```

2.1.37 Terna pitagorica

Dati tre numeri positivi verificare che questi tre numeri siano una terna pitagorica (una terna pitagorica è un insieme di 3 numeri per cui la somma del quadrato di due numeri sia uguale al quadrato del terzo numero, in altre parole sono le lunghezze dei lati di un triangolo rettangolo)

2.1.38 Calcolo spesa con sconti 2

Scrivere l'algoritmo per il pagamento della spesa che consiste nel chiedere inizialmente se si è in possesso della carta fedeltà, poi chiedere tutti i prezzi dei prodotti acquistati terminando la sequenza con un prezzo uguale a zero. Il programma deve sommare i prezzi, e se si è in possesso della carta, scontare del 10% i prezzi minori di 50 e del 5% i prezzi maggiori di 50

2.1.39 Restituzione in biblioteca

Un libro deve essere restituito in biblioteca dopo 15 giorni di prestito altrimenti si è multati di 0,80€ al giorno di ritardo. Ricevuto in ingresso il numero di giorni di un prestito, visualizza se il socio deve essere multato per il ritardo e a quanto ammonta la multa da pagare.

2.1.40 Calcolo bolletta

Calcolare il costo della bolletta telefonica sapendo che i primi 30 scatti costano 20 centesimi l'uno, gli scatti dal 31 al 100 costano 15 centesimi l'uno, mentre gli ulteriori scatti costano 10 centesimi l'uno. Aggiungere infine una tassa fissa di 2,50€ per le spese telefoniche. In input al programma è dato il numero di scatti effettuati.

2.1.41 Indovina il numero segreto

Costruire uno schema di flusso che rappresenti l'algoritmo per il seguente gioco: il computer genera un numero segreto (usa la funzione randint(0, 99) che genera un numero casuale tra 0 e 99, devi importarla da random con "from random import randint") e il giocatore deve individuarlo seguendo le indicazioni fornite dal computer (ti dice se il numero da trovare è più grande o più piccolo di quello che hai provato); una volta trovato il numero segreto, il numero di tentativi effettuati è visualizzato a video.

Soluzione

```
from random import randint

segreto = randint(0, 99)

ntentativi = 0
while True:
    tentativo = int(input("Inserisci un numero tra 0 e 99: "))
    ntentativi += 1
    if tentativo < segreto:
        print("Il numero segreto è più grande")
    elif tentativo > segreto:
        print("Il numero segreto è più piccolo")
    else:
        print("Complimenti! Hai indovinato il numero segreto in", ntentativi, "tentativi")
        break
```

2.1.42 Quoziente e resto con le differenze

Calcolare il quoziente e il resto della divisione intera di due numeri interi positivi forniti in ingresso chiamati dividendo e divisore, applicando il metodo delle sottrazioni successive. Per esempio, se dividendo=13 e divisore=5, il programma dovrà restituire Quoziente=2 e Resto=3, calcolati sottraendo successivamente il valore di divisore dal valore di dividendo

2.1.43 Successione di Fibonacci 2

Visualizza i termini della successione di Fibonacci compresi nell'intervallo tra due interi positivi N1 e N2, entrambi forniti in ingresso, con N2>N1 (controlla bene gli input)

2.1.44 Verifica probabilità del 6

Simula il lancio di un dado per N volte (con N intero e positivo in ingresso e usando la funzione random(7) per il lancio) e verifica che effettivamente la probabilità che esca 6 è 1/6. Per farlo fai tanti tiri e vedi se esce 6 un sesto delle volte (più tiri fai più dovresti avere un risultato positivo).

2.1.45 Calcolo probabilità dado

Simula il lancio di due dadi e verifica le probabilità che escano: una coppia di 6, un valore totale uguale a 7 (due diverse probabilità)

2.1.46 Calcolo probabilità dadi

2.1.47 Multipli del minore tra x e y

Dati in ingresso due numeri positivi x e y, visualizza in ordine decrescente la sequenza di numeri interi compresi tra x e y che sono divisibili per il minore tra x e y. Ad esempio, se x = 7 e y = 35, la sequenza è 35 28 21 14 7.

Soluzione

```
x = int(input("Scrivi il primo numero: "))
while x <= 0:
    x = int(input("Scrivi il primo numero: "))

y = int(input("Scrivi il secondo numero: "))
while y <= 0:
    y = int(input("Scrivi il primo numero: "))

if x < y:
    min = x
    max = y
else:
    min = y
    max = x

for i in range(max, min-1, -1):
    if i % min == 0:
        print(i, end=" ")
```

2.1.48 Sequenza numeri con condizioni

Dati in ingresso due numeri positivi x e y, visualizza in ordine decrescente la sequenza di numeri interi compresi tra x e y che sono divisibili per il minore tra x e y. Ad esempio, se x = 7 e y = 35, la sequenza è 35 28 21 14 7.

2.1.49 Calcolo probabilità dado 2

Simula una serie di lanci di un dado a 6 facce (d6) e un dado da 20 facce (d20). Calcola il valore medio che si ottiene lanciando il d6 e quello ottenuto lanciando il d20.

2.1.50 Calcolo probabilità dadi 2

Valuta se è più probabile ottenere 7 oppure ottenere 8 lanciando 2 dadi da 6 e sommando i 2 valori ottenuti.

2.1.51 Calcolo probabilità dadi 3

Voglio valutare la probabilità che tirando due dadi da 6 esca il valore 2. Il valore calcolato deve essere abbastanza preciso e decido di calcolarlo misurando la media di tanti tiri di dado e fermandomi solo quando vedo che facendo uscire un nuovo 2 valore calcolato non varia di più di 0,01%

2.1.52 media condizionale in sequenza in input lunga n

Vengono dati in input i valori delle altezze di n persone (n chiesto all'utente). Per ogni persona oltre all'altezza viene indicato anche il sesso. Deve essere stampato il valore medio di altezza separatamente per i maschi e per le femmine. L'utente può decidere di non indicare né maschio né femmina, in quel caso non contare quella persona.

2.1.53 Gioco coi dadi

Scrivi il codice di un gioco per 2 persone basato sul lancio di dadi. All'inizio del gioco si decide che tipo di dado usare, si possono scegliere i dadi da 4, 6, 8 o 20 facce. Il gioco consiste nel tirare uno per volta il dado e sommando ogni volta i numeri usciti finchè uno dei due giocatori non supera un valore limite che è 4 volte il valore massimo rappresentato sul dado scelto (ad esempio se si sceglie il dado da 6 il limite è 24). Per rendere il gioco equo si può fare in modo che i due giocatori debbano tirare per forza lo stesso numero di tiri e che entrambi i giocatori possano superare il limite facendo finire il gioco in parità

2.1.54 If con espressione logica

Scrivi una variante del gioco precedente in cui l'obiettivo è avvicinarsi il più possibile al valore limite senza superarlo. Chi si avvicina di più vince. In ogni momento un giocatore può decidere di accontentarsi del valore ottenuto e smettere di tirare.

2.1.55 If con espressione logica 2

Leggi un numero positivo e controlla se esso è divisibile per 2 o divide 3 e non è compreso tra 10 e 20

2.1.56 If con espressione logica 3

Leggi un numero positivo e controlla se esso è divisibile per 2 o è un multiplo di 3 ed è compreso tra 10 e 20

2.1.57 If con espressione logica in sequenza

dato un numero n stampa tutti i numeri da 1 a n che sono divisibili da 2 o 3 o 5 ma non sono divisibili per 4

2.1.58 Figure geometriche

Dato un numero positivo n disegna le figure geometriche riportate di seguito:

```
per n = 5
xxxxx                    xxxxx                    x
xxxx                     xxxxx                    xx
xxx                      xxxxx                    xxx
xx                       xxxxx                    xxxx
x                        xxxxx                    xxxxx
```

Soluzione

```python
def f1(n):
    for i in range(n):
        for j in range(n-i):
            print("x", end="")
        print()

def f2(n):
    for i in range(n):
        for j in range(n):
            print("x", end="")
        print()

def f3(n):
    for i in range(n):
        for j in range(1+i):
            print("x", end="")
        print()


f1(5)
f2(5)
f3(5)
```

2.1.59 Figure geometriche 2

Esegui un programma che dato in input un numero n produca in output un quadrato n x n di numeri e + analogo al seguente (esempio con n = 6):

```
1 2 3 4 5 6
```

```
2 3 4 5 6 +
3 4 5 6 + +
4 5 6 + + +
5 6 + + + +
6 + + + + +
```

Soluzione

```python
n = 6

for i in range(n):
    for j in range(n):
        x = j+1+i
        if x <= n:
            print(x, end=" ")
        else:
            print("+", end=" ")
    print()
```

2.1.60 Indovina l'età

Scrivere una funzione guessMyAge() che deve indovinare la mia età e stampare in quanti tentativi ha indovinato la mia età.

Nel programma principale chiedere all'utente di inserire la sua età e poi invoca la funzione guessMyAge() per indovinare il valore inserito.

2.1.61 Gioco coi dadi 2

Simula l'esecuzione di un gioco di dadi in cui due persone si scontrano lanciando ognuno un dado. Il gioco è diviso in round. Al primo round i due giocatori lanciano ognuno un dado da venti facce e vince chi fa il numero più alto, in caso di pareggio ritirano i dadi finchè uno dei due non vince. Chi perde il round deve affrontare i round successivi con un dado con una faccia in meno. Perde la partita chi arriva ad avere un dado con 0 facce. Mostra ad ogni round quante facce hanno i dadi dei due giocatori e mostra il risultato di ogni lancio dei dadi. Dichiara infine il vincitore.

2.1.62 Figure geometriche 3

Scrivi un programma che dato un numero n stampi una piramide come nel seguente esempio:

```
Con n = 3
  1
 121
12321
```

2.1.63 Figure geometriche 4

Scrivere un programma che legga due interi n e m con valori compresi tra 1 e 9, i cui prodotto sia inferiore a 35, e stampi m piramidi di altezza n. L'esempio si riferisce al caso n=3 e m=4.

```
  1    1    1    1
 121  121  121  121
12321123211232112321
```

2.1.64 Input e condizione

Scrivi un programma che permetta di controllare i dati di input immessi dall'utente e:

1. se l'utente inserisce un intero N compreso tra 1 e 10 (inclusi), il programma deve stampare a video il valore N^N,
2. se l'intero N è compreso tra 11 e 20 (inclusi), il programma deve stampare a video la somma 1 + 2 + 3 +… + N
3. altrimenti deve dare un segnale di errore.

2.1.65 Calcolo sommatoria matematica

Si realizzi un programma che legga un intero N da tastiera, e stampi a video il risultato della seguente sommatoria:

$$\sum_{i=0}^{N} \left[(-1)^i \frac{4}{2 * i + 1} \right]$$

Una volta calcolato e stampato il valore a video, il programma deve chiedere un nuovo numero all'utente e ripetere il calcolo. Il programma deve terminare solo qualora l'utente inserisca un valore negativo.

2.1.66 Calcolo interesse cumulato

Si progetti in C un programma che legge un float, rappresentante un ammontare di euro; di seguito il programma deve leggere un tasso d'interesse (in percentuale), ed un numero di anni. Il programma deve stampare, in uscita, per ogni anno, come l'ammontare cresce con gli interessi. Si ricordi che l'interesse si calcola con la seguente formula:

$$C_{fin} = C_{in} (1 + (r/100))^N$$

dove C_{fin} è il capitale finale, C_{in} è quello iniziale, r è l'interesse, e N rappresenta il numero di anni in cui si applicano gli interessi.

Ad esempio supponiamo che il capitale iniziale sia di 1000.0 €, con un tasso del 3%, per un periodo di 3 anni. L'output stampato deve avere all'incirca questo aspetto:

```
Capitale iniziale: 1000.00€
Dopo 1 anno: 1030.00 €
Dopo 2 anni: 1060.90 €
Dopo 3 anni: 1092.73 €
```

2.1.67 Input sequenza non 0, ASCII code

Realizzare un programma che legga da input un carattere dell'alfabeto e stampi a video il carattere stesso ed il suo valore ASCII. Il programma deve controllare che il carattere inserito sia compreso tra 'a' e 'z' o tra 'A' e 'Z' (in caso contrario si stampi un messaggio di errore). Dopo la stampa, il programma deve continuare a chiedere nuovi caratteri, finché l'utente non inserisce il carattere corrispondente al numero zero ('0'): in tal caso il programma termina.

2.1.68 Input sequenza, calcolo più lunga sottosequenza

Realizzare un programma che prende in input una sequenza di caratteri '0' e '1' e conta la lunghezza della più lunga sotto-sequenza di '0' di fila L'inserimento della sequenza termina

quando si inserisce un carattere diverso da ‘0’ e ‘1’ A quel punto, si stampa a video il valore trovato

2.1.69 Input sequenza, calcolo più lunga sottosequenza 2

Realizzare un programma che prenda in input una sequenza di cifre (tra 1 e 9) e calcoli la somma massima fra le sottosequenze di cifre non decrescenti Il programma termina quando viene inserito lo 0

Esempio:

2	2	4	5	3	9	3	1	5	0
13				12		3	6		

2.1.70 Input sequenza lunga n, quadrati perfetti

Data in input una sequenza di n numeri (n dato dall’utente), stampare per ognuno il numero di quadrati perfetti minori di quel numero.

1.1.2 Lancio moneta

Conta quanti tiri di moneta devi fare prima di vedere uscire testa per 10 volte di fila.

2.1.71 Calcolo probabilità Dado 3

Conta quanti tiri di un dado da 6 devi fare prima di vedere uscire lo stesso numero 5 volte di fila.

3 Funzioni

3.1 Funzioni semplici

3.1.1 Funzione somma

Scrivi e utilizza una funzione per calcolare e restituire la somma tra due numeri

```
def somma(a, b):
    return a+b

print(somma(3,5))
print(somma(3,7))
print(somma(5,5))
print(somma(1,5))
```

3.1.2 Input multipli e funzione

Scrivere un paio di funzioni che calcolano perimetro e area di un triangolo rettangolo. Chiedere all'utente l'inserimento dei due cateti (i cateti vanno inseriti in una sola riga, in modo da forzare l'utilizzo della funzione split()). Richiamare le funzioni e stampare i risultati.

Variante: Scrivi un'unica funzione che calcola e restituisce i valori di perimetro e area

Soluzione

```
import os
os.system('cls')

import math

def perimetro(c1, c2):
    ipo = math.sqrt(c1**2 + c2**2)
    return (c1+c2+ipo)

def area(c1, c2):
    return c1*c2/2

def perimetro_area(c1, c2):
    ipo = math.sqrt(c1**2 + c2**2)
    p = (c1+c2+ipo)
    a = c1*c2/2
    return (p, a)


# strcateti = input("Inserisci le lunghezze dei cateti: ")
# # [scateto1, scateto2] = strcateti.split()
# strcateti = strcateti.strip()
# scateto1, scateto2 = strcateti.split()
# cateto1 = float(scateto1)
# cateto2 = float(scateto2)

cateto1, cateto2 = list(map(float, (input("Inserisci le lunghezze dei cateti: ").strip().split())))

p = perimetro(cateto1, cateto2)
a = area(cateto1, cateto2)

p, a = perimetro_area(cateto1, cateto2)
print("Perimetro=", p)
print("Area=", a)
```

3.1.3 Stampa del maggiore

Scrivi una funzione che prende due numeri come parametro e manda in print il più grande tra i due.

```
def stampaMaggiore(a, b):
    if a > b:
        print(a)
    else:
        print(b)

stampaMaggiore(2,5)
x = stampaMaggiore(2,5)
print(x)
```

3.1.4 Massimo di una lista

Scrivi e usa una funzione che, passata come parametro una lista di interi, restituisce il maggiore tra i numeri contenuti nella lista.

3.1.5 Le 4 operazioni

Scrivi e utilizza 4 funzioni che effettuano le 4 operazioni e restituiscono il risultato.

3.1.6 Calcolatrice con simbolo

Scrivi e utilizza una funzione che dati due valori e un simbolo di una delle 4 operazioni effettua quella operazione e restituisce il risultato

Soluzione

```
def calcola(a,b,simbolo):
    if simbolo == '+':
        return a+b
    elif simbolo == '-':
        return a-b
    elif simbolo == '*':
        return a*b
    elif simbolo == '/':
        return a/b
    else:
        print(f"Operatore \"{simbolo}\" errato")
        return 0

a = float(input("Inserisci il primo numero: "))
b = float(input("Inserisci il secondo numero: "))
simbolo = (input("Inserisci l'operatore: "))

print(calcola(a,b,simbolo))
```

3.1.7 Punto medio

Funzione che date le coordinate di due punti nel piano, calcolino e restituiscano le coordinate del punto medio tra i due (senza approssimazioni).

Soluzione

```
def punto_medio(a, b):
    medio = ((a[0]+b[0])/2, (a[1]+b[1])/2)
    return medio

a=(1,2)
```

```
b=(2,2)

print(punto_medio(a,b))
```

```
def puntoMedio(a = (0,0), b = (0,0)):
    ris = ((a[0]+b[0])/2, (a[1]+b[1])/2)
    return ris

a = (1,4)
b = (3,4)

print(puntoMedio(a, b))
print(puntoMedio(b = (1,1)))
```

3.1.8 Caduta: altezza e velocità

Funzione che dato il tempo di caduta di un oggetto, calcola l'altezza da cui è caduto l'oggetto e la velocità di impatto. Le formule che ti servono sono: h = 1/2 * g * t^2, con g=9,81 e v=g*t.

Soluzione

```
def altezza_e_velocita(t):
    return (1/2*9.81*t**2, 9.81*t)



t = 1

h, v = altezza_e_velocita(t)

print(
    f"Se il tempo di caduta è {t} secondi, allora l'oggetto è caduto di {h} metri e ha urtato il
suolo ad una velocità di {v} m/s")
```

3.1.9 Oggetto che si rompe cadendo

Funzione che prevede se un oggetto si rompe cadendo da una certa altezza. Quello che ti serve sapere è quindi l'altezza da cui cade l'oggetto e la velocità massima di impatto che l'oggetto può sopportare. Le formule che ti servono sono: h = 1/2 * g * t^2, con g=9,81 e v=g*t.

Soluzione

```
def altezza_e_velocita(t):
    return (1/2*9.81*t**2, 9.81*t)



t = 2

max_v = 10

h, v = altezza_e_velocita(t)

print(
    f"Se il tempo di caduta è {t} secondi, allora l'oggetto è caduto di {h} metri e ha urtato il
suolo ad una velocità di {v} m/s")
```

```
if v > max_v:
    print("L'oggetto si è rotto all'impatto")
else:
    print("Dopo l'impatto l'oggetto è ancora intero")
```

3.1.10 Lunghezze delle parole

Scrivi una funzione che data in ingresso una lista A contenente n parole, restituisca in output una lista B di interi che rappresentano la lunghezza delle parole contenute in A.

3.1.11 Incremento di due numeri

Scrivi ed utilizza una funzione che dati due numeri li aumenta entrambi di un certo valore passato come parametro.

```
def raddoppia(numeri):
    for i, n in enumerate(numeri):
        numeri[i] *= 2
    return numeri

x = 5
y = 8
x,y = raddoppia([x,y])
print(x, y)
```

3.1.12 Massimo e minimo di un array

Scrivi e utilizza una funzione che dato un array di numeri, trova e restituisce il massimo e il minimo numero presenti.

3.1.13 Controllo iniziale maiuscola

Scrivi e utilizza una funzione che, data una stringa, restituisce true se la stringa inizia con una lettera maiuscola.

3.1.14 Caratteri separati da spazi

Funzione che data una stringa, la stampa separando ogni carattere della stringa con uno spazio.

3.1.15 Concatenazione di stringhe

Scrivi e utilizza una funzione che riceve come parametri due stringhe e restituisce una stringa che è la concatenazione delle altre due.

3.1.16 Media senza voti estremi

Scrivi un programma che, dati n voti in un array, li passi ad una funzione che restituisca la media dei voti escludendo il voto più alto e il voto più basso.

3.1.17 Stampa di una stringa

Scrivi e utilizza una funzione che stampa una stringa ricevuta come parametro.

3.1.18 Scambio di due variabili

Scrivi una funzione che scambia il contenuto di due variabili intere (i valori devono rimanere scambiati per chi chiama la funzione).

3.1.19 Ordinamento con funzioni

Scrivi un programma che sappia riordinare una lista. Per farlo dividi le seguenti funzionalità in diverse funzioni:

1. funzione che restituisce una lista di n numeri casuali
2. funzione che stampa una lista
3. funzione che riceve una lista e lo ordina, è possibile indicare se in ordine crescente o decrescente
4. main che usando le funzioni descritte sopra crea una lista, la stampa, la ordina e poi la ristampa.

Soluzione

```python
def creaLista(n = 0, min = 0, max = 99):
    from random import randint
    return [randint(min, max) for _ in range(n)]

# per cambiare non faccio una funzione stampa ma una funzione che restituisce
# la stringa che rappresenta la lista
def listaToStr(lista, parentesi = '[]', separatore = ', '):
    ris = ""
    ris += parentesi[0]
    for i, num in enumerate(lista):
        ris += str(num)
        if i < len(lista) - 1:
            ris += separatore
    ris += parentesi[1]
    return ris

def mySort(lista, crescente = True):
    # if crescente:
    #     lista.sort()
    # else:
    #     lista.sort(reverse = True)

    lista.sort(reverse = not(crescente))

lista = creaLista(10, max = 9)
print(listaToStr(lista, '()', '; '))
mySort(lista, False)
print(listaToStr(lista, separatore = '; '))
```

3.1.20 Massimo e minimo di un array 2

Scrivi e utilizza una funzione che dato un array restituisce i valori massimi e minimi dell'array

3.1.21 Quoziente e resto

Scrivi e utilizza una funzione che calcola il quoziente tra due numeri interi e restituisce, oltre al quoziente anche il resto della divisione.

Soluzione

```
def divisione_intera(a,b):
    quoziente = a // b
    resto = a % b
    return (quoziente, resto)

a = 22
b = 3
q, r = divisione_intera(a,b)
print(f"{a}/{b} = {q} resto {r}")
```

3.1.22 Prima lettera maiuscola

Scrivi e utilizza una funzione che data una stringa, fa in modo che la prima lettera sia maiuscola e tutte le altre minuscole. i caratteri che non sono lettere non devono essere modificati.

3.1.23 MCD e semplificazione frazioni

Scrivi una funzione che calcola il massimo comune divisore tra due numeri.
Scrivi ed utilizza poi una funzione che dati due valori numeratore e denominatore, semplifica la frazione, usando la funzione mcd appena definita.

3.1.24 Tipo di triangolo

Scrivi ed utilizza una funzione che riceva come parametri tre numeri e che restituisca true se questi numeri possono essere le misure di un triangolo. In questo caso la funzione deve anche restituire una stringa in cui c'è scritto il tipo di triangolo (equilatero, isoscele, scaleno).

3.1.25 Parole mescolate a caso

Scrivi ed utilizza una funzione che riceve una stringa e restituisce un'altra stringa ottenuta mischiando in ordine casuale le parole della stringa ricevuta. Attento che la stringa originale non va persa.

3.1.26 Estrazione interrogati

Scrivi ed utilizza una funzione che ti permetta di scegliere a caso quali studenti interrogare da una lista di nomi di studenti ricevuta in input. Oltre alla lista dei nomi la funzione riceve anche il numero di studenti da interrogare. È possibile interrogare tante volte uno studente prima di interrogarne altri. La lista infine restituisce la lista dei nomi degli studenti da interrogare.

3.1.27 Estrazione interrogati a giri

Scrivi ed utilizza una funzione che ti permetta di scegliere a caso quali studenti interrogare da una lista di nomi di studenti ricevuta in input. Oltre alla lista dei nomi la funzione riceve anche il numero di studenti da interrogare. Se il numero di studenti da interrogare supera il numero di studenti, è possibile riestrarre studenti già interrogati, ma non prima di averli interrogati tutti (con questo metodo si possono anche fare tanti "giri" di interrogazioni). La lista infine restituisce la lista dei nomi degli studenti da interrogare.

3.2 Funzioni ricorsive

3.2.1 Stampa lista ricorsiva

Scrivi e utilizza tre diverse versioni di una funzione che stampa una lista elemento per elemento. Le tre versioni sono:

1. Versione iterativa, cioè che usa un ciclo for
2. Versione ricorsiva che riceve ogni volta una lista più piccola della precedente (prima quella intera poi quella senza il primo elemento…)
3. Versione ricorsiva che riceve ogni volta la stessa lista ma sono espressi anche gli estremi della lista (inizio e fine) su cui lavorare.

Soluzione

```
# 6.1.1 - Scrivi e utilizza tre diverse versioni di una funzione
# che stampa una lista elemento per elemento. Le tre versioni sono:

#    1. Versione iterativa, cioè che usa un ciclo for
def stampaIterativa(lista):
    for el in lista:
        print(el, end = " ")

#    2. Versione ricorsiva che riceve ogni volta una lista più
# piccola della precedente (prima quella intera poi quella senza il primo
# elemento…)
def stampaRicorsiva(lista):
    if len(lista) < 1:
        return
    else:
        print(lista[0], end = " ")
        stampaRicorsiva(lista[1:])

#    3. Versione ricorsiva che riceve ogni volta la stessa
# lista ma sono espressi anche gli estremi della lista (inizio e fine) su cui
# lavorare.
def stampaRicorsiva2(lista, inizio, fine):
    if len(lista) < 1 or inizio > fine:
        return
    else:
        print(lista[inizio], end = " ")
        stampaRicorsiva2(lista, inizio+1, fine)
```

```
numeri = [1,2,3,4,5]
stampaIterativa(numeri)
print()
stampaRicorsiva(numeri)
print()
stampaRicorsiva2(numeri, 0, len(numeri)-1)
print()
```

3.2.2 Stampa lista al contrario

Fai la stessa cosa dell'esercizio 3.2.1 ma stampa la lista al contrario.

Soluzione

```
# 6.1.2 - Fai la stessa cosa dell'esercizio 6.1.1
# ma stampa la lista al contrario.

#    1. Versione iterativa, cioè che usa un ciclo for
def stampaIterativa(lista, reverse = False):
    if not reverse:
        for el in lista:
            print(el, end = " ")
    else:
        for el in lista[::-1]:
            print(el, end = " ")

#    2. Versione ricorsiva che riceve ogni volta una lista più
# piccola della precedente (prima quella intera poi quella senza il primo
# elemento…)
def stampaRicorsiva(lista, reverse = False):
    if len(lista) < 1:
        return
    else:
        if not reverse:
            print(lista[0], end = " ")
            stampaRicorsiva(lista[1:], reverse)
        else:
            print(lista[-1], end = " ")
            stampaRicorsiva(lista[:-1], reverse)

#    3. Versione ricorsiva che riceve ogni volta la stessa
# lista ma sono espressi anche gli estremi della lista (inizio e fine) su cui
# lavorare.
def stampaRicorsiva2(lista, inizio = 0, fine = -1, reverse = False):
    # se fine è negativo va trasformato nella versione equivalente positiva
    if fine < 0:
        fine = len(lista)+fine # + perchè il numero è già negativo

    # se no qui fa subito return
    if inizio > fine:
        return
    elif inizio == fine:
        print(lista[inizio], end = " ")
```

```
    else:
        if reverse:
            print(lista[fine], end = " ")
            stampaRicorsiva2(lista, inizio, fine-1, reverse)
        else:
            print(lista[inizio], end = " ")
            stampaRicorsiva2(lista, inizio+1, fine, reverse)

numeri = [1,2,3,4,5]
print("Iterativa")
stampaIterativa(numeri)
print()
stampaIterativa(numeri, True)
print()
stampaIterativa(numeri, reverse = True)
print()
print()
print("Ricorsiva 1")
stampaRicorsiva(numeri)
print()
stampaRicorsiva(numeri, True)
print()
print()

print("Ricorsiva 2")
stampaRicorsiva2(numeri)
print()
stampaRicorsiva2(numeri, reverse = True)
print()
stampaRicorsiva2(numeri, inizio = 2, reverse = True)
print()
```

3.2.3 Fattoriale ricorsivo

Scrivi ed utilizza una funzione che calcola il fattoriale di un numero. Scrivi due versioni della funzione, una iterativa e una ricorsiva.

3.2.4 Massima differenza tra consecutivi

Scrivi e utilizza le versioni iterativa e ricorsiva di una funzione che dato un array di interi, calcola e restituisce la massima differenza tra due numeri consecutivi.

Soluzione

```
# 6.1.4 Scrivi e utilizza le versioni iterativa e ricorsiva di
# una funzione che dato un array di interi, calcola e restituisce
# la massima differenza tra due numeri consecutivi.

from random import randint
import os;
os.system('cls')

def maxdiffIterativa(lista):
    if len(lista) < 2:
```

```
        return 0

    ris = abs(lista[1] - lista[0])
    for i in range(1, len(lista)-1):
        diff = abs(lista[i+1] - lista[i])
        if diff > ris:
            ris = diff
    return ris

def maxdiffRicorsiva(lista):
    if len(lista) < 2:
        return 0
    diff = abs(lista[1]-lista[0])
    return max(diff, maxdiffRicorsiva(lista[1:]))

lista = [randint(0,9) for _ in range(10)]

print(lista)
print(maxdiffIterativa(lista))
print(maxdiffRicorsiva(lista))
```

3.2.5 Fibonacci ricorsivo

Scrivi ed utilizza una funzione che calcola l'ennesimo numero della successione di Fibonacci. Scrivi due versioni della funzione, una iterativa e una ricorsiva.

3.2.6 Moltiplicazione ricorsiva

Scrivi una funzione ricorsiva che effettui la moltiplicazione tra due numeri

Soluzione

```
# 6.1.6 - Scrivi una funzione ricorsiva che effettui la
# moltiplicazione tra due numeri

def moltiIterativa(a, b):
    # controllo dominio
    if b < 0:
        b = -b
        a = -a
    ris = 0
    for i in range(b):
        ris += a
    return ris

def moltiRicorsiva(a, b):
    # controllo dominio
    if b < 0:
        b = -b
        a = -a

    # caso finale
    if a == 0 or b == 0:
        return 0
```

```
    # ricorsione
    return a + moltiRicorsiva(a, b-1)

print(moltiIterativa(2, 5))
print(moltiRicorsiva(2, -5))
```

3.2.7 Divisione ricorsiva

Scrivi una funzione che calcola ricorsivamente il quoziente tra due numeri utilizzando le sottrazioni.

Soluzione

```
# 6.1.7 - Scrivi una funzione che calcola ricorsivamente il
# quoziente tra due numeri utilizzando le sottrazioni.

def divIterativa(a, b):
    ris = 0
    while a > b:
        ris += 1
        a -= b
    return (ris, a)

def divRicorsiva(a, b):
    a = abs(a)
    b = abs(b)

    if a < b:
        return (0, a)

    q, r = divRicorsiva(a-b, b)
    return (1 + q, r)

a = 10
b = 3
q, r = divIterativa(a, b)
print(f"{a}/{b} = {q}  resto {r}")

q, r = divRicorsiva(a, b)
print(f"{a}/{b} = {q}  resto {r}")
```

3.2.8 Inversione lista ricorsiva

Scrivi e utilizza le versioni iterativa e ricorsiva di una funzione che riceve un array e inverte l'ordine dei numeri in esso contenuti.

Soluzione

```
# 6.1.8 - Scrivi e utilizza le versioni iterativa e ricorsiva di
# una funzione che riceve un array e inverte l'ordine dei numeri
# in esso contenuti.

import os;
os.system('cls')
```

```python
def invertiListaI(lista, inizio = 0, fine = -1):
    if fine < 0:
        fine = len(lista)+fine

    for i in range(inizio, fine//2 + 1):
        lista[i], lista[fine-i] = lista[fine-i], lista[i]

def invertiListaR(lista, inizio = 0, fine = -1):
    if fine < 0:
        fine = len(lista)+fine

    if inizio >= fine or len(lista) < 1:
        return

    lista[inizio], lista[fine] = lista[fine], lista[inizio]
    invertiListaR(lista, inizio+1, fine-1)

lista = [1,2,3,4,5,6]
print(lista)
invertiListaI(lista)
print(lista)
invertiListaR(lista, 1, -2)
print(lista)

print()

lista = [1,2,3,4,5,6]
print(lista)
invertiListaR(lista)
print(lista)
invertiListaR(lista, 1, -2)
print(lista)
```

3.2.9 Massimo e minimo ricorsivi

Funzione ricorsiva per trovare i valori massimi e minimi di un vettore.

Soluzione

```python
# 6.1.9 - Funzione ricorsiva per trovare i valori massimi e
# minimi di un vettore.

def massimo(lista):
    if len(lista) < 1:
        return None
    if len(lista) == 1:
        return lista[0]
    else:
        resto = massimo(lista[1:])
        if lista[0] > resto:
            return lista[0]
        else:
            return resto
```

```
print(massimo([1,6,3,8,3]))
```

3.2.10 Ricerca lineare ricorsiva

Funzione ricorsiva per cercare un valore all'interno di un vettore non ordinato (ricerca lineare). Risultato: None se non lo trova, altrimenti l'indice della posizione dove è stato trovato.

3.2.11 Ricerca dicotomica ricorsiva

Scrivere una funzione ricorsiva per cercare un valore all'interno di un vettore ordinato (ricerca dicotomica). Risultato: None o l'indice.

Indicazioni: se il vettore è ordinato, iniziate a controllare il numero a metà del vettore, se il numero controllato è più piccolo di quello da trovare, cercherò nella parte del vettore a sinistra (richiamo la funzione indicando come intervallo in cui cercare solo la parte a sinistra), se è più piccolo dovrò cercare a destra, se è uguale l'ho trovato.

Soluzione

```
# 6.1.11    Scrivere una funzione ricorsiva per cercare un valore all'interno di un
# vettore ordinato (ricerca dicotomica). Risultato: None o l'indice.
# Indicazioni: se il vettore è ordinato, iniziate a controllare il numero a metà del
# vettore, se il numero controllato è più piccolo di quello da trovare, cercherò
# nella parte del vettore a sinistra (richiamo la funzione indicando come intervallo
# in cui cercare solo la parte a sinistra), se è più piccolo dovrò cercare a destra,
# se è uguale l'ho trovato.

from random import randint

def contieneRic(lista, el):
    if len(lista) < 1:
        return False

    imid = len(lista) // 2
    if lista[imid] == el:
        return True

    if el < lista[imid]:
        return contieneRic(lista[:imid], el)
    else:
        return contieneRic(lista[imid+1:], el)


def cercaPosRic(lista, el, i = 0, f = -1):
    if f < 0:
        f = len(lista)+f

    if i >= f:
        if lista[i] == el:
            return i
        else:
            return -1
```

```
    imid = (i+f) // 2
    if lista[imid] == el:
        return imid

    if el < lista[imid]:
        return cercaPosRic(lista, el, i, imid-1)
    else:
        return cercaPosRic(lista, el, imid+1, f)


lista = [randint(0,99) for _ in range(20)]
lista.sort()
print(lista)
print(contieneRic(lista, 10))
print(cercaPosRic(lista, 10))
```

3.2.12 Tutti dispari ricorsivo

Scrivere una funzione ricorsiva che riceve un array di interi e restituisce true se gli elementi sono tutti dispari e false se non lo sono.

Soluzione

```
# Scrivere una funzione ricorsiva che riceve un array di
# interi e restituisce true se gli elementi sono tutti dispari e false se non lo
# sono.

def tuttiDispari(lista):
    if len(lista) < 1:
        return False
    if len(lista) == 1:
        return lista[0] % 2 != 0
    else:
         return lista[0] % 2 != 0 and tuttiDispari(lista[1:])

def tuttiDispariIterativa(lista):
    for num in lista:
        if num % 2 == 0:
            return False
    return True

print(tuttiDispari([1,2,7]))
```

3.2.13 Prodotto dei negativi ricorsivo

Scrivere una funzione ricorsiva che riceve un array di interi e restituisce il prodotto degli elementi negativi.

Soluzione

```
# 6.1.13 - Scrivere una funzione ricorsiva che riceve un array di
# interi e restituisce il prodotto degli elementi negativi.
```

```
def prodottoNegativi(lista):
    if len(lista) < 1:
        return None
    elif len(lista) == 1:
        if lista[0] < 0:
            return lista[0]
        else:
            return 1 # se non metto questo uno ma metto 0 moltiplica per 0 e va sempre a 0, se metto None cerca di moltiplicare sempre per None e da errore
    else:
        if lista[0] < 0:
            return lista[0] * prodottoNegativi(lista[1:])
        else:
            return prodottoNegativi(lista[1:])

print(prodottoNegativi([1,-4,3,-2,7]))
```

3.2.14 Selection sort ricorsivo

Scrivi le versioni iterativa e ricorsiva del selection sort.

Soluzione

```
import os;
os.system('cls')

def posmin(lista, inizio, fine):
    imin = inizio
    for i in range(inizio, fine+1):
        if lista[i] < lista[imin]:
            imin = i
    return imin

def selectionSortR(lista, inizio = 0, fine = -1):
    if fine < 0:
        fine = len(lista) + fine

    if inizio >= fine:
        return

    # imin = inizio
    # for i in range(inizio+1, fine+1):
    #     if lista[i] < lista[imin]:
    #         imin = i

    imin = posmin(lista, inizio, fine)
    lista[imin], lista[inizio] = lista[inizio], lista[imin]
    selectionSortR(lista, inizio+1, fine)


lista = [3,7,4,9,1]
selectionSortR(lista)
print(lista)
```

3.2.15 Bubble sort ricorsivo

Scrivi le versioni iterativa e ricorsiva del bubble sort.

Soluzione

```
# 6.1.15 - Scrivi le versioni iterativa e ricorsiva del bubble
# sort.

def bubbleSortIterativo(lista):
    for i in range(len(lista)-1):
        for j in range(len(lista)-1):
            if lista[j] > lista[j+1]:
                lista[j], lista[j+1] = lista[j+1], lista[j]
    return

def bubbleSortRicorsivo(lista, inizio = 0, fine = -1):
    if fine < 0:
        fine = len(lista)+fine

    if fine <= inizio:
        return

    for j in range(inizio, fine):
         if lista[j] > lista[j+1]:
             lista[j], lista[j+1] = lista[j+1], lista[j]

    return bubbleSortRicorsivo(lista, inizio, fine-1)

def bubbleSortRicorsivo2(lista):
    if len(lista) < 1:
        return []
    for j in range(len(lista)-1):
        if lista[j] > lista[j+1]:
            lista[j], lista[j+1] = lista[j+1], lista[j]

    return bubbleSortRicorsivo2(lista[:-1]) + [lista[-1]]


lista = [2,6,4,8,3]
bubbleSortRicorsivo(lista)
print(lista)
# print(bubbleSortRicorsivo2(lista))
```

3.2.16 Stampa matrice ricorsiva

Scrivi una funzione iterativa e una ricorsiva per la stampa di una matrice.

Soluzione

```
from random import randint
import os;
os.system('cls')


def stampaMatriceIterativo(mat):
    for riga in mat:
        for num in riga:
            print(num, end = " ")
        print()

def stampaRigaRicorsiva(riga, inizio = 0, fine = -1):
    if fine < 0:
        fine = len(riga) + fine

    if inizio > fine or len(riga) < 1:
        return

    print(riga[inizio], end = " ")
    stampaRigaRicorsiva(riga, inizio+1, fine)

def stampaMatriceRicorsiva(mat, inizio = 0, fine = -1):
    if fine < 0:
        fine = len(mat) + fine

    if inizio > fine or len(mat) < 1:
        return

    stampaRigaRicorsiva(mat[inizio])
    print()
    stampaMatriceRicorsiva(mat, inizio+1, fine)



n = randint(3,5)
m = randint(4,6)
mat = [[randint(0,9) for j in range(m)] for i in range(n)]

stampaMatriceIterativo(mat)
print()
stampaMatriceRicorsiva(mat)
```

3.2.17 Fibonacci con memoria

Scrivi e usa una funzione ricorsiva per il calcolo dell'ennesimo numero della successione di fibonacci che faccia uso della memoria per evitare che il tempo di calcolo sia esponenziale

3.2.18 Sottoparole crescenti

Si definisca la funzione ricorsiva (o che usa una vostra funzione ricorsiva) che presa in input una stringa di caratteri restituisce la lista delle diverse "sottoparole crescenti" di tale stringa. Le sottoparole devono comparire nella lista in ordine lessicografico (alfabetico).

Si ricorda che una sottoparola e' quello che si ottiene da una parola concellandone 0 o piu' caratteri (in testa, in coda o nel mezzo). Inoltre una sottoparola si dice crescente se i caratteri che la compongono letti da sinistra a destra risultano ordinati lessicograficamente.

Ad esempio con l'input "zanzara" si ottiene:

['a', 'aa', 'aaa', 'aar', 'an', 'anr', 'anz', 'ar', 'az', 'n', 'nr', 'nz', 'r', 'z', 'zz']

Soluzione

Questa soluzione prevede l'uso degli insiemi (set) poiché è comodo usare il loro metodo union

```
def f(parola):
    # inserisci qui il tuo codice
    return sorted(list(ricorsiva(parola)))

def ricorsiva(parola):
    if len(parola) == 0:
        return set()

    if len(parola) == 1:
        return {parola}

    tmp = "".join(sorted(parola))
    if tmp == parola:
        ris = {parola}
    else:
        ris = set()

    for i in range(len(parola)):
        sottoparola = parola[:i]+parola[i+1:]
        ris = ris.union(ricorsiva(sottoparola))

    return ris


print(f("zanzara"))
```

3.3 Algoritmi di ordinamento avanzati

Per ora questo è un capitolo vuoto, magari tratterò l'argomento in futuro

4 Strutture dati fondamentali: Liste, Stringhe, Matrici e Dizionari

4.1 Liste di soli numeri

4.1.1 Lettura lista voti e calcolo media

Leggi n voti, caricali in una lista, calcolane la media, poi stampa tutti i numeri e la media.

Soluzione

```
numeri = []
n = int(input("Dimmi quanti numeri vuoi inserire:"))
media = 0
for i in range(n):
    numero = int(input(f"Scrivi il {i+1}° numero: "))
    while numero < 1 or numero > 10:
        numero = int(input("\nVoto non valido, reinseriscilo: "))
    numeri.append(numero)
    media += numero
media /= n
print("\nI numeri inseriti sono:")
for numero in numeri:
    print(numero, end=" ")
# print("\nLa media dei numeri è", round(media, 2))
print(f"\nLa media dei numeri è {media:.2f}")
```

4.1.2 Letttura lista voti e stampa voti sufficienti

Continua l'esercizio precedente calcolando e stampando la media dei soli voti sufficienti.

Soluzione

```
voti = input("Inserisci i voti (in una sola riga separata da spazi): ")
voti = voti.strip().split()
print(voti)

# voti = list(map(float, voti))
voti = [float(voto) for voto in voti]
print(voti)
mediatot = sum(voti) / len(voti)
print(mediatot)

# voglio i soli voti sufficienti
suffs = [voto for voto in voti if voto >= 6]
# suffs = []
# for voto in voti:
```

```
#     if voto >= 6:
#         suffs.append(voto)
print(suffs)
mediasuffs = sum(suffs) / len(suffs)
print(mediasuffs)
```

4.1.3 Massimo e minimo dell'array

Dato un array di numeri interi casuali, calcola e stampa il valore massimo e il valore minimo contenuti nell'array

Variante: Definisci tre funzioni, una che restituiece il minimo, una che restituisce il massimo, una che li restituisce entrambi

Soluzione

```
import random

lista = [random.randint(0,9) for _ in range(10)]
print(lista)
max = lista[0]
min = lista[0]
for num in lista:
    if num > max:
        max = num
    if num < min:
        min = num

print(max, min)
```

Versioni un po' più elaborate che usano le funzioni

```
from random import randint

def miomin(a):
    min = None
    for el in a:
        if min == None or el < min:
            min = el
    return min

def miomax(a):
    max = None
    for el in a:
        if max == None or el > max:
            max = el
    return max

def minmaxtupla(a):
    return (min(a), max(a))
def minmaxlista(a):
    return [min(a), max(a)]
```

```
a = [randint(0,9) for _ in range(10)]
print(a)
print("massimo:", max(a))
print("minimo:", min(a))
print("massimo:", miomax(a))
print("minimo:", miomin(a))
print(minmaxtupla(a))
print(minmaxlista(a))
```

4.1.4 Media lista e lista dal fondo

Data una lista di numeri interi casuali, stampa tutti i numeri superiori alla media partendo dal fondo

Soluzione

```
import random

A = [random.randint(0, 99) for _ in range(20)]
print(A)

media = sum(A) / len(A)

print(media)

# for num in reversed(A):
for num in A[::-1]:
    if num > media:
        print(num, end=" ")
```

4.1.5 Inverti lista, lista da A a B, lista come anello

Carica una lista, di dimensione a piacere, di numeri casuali compresi in un range di numeri deciso dall'utente. Esegui poi le seguenti operazioni:

1. inverti l'ordine dei numeri contenuti nell'array,
2. Date due posizioni a e b dell'array (chiaramente controlla che a sia minore di b e che siano compresi tra 0 e N-1) stampa prima i numeri dell'array dalla posizione a alla posizione b poi i numeri dalla posizione b alla posizione a
3. (Difficile) Dato un numero positivo x qualsiasi stampa tutti i numeri dell'array dalla posizione 0 alla posizione x. Se x > N-1, dopo aver stampato tutto l'array ricomincia a stampare dalla posizione 0 e continua fino ad aver stampato x+1 numeri.

Soluzione

```
from random import randint



## Metodo C++ style
# lista = []
# for _ in range(10):
```

```
#     lista.append(randint(0,9))

## Metodo Python style
lista = [randint(0,9) for _ in range(10)]
print(lista)

# 1.    inverti l'ordine dei numeri contenuti nell'array,
lista = lista[::-1]
# lista.reverse()
print(lista)

# 2.    Date due posizioni a e b dell'array (chiaramente controlla che a sia minore di
# b e che siano compresi tra 0 e N-1) stampa prima i numeri dell'array dalla posizione a
# alla posizione b poi i numeri dalla posizione b alla posizione a
a = 2
b = 5
print(lista[a:b+1])
if a == 0:
    print(lista[b::-1])
else:
    print(lista[b:a-1:-1])


# 3.    Dato un numero positivo x qualsiasi stampa tutti i numeri dell'array dalla
# posizione 0 alla posizione x. Se x > N-1, dopo aver stampato tutto l'array ricomincia
# a stampare dalla posizione 0 e continua fino ad aver stampato x+1 numeri.

x = 24
for i in range(x):
    pos = i % len(lista)
    print(lista[pos], end = " ")
print()
```

4.1.6 Numero nella lista

Carica una lista di numeri casuali, chiedi poi all'utente un numero e digli se questo numero è presente nella lista.

Soluzione

```
import random

# numeri = []
# for i in range(20):
#     numeri.append(random.randint(0,99))
numeri = [random.randint(0,99) for i in range(20)]

print(numeri)

numero = int(input("Dimmi un numero da cercare: "))

if numero in numeri:
    print("il numero è presente nella lista")
```

```
else:
    print("il numero non è presente nella lista")
```

Soluzione con le funzioni

```
from random import randint

def listaCasuale(n = 10, minimo = 0, massimo = 99):
    return [randint(minimo, massimo) for _ in range(n)]

def contiene(lista, n):
    return n in lista

lista = listaCasuale()
print(lista)

n = int(input("Scrivi un numero da cercare nella lista: "))

if contiene(lista, n):
    print(f"{n} è contenuto in {lista}")
else:
    print(f"{n} non è contenuto in {lista}")
```

4.1.7 Numero nella lista con funzione

Scrivi una funzione a cui vengono passati come parametro una lista di numeri e un numero singolo, e che ti dica in output se l'elemento singolo sia presente o meno nella lista.

Variante: Qualora l'elemento sia presente nella lista, la funzione dovrà comunicarci l'indice dell'elemento.

Soluzione

```
# 3.1.5 Scrivi una funzione a cui vengono passati come parametro un
# elemento e una lista di elementi, e che ti dica in output se l'elemento
# passato sia presente o meno nella lista.
# Qualora l'elemento sia presente nella lista, la funzione dovrà
# inoltre comunicarci l'indice dell'elemento.

from random import randint

# funzione che restituisce la posizione dell'elemento nella lista
# se non lo trova restituisce None cioè niente
def f1(cercare, lista):
    for i,el in enumerate(lista):
        if cercare == el:
            return i

# funzione che restituisce la lista delle posizioni dell'elemento nella lista
# se non lo trova mai restituisce una lista vuota
def f2(cercare, lista):
    posizioni = []
    for i,el in enumerate(lista):
        if cercare == el:
            posizioni.append(i)
    return posizioni
```

```
# a = []
# for _ in range(10):
#     a.append(randint(0,9))
a = [randint(0,9) for _ in range(10)]


el = 4

print(a)
print(f1(el, a))
print(f2(el, a))
```

4.1.8 Liste con funzioni

Scrivi un programma che definisca e utilizzi le seguenti funzioni:

1. Funzione che deve creare e restituire una lista di numeri casuali, per farlo riceve la quantità di numeri che deve contenere, e i valori minimi e massimi dei numeri da generare
2. Una funzione che restituisce il valor medio dei numeri della lista che riceve come parametro
3. Una funzione che restituisce la mediana tra i numeri della lista che riceve come parametro

4.1.9 Istogramma con base verticale

Riempi una lista di numeri casuali poi rappresenta la lista come un istogramma. Ad es.la lista [1, 0, 4, 6, 3, 9] viene rappresentata come segue

```
1: x
0:
4: xxxx
6: xxxxxx
3: xxx
9: xxxxxxxxx
```

Soluzione base

```
from random import randint

numeri = []
for _ in range(6):
    numeri.append(randint(0, 9))

for numero in numeri:
    print(f"{numero}: ", end = '')
    for _ in range(numero):
        print("x", end = '')
    print()
```

Soluzione con funzioni varie

```
import random

def stampa_istogramma(lista):
    for num in lista:
        print(f"{num}: ", end="")
        for _ in range(num):
            print("x", end="")
```

```
        print()

def crea_istogramma(lista): ## un po strano
    ris = ""
    for num in lista:
        ris += f"{num}: "
        for _ in range(num):
            ris += "x"
        ris += "\n"
    return ris

lista = [random.randint(0,9) for _ in range(6)]
print("Stampa con la prima versione:")
stampa_istogramma(lista)
print("\nStampa con la seconda versione:")
print(crea_istogramma(lista))
```

4.1.10 Istogramma con base orizzontale

Riempi una lista di 10 numeri casuali poi stampa un istogramma basato sui numeri inseriti come nel seguente esempio (che ha solo 5 numeri [2,4,0,1,6]):

```
          x
          x
    x     x
    x     x
  x x     x
  x x   x x
  ---------
  2 4 0 1 6
```

4.1.11 Sottosequenze non decrescenti

Riempi una lista di numeri casuali tra 0 e 9 e calcola la somma massima fra le sottosequenze di cifre non decrescenti. Il valore deve essere calcolato da un'apposita funzione.

Esempio:

2	2	4	5	3	9	3	1	5	0
13				12		3	6		

la prima riga rappresenta la lista e la seconda i vari valori calcolati (viene restituito 13 che è il più grande)

Soluzione

```
def funzione(lista):
    somma = lista[0]
    massimo = somma
    for i in range(1, len(lista)):
        if lista[i] >= lista[i - 1]:
            somma += lista[i]
        else:
            somma = lista[i]
```

```
        if somma > massimo:
            massimo = somma

    return massimo


from random import randint

lista = [randint(0, 9) for _ in range(10)]
print("Lista:", lista)
print("Massimo:", funzione(lista))
```

4.1.12 Sottosequenze non decrescenti 2

Riempi una lista di numeri casuali tra 0 e 9, poi usa una funzione da te definita che, data la lista, restituisce la lista delle sottosequenze non decrescenti della lista passata.

Esempio:

2	2	4	5	3	9	3	1	5	0
13				12		3	6		

la prima riga rappresenta la lista e la seconda i vari valori calcolati. Deve essere restituito [[2,2,4,5], [3,9], [3], [1,5]]

4.1.13 Medie e funzioni

Scrivere una funzione che con un ciclo while chiede all'utente di inserire i 4 voti delle ultime verifiche di matematica (ed inserirli in una lista). Se il voto è inferiore a 1 o seperiore a 10, non va considerato. Solo quando l'utente ha inserito 4 voti corretti, uscire dal ciclo. Stampare la media dello studente in Matematica

4.1.14 Media e mediana

Data una lista di numeri interi casuali, calcola e stampa media e mediana della lista

Soluzione

```
from random import randint

lista = [randint(0,99) for _ in range(10)]
media = sum(lista) / len(lista)
ordinata = sorted(lista)
lung = len(ordinata)
if lung % 2 == 0:
    mediana = (ordinata[lung//2 - 1] + ordinata[lung//2]) / 2
else:
    mediana = ordinata[lung//2]

print(lista)
print(ordinata)
print(f"media: {media}")
```

```
print(f"mediana: {mediana}")
```

4.1.15 Lista alternata e ordinamento

Scrivi un programma che riempia una lista di numeri casuali di una cifra in modo tale che ogni numero sia sempre maggiore o minore di entrambi i numeri adiacenti (ad esempio 1 8 3 5 2 7 4 5 3). La lista generata deve essere stampata, poi ordinata e infine ristampata. Per implementare le funzionalità descritte realizza due funzioni: una che riceve una lista e la riempie di numeri casuali come descritto, una che riceve una lista e un'indicazione che dica se ordinare la lista in ordine crescente o decrescente (ad esempio una variabile booleana).

4.1.16 Liste con medie, ordinamenti, splices e input multipli

Scrivi un programma che esegua le seguenti operazioni:

1. Chiedere con un ciclo all'utente di inserire i voti presi in matematica ed inserirli in una lista, uscire dal ciclo quando l'utente inserisce un valore negativo;
2. richiamare la funzione MediaVoti1() che restituisce la media dei voti presenti nella lista;
3. richiamare la funzione MediaVoti2() che restituisce la media di tutti i voti escludendo il più basso e il più alto. (si consiglia di usare il metodo sort() delle liste);

Variante: modificare l'esercizio in modo che i voti vengano inseriti in una sola riga con un solo comando input. Effettua i controlli sugli input tutti insieme.

Soluzione

```
def MediaVoti1(voti):
    media = 0
    for voto in voti:
        media += voto
    media /= len(voti)
    return media

def MediaVoti2(voti):
    # trovo massimo e minimo
    imin, imax = 0, 0
    for i, voto in enumerate(voti):
        if voto > voti[imax]:
            imax = i
        if voto < voti[imin]:
            imin = i

    #calcolo la media
    media = 0
    for i, voto in enumerate(voti):
        if i != imax and i != imin:
            media += voto
```

```
    media /= len(voti) - 2
    return media

def MediaVoti3(voti):
    voti.sort()
    media = 0
    for i in range(1, len(voti)-1):
        media += voti[i]
    media /= len(voti) - 2
    return media


voti = []
while True:
    voto = float(input("Inserisci un voto: "))
    if voto == 0 or voto > 10:
        print("Voto non valido")
        continue
    if voto < 0:
        break
    voti.append(voto)

media = MediaVoti1(voti)
print("Media normale: ", media)
media = MediaVoti2(voti)
print("Media escuso max e min versione 1: ", media)
media = MediaVoti3(voti)
print("Media escuso max e min versione 2: ", media)
```

4.1.17 Estrazione elementi da una lista

Supponendo di avere una classe di 20 alunni (numerati nel registro da 1 a 20), il docente di matematica decide di dedicare i 5 appuntamenti di una settimana ad interrogare 4 studenti casuali, ogni giorno. Sapendo che ogni studente può essere interrogato al massimo due volte, stampare in ognuno dei 5 giorni i quattro interrogati, stampare quali studenti non sono mai stati interrogati.

4.1.18 Algoritmico su lista

Letta in input una sequenza di numeri interi positivi memorizzarla in una lista. Costruire una seconda lista contenente soltanto gli elementi della prima lista che non siano numeri primi. Stampare la seconda lista.

4.1.19 Mixing e ordinamento liste

Date due liste di numeri ordinate (ottienile come vuoi), costruire una terza lista di numeri interi ordinata, ottenuta mediante la “fusione” delle prime due. Stampare la lista.

4.1.20 Liste casuali e calcolo

Riempi una lista di n numeri casuali (n scelto da te), poi verifica se gli elementi della lista sono disposti in ordine crescente, per farlo scrivi e usa una funzione che restituisca la risposta alla domanda.

Continua l’esercizio creando e stampando liste fino a che non ne viene stampata una che rispetta la condizione.

Continua l’esercizio calcolando mediamente quante liste vengono create prima di crearne una che rispetti la condizione; in questo caso puoi stampare solo il numero medio di volte e non ogni lista (dovresti stamparne un’enormità).

Soluzione

```
def crescente(lista):
    for i in range(1, len(lista)):
        if lista[i-1] > lista[i]:
            return False
    return True

from random import randint
n = 5



iterazioni = 1000
media = 0
for _ in range(iterazioni):
    volte = 0
    lista = [randint(1, 100) for _ in range(n)]
    while not crescente(lista):
        # print(lista)
        lista = [randint(1, 100) for _ in range(n)]
        volte += 1
    # print("ci ho messo", volte, "volte")
    media += volte
media /= iterazioni

print("in media ci metto", media, "volte")
```

4.1.21 Verifica se sequenza appartiene a Fibonacci

data una lista di numeri verifica che i numeri in essa contenuti siano una porzione della successione di fibonacci. Fallo con una funzione scritta da te.

Soluzione

```
def verifica_fibonacci(lista):
    if len(lista) == 0:
        return False

    maxval = max(lista)
    successione = [0,1]
    while successione[-1] < maxval:
        successione.append(successione[-1] + successione[-2])
```

```
    # trovo il punto della successione uguale al primo elemento della lista
    i = 0
    while i < len(successione) and successione[i] < lista[0]:
        i += 1
    if i >= len(successione):
        return False

    for j in range(len(lista)):
        if i >= len(successione):
            return False
        if lista[j] != successione[i]:
            return False
        i += 1

    return True

def verifica_fibonacci_2(lista):
    if len(lista) == 0:
        return False

    precedenti = [0, 1]
    if lista[0] == 0:
        lista = lista[1:]
        if len(lista) == 0:
            return True

    while lista[0] > precedenti[-1]:
        precedenti = [precedenti[1], sum(precedenti)]

    for num in lista:
        if num != precedenti[1]:
            return False
        precedenti = [precedenti[1], sum(precedenti)]

    return True


print(verifica_fibonacci([2,3,8,13]))
print(verifica_fibonacci_2([2,3,8,13]))
```

4.2 Stringhe

4.2.1 Codice ASCII di un carattere

Scrivi una funzione che, dato un carattere in ingresso, restituisca in output il codice ASCII associato al carattere passato.

Soluzione

```
def trova_ascii():
    carattere = input("Inserisci il carattere che ti interessa convertire: ")
    valore = ord(carattere)
    output = f"Il valore ASCII associato a '{carattere}' è {valore}"
    return output
```

4.2.2 Stringhe su più righe

Stampa una stringa di una sola riga e una stringa formata da più righe

Soluzione

```
stringa1 = "Ciao a tutti!"

stringa2 = """
Ciao, come va?
Bene grazie
"""

stringa3 = "Ciao, come va?\nBene grazie"

print(stringa1)
print(stringa2)
print(stringa3)
```

4.2.3 Riconoscere una vocale

Scrivi una funzione (puoi farlo anche senza la funzione se non le hai ancora imparate) a cui viene passato un carattere come parametro, e risponde dicendo se il carattere è o meno una vocale.

Soluzione

```
def f(c):
    if c in "aeiouAEIOU":
        return True
    else:
        return False

print(f("a"))
print(f("E"))
print(f("f"))
```

4.2.4 Lunghezza di una stringa

Leggi una stringa da input e poi comunica all'utente di quante lettere è composta la stringa. Il conteggio deve essere fatto in due modi diversi, usando la funzione len e contando i caratteri uno alla volta.

Soluzione

```
stringa = input("Scrivi qualcosa: ")

# print(stringa)

numero_lettere = 0
for lettera in stringa:
    numero_lettere += 1
```

```
print(f"Contando i caratteri uno alla volta ho contato {numero_lettere} lettere")

numero_lettere = len(stringa)
print(f"\nContando i caratteri con la funzione len ho contato {numero_lettere} lettere")
```

4.2.5 Metodi delle stringhe

Data la seguente stringa: " Vediamo come me la CAVO " (attento che ci sono spazi extra all'inizio e alla fine della stringa), stampa:

1. La stringa così com'è
2. La stringa rimuovendo eventuali spazi iniziali e finali
3. La stringa nella versione con tutte le lettere minuscole
4. La stringa nella versione con tutte le lettere maiuscole
5. La stringa con la sola prima lettera maiuscola
6. La stringa con le sole prime lettere di ogni parola maiuscole
7. La seconda parola della stringa
8. La stringa ottenuta sostituendo tutte le 'a' con delle 'e'

Soluzione

```
stringa = "   Vediamo come me la CAVO   "   # attento che ci sono spazi extra alla fine e all'inizio


#     1. La stringa così com'è
print(stringa)


#     2. La stringa rimuovendo eventuali spazi iniziali e finali
# uso quindi le funzioni
# strip(): returns a new string after removing any leading and trailing whitespaces including tabs (\t). - comprende le altre due
# rstrip(): returns a new string with trailing whitespace removed. It’s easier to remember as removing white spaces from “right” side of the string.
# lstrip(): returns a new string with leading whitespace removed, or removing whitespaces from the “left” side of the string.

# print(len(stringa))
stringa = stringa.strip()
# print(len(stringa))
print(stringa)


#     3. La stringa nella versione con tutte le lettere minuscole
print(stringa.lower())


#     4. La stringa nella versione con tutte le lettere maiuscole
```

```
print(stringa.upper())


#     5. La stringa con la sola prima lettera maiuscola
print(stringa.capitalize())

#     6. La stringa con le sole prime lettere di ogni parola maiuscole
lista = stringa.split()
lista = list(map(str.capitalize, lista))
print(" ".join(lista))


#     7. La seconda parola della stringa
print(stringa.split()[1])


#     8. La stringa ottenuta sostituendo tutte le 'a' con delle 'e'
print(stringa.replace('a', 'e')) # così funziona solo con le minuscole
tmp = stringa.replace('a', 'e')
tmp = tmp.replace('A', 'E')
print(tmp)
```

4.2.6 Stampa lettera per lettera

Leggi una stringa da input e poi stampala in due modi diversi, la prima volta tutta insieme poi stampando una lettera per volta separando le lettere con uno spazio.

Soluzione

```
import os
os.system('cls')

stringa = input("Scrivi qualcosa: ")
print(stringa)
for lettera in stringa:
    print(lettera, end=" ")
print("\n\n")
```

4.2.7 Stampa al contrario

Leggi una stringa da input e poi stampala al contrario, cioè partendo dal fondo ma non modificandone il contenuto.

Soluzione

```
import os
os.system('cls')

stringa = input("Scrivi qualcosa: ")
print(stringa)
for lettera in reversed(stringa):
    print(lettera, end="")
```

```
print("\n\n")
```

4.2.8 Stringa inversa

Leggi una stringa da input e poi modificala in modo da ottenere la stringa inversa (ad esempio “ciao” diventa “oaic”). Stampa infine la stringa.

Soluzione

```
import os
os.system('cls')

stringa = input("Scrivi qualcosa: ")
print(stringa)
stringa = stringa[::-1]
print(stringa)
print("\n\n")
```

4.2.9 Scambio di due stringhe

Dichiara e inizializza due stringhe con valori a piacere, stampale. Scambia il contenuto delle due stringhe in modo che la prima contenga il contenuto della seconda e viceversa, poi ristampale.

Soluzione

```
s1 = "ciao"
s2 = "belli"

s1, s2 = s2, s1
print(s1)
print(s2)
```

4.2.10 Confronto alfabetico

Leggi da input due stringhe, se le due stringhe sono uguali comunica che sono uguali altrimenti scrivi quella che viene prima in ordine alfabetico

Soluzione

```
stringa1 = input("Scrivi la prima stringa: ")
stringa2 = input("Scrivi la seconda stringa: ")

s1 = stringa1.lower()
s2 = stringa2.lower()

if s1 == s2:
    print("Le stringhe sono uguali\n\n")
elif s1 < s2:
    prima = stringa1
else:
    prima = stringa2
print(f"La stringa che viene prima in ordine alfabetico è: {prima}\n\n")
print("\n\n")
```

4.2.11 Tre parole a confronto

Leggi usando un'unica funzione input tre parole. Indica poi qual è la parola più lunga, la parola che viene prima in ordine alfabetico e la parola che contiene più vocali. (Questo esercizio è molto bello se si usa la funzione sort)

Soluzione

```
def conta_vocali(s):
    ris = 0
    for l in s:
        if l in "aeiouAEIOU":
            ris += 1
    return ris

parole = input("Scrivi tre parole:")
parole = parole.split()
parole = list(map(str.strip, parole))

print(parole)

parole.sort(key=lambda el: len(el))
print("La parola più lunga:", parole[-1])

parole.sort(key=lambda el: str.lower(el))
print("La parola che viene prima in ordine alfabetico:", parole[0])

parole.sort(key=conta_vocali)
print("La parola che contiene più vocali:", parole[-1])
```

4.2.12 Unione con separatore

Data una lista di stringhe, crea e poi stampa un'unica stringa contenente tutte le stringhe della lista separate da "; " (puntoevirgola e spazio)

Ad esempio dalla lista

["casa", "sedia", "cavallo", "andare in bici", "uscire di casa", "che bello!"]

viene creata la stringa

"casa; sedia; cavallo; andare in bici; uscire di casa; che bello!"

Soluzione

```
lista = ["casa", "sedia", "cavallo", "andare in bici", "uscire di casa", "che bello!"]
stringa = "; ".join(lista)
print(stringa)
```

4.2.13 A capo sul separatore

Scrivi e utilizza una funzione che riceve una stringa e un carattere e che stampi tutto il contenuto della stringa andando a capo ogni volta che trova il carattere dato (il carattere rappresenta un separatore della stringa in sottostringhe).

Variante: la funzione non stampa la stringa ma la restituisce e basta (la stampa falla fuori dalla funzione)

Soluzione

```
def fbase(stringa, carattere):
    for lettera in stringa:
        print(lettera, end = "")
        if lettera == carattere:
            print()

def fvar(stringa, carattere):
    ris = ""
    for lettera in stringa:
        ris += lettera
        if lettera == carattere:
            ris += "\n"
    return ris

fbase("Ciao a tutti, oggi facciamo esercizi sulle stringhe.", 'o')
print("\n")
print(fvar("Ciao a tutti, oggi facciamo esercizi sulle stringhe.", 'o'))
```

4.2.14 A capo sulle virgole

Scrivi un programma che contenga in un array la seguente stringa “Ciao a tutti, mi chiamo Francesco\nOggi vi voglio parlare delle stringhe, siete d’accordo?” e che la stampi in modo tale che quando trova una virgola va a capo(oltre a stampare la virgola) e quando trova un a capo, va a capo due volte.

4.2.15 Maiuscole alternate

data una stringa, trasforma la stringa in modo tale che le lettere in posizione pari siano maiuscole e le altre minuscole,

ad esempio “Ciao a tutti, come state?” diventa “CiAo a tUtTi, CoMe sTaTe?”

Soluzione

```
# data una stringa, trasforma la stringa in modo tale che
# le lettere in posizione pari siano maiuscole e le altre
# minuscole

frase = "Ciao a tutti, come state?"


# metodo 1

lista = list(frase)
```

```
for i in range(1,len(lista),2):
    lista[i] = lista[i].lower()

for i in range(0,len(lista),2):
    lista[i] = lista[i].upper()

# poi devo unire tutti gli elementi e ricostruire la frase

# 1.1 metodo farlocco di ricostruzione
modificata = ""
for el in lista:
    modificata += el
print("Metodo 1.1:", modificata)

# 1.2 metodo veloce di ricostruzione (funzione join)
modificata = "".join(lista) # non puoi fare str(lista)
print("Metodo 1.2:", modificata)



# metodo 2 più veloce
mod = ""
for i, lettera in enumerate(frase):
    if i % 2 == 0:
        mod += frase[i].upper()
    else:
        mod += frase[i].lower()

print("Metodo 2  :", mod)
```

4.2.16 Cifrario di Cesare

Scrivi due funzioni che implementino il cifrario di Cesare cioè un cifrario che per cifrare un testo trasforma ogni lettera nella lettera che si trova tre posizioni più avanti nell'alfabeto. Le due funzioni sono le funzioni "cifra" e "decifra". Scrivi infine un programma che permetta di inserire un testo da cifrare o decifrare a scelta.

Soluzione

```
import string

minuscole = string.ascii_lowercase
maiuscole = string.ascii_uppercase

def cifra(testo, chiave):
    cifrato = ""
    for i in range(len(testo)):
        if testo[i] in minuscole:
            offset = ord('a')
        elif testo[i] in maiuscole:
            offset = ord('A')
        else:
            offset = 0
```

```
        if offset != 0:
            cifrato += chr(((ord(testo[i]) - offset + chiave) % len(minuscole)) + offset)
        else:
            cifrato += testo[i]
    return cifrato

def decifra(testo, chiave):
    decifrato = ""
    for i in range(len(testo)):
        if testo[i] in minuscole:
            offset = ord('a')
        elif testo[i] in maiuscole:
            offset = ord('A')
        else:
            offset = 0
        if offset != 0:
            decifrato += chr(((ord(testo[i]) - offset - chiave) % len(minuscole)) + offset)
        else:
            decifrato += testo[i]
    return decifrato

testo = input("Inserisci un testo: ")
chiave = int(input("Inserisci la chiave: "))
scelta = int(input("Vuoi cifrare o decifrare? scrivi 1 per cifrare, 2 per decifrare: "))
while scelta != 1 and scelta != 2:
    scelta = input("Risposta non prevista, scrivi 1 per cifrare, 2 per decifrare: ")
if scelta == 1:
    testo = cifra(testo, chiave)
    print("Il messaggio cifrato è:")
    print(testo)
else:
    testo = decifra(testo, chiave)
    print("Il messaggio decifrato è:")
    print(testo)
```

4.2.17 Linguaggio dei furfanti

In Svezia, i bambini giocano spesso utilizzando un linguaggio un po' particolare detto "rövarspråket", che significa "linguaggio dei furfanti": consiste nel raddoppiare ogni consonante di una parola e inserire una "o" nel mezzo. Ad esempio la parola "mangiare" diventa "momanongogiarore".

Se sai usare le funzioni, scrivi una funzione in grado di tradurre una parola o frase passata tramite input in "rövarspråket" se no implementa il programma senza funzioni.

4.2.18 Generatore di password

Scrivi una funzione generatrice di password.

La funzione deve generare una stringa alfanumerica di 8 caratteri qualora l'utente voglia una password semplice, o di 20 caratteri qualsiasi (anche simboli) qualora desideri una password più complicata.

Soluzione

```python
import string, random

alfanum = string.ascii_letters + string.digits
tutti = alfanum + string.punctuation

# print(alfanum)

def gen_pass(diff = "facile"):
    if diff == "facile":
        caratteri = alfanum
        lung = 8
    elif diff == "difficile":
        caratteri = tutti
        lung = 20
    else:
        return None

    password = "".join([random.choice(caratteri) for _ in range(lung)])
    return password


print(gen_pass())
print(gen_pass(diff="difficile"))
print(gen_pass(diff="difficilesdfdsf"))


# alternativa con i booleani

insiemesemplice = string.ascii_letters + string.digits
insiemecomplicato = insiemesemplice + string.punctuation

def f(complicato = False):
    ris = ""
    if not complicato:
        for _ in range(8):
            ris += random.choice(insiemesemplice)
    else:
        for _ in range(20):
            ris += random.choice(insiemecomplicato)
    return ris

print(f())
print(f(True))
```

4.2.19 Generatore di indirizzi MAC

Un indirizzo MAC (Media Access Control address) è un indirizzo univoco associato dal produttore, a un chipset per comunicazioni wireless (es WiFi o Bluetooth), composto da 6 coppie di cifre esadecimali separate da due punti.

Un esempio di MAC è 02:FF:A5:F2:55:12.

Scrivi una funzione genera_mac che generi degli indirizzi MAC pseudo casuali.

Soluzione

```
from random import choice

def genera_mac_meno_bello():
    mac = ""
    caratteri = "0123456789ABCDEF"
    for i in range(6):
        mac += choice(caratteri)
        mac += choice(caratteri)
        if i < 5:
            mac += ':'
    return mac

def genera_mac_piu_bello():
    caratteri = "0123456789ABCDEF"
    mac = [choice(caratteri)+choice(caratteri) for _ in range(6)]
    print(mac)
    mac = ":".join(mac)
    return mac

print(genera_mac_piu_bello())
```

4.2.20 Ricerca di sottostringa

Scrivi e utilizza una funzione che riceve due stringhe e sia in grado di dire se la stringa più piccola è una sottostringa di quella più grande. Nel tuo programma il main deve creare due stringhe di lunghezza a piacere contenenti lettere casuali (solo lettere maiuscole e minuscole senza altri simboli o cifre; la funzione non cambia)

Variante: invece di dire se la stringa è contenuta nell'altra stringa restituire la posizione di partenza della sottostringa trovata.

Ad esempio "mi c" è sottostringa di "Ciao a tutti mi chiamo Francesco"

Soluzione

```
import os
os.system('cls')

def sottostringa(piccola, grande):
    if piccola in grande:
        return True
    else:
        return False
```

```python
def sottostringa2(piccola, grande):
    for i in range(len(grande)-len(piccola)):
        if piccola == grande[i:i+len(piccola)]:
            return True

    return False

def posSottostringa(piccola, grande):
    for i in range(len(grande)-len(piccola)):
        if piccola == grande[i:i+len(piccola)]:
            return i

    return -1

def posSottostringaRicorsivo(piccola, grande):
    if len(grande) < len(piccola):
        return -1

    if piccola == grande[:len(piccola)]:
        return 0

    resto = posSottostringaRicorsivo(piccola, grande[1:])
    if resto != -1:
        return 1 + resto
    else:
        return -1

def posSottostringaRicorsivo2(piccola, grande, i = 0, f = -1):
    if f < 0:
        f = len(grande)+f

    if f-i+1 < len(piccola):
        return -1

    if piccola == grande[i:i+len(piccola)]:
        return i

    return posSottostringaRicorsivo2(piccola, grande, i+1, f)

def sottostringaRicorsivo(piccola, grande):
    if len(grande) < len(piccola):
        return False

    if piccola == grande[:len(piccola)]:
        return True

    return sottostringaRicorsivo(piccola, grande[1:])

print('sottostringa')
print(sottostringa('ia', 'Ciao'))
print(sottostringa('asd', 'Ciao'))
print()
```

```
print('posSottostringa')
print(posSottostringa('ia', 'Ciao'))
print(posSottostringa('asd', 'Ciao'))
print()

print('sottostringaRicorsivo')
print(sottostringaRicorsivo('ia', 'Ciao'))
print(sottostringaRicorsivo('asd', 'Ciao'))
print()

print('posSottostringaRicorsivo')
print(posSottostringaRicorsivo('ia', 'Ciao'))
print(posSottostringaRicorsivo('asd', 'Ciao'))
print()

print('posSottostringaRicorsivo2')
print(posSottostringaRicorsivo2('ia', 'Ciao'))
print(posSottostringaRicorsivo2('asd', 'Ciao'))
print()
```

4.2.21 Verifica nome e cognome

Scrivi un programma che chieda all'utente nome e cognome e poi:

1. verifichi che il nome e il cognome siano stati scritti con le iniziali maiuscole (devi controllare se la prima lettera ha i valori corretti secondo la tabella ASCII, i caratteri sono come numeri);
2. crei una stringa che contenga le generalità dell'utente (nome e cognome insieme);
3. stampi le generalità al contrario;
4. chieda nuovamente nome e cognome e verifichi se sono state riscritte le stesse cose di prima.

4.2.22 Città dei paesi europei

Data la seguente lista che contiene alcuni paesi europei

```
eu = ["italia", "germania", "svizzera", "francia", "spagna", "regnoUnito", "belgio"]
```

e la seguente lista di stringhe (fatta da coppie città paese)

```
s = [
    "madrid spagna istanbul turchia bruxelles belgio ostenda belgio hannover germania berlino germania aleppo siria",
    "roma italia milano italia berna svizzera anversa belgio toledo spagna londra regnoUnito",
    "damasco siria  charleroi belgio munchen germania homs siria preston regnoUnito teheran iran",
    "bilbao spagna brema germania zurigo svizzera torino italia beirut libano marsiglia francia",
    "gerusalemme israele ankara turchia kobane siria colonia germania francoforte germania lione francia",
    "lugano svizzera siviglia spagna mosul iraq dortmund germania bordeaux francia manchester regnoUnito"
]
```

stampare i nomi dei due paesi europei, con il maggior numero di città nell'elenco

4.2.23 Province lombarde

Le seguenti stringhe della lista, contengono il nome di una regione e alcune delle sue province:

```
ligureLombardo =
[
"Lombardia Monza Milano Como Lecco Varese",
"Liguria Genova",
"Lombardia Pavia Lodi Cremona Mantova",
"Liguria LaSpezia Savona Imperia",
"Lombardia Brescia Bergamo Sondrio"
]
```

Riconoscere e memorizzare opportunamente le 12 province lombarde.

Stampare le 12 province in ordine alfabetico

4.2.24 Tour di Ligabue e Vasco

Scoprire le città presenti in entrambi i tour di Ligabue e di Vasco

```
tourLigabue =
[
"Prato 10 giugno",
"Lucca 12 giugno",
"Arezzo 15 giugno",
"Siena 16 giugno",
"Firenze 17-19 giugno",
"Viareggio 20 giugno",
"LaSpezia 22 giugno",
"Genova 23-25 giugno",
"Savona 27 giugno"
]

tourVasco =
[
"Modena 3-5 giugno",
"Parma 7 giugno",
"Pavia 9 giugno",
"Genova 10-11 giugno",
"Carrara 14 giugno",
"Lucca 16 giugno",
"Pisa 17 giugno",
"Livorno 18 giugno",
"Firenze 20-23 giugno",
"Arezzo 25 giugno",
"Perugia 27 giugno"
]
```

4.2.25 Virologi italiani

Di seguito è riportata la lista che descrive i più celebri virologi Italiani nell'epoca covid, con il seguente formato: Nome e Cognome - Ospedale - città - (O) Ottimista (P) Pessimista

```
covid= [
    "Roberto Burioni - San Raffaele - Milano (P)",
    "Matteo Bassetti - San Martino - Genova (O)",
    "Alberto Zangrillo - San Raffaele - Milano (O)",
    "Massimo Galli - Luigi Sacco - Milano (P)",
    "MariaRita Gismondo - Luigi Sacco - Milano (O)",
```

```
        "Andrea Crisanti - Università di Padova - Padova (P)",
        "Ilaria Capua - One Health - Florida (P)",
        "Antonella Viola - Città della Speranza - Padova (P)",
        "Fabrizio Pregliasco - Galeazzi - Milano (O)",
        "Walter Ricciardi - Policlino Gemelli - Roma (P)",
        "Franco Locatelli - Bambin Gesù - Roma (P)"
    ]
```

1. stampare nome e cognome dei virologi ordinati per cognome (si consiglia si usare la split() con il separatore "-";
2. stampare il numero di virologi ottimisti e di quelli pessimisti.

Si ricorda che il metodo find() delle stringhe, restituisce -1 se l'elemento non è trovato.

4.2.26 Occorrenze di "tre"

Per ognuno dei seguenti numeri, stampare se è presente la cifra "tre", e nel caso quante volte è presente:

lista = [

"trentatre", "milletrecento","milequattrocentonovantadue", "trecentotredici"

]

4.2.27 Rimario

Scrivi una semplice funzione rimario, a cui viene passato un elenco di parole come parametro e che riceva una parola inserita dall'utente tramite la funzione input.

La funzione rimario dovrà confrontare la parola inserita dall'utente con quelle presenti nell'elenco passato, alla ricerca di rime, intese come parole le cui ultime 3 lettere siano uguali alla parola inserita dall'utente.

Le rime dovranno essere quindi mostrate in output dall'utente.

4.2.28 Parola palindroma casuale

Genera parole di 4 lettere minuscole a caso fino a che non ne ottieni una palindroma. Stampa tutte le parole ottenute. Il controllo sulla parola deve essere fatto con una funzione da te scritta che restituisca true se la parola è palindroma.

Soluzione

```
from random import choice
from string import ascii_lowercase
import os
os.system('cls')


def palindroma(parola):
    if len(parola) == 0:
        return None

    # for i in range(len(parola)/2):
    #     if parola[i] != parola[-1-i]:
```

```
    #         return False
    # return True

    if parola == parola[::-1]:
        return True
    else:
        return False

lettere = ascii_lowercase

# parola = ""
# while not palindroma(parola):
#     parola = "".join([choice(lettere) for _ in range(4)])
#     print(parola)

# oppure
while True:
    parola = "".join([choice(lettere) for _ in range(4)])
    print(parola)
    if palindroma(parola):
        break
```

4.3 Espressioni regolari

Per ognuno dei seguenti numeri, stampare se è presente la cifra "tre", e nel caso quante volte è presente:

lista = [

"trentatre", "milletrecento","milequattrocentonovantadue", "trecentotredici"

]

Soluzione

```
import re # iniziali di regular expressions

lista = ["trentatre", "milletrecento","milequattrocentonovantadue", "trecentotredici"]

# classico 1
trovare = "tre"
for el in lista:
    nmatches = 0
    for i in range(len(el)-len(trovare)+1):
        match = True
        for j in range(len(trovare)):
            if el[i+j] != trovare[j]:
                match = False
                break
```

```
        if match:
            nmatches += 1
    print(el, nmatches)

print("\n")

# classico 2
trovare = "tre"
for el in lista:
    match = 0
    for i in range(len(el)-len(trovare)+1):
        if trovare == el[i:i+len(trovare)]:
            match += 1
    print(el, match)

print("\n")

# con le espressioni regolari
esp = r"tre"
for el in lista:
    ris = re.findall(esp, el)
    print(el, len(ris))
```

4.3.1 Rimario con regex

Scrivi una semplice funzione rimario, a cui viene passato un elenco di parole come parametro e che riceva una parola inserita dall'utente tramite la funzione input.

La funzione rimario dovrà confrontare la parola inserita dall'utente con quelle presenti nell'elenco passato, alla ricerca di rime, intese come parole le cui ultime 3 lettere siano uguali alla parola inserita dall'utente.

Le rime dovranno essere quindi mostrate in output dall'utente.

4.4 Liste miste

4.4.1 Media voti per materia

La seguente lista, contiene i nomi di una materia e i voti conseguiti, (il numero di voti, è diverso a seconda della materia):

```
lista = ["matematica", 5.5, 6, 4, "storia", 8,9, "italiano", 7, 7.5, 7, "inglese", 6.5 ,6, 5.5, "fisica", 6, 5.5]
```

Stampare per ogni materia la media dei voti.

Si può usare la funzione type(x) per verificare il tipo della variabile x

Soluzione

```
lista = ["matematica", 5.5, 6, 4, "storia", 8, 9, "italiano",
         7, 7.5, 7, "inglese", 6.5, 6, 5.5, "fisica", 6, 5.5]

inizio = True
for el in lista:
    if type(el) == str:
        if (not inizio):
            media /= nvoti
            print(f"{materia}: {media:.2f}")

        inizio = False
        materia = el
        media = 0
        nvoti = 0

    else:
        media += el
        nvoti += 1

media /= nvoti
print(f"{materia}: {media:.2f}")
```

4.4.2 Operazioni da stringhe

La seguente lista di stringhe descrive una serie di operazioni:

```
listaOp = ["15.5 + 13.8",
"16.4 / 2",
"7 * 2", "6.5 - 4.1"],
```

stampare i risultati di ogni operazione.

Soluzione

```
listaOp = ["15.5 + 13.8",
           "16.4  /  2",
           "7  *  2",
           "6.5  - 4.1"]

for riga in listaOp:
    riga = riga.split()
    a = float(riga[0])
    op = riga[1]
    b = float(riga[2])
    if op == '+':
        print(f"{a} + {b} = {a+b:.2f}")
    elif op == '-':
        print(f"{a} - {b} = {a-b:.2f}")
    elif op == '*':
        print(f"{a} * {b} = {a*b:.2f}")
    elif op == '/':
        print(f"{a} / {b} = {a/b:.2f}")
```

```
    else:
        print("Operazione sconosciuta.")
```

4.4.3 Voti di storia per alunno

5 alunni (denominati alunno_1, alunno_2 alunno_3 alunno_4 alunno_5)

hanno ricevuto in storia rispettivamente 2, 3, 4, 3, 4 voti

e questa informazione è contenuta nella listaN:

listaN = [2, 3, 4, 3, 4]

la seconda lista riporta i sedici voti

listaVoti = [8.5, 7, 6, 4.5, 7, 5, 5.5, 7, 6.5, 9, 8.5, 7, 7, 6.5, 5, 6.5]

Stampare il nome dell'alunno che ha preso il voto più alto (alunno_3 con voto 9)

stampare i nomi delli alunni e la loro media in storia, ordinati in base alla media

4.4.4 Black Jack

Scrivi un programma che simuli il gioco del Black Jack.

Una mano del gioco è gestita da una funzione che devi chiamare manoBlackJack() che riceve come parametro una lista contenente la puntata fatta da ogni giocatore partecipante (implicitamente quindi anche il numero di giocatori) e restituisce la quantità di denaro da dare alla fine ad ogni giocatore (0 se perde, il doppio di quanto puntato se vince o lo stesso di prima se pareggia)

La funzione deve simulare la gestione di un mazzo di carte con una opportuna struttura dati. Per semplificare il gioco la funzione manoBlackJack() utilizza un nuovo mazzo mischiato ad ogni mano (ben lontano dalla realtà).

La funzione shuffle consente di mescolare il mazzo (restituisce un mazzo di 52 carte pieno e mischiato)

Sia ai giocatori che al banco, vengono assegnate inizialmente 2 carte pescate dal mazzo, entrambe scoperte per i giocatori, una coperta per il banco.

I giocatori giocano a turno e viene chiesto loro inserire la propria decisione che può essere solo di due tipi "vedo" o "continuo". Il singolo giocatore continua a pescare carte finchè perde (somma più di 21) o decide di fermarsi. Finito il turno dei giocatori il banco gioca automaticamente secondo le regole standard del gioco (pesca fintanto che raggiunge meno di 17, da 17 in su si ferma):

Il programma principale chiede se un giocatore si vuole aggiungere alla partita e con quale puntata, finchè si aggiungono giocatori continua a chiederlo (massimo 7 giocatori), poi chiama la funzione per giocare una mano e alla fine chiede se i giocatori vogliono rimanere (se hanno perso ne devono mettere altri) o vogliono ritirare soldi dal tavolo (tutti e se ne vanno, una parte e rimangono). Il programma poi chiede nuovamente se ci sono nuovi giocatori che vogliono partecipare (sempre massimo 7)

Il programma termina quanto tutti i giocatori lasciano il tavolo.

Varianti per completare il programma

Gestisci il mazzo in maniera realistica, queste sono le regole:

“Il Blackjack classico (o francese) si gioca con sei mazzi di carte da poker (o francesi), per un totale di 312 carte. In mezzo ad esse vengono mescolate una carta di plastica e quando, durante il gioco, si arriva ad essa verranno rimescolate tutte le carte.”

“Normalmente una taglia o sabot che dir si voglia è composta da sei mazzi di carte,all'inizio del gioco vengono tolte le prima 5 carte e si mette una carta nera che normalmente viene messa verso la fine della taglia e tiene fuori gioco un trenta quaranta carte(questo nasce al fine di dare filo da torcere a chi usa la conta),si mescolano le carte con l'uscita della carta nera. oggi però è sempre più in uso l'utilizzo delle mescolatrici,delle macchinette nelle quali il croupier mette le carte della mano precedente e che automaticamente vengono mescolate e rimesse in gioco,sono in uso presso i casino svizzeri e quello di montecarlo,altri non saprei,mentre sia a saint vincent che a san remo vengono ancora oggi mescolate a mano,ovviamente con le mescolatrice la conta diventa impossibile.

Tutto questo ovviamente per quello che riguarda i casino reali.”

4.4.5 Elenco di nomi

Scrivi un programma che chieda all'utente una serie di nomi che devono essere tutti memorizzati in una lista. Il numero di nomi da inserire deve essere chiesto all'utente. Quando viene inserito un nome bisogna controllare che inizi con una lettera e se questa è minuscola deve essere fatta diventare maiuscola. Alla fine stampa tutte le stringhe.

Aggiunte: Continua l’esercizio aggiungendo le seguenti funzionalità:

1. Cerca e stampa il nome più lungo
2. Cerca e stampa il nome che in ordine alfabetico è il primo
3. Chiedi all'utente un nome e digli se il nome è presente nell'elenco dei nomi.

4.4.6 Stringa più lunga e più corta

Data una lista di stringhe (quindi una matrice), riempito come preferisci, stampa la stringa più lunga, la più corta e la lunghezza media delle stringhe. I tre risultati devono essere ottenuti per mezzo di apposite funzioni.

4.4.7 Conteggio palindrome

Scrivi e usa una funzione che data una lista di stringhe, restituisca il numero di parole palindrome contenute nell’array. Riempi l’array come preferisci.

4.4.8 Campo minato

Scrivi un programma che permetta di rappresentare il campo di gioco di “Campo minato”. Il campo deve essere formato da una superficie suddivisa in caselle quadrate, quindi avrai una grande tabella (ad esempio di dimensioni 8x12 ma puoi scegliere diversamente). Sul campo devono essere disposte a caso una serie di mine (suggerisco 16 ma puoi cambiare). Fai poi in modo che ogni casella che non sia una mina abbia un numero che rappresenti il numero di mine adiacenti (anche in diagonale quindi un numero da 1 a 8) lasciando vuote le celle che non hanno mine adiacenti (non scrivere 0)

Soluzione

```
# sviluppo questo programma senza creare classi

import random

nmine = 16
nrighe = 8
ncolonne = 12
vuoto = " "
mina = "X"


campo = [[vuoto for j in range(ncolonne)] for i in range(nrighe)]
for _ in range(nmine):
    i = random.randint(0, nrighe-1)
    j = random.randint(0, ncolonne-1)
    while (campo[i][j] != " "):
        i = random.randint(0, nrighe-1)
        j = random.randint(0, ncolonne-1)
    campo[i][j] = mina

for i in range(nrighe):
    for j in range(ncolonne):
        if campo[i][j] != mina:
            val = 0
            if i > 0        and j > 0           and campo[i-1][j-1] == mina:        # ↖
                val +=1
            if i > 0                            and campo[i-1][j] == mina:          # ↑
                val +=1
            if i > 0        and j < ncolonne-1  and campo[i-1][j+1] == mina:        # ↗
                val +=1
            if                  j < ncolonne-1  and campo[i][j+1] == mina:          # →
                val +=1
            if i < nrighe-1 and j < ncolonne-1  and campo[i+1][j+1] == mina:        # ↘
                val +=1
            if i < nrighe-1                     and campo[i+1][j] == mina:          # ↓
                val +=1
            if i < nrighe-1 and j > 0           and campo[i+1][j-1] == mina:        # ↙
                val +=1
            if                  j > 0           and campo[i][j-1] == mina:          # ←
                val +=1

            if val > 0:
                campo[i][j] = val

        print(campo[i][j], end = " ")
    print()

# ← ↑   →   ↓   ↔   ↕   ↖   ↗   ↘   ↙
```

4.4.9 Agenda

Crea un programma di gestione di una agenda. L'agenda deve permettere di visualizzare e inserire i dati riguardanti i propri contatti. Per ogni persona devono essere memorizzati nome cognome e numero di telefono. La visualizzazione dei dati deve avvenire in ordine alfabetico per nome o cognome (scegli tu). Aggiungi poi una funzione per cancellare un contatto in una specifica posizione dell'agenda. (Non avendo ancora fatto la gestione dei files ogni volta l'agenda partirà da uno stato predefinito che puoi decidere tu). Aggiungi anche un'interfaccia utilizzabile.

4.4.10 Parole di una frase

Scrivi un programma che chieda all'utente di scrivere una frase da leggere in un unico input. Le parole della frase devono poi essere stampate una per volta andando sempre a capo tra una parola e l'altra.

4.4.11 Parole unite in una stringa

Scrivi un programma che chieda all'utente di scrivere una serie di parole lette una ad una. Queste parole devono essere unite a formare un'unica stringa che deve essere infine stampata.

4.4.12 Parole in ordine alfabetico

Scrivi un programma che chieda all'utente di scrivere una frase. Le parole della frase devono poi essere ordinate e stampate in ordine alfabetico.

4.5 Matrici di soli numeri

4.5.1 Tabelline

Creare una matrice che contiene le tabelline dei numeri dall' 1 al 10. Alla fine stamparla.

Soluzione

```
# metodo più leggibile
tab = []
for i in range(11):
    riga = []
    for j in range(11):
        riga.append(j*i)
    tab.append(riga)

# metodo un po poù brutto e senza la riga e la colonna 0
tabelline = []
for i in range(10):
    tabelline.append([])
    for j in range(10):
        tabelline[i].append((i+1)*(j+1))

tab = [[j*i for j in range(11)] for i in range(11)]

for riga in tab:
    for num in riga:
        print(f"{num:3d} ", end='')
    print()
print(f"5 * 5 = {tab[5][5]}")
```

4.5.2 Matrice quadrata casuale e stampa riga e colonna

Scrivi un programma che istanzia una matrice n x m di interi casuali (n e m scelti da te). Oltre a stampare l'intera matrice, chiedi all'utente quale riga o quale colonna stampare e stampala sullo schermo

Soluzione base

```
from random import randint

n = 4
m = 6

mat = [[randint(1,99) for _ in range(m)] for _ in range(n)]

for riga in mat:
    for num in riga:
        print(f"{num:2d} ", end = '')
    print()

print()

# stampa della riga 3
riga = 3
for num in mat[riga]:
    print(f"{num:2d} ", end = '')
print()
print()

# stampa della colonna 2
col = 2
for riga in mat:
    print(f"{riga[col]:2d} ")
print()
```

Soluzione con tanta interazione con l'utente e controlli vari

```
import random
import os
os.system('cls')


# lettura delle dimensioni della matrice
n = int(input("Scrivi il numero di righe (min 1 max 20): "))
while n<1 or n>20:
    n = int(input("Input errato, inserisci un numero tra 1 e 20: "))

m = int(input("Scrivi il numero di colonne: "))
while m<1 or m>20:
    m = int(input("Input errato, inserisci un numero tra 1 e 20: "))

# creazione della matrice e inserimento dei numeri casuali


# metodo C++
```

```
# matrice = []
# for i in range(n):
#     riga = []
#     for j in range(m):
#         riga.append(random.randint(1, 99))
#     matrice.append(riga)

matrice = [[random.randint(1, 99) for i in range(m)] for j in range(n)]


# stampa della matrice
for riga in matrice:
    for numero in riga:
        print(f"{numero:3d}", end=" ")
    print("")

scelta_riga_colonna = int(input(
    "Scegli se stampare una riga o una colonna (scrivi 0 per riga, 1 per colonna) "))
while scelta_riga_colonna != 1 and scelta_riga_colonna != 0:
    scelta_riga_colonna = int(input("Input errato, reinserisci: "))

if scelta_riga_colonna == 0:
    numero_riga_colonna = int(input(f"Scegli la riga da stampare (le righe vanno da 0 a {n-1}):
"))
    while numero_riga_colonna < 0 or numero_riga_colonna >= n:
        numero_riga_colonna = int(input("Input errato, reinserisci: "))
    for numero in matrice[numero_riga_colonna]:
        print(numero, end=" ")

else:
    numero_riga_colonna = int(input(f"Scegli la colonna da stampare (le righe vanno da 0 a {m-
1}): "))
    while numero_riga_colonna < 0 or numero_riga_colonna >= m:
        numero_riga_colonna = int(input("Input errato, reinserisci: "))
    for riga in matrice:
        print(riga[numero_riga_colonna])


print("\n\n")
```

4.5.3 Somma e moltiplicazione matrici

Scrivere un programma che riempia due matrici A e B di dimensione n x m e calcoli la matrice C = A + B e la matrice D = A x B (in realtà è un finto prodotto di matrici). Stampa le matrici ottenute. Per il calcolo della somma calcola normalmente C[i][j] = A[i][j] + B[i][j] mentre per la moltiplicazione hai due possibilità:

Versione semplice se non riesci proprio a fare la versione corretta: C[i][j] = A[i][j] x B[i][j].

Versione complicata: vai a guardare su internet come si calcola e scrivi l'algoritmo per farlo.

Soluzione

```
import random
import os
os.system('cls')

def stampa_matrice(M):
    for riga in M:
        for numero in riga:
            print(f"{numero:3d}", end=" ")
        print("")


# creazione e riempimento di A e B
n = 2
m = 2
A = []
B = []
for i in range(n):
    rigaA = []
    rigaB = []
    for j in range(m):
        rigaA.append(random.randint(1, 9))
        rigaB.append(random.randint(1, 9))
    A.append(rigaA)
    B.append(rigaB)

print("Matrice A:")
stampa_matrice(A)
print("\nMatrice B:")
stampa_matrice(B)

# creazione e calcolo di C
C = []
for i in range(n):
    riga = []
    for j in range(m):
        riga.append(A[i][j] + B[i][j])
    C.append(riga)
# print(f"C: {C}")
print("\nMatrice C:")
stampa_matrice(C)

# creazione e calcolo di D (semplice)
D = []
for i in range(n):
    riga = []
    for j in range(m):
        riga.append(A[i][j] * B[i][j])
    D.append(riga)
```

```
# print(f"D semplice: {D}")
print("\nMatrice D semplice:")
stampa_matrice(D)

# calcolo di D normale
if m != n:
    print("Il prodotto tra A e B non si può calcolare")
else:
    D = []
    for i in range(n):
        riga = []
        for j in range(m):
            somma = 0
            for k in range(m):
                somma += A[i][k] * B[k][j]
            riga.append(somma)
        D.append(riga)
    # non sono sicuro del risultato
    # print(f"D completo: {D}")
    print("\nMatrice D normale: (da rivedere il procedimento)")
    stampa_matrice(D)


print("\n\n")
```

4.5.4 Scambio righe pari e dispari

Scrivi un programma che data una matrice n x m di interi, scambia le righe pari con quelle dispari variante: le dimensioni della matrice vengono scelte dall'utente tra i valori massimi 10 x 20.

Soluzione

```
# 3.5.4 - Scrivi un programma che data una matrice n x m di
# interi, scambia le righe pari con quelle dispari
# variante: le dimensioni della matrice vengono scelte dall'utente tra i valori
# massimi 10 x 20.

import os;
os.system('cls')
from random import randint

def stampaMatrice(mat):
    for riga in mat:
        for num in riga:
            print(num, end = " ")
        print()

def scambiaRighe(mat):
    for i in range(0, len(mat)-1, 2):
        mat[i], mat[i+1] = mat[i+1], mat[i]

maxr = 10
maxc = 20
```

```
nrig = int(input(f"Scrivi il numero di righe (massimo {maxr}): "))
while nrig < 1 or nrig > maxr:
    nrig = int(input(f"Scrivi il numero di righe (massimo {maxr}): "))

ncol = int(input(f"Scrivi il numero di colonne (massimo {maxc}): "))
while ncol < 1 or ncol > maxc:
    ncol = int(input(f"Scrivi il numero di colonne (massimo {maxc}): "))

mat = [[randint(0,9) for j in range(ncol)] for i in range(nrig)]

# # in alternativa
# mat = []
# for i in range(nrig):
#     mat.append([])
#     for j in range(ncol):
#         mat[i].append(randint(0,9))

stampaMatrice(mat)
scambiaRighe(mat)
print()
stampaMatrice(mat)
```

4.5.5 Matrice trasposta

Scrivi un programma che data una matrice n x n di interi, con n scelto dall'utente, scambia le righe con le colonne

Variante: La matrice non è quadrata ma è una generica matrice n x m

Soluzione senza funzioni

```
from random import randint
n = 5
mat = [[randint(0,99) for _ in range(n)] for _ in range(n)]

print("Matrice originale:")
for riga in mat:
    for num in riga:
        print(f"{num:2d} ", end = '')
    print()

# inversione righe colonne
for i in range(len(mat)):
    # for j in range(i):
    for j in range(i+1, len(mat[i])):
        mat[i][j], mat[j][i] = mat[j][i], mat[i][j]

print("\nMatrice trasposta:")
for riga in mat:
    for num in riga:
        print(f"{num:2d} ", end = '')
    print()
```

Soluzione variante con uso di funzioni

```
def generaMatriceCasualeNM(n, m, min = 0, max = 99):
    from random import randint
```

```
    return [[randint(min, max) for _ in range(m)] for _ in range(n)]

def stampaMatrice(mat):
    for riga in mat:
        for num in riga:
            print(f"{num:3d} ", end = '')
        print()

def trasposta(mat):
    ris = []
    for j in range(len(mat[0])):
        nuovariga = []
        for i in range(len(mat)):
            nuovariga.append(mat[i][j])
        ris.append(nuovariga)

    # oppure con una list comprehension
    ris = [[mat[i][j] for i in range(len(mat))] for j in range(len(mat[0]))]

    return ris

mat = generaMatriceCasualeNM(4, 6)

print("Matrice originale:")
stampaMatrice(mat)

mat2 = trasposta(mat)
print("\nMatrice trasposta:")
stampaMatrice(mat2)
```

4.5.6 Matrice casuale (parametri utente), numero più frequente

Scrivi un programma che riempia una matrice n x m di numeri interi a caso, n e m sono scelti dall'utente e il range dei numeri è da 0 a x scelto anch'esso dall'utente. Determina poi il numero più frequente generato nella matrice. Il riempimento delle matrici e la ricerca del numero più frequente devono essere svolti da due apposite funzioni, mentre l'interazione con l'utente dev'essere lasciata al main.

Soluzione

```
# 3.5.6 - Scrivi un programma che riempia una matrice n x m di
# numeri interi a caso, n e m sono scelti dall'utente e il range dei numeri è da
# 0 a x scelto anch'esso dall'utente. Determina poi il numero più frequente
# generato nella matrice. Il riempimento delle matrici e la ricerca del numero
# più frequente devono essere svolti da due apposite funzioni, mentre
# l'interazione con l'utente dev'essere lasciata al main.


import os;
os.system('cls')
from random import randint

def generaMatrice(nrig, ncol, maxint):
    return [[randint(0,maxint) for j in range(ncol)] for i in range(nrig)]

def stampaMatrice(mat):
    for riga in mat:
        for num in riga:
```

```
            print(num, end = " ")
        print()

def piuFrequente(mat):
    d = {}
    ris = None
    for riga in mat:
        for num in riga:
            if num in d:
                d[num] += 1
            else:
                d[num] = 1
            if ris == None or d[num] > d[ris]:
                ris = num
    return ris

maxr = 10
maxc = 20

nrig = int(input(f"Scrivi il numero di righe (massimo {maxr}): "))
while nrig < 1 or nrig > maxr:
    nrig = int(input(f"Scrivi il numero di righe (massimo {maxr}): "))

ncol = int(input(f"Scrivi il numero di colonne (massimo {maxc}): "))
while ncol < 1 or ncol > maxc:
    ncol = int(input(f"Scrivi il numero di colonne (massimo {maxc}): "))

x = int(input(f"Scrivi il massimo valore generabile (minimo 0): "))
while x < 0:
    x = int(input(f"Scrivi il massimo valore generabile  (minimo 0): "))

mat = generaMatrice(nrig, ncol, x)
stampaMatrice(mat)
print()
print("Il numero più frequente è", piuFrequente(mat))
```

4.5.7 Riga e colonna con somma più grande

Scrivi un programma che riempia una matrice n x m di numeri interi a caso, n e m sono scelti dall'utente e il range dei numeri è da 0 a x scelto anch'esso dall'utente. Determina poi la riga e la colonna con la somma dei numeri più grande. Il riempimento delle matrici e la ricerca della riga o della colonna devono essere svolti da apposite funzioni, mentre l'interazione con l'utente dev'essere lasciata al main.

Soluzione

```
n = int(input("Scrivi il numero di righe della matrice: "))
while n < 1:
    n = int(input("Scrivi il numero di righe della matrice: "))

m = int(input("Scrivi il numero di colonne della matrice: "))
while m < 1:
    m = int(input("Scrivi il numero di colonne della matrice: "))
```

```python
x = int(input("Scrivi il valore massimo da generare: "))
while x < 1:
    x = int(input("Scrivi il valore massimo da generare: "))

# creo la matrice
from random import randint
mat = [[randint(0, x) for _ in range(m)] for _ in range(n)]

# stampo la matrice
for riga in mat:
    for num in riga:
        print(f"{num:3d} ", end = '')
    print()

# cerco la riga con la somma di numeri maggiore
imax = 0
sommamax = 0
for num in mat[0]:
    sommamax += num

for i, riga in enumerate(mat):
    somma = 0
    for num in riga:
        somma += num

    if somma > sommamax:
        sommamax = somma
        imax = i
print(f"La riga con la somma maggiore è la riga {imax}")

# cerco la colonna
jmax = 0
sommamax = 0
for i in range(len(mat)):
    sommamax += mat[i][0]

for j in range(len(mat[0])):
    somma = 0
    for i in range(len(mat)):
        somma += mat[i][j]

    if somma > sommamax:
        sommamax = somma
        jmax = j
print(f"La colonne con la somma maggiore è la colonna {jmax}")
```

4.5.8 Ricerca coordinate elemento

Scrivi un programma che:

1. crei e riempia di valori casuali una matrice di dimensioni a tua scelta,
2. ne stampi il contenuto sullo schermo,
3. chieda all'utente un numero e cerchi se il numero è presente nella matrice
4. se il numero è presente si stampino le coordinate della posizione nella matrice

Soluzione

```python
from random import randint

def stampaMatrice(mat):
    for riga in mat:
        for numero in riga:
```

```
            print(f"{numero:2d} ", end='')
        print()
    print()

# 1.  crei e riempia di valori casuali una matrice di dimensioni a tua
# scelta,
n = 4
m = 6
mat = [[randint(0,9) for _ in range(m)] for _ in range(n)]

# 2.  ne stampi il contenuto sullo schermo,
stampaMatrice(mat)

# 3.  chieda all'utente un numero e cerchi se il numero è presente nella
# matrice
cercare = int(input("Scrivi un numero da cercare: "))

def contiene(mat, num):
    for riga in mat:
        for numero in riga:
            if numero == num:
                return True
    return False

print(contiene(mat, cercare))

# 4.  se il numero è presente si stampino le coordinate della posizione
# nella matrice

def posizione(mat, num):
    for i,riga in enumerate(mat):
        for j,numero in enumerate(riga):
            if numero == num:
                return (i,j)

def posizioni(mat, num):
    ris = []
    for i,riga in enumerate(mat):
        for j,numero in enumerate(riga):
            if numero == num:
                ris.append((i,j))
    return ris

print(posizione(mat, cercare))
print(posizioni(mat, cercare))
```

4.5.9 Stampa specifica riga o colonna

Scrivi un programma che istanzia una matrice n x m di interi casuali (n e m scelti da te). Oltre a stampare l'intera matrice, chiedi all'utente quale riga o quale colonna stampare e stampala sullo schermo

Soluzione

```python
# 3.5.9 - Scrivi un
# programma che istanzia una matrice n x m di interi casuali (n e m scelti da
# te). Oltre a stampare l'intera matrice, chiedi all'utente quale riga o quale
# colonna stampare e stampala sullo schermo

from random import randint

def stampaMatrice(mat):
    for riga in mat:
        for numero in riga:
            print(f"{numero:2d} ", end='')
        print()
    print()

def stampaRiga(mat, nriga):
    for numero in mat[nriga]:
        print(f"{numero:2d}", end='')

def stampaColonna(mat, ncol):
    for riga in mat:
        print(f"{riga[ncol]:2d}")

n = 4
m = 6
mat = [[randint(0,9) for _ in range(m)] for _ in range(n)]

stampaMatrice(mat)
n = int(input("Vuoi stampare una riga o una colonna? (1. riga, 2. colonna) "))
while n != 1 and n != 2:
    n = int(input("Vuoi stampare una riga o una colonna? (1. riga, 2. colonna) "))

if n == 1:
    n = int(input("Scrivi l'indice della riga da stampare: "))
    if n < 0 or n >= len(mat):
        print("La riga non esiste.")
    else:
        stampaRiga(mat, n)
        print()
else:
    n = int(input("Scrivi l'indice della colonna da stampare: "))
    if n < 0 or n >= len(mat[0]):
        print("La colonna non esiste.")
    else:
        stampaColonna(mat, n)
        print()
nnaMassima(mat))
```

4.5.10 Conteggio pari per ogni riga, dispari per ogni colonna

Scrivi un programma che per mezzo di un'apposita funzione riempia una matrice di numeri casuali. Stampa poi il numero di numeri pari contenuti in ogni riga della matrice (riga per riga

separatamente) e il numero di numeri dispari contenuti in ogni colonna della matrice. Anche in questo caso definisci una o due funzioni per farlo (a scelta).

Soluzione

```
from random import randint

def creaMat(n, m):
    return [[randint(0,99) for _ in range(m)] for _ in range(n)]

mat = creaMat(4, 6)

def pariEDispariPerRigaEColonna(mat):
    pari = [0 for _ in range(len(mat))]
    dispari = [0 for _ in range(len(mat[0]))]

    for i, riga in enumerate(mat):
        for j, num in enumerate(riga):
            if num % 2 == 0:
                pari[i] += 1
            else:
                dispari[j] += 1

    return [pari, dispari]

pari, dispari = pariEDispariPerRigaEColonna(mat)

for i, riga in enumerate(mat):
    for j, num in enumerate(riga):
        print(f"{num:2d} ", end = "")
    print(f" | {pari[i]:2d}")

for _ in range(len(mat[0])):
    print("---", end = "")
print("-+----")

for j in range(len(mat[0])):
    print(f"{dispari[j]:2d} ", end = '')
print(" |")
```

4.5.11 Stampa pari e buchi al posto dei dispari

Riempi una matrice di numeri casuali, poi scrivi una funzione che stampa i soli numeri pari della matrice in modo che la matrice compaia con la stessa forma ma senza i numeri dispari al posto dei quali compariranno spazi vuoti.

Soluzione

```
# 3.5.16 - Riempi una matrice di numeri casuali, poi scrivi una
# funzione che stampa i soli numeri pari della matrice in modo che la matrice
# compaia con la stessa forma ma senza i numeri dispari al posto dei quali
# compariranno spazi vuoti.

from random import randint
```

```
n = 4
m = 6

mat = [[randint(0,99) for _ in range(m)] for _ in range(n)]

def stampaMat(mat):
    for riga in mat:
        for num in riga:
            print(f'{num:2d} ', end='')
        print()

stampaMat(mat)

def stampaMatPari(mat):
    for riga in mat:
        for num in riga:
            if num % 2 == 0:
                print(f'{num:2d} ', end='')
            else:
                print('   ', end='')
        print()

print()
stampaMatPari(mat)
```

4.5.12 Stampa diagonali di matrice quadrata, a X

Riempi una matrice quadrata di numeri casuali, poi stampa la matrice in modo che siano visibili solo i numeri appartenenti alle diagonali

Soluzione

```
# 3.5.15    Riempi una matrice quadrata di numeri casuali,
# poi stampa la matrice in modo che siano visibili solo
# i numeri appartenenti alle diagonali


from random import randint

n = 4
m = 4

mat = [[randint(0,99) for _ in range(m)] for _ in range(n)]

for riga in mat:
    for num in riga:
        print(f'{num:2d} ', end='')
    print()

print()

# stampo solo le diagonali
for i, riga in enumerate(mat):
```

```
    for j, num in enumerate(riga):
        if i == j or i+j == n-1:
            print(f'{num:2d} ', end='')
        else:
            print('   ', end='')
    print()
```

4.5.13 Diagonali come 4 liste

Scrivi un programma che riempia una matrice quadrata di numeri casuali e poi stampi le due diagonali in entrambi i versi

Variante: scrivi una funzione che restituisca quattro liste che contengono i valori delle due diagonali nei due versi

Soluzione

```
# 3.5.14    Scrivi un programma che riempia una matrice quadrata
# di numeri casuali e poi stampi le due diagonali in entrambi i
# versi
# Variante: scrivi una funzione che restituisca quattro liste
# che contengono i valori delle due diagonali nei due versi

from random import randint

n = 4
m = 4

mat = [[randint(0,99) for _ in range(m)] for _ in range(n)]

def stampaMat(mat):
    for riga in mat:
        for num in riga:
            print(f'{num:2d} ', end='')
        print()

stampaMat(mat)
print()

# ← ↑   →   ↓   ↔   ↕   ↖   ↗   ↘   ↙

# diagonale ↘
for i in range(n):
    print(mat[i][i],end=' ')
print()

# diagonale ↖
for i in range(n-1,-1,-1):
    print(mat[i][i],end=' ')
print()

# diagonale ↙
for i in range(n):
    print(mat[i][n-1-i],end=' ')
```

```
print()

# diagonale ↗
for i in range(n):
    print(mat[n-1-i][i],end=' ')
print()

# diagonale ↘
def diag_SuSx_GiuDx(mat):
    n = len(mat)
    # ris = []
    # for i in range(n):
    #     ris.append(mat[i][i])
    # return ris
    return [mat[i][i] for i in range(n)]

# diagonale ↖
def diag_GiuDx_SuSx(mat):
    # n = len(mat)
    # ris = []
    # for i in range(n-1,-1,-1):
    #     ris.append(mat[i][i])
    # return ris

    # return [mat[i][i] for i in range(n-1,-1,-1)]

    return diag_SuSx_GiuDx(mat)[::-1]

# diagonale ↙
def diag_SuDx_GiuSx(mat):
    n = len(mat)
    return [mat[i][n-1-i] for i in range(n)]

# diagonale ↗
def diag_GiuSx_SuDx(mat):
    return diag_SuDx_GiuSx(mat)[::-1]

print('↘', diag_SuSx_GiuDx(mat))
print('↖', diag_GiuDx_SuSx(mat))
print('↙', diag_SuDx_GiuSx(mat))
print('↗', diag_GiuSx_SuDx(mat))
```

4.5.14 Stampa di tutte le diagonali (anche non principali)

(Difficile) Scrivi un programma che riempia una matrice di numeri casuali e che stampi i numeri contenuti in ogni diagonale, prima delle diagonali che vanno da in basso a sinistra a in alto a destra, poi da in alto a sinistra a in basso a destra.

Variante: invece che stampare solamente i numeri, crea una lista di liste che contenga per ogni riga i valori contenuti in una diagonale. Devono essere create due matrici, una per ogni diversa direzione delle diagonali.

Soluzione

```python
# 3.5.13    Scrivi un programma che riempia una matrice di numeri
# casuali e che stampi i numeri contenuti in ogni diagonale,
# prima delle diagonali che vanno da in basso a sinistra a in
# alto a destra, poi da in alto a sinistra a in basso a destra.

# Variante: invece che stampare solamente i numeri, crea una
# lista di liste che contenga per ogni riga i valori contenuti
# in una diagonale. Devono essere create due matrici, una per
# ogni diversa direzione delle diagonali.

from random import randint

n = 4
m = 2
mat = [[randint(0,9) for _ in range(m)] for _ in range(n)]

# stampa matrice
for riga in mat:
    for num in riga:
        print(f"{num} ", end='')
    print()
print()

# Metodo 1 base che pensa alle diagonali che escono dalla matrice
# di cui stampo solo le parti dentro alla matrice
def diagonaliAltoBasso1(mat):
    ris = []
    for j in range(-n+1, m):
        diag = []
        for i in range(n):
            if j+i >= 0 and j+i < m:
                diag.append(mat[i][j+i])
        ris.append(diag)
    return ris

# Metodo 2 (difficile, più adatto alle altre diagonali)
# che non costruisce le diagonali una alla volta ma man mano
# che si scorre la matrice aggiunge i valori alle varie diagonali che all'inizio
# sono vuote, sapendo che per ogni diagonale la differenza tra
# gli indici i e j dei valori è costante
def diagonaliAltoBasso2(mat):
    ris = [[] for _ in range(n+m-1)]
    for j in range(m):
        for i in range(n):
            ris[(j-i)+(n-1)].append(mat[i][j])
    return ris

# Metodo 1: complicato ma che segue il ragionamento
# del metodo 1 per le diagonali alto basso
# qui i valori di j sono difficili
def diagonaliBassoAlto1(mat):
    ris = []
```

```
    for j in range(-n+1, m):
        diag = []
        for i in range(n-1,-1,-1):
            if j+(n-1-i) >= 0 and j+(n-1-i) < m:
                diag.append(mat[i][j+(n-1-i)])
        ris.append(diag)
    return ris

def diagonaliBassoAlto2(mat):
    ris = []
    for i in range(n + m - 1):
        diag = []
        for j in range(m):
            if i-j < n and i-j >= 0:
                diag.append(mat[i-j][j])
        ris.append(diag)
    return ris

# metodo 3 (astuto, ideato da Alessandro Beretta)
# che considera che gli elementi di una stessa diagonale
# basso-sinistra a in alto-destra,
# hanno la somma delle coordinate costante
def diagonaliBassoAlto3(mat):
    ris = [[] for _ in range(n+m-1)]
    for j in range(m):
        for i in range(n):
            ris[i+j].append(mat[i][j])
    return ris



print(diagonaliAltoBasso1(mat))
print(diagonaliAltoBasso2(mat))
print(diagonaliBassoAlto1(mat))
print(diagonaliBassoAlto2(mat))
print(diagonaliBassoAlto3(mat))
```

4.5.15 Verifica matrice crescente

Riempi una matrice n x m di numeri casuali (n e m scelti da te), poi verifica se gli elementi della matrice sono disposti in modo tale che ogni numero sia maggiore di quelli che si trovano immediatamente alla sua sinistra e immediatamente sopra.

Variante: scrivi e usa una funzione che restituisca la risposta alla domanda

Continua l'esercizio creando e stampando matrici fino a che non ne viene stampata una che rispetta la condizione.

Continua l'esercizio calcolando mediamente quante matrici vengono create prima di crearne una che rispetti la condizione; in questo caso puoi stampare solo il numero medio di volte e non ogni lista (dovresti stamparne un'enormità).

4.5.16 Calcolo frequenze matrice

Genera una matrice di numeri casuali poi usa una funzione che restituisce il numero che compare più di frequente nella matrice.

Soluzione

```python
from random import randint

n = 4
m = 6
mat = [[randint(0, 9) for _ in range(m)] for _ in range(n)]

for riga in mat:
    for num in riga:
        print(num, end = ' ')
    print()

# print()

# for i in range(n):
#     for j in range(m):
#         print(mat[i][j], end = ' ')
#     print()

# print()

# for i, riga in enumerate(mat):
#     for j, num in enumerate(riga):
#         # print(num, end = ' ')
#         print(mat[i][j], end = ' ')
#     print()

def piuFrequente(mat):
    d = {}
    for riga in mat:
        for num in riga:
            if num not in d:
                d[num] = 1
            else:
                d[num] += 1

    # # ricerca del massimo (opzione 1)
    # max = None
    # freq = 0
    # for num in d:
    #     if max == None or d[num] > freq:
    #         max = num
    #         freq = d[num]

    # # ricerca del massimo con lista (opzione 2, brutto)
    # lista = list(d.items())
    # max = lista[0][0]
    # freq = lista[0][1]
    # for coppia in lista:
    #     if coppia[1] > freq:
    #         max = coppia[0]
    #         freq = coppia[1]

    # trovo il massimo ordinando una lista (opzione 3)
    lista = list(d.items())
    lista.sort(key = lambda x: -x[1])
    return lista[0]

numero, frequenza = piuFrequente(mat)
print(f"Il numero più frequente è {numero} che compare {frequenza} volte")
```

4.5.17 Calcolo frequenze matrice 2

Genera una matrice di numeri casuali poi usa una funzione che restituisce una lista che contiene il numero che compare più di frequente in ogni colonna della matrice.

Soluzione

```
from random import randint

n = 8
m = 8
mat = [[randint(0, 9) for _ in range(m)] for _ in range(n)]

for riga in mat:
    for num in riga:
        print(f"{num}      ", end = '')
    print()

def piuFrequente(mat):
    n = len(mat)
    m = len(mat[0])

    ris = []

    for j in range(m):
        d = {}
        for i in range(n):
            num = mat[i][j]
            if num in d:
                d[num] += 1
            else:
                d[num] = 1

        max = None
        freqmax = 0
        for num, freq in d.items():
            if max == None or freq > freqmax:
                max = num
                freqmax = freq
        ris.append([max, freqmax])

    return ris


print()

lista = piuFrequente(mat)
for el in lista:
    print(el, end = ' ')
```

4.5.18 Ordinamento righe e colonne

Scrivi un programma che crei una matrice di numeri casuali nel modo che preferisci. Scrivi e usa anche due funzioni:

1. Funzione che ordina in ordine decrescente i numeri all'interno di ogni riga (separatamente)
2. Funzione che ordina in ordine crescente i numeri all'interno di ogni colonna.

Soluzione

```
def stampaMat(mat):
    for riga in mat:
        for num in riga:
```

```
            print(f"{num:2d} ", end='')
        print()

def ordinaRigheDec(mat):
    n = len(mat)
    for i in range(n):
        mat[i].sort(reverse = True)

def ordinaColonneCresc(mat):
    n = len(mat)
    m = len(mat[0])
    for j in range(m):
        colonna = [mat[i][j] for i in range(n)]
        colonna.sort()
        for i in range(n):
            mat[i][j] = colonna[i]


from random import randint
n = 4
m = 6
mat = [[randint(0, 99) for _ in range(m)] for _ in range(n)]

stampaMat(mat)
print()
ordinaRigheDec(mat)
stampaMat(mat)
print()
ordinaColonneCresc(mat)
stampaMat(mat)
print()
```

4.6 Matrici miste

4.6.1 Esploratori famosi

Un file di testo contiene i seguenti dati:

```
Marco Polo 1254 1324
Cristoforo Colombo 1451 1506
Amerigo Vespucci 1454 1512
Francisco Pizarro 1475 1541
Ferdinando Magellano 1480 1521
Hernan Cortez 1485 1547
Walter Raleigh 1552 1618
Henry Hudson 1570 1611
James Cook 1728 1779
Charles Darwin 1809 1882
Kit Carson 1809 1866
David Livingstone 1813 1873
Charles Foucauld 1858 1916
Ronald Amundsen 1872 1928
Ernest Shakleton 1874 1922
```

Scrivi un programma che legga i dati contenuti nel file, li memorizzi opportunamente, e poi scrivi e utilizza le seguenti funzioni:

1. una funzione che stampa tutti i dati.
2. una funzione che ordina i dati in ordine alfabetico secondo il nome degli esploratori

3. una funzione che ordina i dati in ordine crescente secondo le date di nascita degli esploratori
4. una funzione che calcola la durata della vita di ogni esploratore e aggiunge questo dato agli altri
5. una funzione che ordina gli esploratori in base alla durata della loro vita in ordine decrescente e in caso di parità in ordine crescente secondo la data di nascita
6. una funzione che riscrive tutti i dati in un secondo file

Soluzione

```
import os
os.system("cls")

# 1.    una funzione che stampa tutti i dati.
def f1(dati):
    for esp in dati:
        print(esp)

# 2.    una funzione che ordina i dati in ordine alfabetico secondo il nome degli esploratori
def f2(dati):
    dati.sort()

# 3.    una funzione che ordina i dati in ordine crescente secondo le date di nascita degli
esploratori
def f3(dati):
    dati.sort(key = lambda x: (x[2]))

# 4.    una funzione che calcola la durata della vita di ogni esploratore e agiunge questo dato
agli altri
def f4(dati):
    for esp in dati:
        esp.append(esp[3]-esp[2])

# 5.    una funzione che ordina gli esploratori in base alla durata della loro vita in ordine
decrescente e in caso di
#       parità in ordine crescente secondo la data di nascita
def f5(dati):
    dati.sort(key = lambda x: (-x[4], x[2]))

# 6.    una funzione che riscrive tutti i dati in un secondo file
def f6(dati, nomefile):
    with open(nomefile,'w',encoding = 'utf-8') as f:
        for esp in dati:
            for dato in esp:
                f.write(f"{dato} ")
            f.write("\n")

dati = []
with open("8-2-1.txt",'r',encoding = 'utf-8') as f:
    for riga in f:
        riga = riga.strip()
        riga = riga.split()
```

```
        riga[2] = int(riga[2])
        riga[3] = int(riga[3])
        dati.append(riga)

print("Dati letti dal file:")
f1(dati)

f2(dati)
print("Esploratori in ordine alfabetico:")
f1(dati)

f4(dati)
f5(dati)

print("\nClassifica per durata di vita:")
for el in dati:
    print(el)

# scrivo nel secondo file
f6(dati, "8-2-1-out.txt")
```

4.6.2 Voti da lista di stringhe

Scrivi un programma che legga i dati contenuti in una lista di stringhe (riportata di seguito) e li inserisca in opportune strutture. Di questi dati il programma deve fare le seguenti cose:

1. stamparli sullo schermo
2. ordinarli in ordine decrescente di voto
3. stamparli nuovamente in ordine

Dati:

```
dati = [
    'Amici 7.00',
    'Biella 9.00',
    'Brescia 6.00',
    'Carolla 8.00',
    'Cunegatti 7.00',
    'DeBella 4.00',
    'DeVecchi 9.00',
    'Fumagalli 7.00',
    'Galimberti 9.00',
    'Germanò 9.00',
    'Gubellini 9.00',
    'Lepore 5.00',
    'Maconi 6.00',
    'Mariani 8.00',
    'Mattavelli 9.00',
    'Oggiano 4.00',
    'Passoni 5.00',
    'Pastori 4.00',
    'Pirovano 7.00',
    'Rudi 10.00',
    'Russell 6.00',
    'Sogos 4.00',
    'Tezza 6.00',
    'Varisco 8.00',
]
```

4.6.3 Registro voti da stringa

Scrivi un programma che legga i dati contenuti in una stringa di testo (riportata di seguito) e li inserisca in opportune strutture dati che rappresentino il registro dei voti per una materia. Il programma deve poi calcolare la media dei voti per ogni singolo studente e aggiungerla alla struttura dati. Il programma deve mostrare tutti i dati raccolti e calcolati sullo schermo.

```
dati = '''Amici 7.00 8.00 7.00
Biella 8.00 9.50 7.00
Brescia 5.00 7.50 9.00
Carolla 7.00 8.50 8.50
Cunegatti 7.00 7.00 5.50
DeBella 4.00 4.00 4.50
DeVecchi 8.00 10.00 6.00
Fumagalli 8.50 6.00 4.50
Galimberti 9.00 9.00 9.00
Germanò 8.50 9.00 7.50
Gubellini 8.00 10.00 7.00
Lepore 5.50 4.00 3.00
Maconi 7.50 5.00 7.50
Mariani 8.00 8.00 7.00
Mattavelli 9.00 9.50 8.50
Oggiano 5.00 3.00 3.00
Passoni 5.00 6.00 6.00
Pastori 3.50 5.00 7.00
Pirovano 6.50 7.50 7.50
Rudi 9.50 10.00 10.00
Russell 7.50 4.50 5.50
Sogos 4.00 3.50 3.50
Tezza 7.00 5.50 5.50
Varisco 8.00 9.00 8.50'''
```

4.6.4 Registro voti da stringa 2

Scrivi un programma che legga i dati contenuti in una stringa di testo (riportata di seguito) e li inserisca in opportune strutture dati che rappresentino il registro dei voti per una materia. Il programma deve poi calcolare la media dei voti per ogni singolo studente e aggiungerla alla struttura dati. Il programma deve mostrare tutti i dati raccolti e calcolati sullo schermo. Questa versione dell'esercizio è diversa e più difficile della precedente perché si può notare che i cognomi possono essere formati da più parole e sono separati dai voti da un ":", inoltre il numero di voti varia per ogni studente.

```
dati = '''Amici: 7.00 8.00 7.00
Biella: 8.00 9.50 7.00
Brescia: 5.00 7.50 9.00
Carolla: 7.00 8.50 8.50
Cunegatti: 7.00 7.00
De Bella: 4.00 4.00 4.50 5.00
De Vecchi: 8.00 10.00 6.00
Fumagalli: 8.50 6.00 4.50
Galimberti: 9.00 9.00 9.00
Germanò: 8.50 9.00 7.50
Gubellini: 8.00 10.00 7.00
Lepore: 5.50 4.00 3.00 5.00
Maconi: 7.50 5.00
Mariani: 8.00 8.00 7.00
Mattavelli: 9.00 9.50 8.50
Oggiano: 5.00 3.00 3.00 2.00
Passoni: 5.00 6.00 6.00 7.50
```

```
Pastori: 3.50 5.00 7.00 2.00
Pirovano: 6.50 7.50 7.50
Rudi: 9.50 10.00 10.00
Russell: 7.50 4.50
Sogos: 4.00 3.50 3.50 2.00
Tezza: 7.00 5.50 5.50
Varisco: 8.00 9.00 8.50
Pirovano: 6.50 7.50 7.50
Rudi: 9.50 10.00 10.00
Russell: 7.50 4.50 5.50
Sogos: 4.00 3.50 3.50
Tezza: 7.00 5.50 5.50
Varisco: 8.00 9.00 8.50'''
```

Soluzione normale con le liste

```
dati = '''Amici: 7.00 8.00 7.00
Biella: 8.00 9.50 7.00
Brescia: 5.00 7.50 9.00
Carolla: 7.00 8.50 8.50
Cunegatti: 7.00 7.00
De Bella: 4.00 4.00 4.50 5.00
De Vecchi: 8.00 10.00 6.00
Fumagalli: 8.50 6.00 4.50
Galimberti: 9.00 9.00 9.00
Germanò: 8.50 9.00 7.50
Gubellini: 8.00 10.00 7.00
Lepore: 5.50 4.00 3.00 5.00
Maconi: 7.50 5.00
Mariani: 8.00 8.00 7.00
Mattavelli: 9.00 9.50 8.50
Oggiano: 5.00 3.00 3.00 2.00
Passoni: 5.00 6.00 6.00 7.50
Pastori: 3.50 5.00 7.00 2.00
Pirovano: 6.50 7.50 7.50
Rudi: 9.50 10.00 10.00
Russell: 7.50 4.50
Sogos: 4.00 3.50 3.50 2.00
Tezza: 7.00 5.50 5.50
Varisco: 8.00 9.00 8.50
Pirovano: 6.50 7.50 7.50
Rudi: 9.50 10.00 10.00
Russell: 7.50 4.50 5.50
Sogos: 4.00 3.50 3.50
Tezza: 7.00 5.50 5.50
Varisco: 8.00 9.00 8.50'''


lista = dati.split('\n')
for i, riga in enumerate(lista):
    dati = riga.split(':')
    nome = dati[0]

    voti = dati[1].strip()
    voti = voti.split()
    voti = [float(voto) for voto in voti]

    media = sum(voti) / len(voti)

    dati = [nome, voti, media]

    lista[i] = dati
```

```
for el in lista:
    print(el)
```

Soluzione coi dizionari

```
lista = dati.split('\n')
dizionario = {}
for i, riga in enumerate(lista):
    dati = riga.split(':')
    nome = dati[0]
    voti = dati[1].strip()

    voti = voti.split()
    voti = [float(voto) for voto in voti]

    media = sum(voti) / len(voti)

    dati = {'voti': voti, 'media': media}

    dizionario[nome] = dati


print("Stampa risultato come dizionario:")
for nome, voti in dizionario.items():
    print(nome, voti)



# voglio stampare la media dei voti di Rudi
print(f"Media di Rudi: {dizionario['Rudi']['media']}")
```

4.6.5 Virologi italiani 2

La seguente lista descrive i più celebri virologi Italiani nell'epoca covid.

Formato: (Nome e Cognome - Ospedale - città - (O) Ottimista (P) Pessimista

```
lista = [
    'Roberto Burioni - San Raffaele - Milano (P)',
    'Matteo Bassetti - San Martino - Genova (O)',
    'Alberto Zangrillo - San Raffaele - Milano (O)',
    'Massimo Galli - Luigi Sacco - Milano (P)',
    'MariaRita Gismondo - Luigi Sacco - Milano (O)',
    'Andrea Crisanti - Università di Padova - Padova (P)',
    'Ilaria Capua - One Health - Florida (P)',
    'Antonella Viola - Città della Speranza - Padova (P)',
    'Fabrizio Pregliasco - Galeazzi - Milano (O)',
    'Walter Ricciardi - Policlino Gemelli - Roma (P)',
    'Franco Locatelli - Bambin Gesù - Roma (P)'
]
```

1. stampare nome e cognome dei virologi ordinati per cognome (si consiglia si usare la split() con il separatore "-";
2. stampare il numero di virologi ottimisti e di quelli pessimisti.

4.6.6 Autostrade e regioni

Data la seguente lista di liste

```
autostrade = [
```

```
["A1", "Milano-Napoli", ["Lombardia", "Emilia Romagna", "Toscana",
"Umbria", "Lazio", "Campania"], 759],
["A2","Salerno-ReggioCalabria", ["Campania", "Basilicata", "Calabria"],
442],
["A3","Napoli-Salerno", ["Campania"], 52],
["A4","Torino-Trieste", ["Piemonte", "Lombardia", "Veneto", "Friuli"],
524],
["A6","Torino-Savona", ["Piemonte", "Liguria"], 124],
["A7","Milano-Genova", ["Lombardia", "Liguria"], 133],
["A8","Milano-Varese", ["Lombardia"], 43],
["A10","Savona-Ventimigla", ["Liguria"], 113],
["A11","Firenze-Pisa", ["Toscana"], 81],
["A12","Genova-Cecina", ["Liguria", "Toscana"], 210],
["A13","Bologna-Padova", ["Emilia Romagna", "Veneto"], 116],
["A14","Bologna-Taranto", ["Emilia Romagna", "Marche", "Abruzzo", "Molise",
"Puglia"], 743],
["A15","Parma-LaSpezia", ["Emilia Romagna", "Liguria"], 108],
["A16","Napoli-Canosa", ["Campania", "Puglia"], 172],
["A19","Palermo-Catania", ["Sicilia"], 191],
["A21","Torino-Piacenza-Brescia", ["Piemonte", "Emilia Romagna",
"Lombardia"], 238],
["A22","Brennero-Modena", ["Alto Adige", "Trentino", "Veneto", "Emilia
Romagna"], 315],
["A23","Palmanova-Tarvisio", ["Friuli"], 119],
["A24","Roma-Teramo", ["Lazio", "Abruzzo"], 159],
["A25","Torano-Pescara", ["Lazio", "Abruzzo"], 115],
["A26","Genova-Gravellona", ["Liguria", "Piemonte"], 197]
]
```

1. stampare tutte le autostrade che attraversano la Lombardia
2. stampare l'autostrada che attraversa più regioni
3. stampare, tutte le autostrade, ordinate in base al numero di regioni attraversate, e a parità, in base alla loro lunghezza
4. stampare per ogni regione, il numero di autostrade che la attraversano
5. stampare per ogni regione tutte le autostrade che la attraversano
6. stampare per ogni regione tutte le autostrade che la attraversano, ordinandole in base al numero di regioni attraversate

Soluzione

```python
# Data la seguente lista di liste

highways2 = [
    ["A1", "Milano-Napoli", ["Lombardia", "EmiliaRomagna", "Toscana", "Umbria", "Lazio",
"Campania"],759],
    ["A2","Salerno-ReggioCalabria",["Campania", "Basilicata","Calabria"], 442],
    ["A3","Napoli-Salerno",["Campania"], 52],
    ["A4","Torino-Trieste",["Piemonte", "Lombardia", "Veneto", "Friuli"], 524],
    ["A6","Torino-Savona",["Piemonte", "Liguria"], 124],
    ["A7","Milano-Genova",["Lombardia", "Liguria"], 133],
    ["A8","Milano-Varese",["Lombardia"], 43],
    ["A10","Savona-Ventimigla",["Liguria"], 113],
    ["A11","Firenze-Pisa",["Toscana"], 81],
    ["A12","Genova-Cecina",["Liguria", "Toscana"], 210],
    ["A13","Bologna-Padova",["EmiliaRomagna", "Veneto"], 116],
    ["A14","Bologna-Taranto",["EmiliaRomagna", "Marche", "Abruzzo", "Molise", "Puglia"], 743],
```

```
    ["A15","Parma-LaSpezia",["EmiliaRomagna", "Liguria"], 108],
    ["A16","Napoli-Canosa",["Campania", "Puglia"], 172],
    ["A19","Palermo-Catania",["Sicilia"], 191],
    ["A21","Torino-Piacenza-Brescia",["Piemonte","EmiliaRomagna","Lombardia"], 238],
    ["A22","Brennero-Modena",["AltoAdige", "Trentino", "Veneto","EmiliaRomagna"], 315],
    ["A23","Palmanova-Tarvisio",["Friuli"], 119],
    ["A24","Roma-Teramo",["Lazio", "Abruzzo"], 159],
    ["A25","Torano-Pescara",["Lazio", "Abruzzo"], 115],
    ["A26","Genova-Gravellona",["Liguria", "Piemonte"], 197]
]

# 1) stampare tutte le autostrade che attraversano la Lombardia
print("Autostrade che attraversano la Lombardia:")
for autostrada in highways2:
    if "Lombardia" in autostrada[2]:
        print(autostrada)

# 2) stampare l'autostra che attraversa più regioni
print("\nAutostrada che attraversa più regioni:")
maxi = 0
max = len(highways2[0][2])
for i, el in enumerate(highways2):
    lung = len(el[2])
    if lung > max:
        max = lung
        maxi = i
print(highways2[maxi])

# 3) stampare, tutte le autostrade, ordinate in base al numero di regioni attraversate, e a
parità,
# in base alla loro lunghezza
print("\nAutostrade ordinate in base al numero di regioni attraversate, e a parità, in base alla
loro lunghezza:")
highways2.sort(key = lambda el: (len(el[2]), el[3]))
for el in highways2:
    print(el)

# 4) stampare per ogni regione, il numero di autostrade che la attraversano
print("\nPer ogni regione, il numero di autostrade che la attraversano:")
regioni = {}
for aut in highways2:
    for reg in aut[2]:
        if reg not in regioni:
            regioni[reg] = 1
        else:
            regioni[reg] += 1

for key, val in regioni.items():
    print(key, val)

# 5) stampare per ogni regione tutte le autostrade che la attraversano
print("\nPer ogni regione, tutte le autostrade che la attraversano:")
regioni = {}
```

```
for aut in highways2:
    for reg in aut[2]:
        if reg not in regioni:
            regioni[reg] = [aut]
        else:
            regioni[reg].append(aut)

for key, val in regioni.items():
    print (f"{key}:")
    for aut in val:
        print(" -", aut)

# 6) stampare per ogni regione tutte le autostrade che la attraversano, ordinandole in base al numero di
# regioni attraversate
print("\n\nPer ogni regione, tutte le autostrade che la attraversano, ordinate in base al numero di regioni attraversate:")
regioni = {}
for aut in highways2:
    for reg in aut[2]:
        if reg not in regioni:
            regioni[reg] = [aut]
        else:
            regioni[reg].append(aut)

for key, val in regioni.items():
    print (f"{key}:")
    val.sort(key = lambda el: -len(el[2]))
    for aut in val:
        print(" -", aut)
```

Senza dizionario

```
# 3.5.12    Data la seguente lista di liste
highways2 = [
    ["A1", "Milano-Napoli", ["Lombardia", "Emilia Romagna", "Toscana", "Umbria", "Lazio", "Campania"], 759],
    ["A2","Salerno-ReggioCalabria", ["Campania", "Basilicata", "Calabria"], 442],
    ["A3","Napoli-Salerno", ["Campania"], 52],
    ["A4","Torino-Trieste", ["Piemonte", "Lombardia", "Veneto", "Friuli"], 524],
    ["A6","Torino-Savona", ["Piemonte", "Liguria"], 124],
    ["A7","Milano-Genova", ["Lombardia", "Liguria"], 133],
    ["A8","Milano-Varese", ["Lombardia"], 43],
    ["A10","Savona-Ventimigla", ["Liguria"], 113],
    ["A11","Firenze-Pisa", ["Toscana"], 81],
    ["A12","Genova-Cecina", ["Liguria", "Toscana"], 210],
    ["A13","Bologna-Padova", ["Emilia Romagna", "Veneto"], 116],
    ["A14","Bologna-Taranto", ["Emilia Romagna", "Marche", "Abruzzo", "Molise", "Puglia"], 743],
    ["A15","Parma-LaSpezia", ["Emilia Romagna", "Liguria"], 108],
    ["A16","Napoli-Canosa", ["Campania", "Puglia"], 172],
    ["A19","Palermo-Catania", ["Sicilia"], 191],
    ["A21","Torino-Piacenza-Brescia", ["Piemonte", "Emilia Romagna", "Lombardia"], 238],
    ["A22","Brennero-Modena", ["Alto Adige", "Trentino", "Veneto", "Emilia Romagna"], 315],
    ["A23","Palmanova-Tarvisio", ["Friuli"], 119],
```

```
    ["A24","Roma-Teramo", ["Lazio", "Abruzzo"], 159],
    ["A25","Torano-Pescara", ["Lazio", "Abruzzo"], 115],
    ["A26","Genova-Gravellona", ["Liguria", "Piemonte"], 197]
]
# 1.    stampare tutte le autostrade che attraversano la Lombardia
for el in highways2:
    if 'Lombardia' in el[2]:
        print(el)
print()

# 2.    stampare l'autostrada che attraversa più regioni
# lunghezze = [len(el[2]) for el in highways2]
# # print(lunghezze)
# imax = 0
# for i, num in enumerate(lunghezze):
#     if num > lunghezze[imax]:
#         imax = i
# print(highways2[imax])

imax = 0
for i, el in enumerate(highways2):
    if len(el[2]) > len(highways2[imax][2]):
        imax = i
print(highways2[imax])
print()

# 3.    stampare, tutte le autostrade, ordinate in base al numero di regioni attraversate,
# e a parità, in base alla loro lunghezza
highways2.sort(key= lambda x: (-len(x[2]), -x[-1]))
for el in highways2:
    print(el)
print()

# 4.    stampare per ogni regione, il numero di autostrade che la attraversano
regioni = []
for el in highways2:
    for regione in el[2]:
        if regione not in regioni:
            regioni.append(regione)
regioni = [[regione, 0] for regione in regioni]
for regione in regioni:
    for el in highways2:
        if regione[0] in el[2]:
            regione[1] += 1
for regione in regioni:
    print(regione)
print()

# 5.    stampare per ogni regione tutte le autostrade che la attraversano
for regione in regioni:
    nome = regione[0]
    print(f'{nome}: ', end='')
```

```
    # con list comprehension
    # autostrade = [el[0] for el in highways2 if nome in el[2]]

    # senza list comprehension
    autostrade = []
    for el in highways2:
        if nome in el[2]:
            autostrade.append(el[0])

    print(', '.join(autostrade))

# 6.    stampare per ogni regione tutte le autostrade che la attraversano, ordinandole
# in base al numero di regioni attraversate

for regione in regioni:
    nome = regione[0]

    # con list comprehension
    autostrade = [el for el in highways2 if nome in el[2]]
    autostrade.sort(key= lambda x: -len(x[2]))
    print(f'{nome}:')
    for el in autostrade:
        print(' -', el)
```

4.7 Dizionari

4.7.1 Frequenza dei caratteri

Scrivi una funzione a cui passare una stringa come parametro, e che restituisca un dizionario rappresentante la "frequenza di comparsa" di ciscun carattere componente la stringa.

Data una stringa "ababcc", otterremo in risultato {"a": 2, "b": 2, "c": 2}

Soluzione

```
def f(stringa):
    d = {}
    for lettera in stringa:
        if lettera in d:
            d[lettera] += 1
        else:
            d[lettera] = 1
    return d


d = f("Ciao a tutti, come state?")
for key in d:
    print(f"{key}: {d[key]}")
```

4.7.2 Numero più frequente

Scrivi un programma che utilizzi una funzione che, ricevuto una lista di numeri interi, restituisce il numero che compare con più frequenza (in caso di parità restituisce il primo numero trovato con la frequenza maggiore). Puoi risolvilo in due modi:

1. Con sole liste (più lungo e complicato)
2. Con un dizionario

Soluzione

```
def numero_con_frequenza_massima_lista(numeri):
    frequenze = [0 for _ in range(len(numeri))]
    for i in range(len(numeri)):
        if frequenze[i] == 0:
            frequenze[i] = 1
            for j in range(i+1, len(numeri)):
                if(numeri[i] == numeri[j]):
                    frequenze[i] += 1
                    frequenze[j] += 1

    val_max = frequenze[0]
    pos_max = 0
    for i in range(1, len(numeri)):
        if frequenze[i] > val_max:
            val_max = frequenze[i]
            pos_max = i

    return numeri[pos_max]


def numero_con_frequenza_massima_dizionario(lista):
   d = {}
    for num in lista:
        if num in d:
            d[num] += 1
        else:
            d[num] = 1

    max = None
    freqmax = 0
    for key in d:
        if d[key] > freqmax:
            max = key
            freqmax = d[key]

    return max, freqmax

numeri = [1,1,2,2,2,3,4,8,6,4,4]
print(numeri)
print("Primo numero con frequenza massima (liste):", numero_con_frequenza_massima_lista(numeri))
print("Primo numero con frequenza massima (dizionario):",
numero_con_frequenza_massima_dizionario(numeri))
```

4.7.3 Rimozione stringhe e conteggio

Scrivi e usa una funzione che presa in input una lista contenente interi e stringhe, modifica la lista distruttivamente e restituisce un dizionario. Al termine della funzione dalla lista devono risultare cancellate tutte le stringhe e il dizionario restituito deve contenere come chiavi le stringhe cancellate ciascuna con attributo il numero di volte in cui occorrevano nella lista.

Ad esempio per lista=[1,'a',2,'b','a',8,'d',8] la funzione al termine restituisce il dizionario {'a':2,'b':1,'d':1} e la lista diventa [1,2,8,8]

Soluzione

```
def f(lista):
    ris = {}
    i = 0
    while i < len(lista):
        if type(lista[i]) == str:
            if lista[i] in ris:
                ris[lista[i]] += 1
            else:
                ris[lista[i]] = 1

            lista.pop(i)
        else:
            i += 1

    return ris



print(f([1,'a',2,'b','a',8,'d',8]))
```

4.7.4 Girone di Champions

La seguente lista di tuple rappresenta un girone di Champions: ogni tupla contiene, nell'ordine, la squadra di casa, la squadra in trasferta e i gol segnati da ciascuna delle due.

```
champions = [
("Sheriff Tiraspol", "Shaktar Donetsk", 2, 0),
("Inter", "Real Madrid", 0, 1),
("Shaktar Donetsk", "Inter", 0, 0),
("Real Madrid", "Sheriff Tiraspol", 1, 2),
("Shaktar Donetsk", "Real Madrid", 0, 5),
("Inter", "Sheriff Tiraspol", 3, 1),
("Real Madrid", "Shaktar Donetsk", 2, 1),
("Sheriff Tiraspol", "Inter", 1, 3),
("Inter", "Shaktar Donetsk", 2, 0),
("Sheriff Tiraspol", "Real Madrid", 0, 3),
("Shaktar Donetsk", "Sheriff Tiraspol", 1, 1),
("Real Madrid", "Inter", 2, 0)
]
```

Sapendo che la vittoria vale 3 punti, il pareggio 1 punto e la sconfitta 0 punti, calcola e stampa per ogni squadra quanti punti ha fatto.

Variante: stampa la classifica ordinata in base ai punti.

Soluzione

```
champions = [
    ("Sheriff Tiraspol", "Shaktar Donetsk", 2, 0),
    ("Inter", "Real Madrid", 0, 1),
    ("Shaktar Donetsk", "Inter", 0, 0),
    ("Real Madrid", "Sheriff Tiraspol", 1, 2),
    ("Shaktar Donetsk", "Real Madrid", 0, 5),
    ("Inter", "Sheriff Tiraspol", 3, 1),
    ("Real Madrid", "Shaktar Donetsk", 2, 1),
    ("Sheriff Tiraspol", "Inter", 1, 3),
    ("Inter", "Shaktar Donetsk", 2, 0),
    ("Sheriff Tiraspol", "Real Madrid", 0, 3),
    ("Shaktar Donetsk", "Sheriff Tiraspol", 1, 1),
    ("Real Madrid", "Inter", 2, 0)
]

d = {}
for sq1, sq2, g1, g2 in champions:
    if g1 > g2:
        p1 = 3
        p2 = 0
    elif g2 > g1:
        p1 = 0
        p2 = 3
    else:
        p1 = 1
        p2 = 1

    if sq1 not in d:
        d[sq1] = p1
    else:
        d[sq1] += p1

    if sq2 not in d:
        d[sq2] = p2
    else:
        d[sq2] += p2

classifica = list(d.items())
classifica.sort(key = lambda x: -x[1])
for el in classifica:
    print(el)
```

4.7.5 Paesi e lingue

Dato il seguente dizionario:

d = {

"senegal": ("africa", "francese"), "quebec": ("nordAmerica", "francese"),
"stati uniti": ("nordAmerica", "inglese"), "canada": ("nordAmerica", "inglese"),
"brasile": ("sudAmerica", "portoghese"), "messico": ("nordAmerica", "spagnolo"),
"australia": ("oceania", "inglese"), "haiti": ("centroAmerica", "francese"),
"angola": ("africa", "portoghese"), "ciad": ("africa", "francese"),
"filippine": ("asia", "spagnolo"), "argentina": ("sudAmerica", "spagnolo"),
"neoZelanda": ("oceania", "inglese"), "cuba": ("centroAmerica", "spagnolo"),
"mozambico": ("africa", "portoghese"), "mali": ("africa", "francese"),
"nigeria": ("africa", "inglese"), "martinica": ("centroAmerica", "francese"),
"uruguay": ("sudAmerica", "spagnolo"), "cile": ("sudAmerica", "spagnolo"),
"caienna": ("sudAmerica", "francese"), "perù": ("sudAmerica", "spagnolo")
}

1. stampare tutti i paesi di lingua francese
2. per ogni lingua, stampare il numero di paesi
3. per ogni lingua stampare la lista ordinata dei paesi che parlano appunto quella lingua

Soluzione

```
import os
os.system("cls")

d = {
    "senegal": ("africa", "francese"), "quebec": ("nordAmerica", "francese"),
    "stati uniti": ("nordAmerica", "inglese"), "canada": ("nordAmerica", "inglese"),
    "brasile": ("sudAmerica", "portoghese"), "messico": ("nordAmerica", "spagnolo"),
    "australia": ("oceania", "inglese"), "haiti": ("centroAmerica", "francese"),
    "angola": ("africa", "portoghese"), "ciad": ("africa", "francese"),
    "filippine": ("asia", "spagnolo"), "argentina": ("sudAmerica", "spagnolo"),
    "neoZelanda": ("oceania", "inglese"), "cuba": ("centroAmerica", "spagnolo"),
    "mozambico": ("africa", "portoghese"), "mali": ("africa", "francese"),
    "nigeria": ("africa", "inglese"), "martinica": ("centroAmerica", "francese"),
    "uruguay": ("sudAmerica", "spagnolo"), "cile": ("sudAmerica", "spagnolo"),
    "caienna": ("sudAmerica", "francese"), "perù": ("sudAmerica", "spagnolo")
}

# 1.    stampare tutti i paesi di lingua francese
print("Paesi in cui si parla francese:")
for key, val, in d.items():
    if val[1] == "francese":
        print(key)
print("")

print("Dizionario dopo l'inserimento:")
for key, val, in d.items():
    print(f"{key}: {val}")
print("")

# 2.    per ogni lingua, stampare il numero di paesi
```

```
lingue = {}
for nazione in d:
    lingua = d[nazione][1]
    if lingua in lingue:
        lingue[lingua] += 1
    else:
        lingue[lingua] = 1

print("Numero di paesi per ogni lingua:")
for tupla in lingue.items():
    print(tupla)
print("")

# 3.    per ogni lingua stampare la lista ordinata dei paesi che parlano appunto quella lingua
lingue = {}
for nazione in d:
    lingua = d[nazione][1]
    if lingua in lingue:
        lingue[lingua].append(nazione)
    else:
        lingue[lingua] = [nazione]

print("Lista ordinata di paesi per ogni lingua:")
for lingua in lingue:
    lingue[lingua].sort()
    print(f"{lingua}: {lingue[lingua]}")
```

4.7.6 Cifra più presente

Data la seguente lista di numeri

lista = [882, 1024, 917, 777, 89, 7, 274, 112, 898, 666, 76, 75737, 6767, 39]

stampare la cifra più presente

(Risultato: la cifra 7 è presente 12 volte)

Soluzione

```
import os
os.system("cls")

lista = [882, 1024, 917, 777, 89, 7, 274, 112, 898,  666, 76, 75737, 6767, 39]

d = {}
for numero in lista:
    s = str(numero)
    for cifra in s:
        cifra = int(cifra)
        if cifra in d:
            d[cifra] += 1
        else:
            d[cifra] = 1
```

```
print(d)

classifica = list(d.items())
print(classifica)
classifica.sort(key = lambda x: (-x[1], x[0]))

for el in classifica:
    print(f"{el[0]}: {el[1]}")
print()

classifica = dict(classifica)
for key in classifica:
    print(f"{key}: {classifica[key]}")
```

4.7.7 Squadre e città

(dizionario di dizionario --> il secondo dizionario è in pratica usato come fosse una struct)

```
d = {
"Atalanta": {"city": "Bergamo", "country": "Italia"},
"Hellas": {"city": "Verona", "country": "Italia"},
"Sampdoria": {"city": "Genova", "country": "Italia"},
"Genoa": {"city": "Genova", "country": "Italia"},
"Liverpool": {"city": "Liverpool", "country": "RegnoUnito"},
"Everton": {"city": "Liverpool", "country": "RegnoUnito"},
"Espanol": {"city": "Barcellona", "country": "Spagna"},
"Barcellona": {"city": "Barcellona", "country": "Spagna"},
"Siviglia": {"city": "Siviglia", "country": "Spagna"},
"Betis": {"city": "Siviglia", "country": "Spagna"},
"Juventus": {"city": "Torino", "country": "Italia"},
"YoungBoys": {"city": "Berna", "country": "Svizzera"},
"Chievo": {"city": "Verona", "country": "Italia"}
}
```

1. Trovare la città dell'Hellas, e stampare il nome dell'altra squadra della stessa città
2. Stampare in ordine alfabetico tutte le squadre italiane
3. Stampare le sole squadre italiane, ordinate in base alla città, e a parità di città in ordine alfabetico

Soluzione

```
# Riprendendo il dizionario dell'esercizio precedente:
# 1) Trovare la città dell'Hellas, e stampare il nome dell'altra squadra della stessa città
# 2) stampare in ordine alfabetico tutte le squadre italiane
# 3)Stampare le sole squadre italiane , ordinate in base alla città, e a parita di città in
ordine alfabetico

d = {
    "Atalanta": {"city": "Bergamo", "country": "Italia"},
    "Hellas":    {"city": "Verona", "country": "Italia"},
    "Sampdoria": {"city": "Genova", "country": "Italia"},
    "Genoa":    {"city": "Genova", "country": "Italia"},
    "Liverpool": {"city": "Liverpool", "country": "RegnoUnito"},
    "Everton": {"city": "Liverpool", "country": "RegnoUnito"},
    "Espanol": {"city": "Barcellona", "country": "Spagna"},
```

```
    "Barcellona": {"city": "Barcellona", "country": "Spagna"},
    "Siviglia": {"city": "Siviglia", "country": "Spagna"},
    "Betis":    {"city": "Siviglia", "country": "Spagna"},
    "Juventus":    {"city": "Torino", "country": "Italia"},
    "YoungBoys":    {"city": "Berna", "country": "Svizzera"},
    "Chievo":    {"city": "Verona", "country": "Italia"}
}

# 1) Trovare la città dell'Hellas, e stampare il nome dell'altra squadra della stessa città
print("1) Trovare la città dell'Hellas, e stampare il nome dell'altra squadra della stessa città")
citta = d["Hellas"]["city"]
for key, value in d.items():
    if key != "Hellas" and value["city"] == citta:
        print(key)

# 2) stampare in ordine alfabetico tutte le squadre italiane
print("\n2) stampare in ordine alfabetico tutte le squadre italiane")
italiane = [key for key, value in d.items() if value["country"] == "Italia"]
print(sorted(italiane))

# 3)Stampare le sole squadre italiane , ordinate in base alla città, e a parita di città in ordine alfabetico
print("\n3)Stampare le sole squadre italiane , ordinate in base alla città, e a parita di città in ordine alfabetico")
## metodo con le liste
# italiane = [(key, list(value.values())[0], list(value.values())[1]) for key, value in d.items() if d[key]["country"] == "Italia"]
italiane = [(key, list(value.values())) for key, value in d.items() if d[key]["country"] == "Italia"]
italiane.sort(key=lambda el: (el[1], el[0]))
print(italiane)
```

4.7.8 Autostrade e gestori

Dato il seguente dizionario di dizionari:

```
d = {
   "A1":   {"percorso":"Milano-Napoli",   "gestore":"Autostrade   per   l'Italia",
   "km":759},
   "A2": {"percorso":"Salerno-ReggioCalabria", "gestore":"Anas", "km":442},
   "A3":   {"percorso":"Napoli-Salerno",   "gestore":"Autostrade   per   l'Italia",
   "km":52},
   "A4":   {"percorso":"Torino-Trieste",   "gestore":"Autostrade   per   l'Italia",
   "km":524},
   "A6": {"percorso":"Torino-Savona", "gestore":"Gruppo Gavio", "km":124},
   "A7": {"percorso":"Milano-Genova", "gestore":"Enti Pubblici", "km":133},
   "A8":   {"percorso":"Milano-Varese",   "gestore":"Autostrade   per   l'Italia",
   "km":43},
   "A10": {"percorso":"Savona-Ventimiglia", "gestore":"Gruppo Gavio", "km":113},
   "A11":   {"percorso":"Firenze-Pisa",   "gestore":"Autostrade   per   l'Italia",
   "km":81},
   "A12": {"percorso":"Genova-Cecina", "gestore":"Gruppo Gavio", "km":210},
   "A13":   {"percorso":"Bologna-Padova",   "gestore":"Autostrade   per   l'Italia",
   "km":116},
```

```
    "A14":  {"percorso":"Bologna-Taranto",  "gestore":"Autostrade  per  l'Italia",
    "km":743},
    "A15": {"percorso":"Parma-LaSpezia", "gestore":"Gruppo Gavio", "km":108},
    "A16":   {"percorso":"Napoli-Canosa",   "gestore":"Autostrade   per   l'Italia",
    "km":172},
    "A19": {"percorso":"Palermo-Catania", "gestore":"Anas", "km":191},
    "A21":    {"percorso":"Torino-Piacenza-Brescia",    "gestore":"Gruppo    Gavio",
    "km":238},
    "A22": {"percorso":"Brennero-Modena", "gestore":"Enti Pubblici", "km":315},
    "A23": {"percorso":"Palmanova-Tarvisio", "gestore":"Enti Pubblici", "km":119},
    "A24": {"percorso":"Roma-Teramo", "gestore":"Gruppo Toto", "km":159},
    "A25": {"percorso":"Torano-Pescara", "gestore":"Gruppo Toto", "km":115},
    "A26":  {"percorso":"Genova-Gravellona",  "gestore":"Autostrade  per  l'Italia",
    "km":197},
}
```

1. Stampare il gestore dell'autostrada A8
2. stampare il gestore dell'autostrada "Savona-Ventimiglia"
3. stampare l'autostrada più lunga, con il suo percorso e il suo gestore
4. Stampare quanti differenti gestori sono presenti nel dizionario (per la soluzione utilizzare una semplice lista)
5. Per ognuno dei gestori presenti stampare il numero di autostrade gestite, e il numero di km gestiti

Soluzione

```
# Dato il seguente dizionario di dizionario:

d = {
    "A1" : {"percorso":"Milano-Napoli", "gestore":"Autostrade per l'Italia", "km":759},
    "A2" : {"percorso":"Salerno-ReggioCalabria", "gestore":"Anas", "km":442},
    "A3" : {"percorso":"Napoli-Salerno", "gestore":"Autostrade per l'Italia", "km":52},
    "A4" : {"percorso":"Torino-Trieste", "gestore":"Autostrade per l'Italia",
"km":524},
    "A6" : {"percorso":"Torino-Savona", "gestore":"Gruppo Gavio", "km":124},
    "A7" : {"percorso":"Milano-Genova", "gestore":"Enti Pubblici", "km":133},
    "A8" : {"percorso":"Milano-Varese", "gestore":"Autostrade per l'Italia", "km":43},
    "A10" : {"percorso":"Savona-Ventimiglia", "gestore":"Gruppo Gavio", "km":113},
    "A11" : {"percorso":"Firenze-Pisa", "gestore":"Autostrade per l'Italia", "km":81},
    "A12" : {"percorso":"Genova-Cecina", "gestore":"Gruppo Gavio", "km":210},
    "A13" : {"percorso":"Bologna-Padova", "gestore":"Autostrade per l'Italia",
"km":116},
    "A14" : {"percorso":"Bologna-Taranto", "gestore":"Autostrade per l'Italia",
"km":743},
    "A15" : {"percorso":"Parma-LaSpezia", "gestore":"Gruppo Gavio", "km":108},
    "A16" : {"percorso":"Napoli-Canosa", "gestore":"Autostrade per l'Italia",
"km":172},
    "A19" : {"percorso":"Palermo-Catania", "gestore":"Anas", "km":191},
```

```
    "A21" : {"percorso":"Torino-Piacenza-Brescia", "gestore":"Gruppo Gavio", "km":238},
    "A22" : {"percorso":"Brennero-Modena", "gestore":"Enti Pubblici", "km":315},
    "A23" : {"percorso":"Palmanova-Tarvisio", "gestore":"Enti Pubblici", "km":119},
    "A24" : {"percorso":"Roma-Teramo", "gestore":"Gruppo Toto", "km":159},
    "A25" : {"percorso":"Torano-Pescara", "gestore":"Gruppo Toto", "km":115},
    "A26" : {"percorso":"Genova-Gravellona", "gestore":"Autostrade per l'Italia",
"km":197}
}

# 1) Stampare il gestore dell'autostrada A8
print("\n\nGestore dell'autostrada A8:")
print(" -", d["A8"]["gestore"])

# 2) stampare il gestore dell'autostrada "Savona-Ventimiglia"
print("\n\nGestore dell'autostrada \"Savona-Ventimiglia\":")
for aut, dat in d.items():
    if dat["percorso"] == "Savona-Ventimiglia":
        print(" -", dat["gestore"])

# 3) stampare l'autostrada più lunga, con il suo percorso e il suo gestore
print("\n\nL'autostrada più lunga, con il suo percorso e il suo gestore:")
kmax = None
lungmax = None
for aut, dat in d.items():
    if lungmax == None or lungmax < dat["km"]:
        kmax = aut
        lungmax = dat["km"]
print(" -", kmax, d[kmax])

# 4) Stampare quanti differenti gestori sono presenti nel dizionario (per la soluzione
utilizzare una semplice lista)
print("\n\nNumero di gestori:")
gestori = []
for aut, dat in d.items():
    if dat["gestore"] not in gestori:
        gestori.append(dat["gestore"])
print(" -", len(gestori))

# 5) Per ognuno dei gestori presenti stampare il numero di autostrade gestite, e il
numero di km gestiti
print("\n\nPer ognuno dei gestori presenti: il numero di autostrade gestite, e il
numero di km gestiti:")
```

```
gestori = {}
for aut, dat in d.items():
    if dat["gestore"] not in gestori:
        gestori[dat["gestore"]] = [1, dat["km"]]
    else:
        gestori[dat["gestore"]][0] += 1
        gestori[dat["gestore"]][1] += dat["km"]

for gestore, dati in gestori.items():
    print(" -", gestore, dati[0], dati[1])
```

4.7.9 Ore lavorate

Il seguente dizionario descrive quante ore hanno lavorato in una settimana, una serie di dipendenti: ad esempio Mr White ha lavorato 8 ore il lunedì, 9 ore il martedì, 8 ore il mercoledì, 9 ore il giovedì e 4 ore il venerdì

```
d2 = {
   "Mr White": [8, 9, 8, 9, 4],
   "Mr Brown": [0, 8, 7, 10, 8],
   "Mr Blonde": [8, 8, 8, 9, 9],
   "Mr Black": [6, 9, 9, 8, 8],
   "Mr Red": [ 8, 7, 8, 8, 8],
   "Mr Green": [ 4, 8, 8, 8, 4]
}
```

1. Stampare la classifica dei lavoratori, in base al numero di ore lavorate
2. Stampare il giorno con più ore lavorate, e quante ore sono state lavorate in quel giorno

Soluzione

```
giorni = ['Lunedì', 'Martedì', 'Mercoledì', 'Giovedì', 'Venerdì']

d = {
    "Mr White": [8, 9, 8, 9, 4],
    "Mr Brown": [0, 8, 7, 10, 8],
    "Mr Blonde": [8, 8, 8, 9, 9],
    "Mr Black": [6, 9, 9, 8, 8],
    "Mr Red": [ 8, 7, 8, 8, 8],
    "Mr Green": [ 4, 8, 8, 8, 4]
}

# 1.  Stampare la classifica dei lavoratori, in base al numero di ore lavorate
mat = [[key, val, sum(val)] for key, val in d.items()]
mat.sort(key = lambda x: -x[2])
for riga in mat:
    print(f"{riga[0]}: {riga[2]}")

# 2.  Stampare il giorno con più ore lavorate, e quante ore sono state lavorate
# in quel giorno
jmax = None
max = None
```

```
for j in range(len(mat[0][1])):
    somma = 0
    for i in range(len(mat)):
        somma += mat[i][1][j]
    if max == None or somma > max:
        jmax = j
        max = somma

print(f"Il giorno in cui si è lavorato di più è {giorni[jmax]} in cui si è lavorato in torale per
{max} ore.")
```

4.7.10 Rubrica telefonica

Data la seguente rubrica telefonica:

```
rubrica = {
   "Anna": "3471234567",
   "Bruno": "3289876543",
   "Carla": "3335556677",
   "Dario": "3401122334"
}
```

scrivi un programma che mostri ripetutamente un menu con le seguenti operazioni, finché l'utente non sceglie di uscire:

1. Cercare un contatto per nome e stamparne il numero (se il nome non è presente stampare "Contatto non presente" senza usare if, ma sfruttando il metodo get)
2. Aggiungere un nuovo contatto (se il nome è già presente il numero non deve essere sovrascritto)
3. Eliminare un contatto, stampando il numero eliminato (usa il metodo pop)
4. Stampare tutti i contatti in ordine alfabetico

Soluzione

```
rubrica = {
    "Anna": "3471234567",
    "Bruno": "3289876543",
    "Carla": "3335556677",
    "Dario": "3401122334"
}

scelta = ""
while scelta != "0":
    print("\n1) Cerca  2) Aggiungi  3) Elimina  4) Stampa  0) Esci")
    scelta = input("Scelta: ")
    if scelta == "1":
        nome = input("Nome da cercare: ")
        print(rubrica.get(nome, "Contatto non presente"))
    elif scelta == "2":
        nome = input("Nome: ")
        if nome in rubrica:
            print("Contatto già presente, numero non modificato")
        else:
```

```
            rubrica[nome] = input("Numero: ")
    elif scelta == "3":
        nome = input("Nome da eliminare: ")
        if nome in rubrica:
            numero = rubrica.pop(nome)
            print(f"Eliminato {nome} ({numero})")
        else:
            print("Contatto non presente")
    elif scelta == "4":
        for nome in sorted(rubrica):
            print(f"{nome}: {rubrica[nome]}")
```

4.7.11 Scontrino della spesa

Il dizionario listino associa ad ogni prodotto di un negozio il suo prezzo in euro, mentre la lista carrello contiene le tuple (prodotto, quantità) acquistate da un cliente. Lo stesso prodotto può comparire più volte nel carrello e alcuni prodotti del carrello potrebbero non essere presenti nel listino.

```
listino = {
   "pane": 2.50, "latte": 1.20, "uova": 3.10, "pasta": 1.45,
   "mele": 2.20, "caffè": 4.90, "biscotti": 2.75
}
carrello = [
   ("pane", 2), ("latte", 3), ("cioccolato", 1), ("pasta", 4),
   ("caffè", 1), ("banane", 2), ("pasta", 2), ("mele", 1)
]
```

1. Stampare lo scontrino (prodotto, quantità, importo) e il totale da pagare, ignorando i prodotti non presenti nel listino
2. Stampare la lista dei prodotti non disponibili, senza ripetizioni
3. Costruire un dizionario che associ ad ogni prodotto disponibile la quantità totale acquistata e stamparlo (usa il metodo get con valore di default)
4. Aumentare del 10% il prezzo di tutti i prodotti del listino che costano meno di 2 euro e stampare il listino aggiornato

(Risultato del punto 1: totale 24.40 €)

Soluzione

```
listino = {
    "pane": 2.50, "latte": 1.20, "uova": 3.10, "pasta": 1.45,
    "mele": 2.20, "caffè": 4.90, "biscotti": 2.75
}

carrello = [
    ("pane", 2), ("latte", 3), ("cioccolato", 1), ("pasta", 4),
    ("caffè", 1), ("banane", 2), ("pasta", 2), ("mele", 1)
]

# 1) scontrino e totale
totale = 0
for prodotto, quantita in carrello:
    if prodotto in listino:
        importo = listino[prodotto] * quantita
```

```
        totale += importo
        print(f"{prodotto} x{quantita}: {importo:.2f} €")
print(f"Totale: {totale:.2f} €")

# 2) prodotti non disponibili
non_disponibili = []
for prodotto, quantita in carrello:
    if prodotto not in listino and prodotto not in non_disponibili:
        non_disponibili.append(prodotto)
print("Non disponibili:", non_disponibili)

# 3) quantità totale per prodotto
quantita_totali = {}
for prodotto, quantita in carrello:
    if prodotto in listino:
        quantita_totali[prodotto] = quantita_totali.get(prodotto, 0) + quantita
for prodotto, quantita in quantita_totali.items():
    print(f"{prodotto}: {quantita}")

# 4) aumento del 10% dei prodotti che costano meno di 2 €
for prodotto in listino:
    if listino[prodotto] < 2:
        listino[prodotto] = round(listino[prodotto] * 1.1, 2)
print(listino)
```

4.7.12 Traduttore parola per parola

Dato il seguente dizionario italiano-inglese:

```
it_en = {
   "il": "the", "la": "the", "gatto": "cat", "cane": "dog",
   "mangia": "eats", "beve": "drinks", "pesce": "fish",
   "latte": "milk", "nero": "black", "piccolo": "small"
}
```

1. Tradurre parola per parola la frase "il gatto nero mangia il pesce e beve il latte": le parole che non compaiono nel dizionario vanno lasciate in italiano tra parentesi quadre
2. Costruire il dizionario inverso inglese-italiano: siccome più parole italiane possono avere la stessa traduzione, ad ogni parola inglese va associata la lista delle parole italiane corrispondenti
3. Stampare le parole inglesi che hanno più di una traduzione italiana

(Risultato del punto 1: the cat black eats the fish [e] drinks the milk)

Soluzione

```
it_en = {
    "il": "the", "la": "the", "gatto": "cat", "cane": "dog",
    "mangia": "eats", "beve": "drinks", "pesce": "fish",
    "latte": "milk", "nero": "black", "piccolo": "small"
}

# 1) traduzione parola per parola
frase = "il gatto nero mangia il pesce e beve il latte"
tradotta = []
```

```
for parola in frase.split():
    tradotta.append(it_en.get(parola, f"[{parola}]"))
print(" ".join(tradotta))

# 2) dizionario inverso
en_it = {}
for ita, ing in it_en.items():
    if ing in en_it:
        en_it[ing].append(ita)
    else:
        en_it[ing] = [ita]
print(en_it)

# 3) parole inglesi con più di una traduzione italiana
for ing, parole in en_it.items():
    if len(parole) > 1:
        print(f"{ing}: {parole}")
```

4.7.13 Registro di classe

La seguente lista di dizionari rappresenta gli studenti di alcune classi con i rispettivi voti:

```
studenti = [
   {"nome": "Alice", "classe": "3A", "voti": [7, 8, 6.5, 9]},
   {"nome": "Bruno", "classe": "3B", "voti": [5, 6, 5.5]},
   {"nome": "Chiara", "classe": "3A", "voti": [8, 8.5, 9, 10]},
   {"nome": "Davide", "classe": "3C", "voti": [6, 7, 6]},
   {"nome": "Elena", "classe": "3B", "voti": [9, 8, 9.5, 8]},
   {"nome": "Fabio", "classe": "3A", "voti": [4, 5.5, 6]},
   {"nome": "Giulia", "classe": "3C", "voti": [7.5, 8, 7]},
   {"nome": "Marco", "classe": "3B", "voti": [6, 6.5, 7, 5]}
]
```

1. Aggiungere ad ogni studente la chiave "media" con la media dei suoi voti e stampare nome, classe e media di ciascuno
2. Stampare i nomi degli studenti con media insufficiente
3. Costruire un dizionario che associ ad ogni classe la lista dei suoi studenti; per ogni classe, in ordine alfabetico, stampare i nomi degli studenti e la media della classe (media delle medie)
4. Per ogni classe stampare lo studente con la media più alta

Soluzione

```
studenti = [
    {"nome": "Alice", "classe": "3A", "voti": [7, 8, 6.5, 9]},
    {"nome": "Bruno", "classe": "3B", "voti": [5, 6, 5.5]},
    {"nome": "Chiara", "classe": "3A", "voti": [8, 8.5, 9, 10]},
    {"nome": "Davide", "classe": "3C", "voti": [6, 7, 6]},
    {"nome": "Elena", "classe": "3B", "voti": [9, 8, 9.5, 8]},
    {"nome": "Fabio", "classe": "3A", "voti": [4, 5.5, 6]},
    {"nome": "Giulia", "classe": "3C", "voti": [7.5, 8, 7]},
    {"nome": "Marco", "classe": "3B", "voti": [6, 6.5, 7, 5]}
]

# 1) media di ogni studente
for s in studenti:
```

```
    s["media"] = sum(s["voti"]) / len(s["voti"])
    print(f"{s['nome']} ({s['classe']}): {s['media']:.2f}")

# 2) studenti insufficienti
print("Insufficienti:", [s["nome"] for s in studenti if s["media"] < 6])

# 3) per ogni classe la lista degli studenti e la media della classe
classi = {}
for s in studenti:
    if s["classe"] in classi:
        classi[s["classe"]].append(s)
    else:
        classi[s["classe"]] = [s]
for classe in sorted(classi):
    nomi = [s["nome"] for s in classi[classe]]
    media = sum([s["media"] for s in classi[classe]]) / len(classi[classe])
    print(f"{classe}: {nomi}, media {media:.2f}")

# 4) per ogni classe lo studente con la media più alta
for classe in sorted(classi):
    migliore = classi[classe][0]
    for s in classi[classe]:
        if s["media"] > migliore["media"]:
            migliore = s
    print(f"{classe}: {migliore['nome']} ({migliore['media']:.2f})")
```

4.7.14 Prenotazioni al cinema

Una sala cinematografica ha 5 file da 8 posti ciascuna. Le prenotazioni sono memorizzate in un dizionario che ha come chiave una tupla (fila, posto) e come valore il nome di chi ha prenotato. Le file e i posti sono numerati a partire da 0.

```
prenotazioni = {
   (0, 3): "Rossi", (0, 4): "Rossi",
   (1, 0): "Bianchi",
   (2, 2): "Verdi", (2, 3): "Verdi", (2, 4): "Verdi",
   (4, 6): "Neri", (4, 7): "Neri"
}
```

1. Stampare la mappa della sala, una fila per riga, usando X per i posti occupati e . per quelli liberi
2. Stampare per ogni fila il numero di posti liberi
3. Chiedere all'utente fila e posto da prenotare: se il posto è libero chiedere il nome e registrare la prenotazione, altrimenti stampare il nome di chi lo ha già prenotato
4. Costruire un dizionario che associ ad ogni nome la lista dei posti prenotati e stamparlo

Soluzione

```
FILE = 5
POSTI = 8

prenotazioni = {
    (0, 3): "Rossi", (0, 4): "Rossi",
    (1, 0): "Bianchi",
```

```
    (2, 2): "Verdi", (2, 3): "Verdi", (2, 4): "Verdi",
    (4, 6): "Neri", (4, 7): "Neri"
}

# 1) mappa della sala
for fila in range(FILE):
    riga = ""
    for posto in range(POSTI):
        if (fila, posto) in prenotazioni:
            riga += "X "
        else:
            riga += ". "
    print(f"Fila {fila}: {riga}")

# 2) posti liberi per fila
for fila in range(FILE):
    liberi = 0
    for posto in range(POSTI):
        if (fila, posto) not in prenotazioni:
            liberi += 1
    print(f"Fila {fila}: {liberi} posti liberi")

# 3) prenotazione di un posto
fila = int(input("Fila: "))
posto = int(input("Posto: "))
if (fila, posto) in prenotazioni:
    print("Posto già occupato da", prenotazioni[(fila, posto)])
else:
    prenotazioni[(fila, posto)] = input("Nome: ")
    print("Prenotazione effettuata")

# 4) posti prenotati da ogni persona
persone = {}
for posizione, nome in prenotazioni.items():
    if nome in persone:
        persone[nome].append(posizione)
    else:
        persone[nome] = [posizione]
for nome, posti in persone.items():
    print(f"{nome}: {sorted(posti)}")
```

4.8 Dizionari con stringhe

4.8.1 Gruppi di anagrammi

Due parole sono anagrammi se sono formate dalle stesse lettere, eventualmente in ordine diverso (ad esempio "roma" e "amor"). Data la seguente lista di parole:

```
parole = ["roma", "amor", "mora", "lago", "gola", "gatto", "ramo", "pera", "rape",
"arpe", "cane", "toga", "sole", "pare"]
```

1. Raggruppare le parole che sono anagrammi tra loro usando un dizionario (suggerimento: due anagrammi, se ne ordini le lettere, diventano la stessa stringa, che puoi usare come chiave)
2. Stampare i gruppi formati da almeno due parole, dal più numeroso al meno numeroso
3. Stampare le parole che non hanno anagrammi nella lista

Soluzione

```
parole = ["roma", "amor", "mora", "lago", "gola", "gatto", "ramo",
          "pera", "rape", "arpe", "cane", "toga", "sole", "pare"]

# 1) raggruppamento per lettere ordinate
gruppi = {}
for parola in parole:
    chiave = "".join(sorted(parola))
    if chiave in gruppi:
        gruppi[chiave].append(parola)
    else:
        gruppi[chiave] = [parola]

# 2) gruppi con almeno due parole, dal più numeroso
anagrammi = [gruppo for gruppo in gruppi.values() if len(gruppo) > 1]
anagrammi.sort(key=lambda g: -len(g))
for gruppo in anagrammi:
    print(gruppo)

# 3) parole senza anagrammi
print("Senza anagrammi:", [g[0] for g in gruppi.values() if len(g) == 1])
```

4.8.2 Girone di Champions da stringhe

Data la seguente lista di stringhe che rappresenta un girone di Champions:

```
champions= [
   "Sheriff Tiraspol - Shaktar Donetsk; 2 0",
   "Inter - Real Madrid; 0 1",
   "Shaktar Donetsk - Inter; 0 0",
   "Real Madrid - Sheriff Tiraspol; 1 2",
   "Shaktar Donetsk - Real Madrid; 0 5",
   "Inter - Sheriff Tiraspol; 3 1",
   "Real Madrid - Shaktar Donetsk; 2 1",
   "Sheriff Tiraspol – Inter; 1 3",
   "Inter - Shaktar Donetsk; 2 0",
   "Sheriff Tiraspol - Real Madrid; 0 3",
   "Shaktar Donetsk - Sheriff Tiraspol; 1 1",
   "Real Madrid – Inter; 2 0"
]
```

Sapendo che la vittoria vale 3 punti, il pareggio 1 punto e la sconfitta 0 punti, calcola e stampa per ogni squadra, quanti punti ha fatto

Variante: togli dalla stringa in input i punti e virgola che separano i nomi delle squadre dai goal fatti.

Soluzione

```
# 3.8.4 - Data la seguente lista di stringhe che rappresenta un girone di Champions:
```

```
champions= [
    "Sheriff Tiraspol - Shaktar Donetsk;  2 0",
    "Inter - Real Madrid; 0 1",
    "Shaktar Donetsk  - Inter; 0 0",
    "Real Madrid - Sheriff Tiraspol; 1 2",
    "Shaktar Donetsk  - Real Madrid;  0 5",
    "Inter - Sheriff Tiraspol;  3 1",
    "Real Madrid - Shaktar Donetsk;  2 1",
    "Sheriff Tiraspol - Inter;  1 3",
    "Inter - Shaktar Donetsk;  2 0",
    "Sheriff Tiraspol - Real Madrid;  0 3",
    "Shaktar Donetsk  -  Sheriff Tiraspol;  1 1",
    "Real Madrid - Inter;   2 0"
]

# Sapendo che la vittoria vale 3 punti, il pareggio 1 punto e la sconfitta 0 punti,
# calcola e stampa per ogni squadra, quanti punti ha fatto
# Variante: togli dalla stringa in input i punti e virgola che
# separano i nomi delle squadre dai goal fatti.

d = {}
for riga in champions:
    riga = riga.strip().split(';')
    squadre = riga[0].strip().split('-')
    sq1 = squadre[0].strip()
    sq2 = squadre[1].strip()
    goals = riga[1].strip().split()
    g1 = int(goals[0].strip())
    g2 = int(goals[1].strip())

    if g1 > g2:
        p1 = 3
        p2 = 0
    elif g2 > g1:
        p2 = 3
        p1 = 0
    else:
        p1 = 1
        p2 = 1

    if sq1 not in d:
        d[sq1] = p1
    else:
        d[sq1] += p1

    if sq2 not in d:
        d[sq2] = p2
    else:
        d[sq2] += p2

classifica = list(d.items())
```

```
classifica.sort(key = lambda x: -x[1])
for el in classifica:
    print(el)
```

4.8.3 Lettere più frequenti per posizione

progettare la funzione f(ftesto,k) che, presi in input il nome (o l'indirizzo) di un file di testo ed un intero k, restituisce una stringa di caratteri lunga k. Il file di testo contiene stringhe di diversa lunghezza (una per riga ed ogni riga termina con '\n'), un possibile testo da inserire nel file di input è riportato di seguito.

I k caratteri della stringa restituita dalla funzione si ottiengono considerando le stringhe lunghe k presenti nel file di testo. L'i-mo carattere della stringa sara' il carattere che compare con maggior frequenza come i-mo carattere delle stringhe lunghe k nel file di testo (in caso di parita' di occorrenze viene scelto il carattere che precede gli altri lessicograficamente).

Nel caso il file di testo non contenga parole lunghe k allora viene restituita la stringa vuota.

Ad Esempio, per il file di testo:

```
porta
rigore
ora
giacca
follia
gara
finale
salute
fiore
lampada
cane
erba
palo
tre
due
fionda
limite
polo
soglia
amo
```

e k=3 la funzione restituisce la stringa 'are' a seguito della presenza delle seguenti 4 stringhe lunghe 3:

```
tre
due
amo
ora
```

Soluzione

```
def f(ftesto,k):

    with open(ftesto) as f:
        parole = f.read().split()

    parole = [parola for parola in parole if len(parola) == k]

    ris = ""
```

```
    for j in range(k):
        freq = {}
        for parola in parole:
            if parola[j] in freq:
                freq[parola[j]] += 1
            else:
                freq[parola[j]] = 1

        lista = list(freq.items())
        lista.sort(key=lambda el: (-el[1], el[0]))
        ris += lista[0][0]

    return ris


    ## alternativa

    # with open(ftesto) as f:
    #     parole = f.read().split()

    # parole = [parola for parola in parole if len(parola) == k]
    # if len(parole) == 0:
    #     return ''
    # # print(parole)
    # lettere_per_posizione = [[parola[i] for parola in parole] for i in range(k)]
    # # print(lettere_per_posizione)
    # frequenze = [{c: 0 for c in posizione} for posizione in lettere_per_posizione]
    # for i in range(k):
    #     for lettera in lettere_per_posizione[i]:
    #         frequenze[i][lettera] += 1
    # # print(frequenze)
    # frequenze = [[(c, posizione[c]) for c in posizione] for posizione in frequenze]
    # # print(frequenze)
    # for posizione in frequenze:
    #     posizione.sort(key=lambda x: (-x[1], x[0]))
    # # print(frequenze)
    # parola = [posizione[0][0] for posizione in frequenze]
    # parola = ''.join(parola)
    # # print(parola)
    # return parola


ftesto, k = 'nomefile.txt', 3
print(f(ftesto,k))
```

5 Algoritmi di ordinamento base

5.1.1 Selection sort

Carica un array di numeri casuali e utilizza il selection sort per ordinarlo. Deve essere fatta una stampa dell'array prima e dopo l'ordinamento

5.1.2 Bubble sort

Carica un array di numeri casuali e utilizza il bubble sort per ordinarlo. Deve essere fatta una stampa dell'array prima e dopo l'ordinamento

5.1.3 Ordinamento con sort

Carica un array di numeri casuali e utilizza la funzione sort della libreria standard per ordinarlo. Deve essere fatta una stampa dell'array prima e dopo l'ordinamento

5.1.4 Array palindromo e inversione

Scrivi un programma che dato un array di numeri interi, sia in grado di:

1. controllare se l'array è palindromo,
2. invertire l'ordine dei numeri dell'array,
3. ordinare in ordine decrescente l'array.

Soluzione

```
numeri = [10,13,6,13,7]


# controllare se l'array è palindromo
if numeri == numeri[::-1]:
    print(f"{numeri} è palindromo")
else:
    print(f"{numeri} non è palindromo")


# invertire l'ordine dei numeri dell'array,
numeri = numeri[::-1]
print(numeri)


# ordinare in ordine decrescente l'array.
numeri.sort(reverse=True)
print(numeri)
```

5.1.5 Selection sort nei due versi

Scrivi un programma che ordini un array con l'algoritmo selection sort, prima in ordine crescente poi in ordine decrescente.

5.1.6 Bubble sort nei due versi

Scrivi un programma che ordini un array con l'algoritmo bubble sort, prima in ordine crescente poi in ordine decrescente.

5.1.7 Ordinamento di un intervallo

Scrivi un programma che dato un array di dimensione n e due valori x y tali che x<=y<=n, ordini l'array dalla posizione x alla posizione y comprese.

5.1.8 Ordinamento di stringhe

Dato un array di stringhe, che puoi riempire come preferisci, ordina l'array in ordine alfabetico prima crescente e poi decrescente. Devono essere inserite le stampe dell'intero array prima e dopo ogni ordinamento.

Soluzione

```
elenco = ["ciao belli", "come state?", "Io bene"]
print(elenco)


elenco.sort()
print(elenco)


elenco.sort(reverse=True)
print(elenco)
```

5.1.9 Agenda ordinata

Crea un programma di gestione di una agenda. L'agenda deve permettere di visualizzare e inserire i dati riguardanti i propri contatti. Per ogni persona devono essere memorizzati nome cognome e numero di telefono. La visualizzazione dei dati deve avvenire in ordine alfabetico per nome o cognome (scegli tu). (Non avendo ancora fatto la gestione dei files ogni volta l'agenda partirà da uno stato predefinito che puoi decidere tu).

5.1.10 Bubble sort su numeri e capitali

Con la tecnica del Bubble Sort (2 cicli FOR e una IF):

1. ordinare la seguente lista di numeri in ordine crescente
   lista1 = [8, 5, 7, 12, 18, 17, 3, 5]
2. ordinare la seguente lista di capitali in ordine alfabetico
   lista2 = ["Tallinn", "Vilnius", "Riga", "Copenaghen", "Stoccolma", "Helsinki"]
3. ordinare la lista delle capitali in base alla lunghezza del nome della Capitale (e a parità di lunghezza in base al nome della capitale)

5.1.11 Immunologi celebri

La seguente lista di stringhe contiene una serie di stringhe che descrivono alcuni celebri immunologi:

```
imLista = [
"Pasteur 1822 1895 Francia", "Koch 1843 1910 Germania", "Fleming 1881 1955 RegnoUnito",
"Dulbecco 1914 2012 Italia", "Montagnier 1932 2022 Francia", "Gallo 1937 1982 Usa",
"Gram 1853 1938  Danimarca", "Jenner 1749 1823 RegnoUnito", "Sabin 1906 1993 Usa",
]
```

1. stampare gli immunologi ordinati in base a quanti anni sono vissuti
2. stampare gli immunologi ordinati in base alla nazione, e a parità di nazione in base al nome

Soluzione

```
# La seguente lista di stringhe contiene una serie di stringhe
# che descrivono alcuni celebri immunologi:

# 1) stampare gli immunologi ordinati in base a quanti anni sono vissuti
# 2) stampare gli immunologi ordinati in base alla nazione, e a parità di nazione in base al nome

imLista = [
    "Pasteur 1822 1895 Francia", "Koch 1843 1910 Germania", "Fleming 1881 1955 RegnoUnito",
    "Dulbecco 1914 2012 Italia", "Montagnier 1932 2022 Francia", "Gallo 1937 1982 Usa",
    "Gram 1853 1938 Danimarca", "Jenner 1749 1823 RegnoUnito", "Sabin 1906 1993 Usa",
]

lista = [el.split(" ") for el in imLista]
lista = [[el[0], int(el[2])-int(el[1]), el[3]] for el in lista]
# ordino in base all'età
lista.sort(key=lambda el: el[1])
print(lista)
#ordino in base a nazione e poi nome
lista.sort(key=lambda el: (el[2], el[0]))
print(lista)
```

5.1.12 Autostrade da stringhe

Data la seguente lista:

```
highways = [
"A1 Milano-Napoli 759",
"A2 Salerno-ReggioCalabria 442",
"A3 Napoli-Salerno 52",
"A4 Torino-Trieste 524",
"A6 Torino-Savona 124",
"A7 Milano-Genova 133",
"A8 Milano-Varese 43",
"A10 Savona-Ventimigla 113",
"A11 Firenze-Pisa 81",
```

```
"A12 Genova-Cecina 210",
"A13 Bologna-Padova 116",
"A14 Bologna-Taranto 743",
"A15 Parma-LaSpezia 108",
"A16 Napoli-Canosa 172",
"A19 Palermo-Catania 191",
"A21 Torino-Piacenza-Brescia 238",
"A22 Brennero-Modena 315",
"A23 Palmanova-Tarvisio 119",
"A24 Roma-Teramo 159",
"A25 Torano-Pescara 115",
"A26 Genova-Gravellona 197"
]
```

1. Stampare l'autostrada più corta e quella più lunga (Risultato A1 Milano-Napoli 759 A8 Milano-Varese 43)
2. stampare la classifica delle autostrade ordinate per lunghezza
3. stampare le prime tre città che sono capolinea di una autostrada

5.1.13 Materie ordinate per voto

La seguente lista, contiene alternati i nomi di una materia e il voto finale nella materia

```
lista = ["matematica", 5.5, "storia", 8, "italiano", 7, "inglese", 6.5, "fisica", 6]
```

1. stampare la materia che ha preso il voto più alto utilizzando la tecnica del bubble sort
2. stampare le materie ordinate in base al voto
3. stampare le materie ordinate in base al voto

6 Files

6.1 Spiegazioni ed esempi

Esempio di semplici letture con le diverse funzioni:

```
print("Lettura 1:")
with open("prove.txt", "r", encoding='utf-8') as f:
    for riga in f:
        print(riga, end = '')


print("\n\nlettura 2:")
with open("prove.txt", "r", encoding='utf-8') as f:
    righe = f.readlines()
print(righe)
```

```
print("\n\nlettura 3:")
with open("prove.txt", "r", encoding='utf-8') as f:
    riga = f.readline()
    while riga != '':
        print(riga, end='')
        riga = f.readline()

print("\n\nlettura 4:")
with open("prove.txt", "r", encoding='utf-8') as f:
    contenuto = f.read()
print(contenuto)
```

Esempio di lettura di un file e parsing delle singole righe:

```
with open("prove.txt", "r", encoding='utf-8') as f:
    primariga = f.readline()  # leggo e scarto la prima riga
    lista = []
    for riga in f:
        dati = riga.strip().split(',')
        lista.append([dati[0].strip(), dati[1].strip(), int(dati[2].strip())])

prima = primariga.strip().split()
print(f"{prima[0]:15s} {prima[1]:20s} {prima[2]}")
for el in lista:
    print(f"{el[0]:15s} {el[1]:20s} {str(el[2]):>2s}")
```

6.2 Esercizi

6.2.1 Scrittura e lettura di nomi

Scrivi un programma che scriva su un file un elenco di nomi, chiuda il file, lo riapra in lettura lo legga tutto e stampi ciò che ha letto.

6.2.2 Conteggio caratteri di un file

Scrivi e usa una funzione che conta il numero di caratteri contenuti in un file

6.2.3 Righe e caratteri per riga

Scrivi una funzione che conta e restituisce il numero di righe di un file.
Scrivi poi un'altra funzione che calcola per ogni riga il numero di caratteri contenuti nella riga. I valori vanno inseriti in una lista o una tupla da restituire al main.

Variante: invece dei caratteri conta il numero di parole per riga

Soluzione

```
# 7.1.3 Scrivi una funzione che conta e restituisce il numero
# di righe di un file. Scrivi poi un'altra funzione che calcola
# per ogni riga il numero di caratteri contenuti nella riga.
# I valori vanno inseriti in una lista o una tupla da restituire
# al main.
# Variante: invece dei caratteri conta il numero di
# parole per riga
```

```python
def contaRighe(nomefile):
    with open(nomefile, 'r', encoding='utf-8') as f:
        ris = 0
        for riga in f:
            # if len(riga) > 1 or (len(riga) == 1 and riga[-1] != '\n'):
            if riga != '\n':
                ris += 1
    return ris

def contaRighe2(nomefile):
    with open(nomefile, 'r', encoding='utf-8') as f:
        testo = f.read()
        testo = testo.split('\n')
        testo = [el for el in testo if el != '']
        return len(testo)

def caratteriPerRiga(nomefile):
    with open(nomefile, 'r', encoding='utf-8') as f:
        return [len(riga) for riga in f if riga != '\n']

def parolePerRiga(nomefile):
    with open(nomefile, 'r', encoding='utf-8') as f:
        return [len(riga.split()) for riga in f if riga != '\n']

def parolePerRigaFacile(nomefile):
    with open(nomefile, 'r', encoding='utf-8') as f:
        ris = []
        for riga in f:
            if riga != '\n':
                riga = riga.split()
                ris.append(len(riga))
        return ris



print("numero di righe:", contaRighe("7-1-3.txt"))
print("numero di righe:", contaRighe2("7-1-3.txt"))
print("numero di caratteri per riga:", caratteriPerRiga("7-1-3.txt"))
print("numero di caratteri per riga:", parolePerRiga("7-1-3.txt"))
print("numero di caratteri per riga:", parolePerRigaFacile("7-1-3.txt"))
```

6.2.4 Parole ordinate su file

Scrivi e usa una funzione che legga un file e scriva in un altro file tutte le parole del primo file in ordine alfabetico

Soluzione

```python
def f(nomefile, nomefileout):
    with open(nomefile, 'r', encoding='utf-8') as f:
        testo = f.read()
        parole = testo.split()
```

```
        # per mettere tutto minuscolo
        for i in range(len(parole)):
            parole[i] = parole[i].lower()

        parole.sort()
        with open(nomefileout, 'w', encoding='utf-8') as f:
            for parola in parole:
                f.write(parola)
                f.write("\n")

f("8-1-4.txt", "8-1-4-out.txt")
```

6.2.5 Copia in minuscolo

Scrivi un programma che legga un file di testo e che scriva in un secondo file, lo stesso testo modificato in modo che tutte le lettere siano minuscole. Il file di testo iniziale generalo tu come vuoi.

6.2.6 Aggiunta in minuscolo

Scrivi un programma che legga un file di testo e che aggiunga alla fine di tale file, dopo un paio di righe vuote, lo stesso testo modificato in modo che tutte le lettere siano minuscole. Il file di testo iniziale generalo tu come vuoi. Attento a fare in modo che se il programma viene eseguito più volte, le righe da considerare sono solo quelle iniziali.

6.2.7 Cancella righe dopo virgola

Scrivi una funzione parametrica in grado di modificare un file di testo di nome "miofile.txt" letto da disco in modo tale che, se l'ultimo elemento della linea è una virgola, la linea successiva venga eliminata

Soluzione

```
# 7.1.7 - Scrivi una funzione parametrica in grado di modificare
# un file di testo di nome "miofile.txt" letto da disco in modo tale
# che, se l'ultimo elemento della linea è una virgola, la linea successiva venga
# eliminata

def modificaFile(nomefile):
    with open(nomefile, 'r', encoding='utf-8') as fin:
        righe = fin.readlines()

    righe = [riga.strip() for riga in righe]
    print(righe)
    print()


    nuove = [righe[0]]
    for i in range(1, len(righe)):
        # print(righe[i])
        if righe[i-1][-1:] != ',':    # [-1:] perchè la riga prima può essere vuota
            nuove.append(righe[i])
        # print(nuove)
```

```
    for riga in nuove:
        print(riga)

    with open(nomefile[:-4]+'-modificato.txt', 'w', encoding='utf-8') as fout:
        fout.write('\n'.join(nuove))


modificaFile("7-1-7.txt")
```

Remake

```
righe = []
with open("7_2_7.txt", "r", encoding='utf-8') as f:
    righe = f.readlines()

for i in range(len(righe)):
    righe[i] = righe[i].strip()

righeSelezionate = []
for i, riga in enumerate(righe):
    if i == 0 or (len(righe[i-1]) != 0 and righe[i-1][-1] != ','):
        righeSelezionate.append(riga)

for riga in righeSelezionate:
    print(riga)
```

6.2.8 Ricerca di un nome nel file

Scrivi un programma che scriva su un file un elenco di nomi (o numeri se preferisci), chieda poi all'utente un nome da cercare nel file e stampi sullo schermo l'esito della ricerca (trovato o no). Le operazioni di scrittura su file devono essere eseguite da una funzione che riceve come parametro la stringa contenente il nome del file. Le operazioni di ricerca devono essere svolte da una funzione che riceve come parametro la stringa contenente il nome del file e la stringa da cercare.

Continua l'esercizio aggiungendo una funzione che dato il nome del file e il numero di riga, cerca, se esiste, la riga e la restituisce.

Aggiungi anche la possibilità di chiedere all'utente se vuole aggiungere dei nomi a quelli già presenti (aprendo poi il file in modalità append e aggiungendo i nomi dati dall'utente)

6.2.9 Virologi italiani da file

Il seguente testo (che devi inserire in un file di testo) descrive i più celebri virologi Italiani nell'epoca covid

Formato: Nome e Cognome - Ospedale - città - (O) Ottimista (P) Pessimista

```
Roberto Burioni - San Raffaele - Milano (P)
Matteo Bassetti - San Martino - Genova (O)
Alberto Zangrillo - San Raffaele - Milano (O)
Massimo Galli - Luigi Sacco - Milano (P)
MariaRita Gismondo - Luigi Sacco - Milano (O)
Andrea Crisanti - Università di Padova - Padova (P)
Ilaria Capua - One Health - Florida (P)
Antonella Viola - Città della Speranza - Padova (P)
Fabrizio Pregliasco - Galeazzi - Milano (O)
```

```
Walter Ricciardi - Policlinico Gemelli - Roma (P)
Franco Locatelli - Bambin Gesù - Roma (P)
```

stampare nome e cognome dei virologi ordinati per cognome (si consiglia si usare la split() con il separatore "-";

stampare il numero di virologi ottimisti e di quelli pessimisti.

Soluzione con le liste

```
virologi = []
with open("7_2_9.txt", "r", encoding='utf-8') as f:
    f.readline()  # leggo e scarto la prima riga
    for riga in f:
        dati = riga.strip().split('-')
        nome_cognome = dati[0].strip()
        nome, cognome = nome_cognome.split()
        ospedale = dati[1].strip()
        ultimo_pezzo = dati[2].strip()
        # Franco Locatelli - Bambin Gesù - Cologno Monzese (P)
        pezzi = ultimo_pezzo.split()
        citta = pezzi[:-1]
        citta = " ".join(citta)
        attitudine = pezzi[-1][1:-1]
        virologi.append([nome, cognome, ospedale, citta, attitudine])
        # print(virologi[-1])

virologi.sort(key = lambda x: x[1])
print("virologi ordinati per cognome:")
for v in virologi:
    print(v)

ottimisti = 0
pessimisti = 0
for v in virologi:
    if v[-1] == 'O':
        ottimisti += 1
    else:
        pessimisti += 1

print("\n\n\nNumero di virologi ottimisti: ", ottimisti)
print("Numero di virologi pessimisti: ", pessimisti)
```

Soluzione col dizionario e scrittura su file

```
with open("7_2_9.txt", 'r', encoding='utf-8') as f:
    lista = []
    f.readline()
    for riga in f:
        riga = riga.split('-')
        riga = [dato.strip() for dato in riga]
        nome, cognome = riga[0].split()
        ospedale = riga[1]
        citta = riga[2][:-4]
        orientamento = riga[2][-2]
        lista.append({'nome': nome, 'cognome': cognome, 'ospedale': ospedale,
                      'citta': citta, 'orientamento': orientamento})

# 1. stampare nome e cognome dei virologi ordinati per
# cognome (si consiglia si usare la split() con il separatore "-";
lista.sort(key = lambda x: x['cognome'])
for riga in lista:
    print(riga)

# 2. stampare il numero di virologi ottimisti e di quelli
# pessimisti.
```

```
nottimisti = 0
npessimisti = 0
for riga in lista:
    if riga['orientamento'] == 'O':
        nottimisti += 1
    if riga['orientamento'] == 'P':
        npessimisti += 1
print(f"Ottimisti: {nottimisti}")
print(f"Pessimisti: {npessimisti}")

# voglio creare due nuovi files uno per gli ottimisti uno per i pessimisti

with open("7_2_9_ottimisti.txt", 'w', encoding='utf-8') as f:
    f.write(f"File degli ottimisti:\n")
    for el in lista:
        if el['orientamento'] == 'O':
            f.write(f"{el['nome']} - {el['cognome']} - {el['ospedale']} - {el['citta']}
({el['orientamento']})\n")

with open("7_2_9_pessimisti.txt", 'w', encoding='utf-8') as f:
    f.write(f"File degli pessimisti:\n")
    for el in lista:
        if el['orientamento'] == 'P':
            f.write(f"{el['nome']} - {el['cognome']} - {el['ospedale']} - {el['citta']}
({el['orientamento']})\n")

# with open("7_2_9_ottimisti.txt", 'w', encoding='utf-8') as fo:
#     with open("7_2_9_pessimisti.txt", 'w', encoding='utf-8') as fp:
#         fp.write(f"File degli pessimisti:\n")
#         fo.write(f"File degli ottimisti:\n")
#         for el in lista:
#             if el['orientamento'] == 'P':
#                 fp.write(f"{el['nome']} - {el['cognome']} - {el['ospedale']} - {el['citta']}
({el['orientamento']})\n")
#             else:
#                 fo.write(f"{el['nome']} - {el['cognome']} - {el['ospedale']} - {el['citta']}
({el['orientamento']})\n")
```

6.2.10 Esploratori famosi da file

Un file di testo contiene i seguenti dati:

```
Marco Polo 1254 1324
Cristoforo Colombo 1451 1506
Amerigo Vespucci 1454 1512
Francisco Pizarro 1475 1541
Ferdinando Magellano 1480 1521
Hernan Cortez 1485 1547
Walter Raleigh 1552 1618
Henry Hudson 1570 1611
James Cook 1728 1779
Charles Darwin 1809 1882
Kit Carson 1809 1866
David Livingstone 1813 1873
Charles Foucauld 1858 1916
Ronald Amundsen 1872 1928
Ernest Shakleton 1874 1922
```

Scrivi un programma che legga i dati contenuti nel file, li memorizzi opportunamente, e poi scrivi e utilizza le seguenti funzioni:

1. una funzione che stampa tutti i dati.

2. una funzione che ordina i dati in ordine alfabetico secondo il nome degli esploratori
3. una funzione che ordina i dati in ordine crescente secondo le date di nascita degli esploratori
4. una funzione che calcola la durata della vita di ogni esploratore e aggiunge questo dato agli altri
5. una funzione che ordina gli esploratori in base alla durata della loro vita in ordine decrescente e in caso di parità in ordine crescente secondo la data di nascita
6. una funzione che riscrive tutti i dati in un secondo file

Soluzione

```
import os
os.system("cls")

# 1.    una funzione che stampa tutti i dati.
def f1(dati):
    for esp in dati:
        print(esp)

# 2.    una funzione che ordina i dati in ordine alfabetico secondo il nome degli esploratori
def f2(dati):
    dati.sort()

# 3.    una funzione che ordina i dati in ordine crescente secondo le date di nascita degli
esploratori
def f3(dati):
    dati.sort(key = lambda x: (x[2]))

# 4.    una funzione che calcola la durata della vita di ogni esploratore e agiunge questo dato
agli altri
def f4(dati):
    for esp in dati:
        esp.append(esp[3]-esp[2])

# 5.    una funzione che ordina gli esploratori in base alla durata della loro vita in ordine
decrescente e in caso di
#       parità in ordine crescente secondo la data di nascita
def f5(dati):
    dati.sort(key = lambda x: (-x[4], x[2]))

# 6.    una funzione che riscrive tutti i dati in un secondo file
def f6(dati, nomefile):
    with open(nomefile,'w',encoding = 'utf-8') as f:
        for esp in dati:
            for dato in esp:
                f.write(f"{dato} ")
            f.write("\n")

dati = []
with open("8-2-1.txt",'r',encoding = 'utf-8') as f:
    for riga in f:
```

```
        riga = riga.strip()
        riga = riga.split()
        riga[2] = int(riga[2])
        riga[3] = int(riga[3])
        dati.append(riga)

print("Dati letti dal file:")
f1(dati)

f2(dati)
print("Esploratori in ordine alfabetico:")
f1(dati)

f4(dati)
f5(dati)

print("\nClassifica per durata di vita:")
for el in dati:
    print(el)

# scrivo nel secondo file
f6(dati, "8-2-1-out.txt")
```

6.2.11 Voti ordinati su file

Scrivi un programma che legga i dati contenuti in un file di testo (testo riportato di seguito) e li inserisca in opportune strutture. Di questi dati il programma deve fare le seguenti cose:

1. stamparli sullo schermo
2. ordinarli in ordine decrescente di voto
3. stamparli nuovamente in ordine
4. scriverli in ordine in un secondo file che puoi nominare come vuoi

Dati da mettere nel file di testo:

```
Amici 7.00
Biella 9.00
Brescia 6.00
Carolla 8.00
Cunegatti 7.00
DeBella 4.00
DeVecchi 9.00
Fumagalli 7.00
Galimberti 9.00
Germanò 9.00
Gubellini 9.00
Lepore 5.00
Maconi 6.00
Mariani 8.00
Mattavelli 9.00
Oggiano 4.00
Passoni 5.00
Pastori 4.00
Pirovano 7.00
Rudi 10.00
Russell 6.00
Sogos 4.00
```

```
Tezza 6.00
Varisco 8.00
```

Soluzione

```python
lista = []
with open("7_2_11.txt", 'r', encoding='utf-8') as f:
    for riga in f:
        riga = riga.strip()
        riga = riga.split()
        riga[1] = float(riga[1])
        lista.append(riga)

# 1.    stamparli sullo schermo
print("Stampa dei dati iniziali: ")
for riga in lista:
    print(riga[0], riga[1])
print()



# 2.    ordinarli in ordine decrescente di voto
lista.sort(key = lambda x: (-x[1], x[0]))

# 3.    stamparli nuovamente in ordine
print("Stampa ordinata per voto: ")
for riga in lista:
    print(riga[0], riga[1])
print()

# 4.    scriverli in ordine in un secondo file che
# puoi nominare come vuoi

with open("7_2_11_output.txt", 'w', encoding='utf-8') as f:
    for riga in lista:
        f.write(f"{riga[0]} {riga[1]}\n")
```

6.2.12 Registro voti su file

Scrivi un programma che legga i dati contenuti in un file di testo (testo riportato di seguito) e li inserisca in opportune strutture dati che rappresentino il registro dei voti per una materia. Il programma deve poi calcolare la media dei voti per ogni singolo studente e aggiungerla alla struttura dati. Il programma deve mostrare tutti i dati raccolti e calcolati sullo schermo e salvarli in un secondo file di testo.

```
Amici 7.00 8.00 7.00
Biella 8.00 9.50 7.00
Brescia 5.00 7.50 9.00
Carolla 7.00 8.50 8.50
Cunegatti 7.00 7.00 5.50
DeBella 4.00 4.00 4.50
DeVecchi 8.00 10.00 6.00
Fumagalli 8.50 6.00 4.50
Galimberti 9.00 9.00 9.00
Germanò 8.50 9.00 7.50
Gubellini 8.00 10.00 7.00
Lepore 5.50 4.00 3.00
Maconi 7.50 5.00 7.50
Mariani 8.00 8.00 7.00
Mattavelli 9.00 9.50 8.50
Oggiano 5.00 3.00 3.00
Passoni 5.00 6.00 6.00
```

```
Pastori 3.50 5.00 7.00
Pirovano 6.50 7.50 7.50
Rudi 9.50 10.00 10.00
Russell 7.50 4.50 5.50
Sogos 4.00 3.50 3.50
Tezza 7.00 5.50 5.50
Varisco 8.00 9.00 8.50
```

Soluzione

```python
lista = []
with open("8-2-3.txt", 'r', encoding='utf-8') as f:
    for riga in f:
        riga = riga.strip()
        riga = riga.split()
        studente = riga[0]
        voti = list(map(float, riga[1:]))
        media = sum(voti)/len(voti)
        lista.append([studente, voti, media])

# Stampa
print("Stampa")
for riga in lista:
    print(riga)
print()

# scrittura su file
with open("8-2-3-out.txt", 'w', encoding='utf-8') as f:
    for riga in lista:
        f.write(f"{riga[0]}: ")
        for voto in riga[1]:
            f.write(f"{voto} ")
        f.write(f"- media: {riga[2]:.2f}\n")
```

6.2.13 Registro voti su file 2

Scrivi un programma che legga i dati contenuti in un file di testo (testo riportato di seguito) e li inserisca in opportune strutture dati che rappresentino il registro dei voti per una materia. Il programma deve poi calcolare la media dei voti per ogni singolo studente e aggiungerla alla struttura dati (che quindi conterrà anche una variabile media). Il programma deve mostrare tutti i dati raccolti e calcolati sullo schermo e salvarli in un secondo file di testo. Questa versione del programma è più difficile della precedente perché si può notare che i cognomi possono essere formati da più parole e sono separati dai voti da un ":", inoltre il numero di voti varia per ogni studente.

```
Amici: 7.00 8.00 7.00
Biella: 8.00 9.50 7.00
Brescia: 5.00 7.50 9.00
Carolla: 7.00 8.50 8.50
Cunegatti: 7.00 7.00
De Bella: 4.00 4.00 4.50 5.00
De Vecchi: 8.00 10.00 6.00
Fumagalli: 8.50 6.00 4.50
Galimberti: 9.00 9.00 9.00
Germanò: 8.50 9.00 7.50
```

```
Gubellini: 8.00 10.00 7.00
Lepore: 5.50 4.00 3.00 5.00
Maconi: 7.50 5.00
Mariani: 8.00 8.00 7.00
Mattavelli: 9.00 9.50 8.50
Oggiano: 5.00 3.00 3.00 2.00
Passoni: 5.00 6.00 6.00 7.50
Pastori: 3.50 5.00 7.00 2.00
Pirovano: 6.50 7.50 7.50
Rudi: 9.50 10.00 10.00
Russell: 7.50 4.50
Sogos: 4.00 3.50 3.50 2.00
Tezza: 7.00 5.50 5.50
Varisco: 8.00 9.00 8.50
```

Soluzione con sole liste

```python
dati = []
with open("7_2_13.txt", "r", encoding='utf-8') as f:
    for riga in f:
        riga = riga.strip()
        riga = riga.split(":")
        # print(riga)
        cognome = riga[0]
        # print(cognome)
        voti = riga[1].strip().split()
        # print(voti)
        voti = [float(voto) for voto in voti]
        # print(voti)
        media = sum(voti)/len(voti)
        media = round(media, 2)
        dati.append([cognome, voti, media])
        print(dati[-1])

with open("7_2_13_output.txt", "w", encoding='utf-8') as f:
    f.write(f"{'Cognomi':15} {'Voti':20} {'Medie'}\n")
    for riga in dati:
        f.write(f"{riga[0]:15} {' '.join(str(voto) for voto in riga[1]):20} {riga[2]}\n")
```

Soluzione con i dizionari

Soluzione con le classi

```python
from statistics import mean


class Studente:
    def __init__(self, cognome = "", voti = []):
        self.cognome = cognome
        self.voti = voti
        self.media = mean(voti) # se no te la calcoli tu con il solito ciclo o con sum(voti) /
len(voti) per cui non serve importare niente


    def __repr__(self) -> str:
        ris = f"{self.cognome}: "
        for voto in self.voti:
            ris += f"{voto} "
        ris += f"- Media: {self.media:.2f}"
```

```
        return ris


with open("6-13-input.txt", encoding = 'utf-8') as fi:
    studenti = []
    for riga in fi:
        dati = list(map(str.strip, riga.split(":")))
        cognome = dati[0]
        voti = list(map(str.strip, dati[1].split()))
        voti = list(map(float, voti))
        studenti.append(Studente(cognome, voti))
        # print(studenti[-1])

    for studente in studenti:
        print(studente)

    with open("6-13-output.txt", "w", encoding = 'utf-8') as fo:
        # due alternative

        # 1
        # for studente in studenti:
            # fo.write(str(studente))
            # fo.write("\n")

        # 2 (qui map non lo userei perchè va aggiunto il \n)
        fo.writelines(str(s) + '\n' for s in studenti)
```

6.2.14 Libri da file

Leggi le informazioni riguardanti una serie di libri da un file (il testo è riportato di seguito) e:

1. carica i dati in una lista di libri (usa i dizionari per costruire una sorta di struttura con le chiavi: titolo, autore, anno)
2. ordina i libri per data di scrittura e riscrivili in un altro file nello stesso formato del file originale
3. scrivi e usa una funzione che riceve i dati già caricati e che restituisce il libro più recente
4. scrivi e usa una funzione che restituisca il secolo in cui sono stati scritti più libri e il numero di libri scritti in quel secolo

```
Divina Commedia;Dante Alighieri;1321
Promessi Sposi;Alessandro Manzoni;1840
Il Decamerone;Alessandro Boccaccio;1350
L'Orlando Furioso;Ludovico Ariosto;1516
Il fu Mattia Pascal;Luigi Pirandello;1904
Ultime lettere di Jacopo Ortis;Ugo Foscolo;1802
Se questo è un uomo;Primo Levi;1947
Il barone rampante;Italo Calvino;1957
I Malavoglia;Giovanni Verga;1881
Il Principe; Niccolò Macchiavelli;1532
La Gerusalemme Liberata;Torquato Tasso;1581
Cuore;Edmondo De Amicis;1886
```

```
Il deserto dei Tartari;Dino Buzzati;1940
La coscienza di Zeno;Italo Svevo;1923
Pinocchio;Carlo Collodi;1883
Il piacere;Gabriele D'Annunzio;1889
Myricae;Giovanni Pascoli;1891
Canti;Giacomo Leopardi;1831
Uno, nessuno e centomila;Luigi Pirandello;1926
Il nome della rosa;Umberto Eco;1980
La casa in collina;Cesare Pavese;1948
Il gattopardo;Giuseppe Tomasi di Lampedusa;1958
I Vicerè;Federico De Roberto;1894
Mastro don Gesualdo;Giovanni Verga;1889
```

Soluzione

```python
class libro:
    def __init__(self, titolo = "", autore = "", anno = 0):
        self.titolo = titolo
        self.autore = autore
        self.anno = anno

    def __repr__(self) -> str:
        return f"{self.titolo};{self.autore};{self.anno}\n"


def piu_recente(libri):
    imin = 0
    min = libri[0].anno
    for i, libro in enumerate(libri):
        if libro.anno < min:
            imin = i
            min = libro.anno
    return libri[i]


def secolo_con_piu_libri(libri):
    secoli = {}
    for libro in libri:
        secolo = (libro.anno // 100) * 100
        if secolo in secoli:
            secoli[secolo] += 1
        else:
            secoli[secolo] = 1

    # print (secoli)

    smax = None
    for secolo, n in secoli.items():
        if smax == None or n > nmax:
            smax = secolo
            nmax = n
    return (smax, nmax)
```

```python
with open("6-14-input.txt", encoding = 'utf-8') as fi:
    libri = []
    for riga in fi:
        # dati = [dato.strip() for dato in riga.split(";")]
        dati = list(map(str.strip, riga.split(";")))
        # print(dati)
        libri.append(libro(dati[0], dati[1], int(dati[2])))
        # print(libri[-1])

    libri.sort(key=lambda el: el.anno)
    print(libri)

    with open("6-14-output.txt", "w", encoding = 'utf-8') as fo:
        # ci sono 3 alternative

        # 1
        # for libro in libri:
        #     fo.write(str(libro))
        #     fo.write("\n")

        # 2
        # fo.writelines(str(libro) for libro in libri)

        # 3
        fo.writelines(map(str, libri))

    print(f"\n\nIl libro più recente è {str(piu_recente(libri))}")

    x = secolo_con_piu_libri(libri)
    print(f"\n\nIl secolo con più libri: {x[0]} con {x[1]} libri")
    # secolo_con_piu_libri(libri)
```

6.2.15 Triangoli da file

Scrivi un programma che sia in grado di leggere da file i dati riportati di seguito e salvarli in opportune strutture dati (punto e triangolo). Il programma deve poi stampare sia su schermo che in un altro file i dati riguardanti ogni triangolo e se esso è scaleno, isoscele o equilatero. Per decidere il tipo di triangolo devono essere usate tre diverse funzioni che controllano ognuna se il triangolo passato come parametro è di uno dei tre tipi richiesti.

```
Input
(1, 0) (3, 3) (-0.6, 3.23)
(1, 0) (1, 4) (-2.46, 2)
(2, 0) (5, 3) (2, 6)
(3, 0) (4.4, 3.3) (2.35, 4.8)
(2, 2) (4, 2) (4, 6)

Output
(1.000000, 0.000000) (3.000000, 3.000000) (-0.600000, 3.230000) - equilatero
(1.000000, 0.000000) (1.000000, 4.000000) (-2.460000, 2.000000) - equilatero
```

```
(2.000000, 0.000000) (5.000000, 3.000000) (2.000000, 6.000000) - isoscele
(3.000000, 0.000000) (4.400000, 3.300000) (2.350000, 4.800000) - scaleno
(2.000000, 2.000000) (4.000000, 2.000000) (4.000000, 6.000000) - scaleno
```

Soluzione

```python
from math import sqrt
import re
import os

os.system("cls")

triangoli = []
with open("8-2-6.txt", 'r', encoding='utf-8') as f:
    for riga in f:
        riga = riga.strip()


        # modalità 1
        # con lo split classico
        riga = riga.split(") (")
        triangolo = []
        for p in riga:
            if p[0] == '(':
                p = p[1:]
            if p[-1] == ')':
                p = p[:-1]
            p = p.split(", ")
            p = tuple(map(float, p))
            triangolo.append(tuple(p))
        triangolo = tuple(triangolo)
        # fine modalità 1


        # # modalità 2
        # # alternativa con lo split che usa le espressioni regolari
        # riga = re.split(r"\(|, |\) \(|\)", riga)
        # riga = list(filter(None, riga))
        # riga = list(map(float, riga))
        # # print(riga)
        # triangolo = []
        # for i in range(0, len(riga), 2):
        #     triangolo.append((riga[i], riga[i+1]))
        # triangolo = tuple(triangolo)
        # # fine alternativa

        triangoli.append(triangolo)


def distanza(p1, p2):
    return sqrt((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)


def circa(x, y):
    if x-y < 0.01 and x-y > -0.01:
```

```
        return True
    else:
        return False


def equilatero(triangolo):
    A = triangolo[0]
    B = triangolo[1]
    C = triangolo[2]
    a = distanza(C, B)
    b = distanza(A, C)
    c = distanza(A, B)
    if circa(a, b) and circa(b, c) and circa(c, a):
        return True
    else:
        return False


def isoscele(triangolo):
    A = triangolo[0]
    B = triangolo[1]
    C = triangolo[2]
    a = distanza(C,B)
    b = distanza(A,C)
    c = distanza(A,B)
    if circa(a, b) or circa(b, c) or circa(c, a):
        return True
    else:
        return False


def scaleno(triangolo):
    A = triangolo[0]
    B = triangolo[1]
    C = triangolo[2]
    a = distanza(C,B)
    b = distanza(A,C)
    c = distanza(A,B)
    if not circa(a, b) and not circa(b, c) and not circa(c, a):
        return True
    else:
        return False

with open("8-2-6-out.txt", 'w', encoding='utf-8') as f:
    for triangolo in triangoli:
        print(triangolo, end = " - ")
        f.write(f"{triangolo} - ")
        if equilatero(triangolo):
            print("equilatero")
            f.write("equilatero\n")
        elif isoscele(triangolo):
            print("isoscele")
            f.write("isoscele\n")
```

```
        elif scaleno(triangolo):
            print("scaleno")
            f.write("scaleno\n")
```

6.2.16 Champions League

Leggi da un file (il testo è riportato di seguito) i dati che descrivono i risultati delle partite di un girone di Champions:

Sapendo che la vittoria vale 3 punti, il pareggio 1 punto e la sconfitta 0 punti, calcola e stampa la classifica del girone comprendente il nome della squadra e i punti fatti.

Contenuto del file:

```
Squadra in casa - Squadra fuori casa goal casa goalfuori casa
Sheriff Tiraspol - Shaktar Donetsk 2 0
Inter - Real Madrid 0 1
Shaktar Donetsk - Inter 0 0
Real Madrid - Sheriff Tiraspol 1 2
Shaktar Donetsk - Real Madrid 0 5
Inter - Sheriff Tiraspol 3 1
Real Madrid - Shaktar Donetsk 2 1
Sheriff Tiraspol - Inter 1 3
Inter - Shaktar Donetsk 2 0
Sheriff Tiraspol - Real Madrid 0 3
Shaktar Donetsk - Sheriff Tiraspol 1 1
Real Madrid - Inter 2 0
```

Variante: invece che stampare solo i punti fatti, stampa anche il numero di vittorie, il numero di pareggi, il numero di sconfitte, il goal fatti, i goal subiti e la differenza reti.

Soluzione

```
d = {}
with open("8-2-7-in.txt", 'r', encoding = 'utf-8') as f:
    for riga in f:
        dati = riga.split()
        squadre = dati[:-2]
        squadre = " ".join(squadre)
        squadre = squadre.split("-")
        squadre = [squadra.strip() for squadra in squadre]
        goal = list(map(int, dati[-2:]))

        if goal[0] > goal[1]:
            if squadre[0] in d:
                d[squadre[0]] += 3
            else:
                d[squadre[0]] = 3

        elif goal[1] > goal[0]:
            if squadre[1] in d:
                d[squadre[1]] += 3
            else:
                d[squadre[1]] = 3

        else:
```

```
            if squadre[0] in d:
                d[squadre[0]] += 1
            else:
                d[squadre[0]] = 1

            if squadre[1] in d:
                d[squadre[1]] += 1
            else:
                d[squadre[1]] = 1

for key, val in d.items():
    print(f"{key}: {val}")

for tupla in d.items():
    print(f"{tupla}")

lista = list(d.items())
lista.sort(key = lambda x: (-x[1], x[0]))

print("\nClassifica:")
for s, g in lista:
    print(f"{s}: {g}")

# d = dict(lista)
# print(d)
```

Soluzione variante:

```
# 5.1.17 - Leggi da un file (il testo è riportato di seguito) i dati
# che descrivono i risultati delle partite di un girone di Champions:
# Sapendo che la vittoria vale 3 punti, il pareggio 1 punto e la
# sconfitta 0 punti, calcola e stampa la classifica del girone
# comprendente: il nome della squadra, i punti fatti, il numero
# di vittorie, il numero di pareggi, il numero di sconfitte, il goal
# fatti, i goal subiti e la differenza reti.

with open('5-1-17.txt', 'r', encoding='utf-8') as f:
    d = {}
    for riga in f:
        tmp = riga.strip().split()
        g1 = int(tmp[-2])
        g2 = int(tmp[-1])
        squadre = ' '.join(tmp[:-2])
        squadre = squadre.split('-')
        sq1 = squadre[0].strip()
        sq2 = squadre[1].strip()

        if g1 > g2:
            v1 = 1
            p1 = 0
            s1 = 0
            punti1 = 3
```

```
            v2 = 0
            p2 = 0
            s2 = 1
            punti2 = 0
        elif g2 > g1:
            v1 = 0
            p1 = 0
            s1 = 1
            punti1 = 0
            v2 = 1
            p2 = 0
            s2 = 0
            punti2 = 3
        else:
            v1 = 0
            p1 = 1
            s1 = 0
            punti1 = 1
            v2 = 0
            p2 = 1
            s2 = 0
            punti2 = 1

       # il nome della squadra, i punti fatti, il numero
        # di vittorie, il numero di pareggi, il numero di sconfitte, il goal
        # fatti, i goal subiti e la differenza reti.
        if sq1 not in d:
            d[sq1] = [punti1, v1, p1, s1, g1, g2, g1-g2]
        else:
            d[sq1] = [d[sq1][0]+punti1, d[sq1][1]+v1, d[sq1][2]+p1, d[sq1]
                      [3]+s1, d[sq1][4]+g1, d[sq1][5]+g2, d[sq1][6]+g1-g2]
        if sq2 not in d:
            d[sq2] = [punti2, v2, p2, s2, g2, g1, g2-g1]
        else:
            d[sq2] = [d[sq2][0]+punti2, d[sq2][1]+v2, d[sq2][2]+p2, d[sq2]
                      [3]+s2, d[sq2][4]+g2, d[sq2][5]+g1, d[sq2][6]+g2-g1]


classifica = list(d.items())
classifica.sort(key = lambda x: -x[1][0])

for el in classifica:
    print(el)
```

6.2.17 Autostrade da file

Dato il seguente file di dati in input:

```
A1: Milano -  Napoli: Autostrade per l'Italia: 759
A2: Salerno - Reggio Calabria: Anas: 442
A3: Napoli - Salerno: Autostrade per l'Italia: 52
A4: Torino - Trieste: Autostrade per l'Italia: 524
A6: Torino - Savona: Gruppo Gavio: 124
A7: Milano - Genova: Enti Pubblici: 133
A8: Milano - Varese: Autostrade per l'Italia: 43
```

```
A10: Savona - Ventimiglia: Gruppo Gavio: 113
A11: Firenze - Pisa: Autostrade per l'Italia: 81
A12: Genova - Cecina: Gruppo Gavio: 210
A13: Bologna - Padova: Autostrade per l'Italia: 116
A14: Bologna - Taranto: Autostrade per l'Italia: 743
A15: Parma - La Spezia: Gruppo Gavio: 108
A16: Napoli - Canosa: Autostrade per l'Italia: 172
A19: Palermo - Catania: Anas: 191
A21: Torino - Piacenza - Brescia: Gruppo Gavio: 238
A22: Brennero - Modena: Enti Pubblici: 315
A23: Palmanova - Tarvisio: Enti Pubblici: 119
A24: Roma - Teramo: Gruppo Toto: 159
A25: Torano - Pescara: Gruppo Toto: 115
A26: Genova - Gravellona: Autostrade per l'Italia: 197
```

1. Stampare il gestore dell'autostrada A8
2. stampare il gestore dell'autostrada "Savona - Ventimiglia"
3. stampare l'autostrada più lunga, con il suo percorso e il suo gestore
4. Stampare quanti differenti gestori sono presenti nell'elenco
5. Per ognuno dei gestori presenti stampare il numero di autostrade gestite, e il numero di km gestiti

Soluzione mettendo i dati un una grande lista

```python
dati = []
with open("7_2_17.txt", "r", encoding='utf-8') as f:
    for riga in f:
        riga = riga.strip()
        auto, tratta, gestore, km = riga.split(':')
        tratta = tratta.strip()
        gestore = gestore.strip()
        km = int(km.strip())
        dati.append([auto, tratta, gestore, km])
        # print(dati[-1])

# 1.    Stampare il gestore dell'autostrada A8
for dato in dati:
    if dato[0] == 'A8':
        print("Gestore dell'autostrada A8:", dato[2])



# 2.    stampare il gestore dell'autostrada "Savona - Ventimiglia"
for dato in dati:
    if dato[1] == 'Savona - Ventimiglia':
        print("Gestore dell'autostrada Savona - Ventimiglia:", dato[2])
# 3.    stampare l'autostrada più lunga, con il suo percorso e il suo gestore
dati.sort(key = lambda x: -x[-1])
print("Autostrada più lunga: ", dati[0])

# 4.    Stampare quanti differenti gestori sono presenti nell'elenco

# gà che devo fare il dizionario lo completo anche per il punto 5
gestori = {}
for dato in dati:
    gestore = dato[2]
    if gestore not in gestori:
        gestori[gestore] = 1
    else:
        gestori[gestore] += 1
print("Numero di gestori:", len(gestori))
```

```
# 5.    Per ognuno dei gestori presenti stampare il numero di autostrade gestite, e il numero di
km gestiti
kmPerGestore = {}
for dato in dati:
    gestore = dato[2]
    km = dato[3]
    if gestore not in kmPerGestore:
        kmPerGestore[gestore] = km
    else:
        kmPerGestore[gestore] += km

for gestore in gestori:
    print(f"{gestore} gestisce {gestori[gestore]} autorstrade per un totale di
{kmPerGestore[gestore]}km")



# provo a mischiare i due dizionari
gestori = {}
for dato in dati:
    gestore = dato[2]
    km = dato[3]
    if gestore not in gestori:
        gestori[gestore] = [1, km]
    else:
        nauto = gestori[gestore][0] + 1
        nuovikm = gestori[gestore][1] + km
        gestori[gestore] = [nauto, nuovikm]
print()
for gestore in gestori:
    print(f"{gestore} gestisce {gestori[gestore][0]} autorstrade per un totale di
{gestori[gestore][1]}km")
```

Soluzione con (quasi) solo dizionari

```
import os
os.system("cls")


with open("8-2-8.txt", 'r', encoding = 'utf-8') as f:
    dati = {}
    for riga in f:
        riga = riga.split(":")
        riga = [dato.strip() for dato in riga]
        riga[3] = int(riga[3])
        dati[riga[0]] = {'tratta': riga[1], 'gestore': riga[2], 'km': riga[3]}

for key, val in dati.items():
    print(f"{key}: {val}")

# 1. Stampare il gestore dell'autostrada A8
print("\nGestore autostrada A8: ", end="")
print(dati['A8'])

# 2. stampare il gestore dell'autostrada "Savona - Ventimiglia"
print("\nGestore autostrada Savona - Ventimiglia: ", end="")
for key, val in dati.items():
    if val['tratta'] == 'Savona - Ventimiglia':
        print(val['gestore'])
```

```
# 3. stampare l'autostrada più lunga, con il suo percorso e
# il suo gestore
print("\nAutostrada più lunga: ", end="")
max = None
for key, val in dati.items():
    if max == None or val['km'] > dati[max]['km']:
        max = key
print(f"{max}: {dati[max]}")

# 4. Stampare quanti differenti gestori sono presenti nell'elenco
# 5. Per ognuno dei gestori presenti stampare il numero di
# autostrade gestite, e il numero di km gestiti

d = {}
for key, val in dati.items():
    gestore = val['gestore']
    if gestore in d:
        d[gestore][0] += 1
        d[gestore][1] += val['km']
    else:
        d[gestore] = [1, val['km']]

print("\nDati sui gruppi:")
for key, val in d.items():
    print(f"{key}: {val}")
# equivale a:
# for key in d:
#     print(f"{key}: {d[key]}")
```

Soluzione mista

```
import os
os.system("cls")

with open("8-2-8.txt", 'r', encoding = 'utf-8') as f:
    dati = []
    for riga in f:
        riga = riga.split(":")
        riga = [dato.strip() for dato in riga]
        riga[3] = int(riga[3])
        dati.append({'nome': riga[0], 'tratta': riga[1], 'gestore': riga[2], 'km': riga[3]})

for dato in dati:
    print(dato)

# 1. Stampare il gestore dell'autostrada A8
print("\nGestore autostrada A8: ", end="")
for dato in dati:
    if dato['nome'] == 'A8':
        print(dato['gestore'])

# 2. stampare il gestore dell'autostrada "Savona - Ventimiglia"
print("\nGestore autostrada Savona - Ventimiglia: ", end="")
```

```python
for dato in dati:
    if dato['tratta'] == 'Savona - Ventimiglia':
        print(dato['gestore'])

# 3. stampare l'autostrada più lunga, con il suo percorso e
# il suo gestore
print("\nAutostrada più lunga: ", end="")
max = None
for dato in dati:
    if max == None or dato['km'] > max['km']:
        max = dato
print(max)

# 4. Stampare quanti differenti gestori sono presenti nell'elenco
# 5. Per ognuno dei gestori presenti stampare il numero di
# autostrade gestite, e il numero di km gestiti

d = {}
for dato in dati:
    gestore = dato['gestore']
    if gestore in d:
        d[gestore][0] += 1
        d[gestore][1] += dato['km']
    else:
        d[gestore] = [1, dato['km']]

print("\nDati sui gruppi:")
for key, val in d.items():
    print(f"{key}: {val}")
```

6.2.18 Squadre e città da file

Dato il file contenente le seguenti informazioni:

```
Atalanta: Bergamo, Italia
Hellas: Verona, Italia
Sampdoria: Genova, Italia
Genoa: Genova, Italia
Liverpool: Liverpool, Regno Unito
Everton: Liverpool, Regno Unito
Espanol: Barcellona, Spagna
Barcellona: Barcellona, Spagna
Siviglia: Siviglia, Spagna
Betis: Siviglia, Spagna
Juventus: Torino, Italia
Young Boys: Berna, Svizzera
Chievo: Verona, Italia
```

1. Leggi i dati e salvali in una struttura formata esclusivamente da dizionari (per i passaggi successivi puoi usare quello che vuoi)
2. Trovare la città dell'Hellas, e stampare il nome dell'altra squadra della stessa città
3. stampare in ordine alfabetico tutte le squadre italiane

4. Stampare le sole squadre italiane, ordinate in base alla città, e a parità di città in ordine alfabetico

Soluzione

```
# 7.1.19 - Dato il file contenente le seguenti informazioni:
# Atalanta: Bergamo, Italia
# Hellas: Verona, Italia
# Sampdoria: Genova, Italia
# Genoa: Genova, Italia
# Liverpool: Liverpool, Regno Unito
# Everton: Liverpool, Regno Unito
# Espanol: Barcellona, Spagna
# Barcellona: Barcellona, Spagna
# Siviglia: Siviglia, Spagna
# Betis: Siviglia, Spagna
# Juventus: Torino, Italia
# Young Boys: Berna, Svizzera
# Chievo: Verona, Italia

# 1. Leggi i dati e salvali in una struttura formata
# esclusivamente da dizionari (per i passaggi successivi puoi usare quello che
# vuoi)
# 2. Trovare la città dell'Hellas, e stampare il nome
# dell'altra squadra della stessa città
# 3. stampare in ordine alfabetico tutte le squadre italiane
# 4.  Stampare le sole squadre italiane, ordinate in base
# alla città, e a parità di città in ordine alfabetico

import os
os.system('cls')

# 1. Leggi i dati e salvali in una struttura formata
# esclusivamente da dizionari (per i passaggi successivi puoi usare quello che
# vuoi)
with open('7-2-9.txt', 'r', encoding='utf-8') as f:
    dati = {}
    for riga in f:
        dato = riga.split(':')
        nome = dato[0]
        dato = ''.join(dato[1])
        dato = dato.strip()
        dato = dato.split(', ')
        citta = dato[0]
        nazione = dato[1]
        dati[nome] = {'citta': citta, 'nazione': nazione}

# for key in dati:
#     print(f'{key}: {dati[key]}')
# print()

# 2. Trovare la città dell'Hellas, e stampare il nome
# dell'altra squadra della stessa città
```

```python
print("Squadra della stessa città dell'Hellas: ", end='')
citta = dati['Hellas']['citta']
for key, value in dati.items():
    if value['citta'] == citta and key != 'Hellas':
        print(key)
print()


# 3. stampare in ordine alfabetico tutte le squadre italiane
print('Squadre in ordine alfabetico:')
lista = list(dati.items())
lista.sort()
for dato in lista:
    print(f'- {dato[0]}')
print()

# 4.  Stampare le sole squadre italiane, ordinate in base
# alla città, e a parità di città in ordine alfabetico
italiane = [el for el in lista if el[1]['nazione'] == 'Italia']
italiane.sort(key = lambda x: (x[1]['citta'], x[0]))
print('Squadre italiane in ordine alfabetico per città e poi per nome della squadra')
for squadra in italiane:
    # print(squadra)
    print(f'- {squadra[0]}: {squadra[1]["citta"]}')
```

6.2.19 Codici identificativi

Devi assegnare ad ogni persona della classe un identificativo univoco formato dalle prime tre consonanti del nome seguite dalle prime tre consonanti del cognome seguito da una serie di 3 caratteri alfanumerici casuali. Se i nomi o i cognomi non contengono 3 consonanti aggiungi vocali a caso per arrivare alle 3 lettere richieste sia per il nome che per il cognome. Nel seguente file di testo è riportato l'elenco dei nomi. In un secondo file riscrivi i nomi seguiti dal codice identificativo.

```
Banani, Fahd
Brambilla, Carlo
Brambilla, Federico
Carrera, Aurora
Crespi, Ryan
Germanò, Riccardo
Grassi, Arianna
Hoxha, Visar
Kuka, Blerina
La Rosa, Abraham
Livieri, Lorenzo
Meterangelo, Chiara
Miele, Serena
Puertas Moreno, Alessandra
Pulvirenti, Giorgia
Roncoroni, Luca Davide
Rossetti, Gabriele
Sambo, Luigi
Sfondrini, Diego
Tarboush, Andrea
Tieni, Ilaria
Tornaghi, Francesca
```

Ad esempio Tieni Ilaria è associata a lrotnaf5u, lr sono scelti dal nome, o è a caso, tn presi dal cognome, a a caso, f5u a caso.

Soluzione

```
# 8.2.10 - Devi assegnare ad ogni persona della classe un
# identificativo univoco formato dalle prime tre consonanti del nome seguite
# dalle prime tre consonanti del cognome seguito da una serie di 3 caratteri
# alfanumerici casuali. Se i nomi o i cognomi non contengono 3 consonanti
# aggiungi vocali a caso per arrivare alle 3 lettere richieste sia per il nome
# che per il cognome. Nel seguente file di testo è riportato l'elenco dei nomi.
# In un secondo file riscrivi i nomi seguiti dal codice identificativo.

# Ad esempio Tieni Ilaria è associata a lrotnaf5u, lr sono
# scelti dal nome, o è a caso, tn presi dal cognome, a a caso, f5u a caso.

from random import choice
from string import ascii_lowercase, digits

def generaCodice(nome, cognome):
    vocali = 'aeiou'
    consonanti = 'bcdfghjklmnpqrstvwxyz'
    alfanum = ascii_lowercase + digits
    codice = []
    nome = nome.lower()
    cognome = cognome.lower()
    i = 0
    for lettera in nome:
        if lettera in consonanti:
            codice.append(lettera)
            i += 1
            if i == 3:
                break
    while i < 3:
        codice.append(choice(vocali))
        i += 1

    i = 0
    for lettera in cognome:
        if lettera in consonanti:
            codice.append(lettera)
            i += 1
            if i == 3:
                break
    while i < 3:
        codice.append(choice(vocali))
        i += 1

    for i in range(3):
        codice.append(choice(alfanum))

    return "".join(codice)
```

```
with open("8-2-10.txt", 'r', encoding='utf-8') as f:
    persone = []
    for riga in f:
        dati = riga.split()
        dati = [dato.strip() for dato in dati]
        dati.append(generaCodice(dati[1],dati[0]))
        persone.append(dati)

for dati in persone:
    print(dati)


with open("8-2-10-out.txt", 'w', encoding='utf-8') as f:
    for dati in persone:
        f.write(f"{dati[1]} {dati[0]} {dati[2]}\n")
```

6.2.20 Probabilità dei dadi su file

Sei un giocatore di giochi di ruolo e ti serve avere delle tabelle che ti indichino le probabilità di ottenere i vari punteggi con dei tiri di diversi dadi. Non conosci molto la matematica e il calcolo combinatorio (e oltretutto calcolare tutto a mano è lungo) ma sai programmare bene in Python, pensi quindi di stimare le probabilità con un programma e di scrivere dei file ognuno che idica le diverse probabilità ottenibili con un diverso dado. I dadi che usi sono: d4 (dado a quattro facce), d6, d8, d10, d12, d20. Per ogni dado vuoi sapere le probabilità di ottenere ogni diverso risultato possibile tirando il dado una volta, o 2, o 3… fino a 10 volte.

Alla fine devi generare 6 file (uno per dado) e ogni file deve avere una prima riga che indica il dado (ad esempio “d6”) e poi diverse righe, ognuna inizia indicando il numero di dadi tirati (da 1 a 10 quindi 10 righe) seguito da un “:” e poi un dizionario che associa ad ogni valore ottenibile la probabilità associata.

Soluzione

```
import os
from random import randint
os.system('cls')

dadi = [4,6,8,10,12,20]

ntiri = 10000
ndadimax = 10

lista = []
for dado in dadi:
    probs = [{} for _ in range(ndadimax+1)]
    for ndadi in range(1, ndadimax+1):
        for caso in range(ndadi, dado*ndadi+1):
            probs[ndadi][caso] = 0
        for lanci in range(ntiri):
            ris = 0
            for tiro in range(ndadi):
                ris += randint(1,dado)
```

```
            # print(ris)
            probs[ndadi][ris] += 1

        for key in probs[ndadi]:
            probs[ndadi][key] /= ntiri
            probs[ndadi][key] *= 100

    lista.append(probs)
    print(f"d{dado} fatto")

# stampa delle probabilità con uno specifico tipo di dado
# for i, ndadi in enumerate(lista[0]):
#     print(f"Con {i} dadi: ",end="")
#     for val in ndadi:
#         print(f"{val}: {ndadi[val]:.3f}   ", end="")
#         # prob = format(ndadi[val], "10.2E")
#         # print(f"{val}: {prob:5f}   ", end="")
#     print()

# scrivo risultati nei files
for i in range(len(lista)):
    with open(f"8-2-11-outputs/d{dadi[i]}.txt", 'w', encoding='utf-8') as f:
        for j, ndadi in enumerate(lista[i]):
            if j > 0:
                if j == 1:
                    f.write(f"Con {j:2d} dado: ")
                else:
                    f.write(f"Con {j:2d} dadi: ")
                for val in ndadi:
                    f.write(f"{val:2d}: {ndadi[val]:7.4f}   ")
                    # prob = format(ndadi[val], "10.2E")
                    # print(f"{val}: {prob:5f}   ", end="")
                f.write("\n")
```

6.2.21 Agenda su file

Crea un programma di gestione di una agenda. L'agenda deve permettere di visualizzare e inserire i dati riguardanti i propri contatti. Per ogni persona devono essere memorizzati nome cognome e numero di telefono. La visualizzazione dei dati deve avvenire in ordine alfabetico per nome o cognome (scegli tu). Tutti i dati devono essere salvati in un file in modo da poterli caricare ogni volta che si apre il programma.

Rendi il programma più modulare possibile dividendo le funzionalità in funzioni separate. Scrivi anche una funzione che ti permetta di cercare un contatto in base a nome o cognome o entrambi.

Variante: L'agenda è ordinata per cogome e la funzione di ricerca cerca il contatto a partire dal cognome con una ricerca dicotomica. Se hai fatto le funzioni ricorsive la funzione deve essere fatta sia nella versione iterativa che ricorsiva

6.2.22 Ore lavorate da file

I seguenti dati, che dovrai inserire in un file di testo, descrivono quante ore hanno lavorato in una settimana, una serie di dipendenti: ad esempio Mr White ha lavorato 8 ore il lunedì, 9 ore il martedì, 8 ore il mercoledì, 9 ore il giovedì e 4 ore il venerdì

```
Mr White: 8, 9, 8, 9, 4
Mr Brown: 0, 8, 7, 10, 8
Mr Blonde: 8, 8, 8, 9, 9
Mr Black: 6, 9, 9, 8, 8
Mr Red: 8, 7, 8, 8, 8
Mr Green: 4, 8, 8, 8, 4
```

1. Stampare la classifica dei lavoratori, in base al numero di ore lavorate
2. Stampare il giorno con più ore lavorate, e quante ore sono state lavorate in quel giorno

Soluzione

```python
dati = []
with open("7_2_22.txt", 'r', encoding='utf-8') as f:
    for riga in f:
        riga = riga.strip().split(':')
        nome = riga[0]
        ore = riga[1].strip()
        ore = ore.split(',')
        ore = [int(ora) for ora in ore]
        # ore = list(map(int, ore)) # alternativa alla list comprehension
        # print(nome, ore)
        dati.append([nome, ore])
        # print(dati[-1])



# 1.    Stampare la classifica dei lavoratori, in base al numero di ore lavorate
print("\nClassifica delle ore lavorate")
dati.sort(key=lambda x:-sum(x[1]))
for el in dati:
    print(f"{el[0]:10s} : {str(el[1]):17s} : {sum(el[1])}")



# 2.    Stampare il giorno con più ore lavorate, e quante ore sono state lavorate in quel giorno

sommecolonne = [0 for _ in range(len(dati[0][1]))]
for riga in dati:
    ore = riga[1]
    for i in range(len(ore)):
        sommecolonne[i] += ore[i]

# print(sommecolonne)

# jmax = 0
# max = sommecolonne[0]
# for j in range(1, len(sommecolonne)):
#     if sommecolonne[j] > max:
#         jmax = j
#         max = sommecolonne[j]

massimo = max(sommecolonne)
jmax = sommecolonne.index(massimo)

giorni = ['lunedì', 'martedì', 'mercoledì', 'giovedì', 'venerdì']
print(giorni[jmax], massimo)
```

6.2.23 Lettere più frequenti per posizione

progettare la funzione f(ftesto,k) che, presi in input il nome (o l'indirizzo) di un file di testo ed un intero k, restituisce una stringa di caratteri lunga k. Il file di testo contiene stringhe di diversa lunghezza (una per riga ed ogni riga termina con '\n'), un possibile testo da inserire nel file di input è riportato di seguito.

I k caratteri della stringa restituita dalla funzione si ottengono considerando le stringhe lunghe k presenti nel file di testo. L'i-mo carattere della stringa sara' il carattere che compare con maggior frequenza come i-mo carattere delle stringhe lunghe k nel file di testo (in caso di parita' di occorrenze viene scelto il carattere che precede gli altri lessicograficamente).

Nel caso il file di testo non contenga parole lunghe k allora viene restituita

la stringa vuota.

Ad Esempio, per il file di testo:

```
porta
rigore
ora
giacca
follia
gara
finale
salute
fiore
lampada
cane
erba
palo
tre
due
fionda
limite
polo
soglia
amo
```

Se k=3 la funzione restituisce la stringa 'are' a seguito della presenza delle seguenti 4 stringhe lunghe 3:

```
tre
due
amo
ora
```

Soluzione

```
def f(ftesto,k):

    with open(ftesto) as f:
        parole = f.read().split()

    parole = [parola for parola in parole if len(parola) == k]

    ris = ""
    for j in range(k):
        freq = {}
```

```
        for parola in parole:
            if parola[j] in freq:
                freq[parola[j]] += 1
            else:
                freq[parola[j]] = 1

        lista = list(freq.items())
        lista.sort(key=lambda el: (-el[1], el[0]))
        ris += lista[0][0]

    return ris



    ## alternativa

    # with open(ftesto) as f:
    #     parole = f.read().split()

    # parole = [parola for parola in parole if len(parola) == k]
    # if len(parole) == 0:
    #     return ''
    # # print(parole)
    # lettere_per_posizione = [[parola[i] for parola in parole] for i in range(k)]
    # # print(lettere_per_posizione)
    # frequenze = [{c: 0 for c in posizione} for posizione in lettere_per_posizione]
    # for i in range(k):
    #     for lettera in lettere_per_posizione[i]:
    #         frequenze[i][lettera] += 1
    # # print(frequenze)
    # frequenze = [[(c, posizione[c]) for c in posizione] for posizione in frequenze]
    # # print(frequenze)
    # for posizione in frequenze:
    #     posizione.sort(key=lambda x: (-x[1], x[0]))
    # # print(frequenze)
    # parola = [posizione[0][0] for posizione in frequenze]
    # parola = ''.join(parola)
    # # print(parola)
    # return parola



ftesto, k = 'nomefile.txt', 3
```

```
print(f(ftesto,k))
```

6.2.24 Regioni e Province

Il seguente file di testo contiene le informazioni riguardanti le province d'Italia raggruppate per regione:

```
Valle d'Aosta: Aosta
Piemonte: Torino, Alessandria, Asti, Biella, Cuneo, Novara, Verbano-Cusio-Ossola,
Vercelli
Liguria: Genova, Imperia, La Spezia, Savona
Lombardia: Milano, Bergamo, Brescia, Como, Cremona, Lecco, Lodi, Mantova, Monza e
Brianza, Pavia, Sondrio, Varese
Trentino-Alto Adige: Trento, Bolzano
Veneto: Venezia, Belluno, Padova, Rovigo, Treviso, Verona, Vicenza
Friuli-Venezia Giulia: Trieste Gorizia, Pordenone, Udine
Emilia-Romagna: Bologna, Ferrara, Forlì-Cesena, Modena, Parma, Piacenza, Ravenna,
Reggio Emilia, Rimini
Toscana: Firenze, Arezzo, Grosseto, Livorno, Lucca, Massa Carrara, Pisa, Pistoia,
Prato, Siena
Umbria: Perugia, Terni
Marche: Ancona, Ascoli Piceno, Fermo, Macerata, Pesaro-Urbino
Lazio: Roma, Frosinone, Latina, Rieti, Viterbo
Abruzzo: L'Aquila, Chieti, Pescara, Teramo
Molise: Campobasso, Isernia
Campania: Napoli, Avellino, Benevento, Caserta, Salerno
Puglia: Bari, Andria, Barletta, Brindisi, Foggia, Lecce, Taranto, Trani
Basilicata: Potenza, Matera
Calabria: Catanzaro, Cosenza, Crotone, Reggio Calabria, Vibo Valentia
Sicilia: Palermo, Agrigento, Caltanissetta, Catania, Enna, Messina, Ragusa,
Siracusa, Trapani
Sardegna: Cagliari, Nuoro, Oristano, Sassari, Sud Sardegna
```

La prima provincia indicata per ogni regione è il capoluogo di regione. Scrivi un programma che memorizzi opportunamente tutti i dati, poi:

1. Scrivi e usa una funzione che calcola quante province sono presenti mediamente in ogni regione
2. Ordina le regioni per numero di province
3. Scrivi i nomi delle regioni con più province e con meno province

Soluzione con solo liste

```
dati = []
with open("7_2_24.txt", "r", encoding='utf-8') as f:
    for riga in f:
        riga = riga.strip()
        regione, listacitta = riga.split(':')
        regione = regione.strip()
        listacitta = listacitta.strip().split(',')
        listacitta = [citta.strip() for citta in listacitta]
        dati.append([regione, listacitta])
        print(dati[-1])

# 1.    Scrivi e usa una funzione che calcola quante province sono presenti
#       mediamente in ogni regione
def mediaCittaPerRegione(lista):
    somma = 0
    # for regione, elenco in lista:
    #     somma += len(elenco)
    #oppure
    for el in lista:
        somma += len(el[1])
    return somma / len(lista)
```

```
print(f"\nLa media del numero di città per regione: {mediaCittaPerRegione(dati)}")

# 2.    Ordina le regioni per numero di province
dati.sort(key = lambda x: -len(x[1]))
print("\nStampa delle regioni ordinate per numero di province:")
for el in dati:
    print(el)

# 3.    Scrivi i nomi delle regioni con più province e con meno province
print("\nLa regione con più province: ", dati[0])
print("La regione con meno province: ", dati[-1])
```

Soluzone con dizionario interno

```
dati = []
with open("7_2_24.txt", "r", encoding='utf-8') as f:
    for riga in f:
        riga = riga.strip()
        regione, listacitta = riga.split(':')
        regione = regione.strip()
        listacitta = listacitta.strip().split(',')
        listacitta = [citta.strip() for citta in listacitta]
        dati.append({"regione": regione, "province": listacitta})
        print(dati[-1])

# 1.    Scrivi e usa una funzione che calcola quante province sono presenti
#       mediamente in ogni regione
def mediaCittaPerRegione(lista):
    somma = 0
    # for regione, elenco in lista:
    #     somma += len(elenco)
    #oppure
    for el in lista:
        somma += len(el["province"])
    return somma / len(lista)

print(f"\nLa media del numero di città per regione: {mediaCittaPerRegione(dati)}")

# 2.    Ordina le regioni per numero di province
dati.sort(key = lambda x: -len(x["province"]))
print("\nStampa delle regioni ordinate per numero di province:")
for el in dati:
    print(el)

# 3.    Scrivi i nomi delle regioni con più province e con meno province
print("\nLa regione con più province: ", dati[0])
print("La regione con meno province: ", dati[-1])
```

Soluzione coi dizionari

```
# Il seguente file di testo contiene le informazioni
# riguardanti le province d'Italia raggruppate per
# regione:

# La prima provincia indicata per ogni regione è il
# capoluogo di regione. Scrivi un programma che memorizzi
# opportunamente tutti i dati, poi:

dati = {}
```

```python
with open("1-25.txt", "r", encoding="utf-8") as f:
    for riga in f:
        riga = riga.strip().split(":")
        regione = riga[0]
        province = riga[1].split(",")
        province = [prov.strip() for prov in province]
        # print(regione, province)
        dati[regione] = province

# for key, val in dati.items():
#     print(key, val)

# 1.    Scrivi e usa una funzione che calcola quante
# province sono presenti mediamente in ogni regione
def mediaProvince(diz):
    media = 0
    for key in diz:
        media += len(diz[key])
    media /= len(diz)
    return media

print(f"Media province per regione: {mediaProvince(dati):.2f}")

# 2.    Ordina le regioni per numero di province
lista = list(dati.items())
lista.sort(key = lambda x:-len(x[1]))
for el in lista:
    print(el)

# 3.    Scrivi i nomi delle regioni con più province
# e con meno province
print()
print(lista[0])
print(lista[-1])
```

6.2.25 Vendite

Scrivi un programma che:

1. legga e memorizzi opportunamente i dati presenti in un file di testo (testo riportato di seguito)
2. Calcola l'incasso totale per ogni prodotto
3. ordini gli acquisti fatti in base alle quantità
4. ordini gli acquisti fatti in base al totale di incasso fatto da ogni prodotto
5. Salva poi i dati ottenuti al punto precedente su un altro file

```
giorno,prodotto,prezzo unitario,quantità
lunedì,mele,1.2,10
lunedì,banane,0.8,5
lunedì,arance,1.5,7
martedì,mele,1.2,6
martedì,banane,0.8,9
martedì,pere,1.7,4
```

```
mercoledì,mele,1.2,3
mercoledì,arance,1.5,8
mercoledì,pere,1.7,6
giovedì,banane,0.8,12
giovedì,mele,1.2,5
giovedì,arance,1.5,4
venerdì,mele,1.2,9
venerdì,banane,0.8,7
venerdì,pere,1.7,3
sabato,mele,1.3,15
sabato,arance,1.6,10
sabato,banane,0.9,11
domenica,mele,1.3,8
domenica,pere,1.8,5
domenica,arance,1.6,6
```

Variante:

Dati più "sporchi"

7 Classi – Spiegazioni ed esempi

7.1 Le classi

La maggior parte dei linguaggi di programmazione moderni sono linguaggi di programmazione ad oggetti. In linguaggi come il C i dati possono essere memorizzati nelle variabili o eventualmente in strutture (struct), cioè insiemi di variabili trattati come una sola variabile. Capita spesso di dover definire funzioni che lavorino su tali strutture. L'idea dell'oggetto è di avere un'unica struttura ordinata ed efficiente che contenga le variabili e le funzioni che lavorano con esse.

Se vogliamo usare oggetti dobbiamo prima definirli, ovvero indicare:

- da quali **attributi** (le variabili) è composto,
- quali sono i suoi **metodi** (le funzioni).

Per essere precisi un oggetto è solamente un'istanza di un'entità più astratta che chiamiamo classe. Ad esempio potremmo definire la classe Punto e poi istanziare tutte le istanze (oggetti) di Punto che vogliamo.

Riassumendo:

- la **classe** è il modello generale,
- **l'oggetto** è un'istanza concreta della classe.

Creiamo ad esempio una classe Punto bidimensionale:

```
class PuntoB:
    def __init__(self, x, y):
        self.x = x
```

```
        self.y = y

    def __str__(self):
        return f"({self.x}, {self.y})"
```

che possiamo usare così:

```
a = PuntoB(3, 4)
b = PuntoB(1, 2)
print(a)  # Output: (3, 4)
print(b)  # Output: (1, 2)
```

Vediamo che PuntoB è la descrizione generale di un punto bidimensionale che possiede alcune funzioni particolari per poter essere creato e rappresentato.

In seguito sono state create due istanze di PuntoB e stampate su terminale.

7.1.1 Metodo __init__

A differenza di altri linguaggi, in Python non è necessario dichiarare esplicitamente il tipo delle variabili. Gli attributi vengono creati direttamente all'interno di un metodo speciale chiamato __init__ (si scrive con due "underscore" all'inizio e alla fine del nome).

Questo metodo:

- viene eseguito quando si crea un oggetto
- serve a inizializzare gli attributi
- viene comunemente chiamato costruttore

Il metodo __init__ riceve:

- i parametri necessari (in questo caso x e y)
- un parametro speciale chiamato self, che deve sempre essere il primo

self rappresenta l'oggetto stesso e permette di accedere ai suoi attributi.

Quando si scrive

```
self.x = x
```

si assegna:

- all'attributo x dell'oggetto (self.x)
- il valore del parametro x

Le due variabili hanno lo stesso nome ma sono diverse:

- una è un attributo dell'oggetto
- l'altra è un parametro della funzione

Nonostante __init__ crei un'istanza della classe e si utilizzi scrivendo:

```
a = PuntoB(3, 4)
```

non va indicato nessun return.

7.1.2 Metodo __str__

Il metodo __str__ serve a restituire una rappresentazione dell'oggetto sotto forma di stringa.

Viene chiamato automaticamente quando:

- si usa print(oggetto)
- si converte l'oggetto in stringa (str(oggetto))

Anche questo metodo riceve self per accedere agli attributi.

7.1.3 Dunder Methods

I metodi come __init__ e __str__ sono chiamati:

- dunder methods (double underscore)
- oppure magic methods

Sono una caratteristica di Python e permettono di definire comportamenti standard per gli oggetti.

Ad esempio:

- __init__ → inizializzazione
- __str__ → rappresentazione leggibile

Python utilizza automaticamente questi metodi quando necessario.

__init__ non è strettamente obbligatorio ma nella pratica è quasi sempre necessario definirlo perché altrimenti Python usa un costruttore di default che non inizializza attributi.

A fine capitolo troverai l'elenco completo di tutti i dunder methods.

7.2 Metodi statici e metodi di classe

Finora abbiamo visto metodi "normali" che ricevono self e lavorano su una singola istanza.

In Python esistono anche altri due tipi di metodi:

- metodi statici (@staticmethod)
- metodi di classe (@classmethod)

Su che cosa lavora ogni tipo di metodo:

Tipo di metodo	**Riceve**	**Lavora su**
Metodo normale	self	oggetto (istanza corrente)
Class method	cls	classe
Static method	—	nessuno in particolare, può essere definito anche esternamente alla classe

7.2.1 Metodi statici

Un metodo statico è una funzione dentro la classe che:

- non usa self
- non usa cls

È semplicemente una funzione “legata” alla classe.

Esempio

```
class PuntoB:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    @staticmethod
    def distanza(p1, p2):
        return ((p1.x - p2.x)**2 + (p1.y - p2.y)**2)**0.5


a = PuntoB(3, 4)
b = PuntoB(1, 2)

print(PuntoB.distanza(a, b))
```

Quando usare @staticmethod:

- la funzione è logicamente collegata alla classe
- ma non ha bisogno né dell’oggetto né della classe

La funzione riguarda i punti, ma non un punto specifico.

Per questo viene definita nella classe, anche se potrebbe essere definita all’esterno.

7.2.2 Metodi di classe

Un metodo di classe riceve come primo parametro la classe stessa (cls).

Attenzione cls è diverso da self, che rappresenta invece l’istanza corrente.

```
class PuntoB:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return f"({self.x}, {self.y})"

    @classmethod
    def origine(cls):
        return cls(0, 0)


p = PuntoB.origine()
print(p)   # (0, 0)
```

Usi @classmethod quando:

- vuoi creare oggetti in modo alternativo
- vuoi lavorare sulla classe invece che sulle istanze

Spesso usato come costruttore alternativo

In realtà è raro usare @classmethod per casi semplici come quello dell'esempio,
spesso si usa il costruttore __init__ con parametri di default.
Ad esempio:

```
class PuntoB:
    def __init__(self, x = 0, y = 0):
        self.x = x
        self.y = y

    def __str__(self):
        return f"({self.x}, {self.y})"


p = PuntoB()
print(p)   # (0, 0)
```

Il risultato è lo stesso dell'esempio precedente. Entrambi i casi hanno i loro vantaggi:

- __init__ con valori di default è molto flessibile
- @classmethod permette di definire metodi con un significato preciso e più chiaro per chi usa la classe

7.3 Classe PuntoB completa

Di seguito vediamo una definizione completa della classe con il suo utilizzo:

```
class PuntoB:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return f"({self.x}, {self.y})"

    def distanza(self, altro_punto):
        return ((self.x - altro_punto.x) ** 2 + (self.y - altro_punto.y) ** 2) ** 0.5

    @staticmethod
    def punto_medio(p1, p2):
        x_medio = (p1.x + p2.x) / 2
        y_medio = (p1.y + p2.y) / 2
        return PuntoB(x_medio, y_medio)

    @classmethod
    def origine(cls):
        return cls(0, 0)

# posso definirla anche fuori dalla classe, ma è più ordinato metterla dentro
def punto_medio(p1, p2):
        x_medio = (p1.x + p2.x) / 2
        y_medio = (p1.y + p2.y) / 2
        return PuntoB(x_medio, y_medio)
```

```
a = PuntoB(3, 4)
b = PuntoB(1, 2)
o = PuntoB.origine()

print(o)  # Output: (0, 0)

print(a)  # Output: (3, 4)
print(b)  # Output: (1, 2)

print(a.distanza(b))  # Output: 2.8284271247461903

punto_medio = PuntoB.punto_medio(a, b)
print(punto_medio)  # Output: (2.0, 3.0)
```

7.4 Ereditarietà

L'ereditarietà permette di creare una nuova classe a partire da una classe esistente.

La nuova classe:

- eredita attributi e metodi
- può aggiungere nuove funzionalità
- può modificare quelle esistenti

Classe di partenza: superclasse o classe base o classe padre

Nuova classe: sottoclasse o classe derivata o classe figlio

Analizziamo il seguente esempio:

```
# Classe base (superclasse)
class Animale:
    def __init__(self, nome):
        self.nome = nome

    def parla(self):
        raise NotImplementedError("Questo metodo deve essere implementato nelle
sottoclassi")

# Classe derivata (sottoclasse)
class Cane(Animale):
    def parla(self):
        return f"{self.nome} dice: Bau!"

# Un'altra classe derivata
class Gatto(Animale):
    def parla(self):
        return f"{self.nome} dice: Miao!"

# Creazione degli oggetti
cane = Cane("Fido")
gatto = Gatto("Miao")
```

```
# Chiamata ai metodi
print(cane.parla())  # Output: Fido dice: Bau!
print(gatto.parla())  # Output: Miao dice: Miao!
```

Nell'esempio si definisce una classe Animale e poi due classi figlie Cane e Gatto.

Animale è la superclasse delle sottoclassi Cane e Gatto.

In questo caso Animale definisce il metodo parla, ma non ne fornisce un'implementazione concreta, iInfatti, se il metodo venisse chiamato direttamente sulla classe base, verrebbe generato un errore.

Le sottoclassi non ridefiniscono __init__ perché l'implementazione della superclasse è già sufficiente.

Guardiamo il secondo esempio:

```
# Classe base (superclasse)
class Persona:
    def __init__(self, nome, eta):
        self.nome = nome
        self.eta = eta

    def __str__(self):
        return f"{self.nome}, {self.eta} anni"

# Classe derivata (sottoclasse)
class Studente(Persona):
    def __init__(self, nome, eta, matricola):
        # Chiamata al costruttore della classe base per inizializzare nome e età
        super().__init__(nome, eta)
        self.matricola = matricola

    def __str__(self):
        # Personalizzazione del metodo __str__ per includere anche la matricola
        return f"{super().__str__()}, Matricola: {self.matricola}"

# Creazione di un oggetto della classe Studente
studente = Studente("Giovanni", 22, "S12345")

# Stampa dei dettagli dello studente
print(studente)
```

In questo esempio Persona è superclasse della sottoclasse Studente.

La sottoclasse:

- aggiunge un nuovo attributo (matricola)
- modifica il metodo __str__, estendendolo

Per farlo utilizza super().

super().__init__(nome, eta) -> inizializza la parte ereditata

super().__str__() -> riutilizza il metodo della superclasse

Ridefinire o estendere un metodo della superclasse si chiama **override**.

7.5 Polimorfismo

Riprendiamo l'esempio degli animali

```
class Animale:
    def __init__(self, nome):
        self.nome = nome

    def parla(self):
        raise NotImplementedError("Metodo da implementare")


class Cane(Animale):
    def parla(self):
        return f"{self.nome} dice: Bau!"


class Gatto(Animale):
    def parla(self):
        return f"{self.nome} dice: Miao!"
```

e aggiungiamo questo codice:

```
animali = [
    Cane("Fido"),
    Gatto("Micio"),
    Cane("Rex")
]

for animale in animali:
    print(animale.parla())

# output:
    # Fido dice: Bau!
    # Micio dice: Miao!
    # Rex dice: Bau!
```

Nel ciclo, l'interprete Python non sa in anticipo se animale è un Cane o un Gatto ma chiama automaticamente il metodo corretto.

Stessa interfaccia, comportamento diverso.

Questo è il **polimorfismo**.

Il polimorfismo funziona perché:

- le classi condividono una struttura comune (la superclasse Animale)
- i metodi sono stati ridefiniti (override)

In realtà il Python è estremamente versatile (a volte troppo pericolosamente versatile!) ed il polimorfismo funziona anche senza ereditarietà, è sufficiente che gli oggetti abbiano gli stessi metodi.

Questo si chiama duck typing: “Se cammina come un'anatra e starnazza come un'anatra, allora è un'anatra”.

7.6 Elenco dei Dunder Methods

Di seguito l’elenco dei dunder methods:

1. Costruttori e Distruttori

- __init__(self, ...) - Inizializza l'oggetto. Chiamato subito dopo la creazione di un'istanza.
- __del__(self) - Distruttore, chiamato quando un oggetto viene eliminato o viene raccolto dal garbage collector.

2. Rappresentazione e Stringhe

- __str__(self) - Definisce la rappresentazione stringa dell'oggetto, utilizzata da str() e print().
- __repr__(self) - Definisce la rappresentazione ufficiale dell'oggetto, utilizzata da repr() e da ambienti di sviluppo interattivi.
- __format__(self, format_spec) - Definisce come formattare l'oggetto utilizzando format().
- __bytes__(self) - Definisce la rappresentazione in bytes dell'oggetto.

3. Operazioni Aritmetiche

- __add__(self, other) - Definisce il comportamento per l'operatore +.
- __sub__(self, other) - Definisce il comportamento per l'operatore -.
- __mul__(self, other) - Definisce il comportamento per l'operatore *.
- __truediv__(self, other) - Definisce il comportamento per l'operatore /.
- __floordiv__(self, other) - Definisce il comportamento per l'operatore //.
- __mod__(self, other) - Definisce il comportamento per l'operatore %.
- __pow__(self, other) - Definisce il comportamento per l'operatore **.
- __neg__(self) - Definisce il comportamento per l'operatore unario -.

4. Confronto

- __eq__(self, other) - Definisce il comportamento per l'operatore ==.
- __ne__(self, other) - Definisce il comportamento per l'operatore !=.
- __lt__(self, other) - Definisce il comportamento per l'operatore <.
- __le__(self, other) - Definisce il comportamento per l'operatore <=.
- __gt__(self, other) - Definisce il comportamento per l'operatore >.
- __ge__(self, other) - Definisce il comportamento per l'operatore >=.

5. Conversione di Tipo

- __int__(self) - Conversione a intero utilizzando int().
- __float__(self) - Conversione a float utilizzando float().
- __bool__(self) - Conversione a booleano utilizzando bool().
- __len__(self) - Definisce la lunghezza dell'oggetto, utilizzata da len().

6. Accesso agli Attributi

- __getattr__(self, name) - Viene chiamato quando si accede a un attributo inesistente.
- __setattr__(self, name, value) - Viene chiamato per impostare un attributo.
- __delattr__(self, name) - Viene chiamato per eliminare un attributo.

7. Operatori Logici

- __and__(self, other) - Definisce il comportamento per l'operatore &.
- __or__(self, other) - Definisce il comportamento per l'operatore |.
- __xor__(self, other) - Definisce il comportamento per l'operatore ^.
- __invert__(self) - Definisce il comportamento per l'operatore ~.

8. Contesto (Gestione delle Risorse)

- __enter__(self) - Definisce l'entrata in un contesto (with statement).
- __exit__(self, exc_type, exc_value, traceback) - Definisce l'uscita da un contesto.

9. Container e Iterazione

- __getitem__(self, key) - Definisce l'accesso tramite indice o chiave, come obj[key].
- __setitem__(self, key, value) - Definisce l'assegnazione tramite indice o chiave, come obj[key] = value.
- __delitem__(self, key) - Definisce l'eliminazione tramite indice o chiave.
- __iter__(self) - Definisce un iteratore per l'oggetto.
- __next__(self) - Definisce il comportamento per il metodo next().

10. Gestione della Lunghezza e Contenimento

- __len__(self) - Definisce la lunghezza dell'oggetto, utilizzata da len().
- __contains__(self, item) - Definisce il comportamento per l'operatore in.

11. Chiamabilità e Operazioni su oggetti

- __call__(self, ...) - Rende l'oggetto chiamabile, come una funzione.
- __hash__(self) - Definisce il comportamento per l'hashing, utilizzato in strutture dati come set e dizionari.
- __copy__(self) - Definisce il comportamento per la copia superficiale (copy.copy()).
- __deepcopy__(self, memo) - Definisce il comportamento per la copia profonda (copy.deepcopy()).
-

8 Classi – Esercizi

8.1 Esercizi senza ereditarietà

8.1.1 Distanza e punto medio

Scrivi un programma che definisca un punto e poi dati due punti scelti da te sia in grado, per mezzo di due apposite funzioni, di calcolare la distanza fra due punti e trovare il punto medio.

Soluzione senza staticmethod

```
class punto:
    def __init__(self, x = 0, y = 0):
        self.x = x
        self.y = y

    def __repr__(self):
        return f'({self.x}, {self.y})'

def distanza(a, b):
    return ((a.x-b.x)**2 + (a.y-b.y)**2)**(1/2)

def puntoMedio(a, b):
    return punto((a.x+b.x)/2, (a.y+b.y)/2)

a = punto()
b = punto(2, 1)

print(f"Distanza tra A{a} e B{b} = {distanza(a,b)}")
print(f"Punto medio tra A{a} e B{b} = C{puntoMedio(a,b)}")
```

Soluzione con staticmethod

```
class Punto:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return f"({self.x}, {self.y})"

    @staticmethod
    def distanza(a, b):
        return ((a.x-b.x)**2 + (a.y-b.y)**2)**0.5

    @staticmethod
    def punto_medio(a, b):
        return Punto((a.x + b.x)/2, (a.y + b.y)/2)

a = Punto(1,1)
b = Punto(3,6)

print(Punto.distanza(a, b))
print(Punto.punto_medio(a, b))
```

8.1.2 Punti e rette

Definisci una classe punto. Scrivi ed utilizza poi le funzioni:

1. Funzione per stampare una rappresentazione del punto (es. A(1,2))
2. Funzione per calcolare il punto medio
3. Funzione per calcolare la retta passante tra due punti. Dovrai anche definire la classe retta che contiene le informazioni necessarie ad identificare una retta e definire una funzione per stamparla in modo standard (y=mx+c)
4. Funzione che verifica se tre punti sono allineati.

Soluzione

```
class Punto:

    def __init__(self, x = 0, y = 0) -> None:
        self.x = x
        self.y = y

    def __repr__(self) -> str:
        return f'({self.x}, {self.y})'

    def distanza(self, b):
        return ((self.x-b.x)**2 + (self.y-b.y)**2)**(1/2)

    @staticmethod
    def distanzaStatico(a, b):
        return ((a.x-b.x)**2 + (a.y-b.y)**2)**(1/2)

def distanzaFuori(a, b):
    return ((a.x-b.x)**2 + (a.y-b.y)**2)**(1/2)

def puntoMedio(a, b):
    return Punto((a.x+b.x)/2, (a.y+b.y)/2)

class Retta:
    def __init__(self, m = 1, q = 0) -> None:
        self.m = m
        self.q = q

    def __repr__(self) -> str:
        ris = 'y = '

        if self.m == 0:
            ris += f'{self.q}'
            return ris

        if self.m == 1:
            ris += 'x'
        else:
            ris += f'{self.m}x'

        if self.q > 0:
```

```
            ris += f' + {self.q}'
        if self.q < 0:
            ris += f' - {abs(self.q)}'

        return ris

    def __eq__(self, r) -> bool:
        return self.m == r.m and self.q == r.q

def rettaPassante(A, B):
    if type(A) is not Punto or type(B) is not Punto:
        return
    m = (A.y - B.y) / (A.x - B.x)
    q = A.y - m*A.x

    return Retta(m ,q)


def puntiAllineati(A, B, C):
    if type(A) is not Punto or type(B) is not Punto or type(C) is not Punto:
        return False
    return rettaPassante(A, B) == rettaPassante(B, C)



A = Punto(1, 1)
B = Punto(2, 2)
print(f'A{A}')
print(f'B{B}')
print(A.distanza(B))
print(Punto.distanzaStatico(A,B))
print(distanzaFuori(A,B))
C = puntoMedio(A,B)
print(f'C{C}')

r1 = Retta(-3, -2)
print(r1)
print()

print(rettaPassante(A, B))
print(puntiAllineati(A, B, Punto(0,5)))
```

8.1.3 Punto N-dimensionale (difficile)

Definisci una classe punto ndimensionale (dimensioni a piacere) poi definisci e usa le funzioni trasla e punto medio.

Soluzione

```
class Punto:
    def __init__(self, coordinate):

        if type(coordinate) == list:
            self.coordinate = coordinate
        else:
```

```
            raise TypeError("Coordinate devono essere una tupla o una lista")
            self.coordinate = [0]

    def __str__(self):
        ris = "("
        ris += ", ".join(str(c) for c in self.coordinate)
        ris += ")"
        return ris

    def trasla(self, deltas):
        if type(deltas) == tuple or type(deltas) == list:
            while len(deltas) > len(self.coordinate):
                self.coordinate.append(0)
            for i in range(len(deltas)):
                self.coordinate[i] += deltas[i]
        else:
            raise TypeError("Deltas devono essere una tupla o una lista")

    @staticmethod
    def punto_medio(a, b):
        if type(a) == Punto and type(b) == Punto:
            x = Punto(a.coordinate)
            y = Punto(b.coordinate)
            if len(x.coordinate) < len(y.coordinate):
                while len(x.coordinate) < len(y.coordinate):
                    x.coordinate.append(0)
            elif len(y.coordinate) < len(x.coordinate):
                while len(y.coordinate) < len(x.coordinate):
                    y.coordinate.append(0)
            coordinate = []
            for i in range(len(x.coordinate)):
                coordinate.append((x.coordinate[i] + y.coordinate[i]) / 2)
            return Punto(coordinate)
        else:
            raise TypeError("a e b devono essere istanze di Punto")

a = Punto([1,1,1])
print(a)

a.trasla([1, 1, 1, 1])
print(a)

b= Punto([3,6,9])
print(Punto.punto_medio(a, b))
```

8.1.4 Vettori 1 (definizione)

Definisci la classe vettore. Scrivi e utilizza le funzioni necessarie a:

1. Stampare su console il vettore,
2. Calcolare il modulo del vettore
3. Calcolare la somma di due vettori (con l'operatore +)
4. Calcolare la differenza tra due vettore (con l'operatore -)

Soluzione

```
# 8.1.3 Definisci la classe vettore. Scrivi e utilizza le funzioni necessarie a:
# 1.     stampare su console il vettore,
# 2.    Calcolare il modulo del vettore
# 3.    Calcolare la somma di due vettori (con l'operatore +)
# 4.    Calcolare la differenza tra due vettore (con l'operatore -)
```

```python
class Vettore:
    def __init__(self, x, y) -> None:
        self.x = x
        self.y = y

    def __repr__(self) -> str:
        return f'({self.x}, {self.y})'

    def modulo(self):
        return (self.x**2 + self.y**2)**(1/2)

    def __add__(self, v2):
        return Vettore(self.x+v2.x, self.y+v2.y)

    def __sub__(self, v2):
        return Vettore(self.x-v2.x, self.y-v2.y)



v1 = Vettore(1, 1)
print(v1)
print(v1.modulo())

v2 = Vettore(1,2)
print(v1+v2)
print(v1-v2)
```

8.1.5 Vettori 2 (file)

Scrivi un programma che:

1. legga dal seguente file di testo un elenco di vettori e li salvi in una lista di oggetti Vector da te definito;
2. ordini I vettori in ordine decrescente di modulo
3. stampi tutti I vettori ordinati ognuno seguito dal valore del suo modulo
4. stampi il vettore risultante dalla somma di tutti i vettori; esegui le somme con l'operatore +

File di testo:

```
(1.2, 3.5)
(-0.7, 4.1)
(2.8, -1.6)
(0.0, 5.9)
(-3.3, 2.2)
(4.75, -0.5)
(-1.1, -3.8)
(6.4, 1.3)
(-2.9, 0.6)
(3.14, 2.71)
(-4.2, -1.9)
(5.5, -4.4)
(0.8, 7.2)
(-6.6, 3.3)
(2.25, -2.25)
(-0.4, 0.9)
(7.7, -1.1)
```

```
(-3.6, 5.8)
(1.9, -6.3)
(4.0, 0.2)
```

Soluzione

```python
class Vettore:
    def __init__(self, x = 0, y = 0, stringa = None):
        self.x = x
        self.y = y

        if type(stringa) == str:
            # (1.2, 3.5)
            self.x, self.y = stringa.strip()[1:-1].split(",")
            self.x = float(self.x.strip())
            self.y = float(self.y.strip())

    def __repr__(self):
        return f"({self.x:.2f}; {self.y:.2f})"

    def modulo(self):
        return (self.x**2 + self.y**2)**0.5

    def __lt__(self, other):
        return self.modulo() < other.modulo()

    def __add__(self, other):
        return Vettore(self.x + other.x, self.y + other.y)


vettori = []
with open("9_1_5.txt", "r", encoding='utf-8') as f:
    for riga in f:
        vettori.append(Vettore(stringa = riga))


# vettori.sort(key = lambda x: -x.modulo())
vettori.sort(reverse=True)

print("Vettori ordinati per modulo:")
for vettore in vettori:
    # print(f"{vettore}: {vettore.modulo()}")
    print(f"{str(vettore):16s}: {vettore.modulo():.2f}")

v = Vettore()
for vettore in vettori:
    v += vettore
print(f"\n\nVettore risultante dalla somma di tutti i vettori: {v}")
```

8.1.6 Gestione di un corso

Scrivi un programma per gestire un corso. Al corso sono iscritte delle persone. Per ogni persona dovete sapere nome e cognome e ad ognuno alla fine del corso viene assegnato un voto da 1 a 10. Se il corso non è finito il voto ha valore None. Crea a parte un file di testo che contenga i nomi degli iscritti (all'inizio senza voti) da cui il programma legge i nomi di tutti gli iscritti. Il programma legge i nomi, li visualizza, chiede se si vuole inserire un voto e gestisce l'inserimento. Alla fine il programma salva tutti i dati nel file di prima. Dalla seconda esecuzione il programma troverà già dei voti nel file.

Soluzione

```python
import os
os.system('cls')

class persona:
    def __init__(self, nome, cognome, voto = None) -> None:
        self.nome = nome
        self.cognome = cognome
        self.voto = voto

    def __repr__(self) -> str:
        ris = f'{self.cognome} {self.nome}'
        if self.voto != None:
            ris += f': {self.voto:g}'
        return ris


def caricaDati(nomefile):
    dati = []
    with open(nomefile, 'r', encoding='utf-8') as f:
        for riga in f:
            riga = riga.split(',')
            nome = riga[1].strip()
            cognome = riga[0].strip()
            if len(riga) > 2 :
                voto = float(riga[2])
            else:
                voto = None
            dati.append(persona(nome, cognome, voto))
    return dati

def aggiornaFile(nomefile, dati):
    with open(nomefile, 'w', encoding='utf-8') as f:
        for dato in dati:
            f.write(f'{dato.cognome}, {dato.nome}')
            if dato.voto != None:
                f.write(f', {dato.voto}')
            f.write('\n')

def stampaDati(dati):
    for i, dato in enumerate(dati):
        print(f'{i+1} - {dato}')

def inserisciVoto(dati, pos, voto):
    if pos < 0 or pos >= len(dati):
        return False
    else:
        dati[pos].voto = voto
        return True

nomefile = '8-1-4.txt'
dati = caricaDati(nomefile)

si = ['si', 'sì', 'Si', 'Sì', 'SI']
```

```
no = ['no', 'NO', 'No']

while True:
    print('Elenco degli studenti con i rispettivi voti:')
    stampaDati(dati)
    scelta = input('\n\nVuoi inserire un voto? ')
    while scelta not in (si+no):
        print("Non capisco la risposta.")
        scelta = input('\nVuoi inserire un voto? ')
    if scelta in no:
        break

    pos = int(input("Inserisci il numero dello studente: "))
    if pos < 1 or pos > len(dati):
        print('Lo studente non esiste.')
    else:
        voto = float(input('Scrivi il voto: '))
        if inserisciVoto(dati, pos-1, voto):
            aggiornaFile(nomefile, dati)
    print('-------------------------------------------------')
```

8.1.7 Esploratori famosi con classi

Scrivi un programma che gestisca una lista di dati riguardanti esploratori famosi. Devi definire le strutture dati adeguate a contenere queste informazioni: una classe esploratore e una lista di esploratori. Scrivi una serie di funzioni che lavorino sulle tue strutture dati:

1. una funzione che stampa la lista di esploratori.
2. una funzione che ordina la lista in ordine alfabetico secondo il nome degli esploratori
3. una funzione che ordina la lista in ordine crescente secondo le date di nascita degli esploratori
4. una funzione che ordina la lista in ordine decrescente secondo le durate delle vite degli esploratori

I dati sugli esploratori vanno inseriti in un file e sono

```
Marco Polo 1254 1324
Cristoforo Colombo 1451 1506
Amerigo Vespucci 1454 1512
Francisco Pizarro 1475 1541
Ferdinando Magellano 1480 1521
Hernan Cortez 1485 1547
Walter Raleigh 1552 1618
Henry Hudson 1570 1611
James Cook 1728 1779
Charles Darwin 1809 1882
Kit Carson 1809 1866
David Livingstone 1813 1873
Charles Foucauld 1858 1916
Ronald Amundsen 1872 1928
Ernest Shakleton 1874 1922
```

Soluzione

```
# 6.1.5 Scrivi un programma che gestisca una lista di dati riguardanti esploratori famosi.
# Devi definire le strutture dati adeguate a contenere queste informazioni:
```

```python
# una classe esploratore e una lista di esploratori.
# Scrivi una serie di funzioni che lavorino sulle tue strutture dati:
# 1.    una funzione che stampa la lista di esploratori.
# 2.    una funzione che ordina la lista in ordine alfabetico secondo il nome degli esploratori
# 3.    una funzione che ordina la lista in ordine crescente secondo le date di nascita degli esploratori
# 4.    una funzione che ordina la lista in ordine decrescente secondo le durate delle vite degli esploratori
# I dati sugli esploratori vanno inseriti in un file e sono

testo = '''
    Marco Polo 1254 1324
    Cristoforo Colombo 1451 1506
    Amerigo Vespucci 1454 1512
    Francisco Pizarro 1475 1541
    Ferdinando Magellano 1480 1521
    Hernan Cortez 1485 1547
    Walter Raleigh 1552 1618
    Henry Hudson 1570 1611
    James Cook 1728 1779
    Charles Darwin 1809 1882
    Kit Carson 1809 1866
    David Livingstone 1813 1873
    Charles Foucauld 1858 1916
    Ronald Amundsen 1872 1928
    Ernest Shakleton 1874 1922
'''

class Esploratore:
    def __init__(self, nome, cognome, nascita, morte) -> None:
        self.nome = nome
        self.cognome = cognome
        if type(nascita) is str:
            nascita = int(nascita)
        if type(morte) is str:
            morte = int(morte)
        self.nascita = nascita
        self.morte = morte

        if type(nascita) is int and type(morte) is int:
            self.eta = morte-nascita
        else:
            self.eta = None

    def __repr__(self) -> str:
        return f'{self.nome} {self.cognome} {self.nascita} {self.morte} - {self.eta}'

esploratori = []
righe = testo.strip().split('\n')
for riga in righe:
    riga = riga.strip()
    # print(f'"{riga}"')
    nome, cognome, nascita, morte = riga.split()
```

```
    esploratori.append(Esploratore(nome, cognome, nascita, morte))

for esploratore in esploratori:
    print(esploratore)
```

8.1.8 Frazioni

Scrivi e utilizza un insieme di funzioni da utilizzare per fare calcoli con le frazioni. In pratica devono essere definite le funzioni per fare almeno le 4 operazioni (magari anche altre operazioni). Deve anche essere fatta una funzione per effettuare la semplificazione delle frazioni (puoi decidere se creare e usare anche un'altra funzione mcd o utilizzare un altro metodo). Scrivi la definizione della classe frazione e tutte le eventuali altre funzioni in un file separato e poi utilizza quanto definito per fare dei calcoli con le frazioni in un programma.

Soluzione

Separo in due file:

nel file frazione metto la classe Frazione

```
class Frazione:
    def __init__(self, num, den):
        if type(num) == str:
            self.num, self.den = [int(x) for x in num.split("/")]
        else:
            self.num = num
            self.den = den
        self.semplifica()

    def __str__(self):
        if self.den == 1:
            return f"{self.num}"
        else:
            return f"{self.num}/{self.den}"

    def __add__(self, other):
        num = self.num * other.den + other.num * self.den
        den = self.den * other.den
        ris = Frazione(num, den)
        ris.semplifica()
        return ris

    def __sub__(self, other):
        num = self.num * other.den - other.num * self.den
        den = self.den * other.den
        ris = Frazione(num, den)
        ris.semplifica()
        return ris

    def __mul__(self, other):
        num = self.num * other.num
        den = self.den * other.den
        ris = Frazione(num, den)
        ris.semplifica()
        return ris

    def __truediv__(self, other):
        num = self.num * other.den
        den = self.den * other.num
        ris = Frazione(num, den)
        ris.semplifica()
        return ris

    def __neg__(self):
```

```
        return Frazione(-self.num, self.den)


    @staticmethod
    def mcd(a, b):
        a = abs(a)
        b = abs(b)
        ris = a
        while ris != 1:
            if a % ris == 0 and b % ris == 0:
                return ris
            ris -= 1
        return 1

    def semplifica(self):
        mcd = Frazione.mcd(self.num, self.den)
        self.num //= mcd
        self.den //= mcd

        # controllo il segno
        if self.num * self.den < 0:
            segno = -1
        else:
            segno = 1
        self.num = abs(self.num) * segno
        self.den = abs(self.den)
```

nel file main uso la classe

```
from frazione import Frazione
f1 = Frazione(4, 1)
f2 = Frazione(1, 2)

print(f1)
print(f2)
print(f"{f1}+{f2} = {f1+f2}")
print(f"{f1}-{f2} = {f1-f2}")
print(f"{f1}*{f2} = {f1*f2}")
print(f"{f1}/{f2} = {f1/f2}")
print(f"-({f1}) = {-f1}")
```

8.1.9 Studente e media voti

Scrivi un programma in cui è definita una struttura studente. Dello studente ci interessano il nome, i voti presi e la media dei voti. Il programma deve istanziare uno o più studenti e aggiungere una serie di voti casuali (anche mezzi voti ma non tutti i decimali possibili). Voglio creare uno studente e poi usare delle funzioni per aggiungere voti e calcolare la media. Infine stampare lo studente con tutti i suoi dati con una funzione apposita.

8.1.10 Triangolo rettangolo

Definire e utilizzare tutte le funzionalità delle la classe TriangoloRettangolo di seguito descritta:

Il costruttore riceve le misure dei due cateti e autonomamente calcola anche la misura dell'ipotenusa. Definire gli ulteriori metodi che servono a:

1. calcolare l'area
2. calcolare il perimetro
3. stampare il triangolo (solo i valori dei lati chiamandoli cateto1 cateto2 e ipotenusa)

Variante: il costruttore deve essere in grado di costruire il triangolo dati due valori qualsiasi tra cateti e ipotenusa

Soluzione

```
class TriangoloRettangolo:
    def __init__(self, c1, c2) -> None:
        self.c1 = c1
        self.c2 = c2
        self.ipo = ((c1**2)+(c2**2))**(1/2)

    def area(self):
        return self.c1 * self.c2 / 2

    def perimetro(self):
        return self.c1 + self.c2 + self.ipo

    def __repr__(self) -> str:
        return f'cateto1: {self.c1:.2f}, cateto2: {self.c2:.2f}, ipotenusa: {self.ipo:.2f}'

t1 = TriangoloRettangolo(1,1)
t2 = TriangoloRettangolo(3,4)

print(t1, "- Area:", t1.area(), "- Perimetro:", t1.perimetro())
print(t2, "- Area:", t2.area(), "- Perimetro:", t2.perimetro())
```

Soluzione variante

```
class TriangoloRettangolo:
    def __init__(self, c1 = None, c2 = None, ipo = None) -> None:
        if c1 == None:
            if c2 != None and ipo != None:
                self.c1 = ((ipo**2)-(c2**2))**(1/2)
                self.c2 = c2
                self.ipo = ipo
            else:
                raise Exception("Non posso creare un triangolo con meno di 2 lati.")

        elif c2 == None:
            if c1 != None and ipo != None:
                self.c1 = c1
                self.c2 = ((ipo**2)-(c1**2))**(1/2)
                self.ipo = ipo
            else:
                raise Exception("Non posso creare un triangolo con meno di 2 lati.")

        elif ipo == None:
            if c1 != None and c2 != None:
                self.c1 = c1
                self.c2 = c2
                self.ipo = ((c1**2)+(c2**2))**(1/2)
            else:
                raise Exception("Non posso creare un triangolo con meno di 2 lati.")
```

```
        else:
            # if ((c1**2)+(c2**2)) == ipo**2: # potrebbe non funzionare se do valori approssimati
            if abs(((c1**2)+(c2**2)) - ipo**2) < 0.0001:
                self.c1 = c1
                self.c2 = c2
                self.ipo = ipo
            else:
                raise Exception("Questi tre lati non formano un triangolo rettangolo")


    def area(self):
        return self.c1 * self.c2 / 2

    def perimetro(self):
        return self.c1 + self.c2 + self.ipo

    def __repr__(self) -> str:
        return f'cateto1: {self.c1}, cateto2: {self.c2}, ipotenusa: {self.ipo}'

t1 = TriangoloRettangolo(1,1)
t2 = TriangoloRettangolo(3,4,5)
t3 = TriangoloRettangolo(c2=4,ipo=5)
t4 = TriangoloRettangolo(c1=3,ipo=5)
t5 = TriangoloRettangolo(1,1,1.414213562373)

print(t1, "- Area:", t1.area(), "- Perimetro:", t1.perimetro())
print(t2, "- Area:", t2.area(), "- Perimetro:", t2.perimetro())
print(t3, "- Area:", t3.area(), "- Perimetro:", t3.perimetro())
print(t4, "- Area:", t4.area(), "- Perimetro:", t4.perimetro())
print(t5, "- Area:", t5.area(), "- Perimetro:", t5.perimetro())
```

8.1.11 Figure ordinate per area

Definite le classi:

1. TriangoloRettangolo
2. Rettangolo
3. Quadrato
4. Rombo

Svolgi le seguenti operazioni:

1. dichiara una serie di oggetti di queste classi e inseriscile in una lista,
2. ordina la lista in base all'Area(),
3. stampa ogni oggetto della lista.

Soluzione

```
# Definite le classi:
# TriangoloRettangolo
```

```python
# Rettangolo
# Quadrato
# Rombo

# nella funzione principale, dichiarare una serie di oggetti di queste classi, e inserirle in una lista
# Ordinare la lista in base all'Area()
# Invocare il metodo Stampa() su ogni oggetto della lista

import math
import random

class TriangoloRettangolo:
    def __init__(self, params) -> None:
        self.c1 = params[0]
        self.c2 = params[1]
        self.i = math.sqrt(self.c1**2 + self.c2**2)
        self.area = self.Area()

    def Area(self):
        return self.c1*self.c2/2.0

    def Perimetro(self):
        return self.c1 + self.c2 + self.i

    def __repr__(self) -> str:
        return f"Cateto 1: {self.c1:.2f} - Cateto 2: {self.c2:.2f} - Ipotenusa: {self.i:.2f}"


class Rettangolo:
    def __init__(self, params) -> None:
        self.base = params[0]
        self.altezza = params[1]
        self.area = self.Area()

    def Area(self):
        return self.base * self.altezza

    def __repr__(self) -> str:
        return f"Base: {self.base:.2f} - Altezza: {self.altezza:.2f} - Area: {self.area:.2f}"
```

```
class Quadrato:
    def __init__(self, params) -> None:
        self.lato = params[0]
        self.area = self.Area()

    def Area(self):
        return self.lato ** 2

    def __repr__(self) -> str:
        return f"Lato: {self.lato:.2f} - Area: {self.area:.2f}"


class Rombo:
    def __init__(self, params) -> None:
        self.dmaggiore = params[0]
        self.dminore = params[1]
        self.area = self.Area()

    def Area(self):
        return self.dmaggiore * self.dminore / 2.0

    def __repr__(self) -> str:
        return f"Diagonale maggiore: {self.dmaggiore:.2f} - Diagonale minore:
{self.dminore:.2f} - Area: {self.area:.2f}"


class FiguraGeometrica:
    tipologie = [(TriangoloRettangolo, 2), (Rettangolo, 2), (Quadrato, 1), (Rombo, 2)]


# elenco = [Quadrato(3), Rettangolo(2,6), Rombo(2, 3), TriangoloRettangolo(2, 4),
Quadrato(4)]
elenco = []
for i in range(5):
    figura, parametri = random.choice(FiguraGeometrica.tipologie)
    parametri = [random.random() * 10 for _ in range(parametri)]
    elenco.append(figura(parametri))

for el in elenco:
    print(el)
print("\n")
```

```
print(type(elenco[0].area))

elenco.sort(key = lambda el: el.area)
for el in elenco:
    print(el)
print("\n")

print("ciao"+ str(elenco[0]))
```

8.1.12 Classe Orario

Definisci e utilizza una classe orario che permetta di:

1. Istanziare orari (ad esempio 8:25)
2. Stampare orari
3. Confrontare orari (<, <=, >, >=, ==)
4. Calcolare differenze di tempo tra un orario ed un altro in due modi:
    a. Numero di minuti (solo minuti ad esempio 122 minuti)
    b. Numero di ore e minuti (misto ad esempio 2 h 2 m che equivale a 122 minuti)

Soluzione

```
class Orario:
    def __init__(self, h = 0, m = 0) -> None:
        # potrei mettere dei controlli su tipo e valore di h e m,
        # ma per ora mi limito a fare il modulo
        self.h = h % 24
        if m >= 60:
            self.h += m // 60
        if m < 0:
            self.h -= (-m) // 60 + 1

        if m >= 0:
            self.m = m % 60
        else:
            self.m = 60 - (-m) % 60

    @staticmethod
    def orario_da_minuti(self, minuti):
        h = minuti // 60
        m = minuti % 60
        return Orario(h, m)

    @staticmethod
    def orario_da_stringa(self, stringa):
        h, m = stringa.split(':')
        return Orario(int(h), int(m))

    def minuti_totali(self):
        return self.h * 60 + self.m

    def __str__(self) -> str:
        return f"{self.h:02d}:{self.m:02d}"

    def __lt__(self, b):
        return (self.h < b.h) or (self.h == b.h and self.m < b.m)
```

```
    def __le__(self, b):
        return (self.h < b.h) or (self.h == b.h and self.m <= b.m)

    def __gt__(self, b):
        return (self.h > b.h) or (self.h == b.h and self.m > b.m)

    def __ge__(self, b):
        return (self.h > b.h) or (self.h == b.h and self.m >= b.m)

    def __eq__(self, b):
        return self.h == b.h and self.m == b.m

    def __sub__(self, b):
        # return self.h * 60 + self.m - b.h * 60 - b.m
        return Orario.orario_da_minuti(self, self.minuti_totali() - b.minuti_totali())

    def __add__(self, b):
        return Orario(self.h + b.h, self.m + b.m)

    def meno(self, b):
        h = self.h-b.h
        m = self.m-b.m
        if m < 0:
            h -= 1
        return Orario(h, m)

prova = Orario(8, 25)
print("Orario(8, 25):", prova)
prova = Orario(24, 15)
print("Orario(24, 15):", prova)
prova = Orario(8, -25)
print("Orario(8, -25):", prova)
prova = Orario.orario_da_stringa("8:25")
print("Orario da stringa('8:25'):", prova)

print("-----------------------------")

o1 = Orario()
o2 = Orario(24,15)

print(o1)
print(o2)

print(f"{o1} <  {o2}: {o1<o2}")
print(f"{o1} <= {o2}: {o1<=o2}")
print(f"{o1} <= {o1}: {o1<=o1}")
print(f"{o1} >  {o2}: {o1>o2}")
print(f"{o1} >= {o2}: {o1>=o2}")
print(f"{o1} == {o2}: {o1==o2}")
print(f"{o1} == {o1}: {o1==o1}")
print(f"{o1} - {o2} = {o1-o2}")
print(f"{o2} - {o1} = {o2-o1}")

print(f"{o1} meno {o2} = {o1.meno(o2)}")
```

8.1.13 Orari dei treni

Dato il seguente file:

```
Milano-Centrale Bologna 7:30 8:34
Milano-Centrale Genova-Principe 8:05 9:44
Milano-Centrale Torino 7:30 8:30
Milano-Centrale Piacenza 8:01 8:53
Milano-Centrale Chiasso 8:10 9:10
```

```
Milano-Centrale Brescia 8:15 8:51
Milano-Centrale Verona 8:45 9:58
Milano-Centrale Varese 8:05 9:00
Brescia Venezia 8:13 9:52
Voghera Torino 7:57 9:25
```

O, se non sai leggere da file, la seguente lista di stringhe

```
listaTreni = [
    "Milano-Centrale Bologna 7:30 8:34",
    "Milano-Centrale Genova-Principe 8:05 9:44",
    "Milano-Centrale Torino 7:30 8:30",
    "Milano-Centrale Piacenza 8:01 8:53",
    "Milano-Centrale Chiasso 8:10 9:10",
    "Milano-Centrale Brescia 8:15 8:51",
    "Milano-Centrale Verona 8:45 9:58",
    "Milano-Centrale Varese 8:05 9:00",
    "Brescia Venezia 8:13 9:52",
    "Voghera Torino 7:57 9:25"
]
```

Definire una classe ViaggioTreno il cui costruttore (init) riceve una delle stringhe contenuta in listaTreni. Definisci anche una classe orario per gestire meglio gli orari.

Esegui poi le seguenti operazioni:

1. Stampa tutti i treni, in base alla durata del tragitto
2. Stampa tutte le stazioni raggiungibili da Milano entro un'ora
3. Stampa il numero di stazioni presenti nella lista dei treni, ed infine stampare tutte le stazioni ordinate in base al nome

Soluzione

```python
# data la seguente lista di stringhe

listaTreni = [
    "Milano-Centrale Bologna  7:30  8:34", "Milano-Centrale Genova-Principe  8:05  9:44",
    "Milano-Centrale Torino  7:30  8:30", "Milano-Centrale Piacenza  8:01  8:53",
    "Milano-Centrale Chiasso  8:10  9:10", "Milano-Centrale Brescia  8:15  8:51",
    "Milano-Centrale Verona  8:45  9:58", "Milano-Centrale Varese  8:05  9:00",
    "Brescia Venezia 8:13  9:52", "Voghera Torino 7:57  9:25"]

# Definire una classe ViaggioTreno che riceve nella __init__  una stringa di listaTreni

class Orario:
    def __init__(self, ore, min) -> None:
        self.ore = ore
        self.min = min
        self.totmin = ore*60+min

    def __le__(self, altro):
        return self.ore < altro.ore or (self.ore == altro.ore and self.min <= altro.min)

    def __repr__(self) -> str:
        return f"{self.ore}:{self.min:02d}"

    def __sub__(self, altro):
        return self.totmin - altro.totmin
```

```python
class ViaggioTreno:
    def __init__(self, info) -> None:
        info = info.split()
        # print(info)
        self.partenza = info[0]
        self.arrivo = info[1]
        ore, min = map(int, info[2].split(":"))
        self.orario_partenza = Orario(ore, min)
        ore, min = map(int, info[3].split(":"))
        self.orario_arrivo = Orario(ore, min)
        self.durata = self.orario_arrivo - self.orario_partenza

    def __repr__(self) -> str:
        return f"{self.partenza} {self.arrivo} {self.orario_partenza} {self.orario_arrivo}"

viaggi = [ViaggioTreno(info) for info in listaTreni]
# print(viaggi[0])

# 1) Stampare tutti i treni, in base alla durata del tragitto
viaggi.sort(key = lambda el: el.durata)
for viaggio in viaggi:
    print(viaggio)

# 2) Stampare tutte le stazioni raggiungibili da Milano entro un'ora
print("\n\nTutte le stazioni raggiungibili da Milano entro un'ora")
ris = [(el.arrivo, el.durata) for el in viaggi if el.durata <= 60 and "Milano" in el.partenza]
for viaggio in ris:
    print(viaggio)

# 3) Stampare il numero di stazioni presenti in listaTreni, ed infine stampare tutte le stazioni ordinate in base al nome
print("\n\nElenco di stazioni ordinate per nome e numero di stazioni presenti in listaTreni")
elenco_stazioni = []
for viaggio in viaggi:
    if viaggio.partenza not in elenco_stazioni:
        elenco_stazioni.append(viaggio.partenza)
    if viaggio.arrivo not in elenco_stazioni:
        elenco_stazioni.append(viaggio.arrivo)

print(sorted(elenco_stazioni))
print(len(elenco_stazioni))
```

8.1.14 Classifica marcatori

La seguente lista, rappresenta i principali goleador del campionato Italiano

```
listaBomber = [
   "Dusan Vlahovic Serbia 24 gol (5) rigori Fiorentina",
   "Gianluca Scamacca Italia 16 gol (1) rigore Sassuolo",
   "Rafael Leao Portogallo 11 gol Milan",
   "Dries Mertens Belgio 11 gol Napoli",
   "Norberto Beto Portogallo 11 gol Udinese",
   "Andrea Pinamonti Italia 13 gol (4) rigori Empoli",
```

```
    "Ciro Immobile Italia 27 gol (7) rigori Lazio",
    "Mario Arnautovic Austria 14 gol (2) rigori Bologna",
    "Lautaro Martinez Argentina 21 gol (3) rigori Inter",
    "Tammy Abraham Inghilterra 17 gol (3) rigori Roma",
    "Domenico Berardi Italia 15 gol (5) rigori Sassuolo",
    "Victor Osimhen Nigeria 14 gol Napoli",
    "Gerard Deulofeu Spagna 13 gol (1) rigore Udinese",
    "Edin Dzeko Bosnia 13 gol Inter",
    "Antonin Barak RepCeca 11 gol (4) rigori Verona",
    "Joao Pedro Brasile 13 gol (2) rigori Cagliari",
    "Mario Pasalic Croazia 13 gol Atalanta",
    "Giovanni Simeone Argentina 17 gol Verona",
    "Gianluca Caprari Italia 12 gol (2) rigori Verona",
    "Lorenzo Insigne Italia 11 gol (9) rigori Napoli"
]
```

Qui di seguito invece la lista delle sole squadre:

```
team = "Milan Inter Napoli Juventus Lazio Roma Fiorentina Atalanta Verona Torino
Sassuolo Udinese Bologna Empoli Sampdoria Spezia Salernitana Cagliari Genoa Venezia"
```

Esegui le seguenti operazioni:

1. Definisci una classe giocatore il cui costruttore riceve una stringa tratta da ListaBomber,
2. Crea una lista di giocatori leggendo tutte le stringhe contenute in ListaBomber
3. Stampa la classifica dei goleador in base al numero di gol segnati
4. stampa i nomi dei primi 3 rigoristi
5. stampa quali squadre hanno almeno due giocatori tra i goleador
6. stampa la classifica dei goleador italiani
7. stampa tutte le squadre che non hanno neanche un goleador nell'elenco
8. stampa la classifica dei goleador contando solo i gol fatti su azione (e non quelli su rigore)

8.1.15 Scelte universitarie

Data la seguente lista:

```
sceltaUniversitaria2018 = [
   "Banfi", "Vimercate", "LSA", 48,
   ["Ingegneria", 40.7, "Scientifica", 23, "Economica", 10.4, "Sociale", 7.4,
   "Giuridico-Politica", 5.2, "Umanistica", 5.2, "Sanitaria", 4.4, "Medica", 3,
   "Scienze Motorie", 0.3],
   "Frisi", "Monza", "LSA", 58,
   ["Ingegneria", 53, "Scientifica", 17.9, "Economica", 12.5, "Sociale", 3,
   "Giuridico-Politica", 2.4, "Umanistica", 1, "Sanitaria", 4.8, "Medica", 5.4,
   "Scienze Motorie", 0],
   "Agnesi", "Merate", "LSA", 41,
   ["Ingegneria", 46.5, "Scientifica", 28.9, "Economica", 10.5, "Sociale", 2.8,
   "Giuridico-Politica", 2.4, "Umanistica", 3.5, "Sanitaria", 2.6, "Medica", 2.6,
   "Scienze Motorie", 2.6],
   "Maiorana", "Desio", "LSA", 56,
   ["Ingegneria", 41.8, "Scientifica", 35.3, "Economica", 7.2, "Sociale", 3.9,
   "Giuridico-Politica", 1.3, "Umanistica", 1.3, "Sanitaria", 7.2, "Medica", 1.3,
   "Scienze Motorie", 0.7],
   "Banfi", "Vimercate", "LS", 97,
   ["Ingegneria", 30.7, "Scientifica", 18.7, "Economica", 18.4, "Sociale", 7.4,
   "Giuridico-Politica", 5.3, "Umanistica", 7.1, "Sanitaria", 6.4, "Medica", 4.2,
   "Scienze Motorie", 0.7],
   "Frisi", "Monza", "LS", 123,
```

```
   ["Ingegneria", 39, "Scientifica", 13.1, "Economica", 20.3, "Sociale", 4.2,
   "Giuridico-Politica", 6.1, "Umanistica", 4.2, "Sanitaria", 1.7, "Medica", 10.9,
   "Scienze Motorie", 0.5],
   "Agnesi", "Merate", "LS", 41,
   ["Ingegneria", 25.4, "Scientifica", 22.3, "Economica", 18, "Sociale", 3.4,
   "Giuridico-Politica", 5.9, "Umanistica", 6.8, "Sanitaria", 8.4, "Medica", 8.7,
   "Scienze Motorie", 1.1],
   "Maiorana", "Desio", "LS", 118,
   ["Ingegneria", 32.2, "Scientifica", 21.9, "Economica", 15.4, "Sociale", 5.6,
   "Giuridico-Politica", 3.6, "Umanistica", 7.7, "Sanitaria", 7.1, "Medica", 5.9,
   "Scienze Motorie", 0.6],
   "Banfi", "Vimercate", "Classico", 24,
   ["Ingegneria", 8.6, "Scientifica", 7.1, "Economica", 11.4, "Sociale", 12.9,
   "Giuridico-Politica", 25.7, "Umanistica", 30, "Sanitaria", 1.4, "Medica", 2.9,
   "Scienze Motorie", 0],
   "Zucchi", "Monza", "Classico", 121,
   ["Ingegneria", 13.8, "Scientifica", 16.1, "Economica", 16.7, "Sociale", 8.5,
   "Giuridico-Politica", 17.9, "Umanistica", 21.7, "Sanitaria", 1.5, "Medica", 3.8,
   "Scienze Motorie", 0.5],
]
```

Definire una classe CorsoStudi e una listaCorsi[] dove travasare le informazioni presenti in sceltaUniversitaria2018

Esegui poi le seguenti operazioni:

1. Stampare il numero totale di studenti per ognuno degli istituti presenti;
2. Costruire un nuovo oggetto della classe CorsoStudi, chiamandolo corsoLsa dove riversare le medie pesate (tenendo conto delle percentuali, ma anche del numero di diplomati) dei vari corsi di scienze applicate, ripetere la stessa cosa per classico e tradizionale;
3. Stampare la classifica dei corsi, in base alla somma delle percentuali di "Ingegneria" più "Scientifica";
4. Stampare la classifica delle scuole, valutando soltanto la somma degli iscritti tra scientifico e scienze applicate;
5. Stampare per ogni indirizzo universitario "Ingegneria Scientifica Economica …), il corso scolastico con la più alta percentuale.

8.1.16 Finali di Wimbledon

La seguente lista, associa a ogni tennista la sua nazionalità

```
listaPlayer= [
   "Roger Federer (Svizzera)", "Mark Philippoussis (Australia)",  "Andy Roddick
   (Usa)", "Rafael Nadal (Spagna)",
   "Tomas Berdich (RepCeca)", "Novak Djokovic (Serbia)", "Andy Murray (RegnoUnito)",
   "Milos Raonic (Canada)",
   "Marin Cilic (Croazia)", "Kevin Anderson (Sudafrica)", "Matteo Berrettini
   (Italia)", "Nick Kyrgyos (Australia)"
]
```

La seguente Lista, descrive le finali di Wimbledon:

```
wimbledon = [
   "2003 Roger Federer - Mark Philippoussis",  "7-6 6-2 7-6",
   "2004 Roger Federer - Andy Roddick",  "4-6 7-5 7-6 6-4",
   "2005 Roger Federer - Andy Roddick",  "6-2 7-6 6-4",
   "2006 Roger Federer - Rafael Nadal",  "6-0 7-6 6-7 6-3",
   "2007 Roger Federer - Rafael Nadal",  "7-6 4-6 7-6 2-6 6-2",
   "2008 Rafael Nadal - Roger Federer",  "7-6 4-6 7-6 2-6 6-2",
   "2009 Roger Federer - Andy Roddick",  "6-2 7-6 6-4",
```

```
    "2010 Rafael Nadal - Tomas Berdich",  "6-3 7-5 6-4",
    "2011 Novak Djokovic  -  Rafael Nadal",  "6-4 6-1 1-6 6-3",
    "2012 Roger Federer - Andy Murray",  "4-6 7-5 6-3 6-4",
    "2013 Andy Murray - Novak Djokovic",  "6-4 7-5 6-4",
    "2014 Novak Djokovic - Roger Federer",  "6-7 6-4 7-6 5-7 6-4",
    "2015 Novak Djokovic - Roger Federer",  "7-6 6-7 6-4 6-3",
    "2016 Andy Murray - Milos Raonic",  "6-4 7-6 7-6",
    "2017 Roger Federer - Marin Cilic",  "6-3 6-1 6-4",
    "2018 Novak Djokovic - Kevin Anderson",  "6-2 6-2 7-6",
    "2019 Novak Djokovic - Roger Federer",  "7-6 1-6 7-6 4-6 13-12",
    "2021 Novak Djokovic - Matteo Berrettini",  "6-7 6-4 6-4 6-3",
    "2022 Novak Djokovic - Nick Kyrgios",  "4-6 6-3 6-4 7-6",
]
```

Il tuo programma deve svolgere i seguenti compiti:

1. Riversare le informazioni di listaPlayer in un una lista di oggetti Player
2. Stampare la nazione ha almeno due player che hanno disputato una finale di Wimbledon
3. Stampare l'anno in cui non è stato disputato il torneo di Wimbledon
4. Stampare l'anno del quinto torneo vinto dai giocatori che hanno vinto Wimbledon almeno 5 volte
5. Definire una classe Finale, che descrive il match di una finale
6. Stampare la classifica dei tre match più combattuti
7. Stampare la classifica dei giocatori che hanno vinto più volte Wimbledon
8. Stampare la classifica dei giocatori che hanno giocato più finali di Wimbledon
9. Stampare il nome del giocatore che ha vinto più finali consecutive di Wimbledon e quante ne ha vinte, e da che anno a che anno

Soluzione

```
listaPlayer= [
"Roger Federer (Svizzera)", "Mark Philippoussis (Australia)",  "Andy Roddick (Usa)", "Rafael Nadal (Spagna)",
"Tomas Berdich (RepCeca)", "Novak Djokovic (Serbia)", "Andy Murray (RegnoUnito)", "Milos Raonic (Canada)",
"Marin Cilic (Croazia)", "Kevin Anderson (Sudafrica)", "Matteo Berrettini (Italia)", "Nick Kyrgyos (Australia)"
]
wimbledon = [
"2003 Roger Federer - Mark Philippoussis",  "7-6 6-2 7-6",
"2004 Roger Federer - Andy Roddick",  "4-6 7-5 7-6 6-4",
"2005 Roger Federer - Andy Roddick",  "6-2 7-6 6-4",
"2006 Roger Federer - Rafael Nadal",  "6-0 7-6 6-7 6-3",
"2007 Roger Federer - Rafael Nadal",  "7-6 4-6 7-6 2-6 6-2",
"2008 Rafael Nadal - Roger Federer",  "7-6 4-6 7-6 2-6 6-2",
"2009 Roger Federer - Andy Roddick",  "6-2 7-6 6-4",
"2010 Rafael Nadal - Tomas Berdich",  "6-3 7-5 6-4",
"2011 Novak Djokovic  -  Rafael Nadal",  "6-4 6-1 1-6 6-3",
"2012 Roger Federer - Andy Murray",  "4-6 7-5 6-3 6-4",
"2013 Andy Murray - Novak Djokovic",  "6-4 7-5 6-4",
"2014 Novak Djokovic - Roger Federer",  "6-7 6-4 7-6 5-7 6-4",
"2015 Novak Djokovic - Roger Federer",  "7-6 6-7 6-4 6-3",
"2016 Andy Murray - Milos Raonic",  "6-4 7-6 7-6",
"2017 Roger Federer - Marin Cilic",  "6-3 6-1 6-4",
"2018 Novak Djokovic - Kevin Anderson",  "6-2 6-2 7-6",
```

```
"2019 Novak Djokovic - Roger Federer",  "7-6 1-6 7-6 4-6 13-12",
"2021 Novak Djokovic - Matteo Berrettini",  "6-7 6-4 6-4 6-3",
"2022 Novak Djokovic - Nick Kyrgios",  "4-6 6-3 6-4 7-6",
]

# 1.    Riversare le informazioni di listaPlayer in un una lista di oggetti
# Player

class Player:
    def __init__(self, nome, cognome, nazione):
        self.nome = nome
        self.cognome = cognome
        self.nazione = nazione

    def __repr__(self):
        return f"{self.nome} {self.cognome} ({self.nazione})"

giocatori = []
for el in listaPlayer:
    el = el.split()
    giocatori.append(Player(el[0], el[1], el[2][1:-1]))
    print(giocatori[-1])

# 5.    Definire una classe Finale, che descrive il match di una finale
class Finale:
    def __init__(self, anno, giocatore1, giocatore2, punteggi):
        self.anno = anno
        self.giocatore1 = giocatore1
        self.giocatore2 = giocatore2
        self.sets = []
        for ris in punteggi.split():
            s = ris.split('-')
            s = [int(num) for num in s]
            self.sets.append(s)

    def __repr__(self):
        ris = f"{self.anno} {self.giocatore1} - {self.giocatore2}, "
        lista = [f"{s[0]}-{s[1]}" for s in self.sets]
        ris += " ".join(lista)
        return ris

    def vincitore(self):
        p1 = 0
        p2 = 0
        for s in self.sets:
            if s[0] > s[1]:
                p1 += 1
            else:
                p2 += 2
        if p1 > p2:
            return self.giocatore1
        else:
```

```
            return self.giocatore2


finali = []
for i in range(0,len(wimbledon),2):
    # "2003 Roger Federer - Mark Philippoussis",  "7-6 6-2 7-6",
    nomi = wimbledon[i].split('-')
    primaparte = nomi[0].split()
    anno = int(primaparte[0])
    nome1 = primaparte[1].strip() + " " + primaparte[2].strip()
    secondaparte = nomi[1].split()
    nome2 = secondaparte[0].strip() + " " + secondaparte[1].strip()
    finali.append(Finale(anno, nome1, nome2, wimbledon[i+1]))
    print(finali[-1])

# 2.    Quale nazione ha almeno due player che hanno disputato una finale di
# wimbledon
nazioni = {}
for giocatore in giocatori:
    nomecognome = giocatore.nome + " " + giocatore.cognome
    for finale in finali:
        if finale.giocatore1 == nomecognome or finale.giocatore2 == nomecognome:
            if giocatore.nazione in nazioni:
                nazioni[giocatore.nazione] += 1
            else:
                nazioni[giocatore.nazione] = 1

print("Nazioni con almeno due giocatori che hanno disputato una finale di Wimbledon")
for nazione, val in nazioni.items():
    if val >= 2:
        print("-", nazione)

# 3.    In quale anno non è stato disputato il torneo di Wimbledon
print("Anni in cui non è stato disputato il torneo di Wimbledon: ")
for i in range(len(finali)-1):
    if finali[i+1].anno - finali[i].anno > 1:
        print("-", finali[i].anno+1)

# 4.    L'anno del quinto torneo vinto, dai giocatori che hanno vinto Wimbledon
# almeno 5 volte
vittorie = {}
for finale in finali:
    vincitore = finale.vincitore()
    if vincitore in vittorie:
        vittorie[vincitore] += 1
    else:
        vittorie[vincitore] = 1
    if vittorie[vincitore] == 5:
        print(f"{vincitore} ha vinto il suo quinto titolo a Wimbledon nel {finale.anno}")

# 6.    Stampare  la classifica dei tre match più combattuti
durate = []
```

```python
for finale in finali:
    games = 0
    for s in finale.sets:
        games += s[0] + s[1]
    durate.append([finale, games])
    print(durate[-1])
durate.sort(key = lambda x:-x[1])
for x in durate[:3]:
    print(x)

# 7.    Stampare la classifica dei giocatori che hanno vinto più volte wimbledon
listavittorie = list(vittorie.items())
listavittorie.sort(key=lambda x: (-x[1], x[0]))
print("Classifica vincitori:")
for i, el in enumerate(listavittorie):
    print(f"{i+1}. {el[0]} con {el[1]} vittorie." )

# 8.    Stampare la classifica dei giocatori che hanno giocato più finali di
# wimbledon)
numFinali = {}
for finale in finali:
    if finale.giocatore1 in numFinali:
        numFinali[finale.giocatore1] += 1
    else:
        numFinali[finale.giocatore1] = 1
    if finale.giocatore2 in numFinali:
        numFinali[finale.giocatore2] += 1
    else:
        numFinali[finale.giocatore2] = 1

listaNumFinali = list(numFinali.items())
listaNumFinali.sort(key=lambda x: (-x[1], x[0]))
print("Classifica delle finali disputate:")
for i, el in enumerate(listaNumFinali):
    print(f"{i+1}. {el[0]} con {el[1]} finali disputate.")


# 9.    Stampare il nome del giocatore che ha vinto più finali consecutive di
# Wimbledon e quante ne ha vinte, e da che anno a che anno

best = finali[0].vincitore()
numbest = 1
# print(best)
if len(finali) > 1:
    i = 0
    while i < len(finali):
        nome = finali[i].vincitore()
        num = 1
        while i < len(finali)-1 and finali[i+1].vincitore() == nome:
            num += 1
            i += 1
        if num > numbest:
            best = nome
```

```
            numbest = num
        i += 1
print(f"Il giocatore che ha vinto più finali vinte consecutivamente è {best} con {numbest} finali vinte consecutivamente")
```

8.1.17 Ore lavorate con classi

I seguenti dati, che dovrai inserire in un file di testo, descrivono quante ore hanno lavorato in una settimana, una serie di dipendenti: ad esempio Mr White ha lavorato 8 ore il lunedì, 9 ore il martedì, 8 ore il mercoledì, 9 ore il giovedì e 4 ore il venerdì

```
Mr White: 8, 9, 8, 9, 4
Mr Brown: 0, 8, 7, 10, 8
Mr Blonde: 8, 8, 8, 9, 9
Mr Black: 6, 9, 9, 8, 8
Mr Red: 8, 7, 8, 8, 8
Mr Green: 4, 8, 8, 8, 4
```

1. Stampare la classifica dei lavoratori, in base al numero di ore lavorate
2. Stampare il giorno con più ore lavorate, e quante ore sono state lavorate in quel giorno
3. Svolgi l'esercizio definendo la classe lavoratore e creando una lista di elementi lavoratore

Soluzione

```
class Lavoratore:
    def __init__(self, nome, ore):
        self.nome = nome
        self.ore = ore

    def __repr__(self):
        numeri = ", ".join([str(ora) for ora in self.ore])
        return f"{self.nome}: {numeri}"

    def __lt__(self, other):
        return sum(self.ore) < sum(other.ore)


dati = []
with open("1-15.txt", "r", encoding="utf-8") as f:
    for riga in f:
        riga = riga.strip().split(":")
        nome = riga[0]
        numeri = [int(num.strip()) for num in riga[1].split(',')]
        dati.append(Lavoratore(nome, numeri))
        # print(dati[-1])

# 1.    Stampare la classifica dei lavoratori, in base al numero di ore lavorate
dati.sort(reverse=True)
for dato in dati:
    print(dato)

# 2.    Stampare il giorno con più ore lavorate, e quante ore sono state lavorate
```

```
# in quel giorno.
giorni = [0 for _ in range(5)]
for dato in dati:
    for i in range(len(dato.ore)):
        giorni[i] += dato.ore[i]

imax = 0
for i in range(len(giorni)):
    giorni[i] /= len(dati)
    if giorni[i] > giorni[imax]:
        imax = i

nomigiorni = ['Lunedì', 'Martedì', 'Mercoledì', 'Giovedì', 'Venerdì']
print(f"Il giorno un cui si sono lavorate più ore è {nomigiorni[imax]}")
```

8.2 Esercizi con Ereditarietà e Polimorfismo

8.2.1 Studenti e docenti

Definisci le seguenti classi:

1. Persona che ha gli attributi nome, cognome
2. Studente, che è una persona e ha anche la matricola e la classe di appartenenza
3. Docente che è una persona e ha anche la lista delle materie insegnate in ogni classe

Genera una lista di studenti e di docenti, poi scrivi e utilizza una funzione che data la matricola di uno studente restituisce la lista dei suoi docenti.

Variante comod: copia dalla soluzione solo la parte in cui si creano i dati e tu vai a definire classi e metodi e funzioni.

Soluzione

```
class Persona:
    def __init__(self, nome, cognome):
        self.nome = nome
        self.cognome = cognome

    def __str__(self):
        return f"{self.nome} {self.cognome}"

class Studente(Persona):
    def __init__(self, nome, cognome, matricola, classe):
        super().__init__(nome, cognome)
        self.matricola = matricola
        self.classe = classe

    def __repr__(self):
        return f"{super().__str__()}, {self.matricola}, {self.classe}"

class Docente(Persona):
    def __init__(self, nome, cognome, classi):
        super().__init__(nome, cognome)
        self.classi = classi

    def __repr__(self):
        ris = ""
        ris += super().__str__()
        ris += "\n"
        for classe in self.classi.items():
            ris += f" - {classe[0]}: {', '.join(classe[1])}\n"
        return ris

# Genera una lista di studenti e di docenti, poi scrivi e utilizza una funzione che
# data la matricola di uno studente restituisce la lista dei suoi docenti.
def funzione(studenti, docenti, matricola):
    classe = None
    for studente in studenti:
        if studente.matricola == matricola:
            classe = studente.classe

    if classe == None:
        return []

    ris = []
    for docente in docenti:
        if classe in docente.classi:
            ris.append(docente)
```

```
    return ris


studenti = []
studenti.append(Studente("Matteo", "Piazza", "14", "3Q"))
studenti.append(Studente("Guglielmo", "Martinelli", "10", "3Q"))
studenti.append(Studente("Diletta", "Bizzaro", "6", "1P"))
studenti.append(Studente("Gabriele", "Rossetti", "11", "5O"))

docenti = []
docenti.append( Docente("Francesco", "Tormene",
                        {"1P": ["Informatica", "Educazione civica"],
                         "3Q": ["Informatica", "Educazione civica"],
                         "5O": ["Informatica", "Educazione civica"]
                        }))
docenti.append( Docente("Fernanda", "Amodeo",
                        {"1P": ["Scienze motorie", "Educazione civica"],
                         "3Q": ["Scienze motorie", "Educazione civica"],
                         }))


# for studente in studenti:
#     print(studente)

# for docente in docenti:
#     print(docente)

# print()
# print()

print("Docenti di Matteo Piazza")
print(funzione(studenti, docenti, "14"))
print()
print("Docenti di Gabriele Rossetti")
print(funzione(studenti, docenti, "11"))
```

8.2.2 Veicoli

definisci le classi Veicolo, Auto, Moto e Camion. Veicolo è la classe da cui derivano le altre. ogni classe ha, oltre a marca e modello anche una funzione muovi specifica per ogni classe. Nella classe Veicolo definisci la classe muovi ma non implementarla. Ogni classe ha anche una funzione __repr__ specifica per rappresentare tutti i dati della classe. La classe Auto ha anche il numero di porte, la classe Moto la cilindrata, il Camion la capacità di carico.

Soluzione

```
# Classe base
class Veicolo:
    def __init__(self, marca, modello):
        self.marca = marca
        self.modello = modello

    def muovi(self):
        raise NotImplementedError("Il metodo muovi() deve essere implementato dalla
sottoclasse.")

# Classe derivata per Auto
```

```
class Auto(Veicolo):
    def __init__(self, marca, modello, numero_porte):
        super().__init__(marca, modello)
        self.numero_porte = numero_porte

    def muovi(self):
        print(f"L'auto {self.marca} {self.modello} sta viaggiando su strada.")

# Classe derivata per Moto
class Moto(Veicolo):
    def __init__(self, marca, modello, cilindrata):
        super().__init__(marca, modello)
        self.cilindrata = cilindrata

    def muovi(self):
        print(f"La moto {self.marca} {self.modello} sta viaggiando su strada.")

# Classe derivata per Camion
class Camion(Veicolo):
    def __init__(self, marca, modello, capacita_carico):
        super().__init__(marca, modello)
        self.capacita_carico = capacita_carico

    def muovi(self):
        print(f"Il camion {self.marca} {self.modello} sta percorrendo la strada con un carico di
{self.capacita_carico} tonnellate.")


# Creazione di istanze di vari veicoli
veicoli = []
veicoli.append(Auto("Fiat", "500", 5))
veicoli.append(Moto("Yamaha", "MT-09", 847))
veicoli.append(Camion("Volvo", "FH16", 25))

# Chiamata del metodo muovi() su diversi tipi di veicoli
for veicolo in veicoli:
    veicolo.muovi()
```

8.2.3 Figure geometriche

Definisci la classe figura geometrica (piana) e poi definisci le classi figlie triangolo, quadrilatero, quadrato, triangolo_equilatero, triangolo_rettangolo. Istanzia poi una serie di figure geometriche e stampa per ognuna il valore del perimetro e dell'area utilizzando le specifiche funzioni definite per le classi.

Soluzione

```
class Figura:
    def __init__(self, lati = []):
        if type(lati) != list:
            raise TypeError("La classe deve ricevere una lista di lati")
        self.lati = lati

    def perimetro(self):
```

```
        return sum(self.lati)

    def __repr__(self):
        ris = "Figura con lati "
        ris += ", ".join([str(lato) for lato in self.lati])
        return ris

    def area(self):
        raise NotImplementedError("Il metodo perimetro() deve essere implementato dalla
sottoclasse.")

class Triangolo(Figura):
    def __init__(self, lati=[]):
        super().__init__(lati)
        if len(lati) != 3:
            raise ValueError("Il triangolo deve avere tre lati")
        ordinati = sorted(self.lati)
        # print(ordinati)
        if ordinati[2] >= ordinati[0] + ordinati[1]:
            raise ValueError("Il triangolo è degenere")

    def __repr__(self):
        ris = "Triangolo con lati "
        ris += ", ".join([str(lato) for lato in self.lati])
        return ris

    def area(self):
        p = sum(self.lati) / 2
        return f"{(p*(p-self.lati[0])*(p-self.lati[1])*(p-self.lati[2]))**0.5}"

class Quadrilatero(Figura):
    def __init__(self, lati=[]):
        super().__init__(lati)
        if len(lati) != 4:
            raise ValueError("Un quadrilatero deve avere quattro lati")

    def __repr__(self):
        ris = "Quadrilatero con lati "
        ris += ", ".join([str(lato) for lato in self.lati])
        return ris


class Quadrato(Quadrilatero):
    def __init__(self, lati=[]):
        super().__init__(lati)
        ordinati = sorted(self.lati)
        if ordinati[0] != ordinati[3]:
            raise ValueError("Un quadrato deve avere tutti i lati congruenti")

    def __repr__(self):
        ris = "Quadrato con lati "
        ris += ", ".join([str(lato) for lato in self.lati])
        return ris
```

```
    def area(self):
        return str(self.lati[0]**2)

class Triangolo_equilatero(Triangolo):
    def __init__(self, lati=[]):
        super().__init__(lati)
        ordinati = sorted(self.lati)
        if ordinati[0] != ordinati[2]:
            raise ValueError("Un triangolo equilatero deve avere tutti i lati congruenti")

    def __repr__(self):
        ris = "Triangolo equilatero con lati "
        ris += ", ".join([str(lato) for lato in self.lati])
        return ris

class Triangolo_rettangolo(Triangolo):
    def __init__(self, lati=[]):
        super().__init__(lati)
        ordinati = sorted(self.lati)
        if ordinati[0]**2 + ordinati[1]**2 != ordinati[2]**2:
            raise ValueError("Un triangolo rettangolo deve avere i lati le cui misure formano una
terna pitagorica")

    def __repr__(self):
        ris = "Triangolo rettangolo con lati "
        ris += ", ".join([str(lato) for lato in self.lati])
        return ris

figure = []
figure.append(Triangolo([1,2,1.5]))
figure.append(Quadrato([2,2,2,2]))
figure.append(Triangolo([2,2,3]))
figure.append(Quadrato([1,1,1,1]))
figure.append(Triangolo_equilatero([2,2,2]))
figure.append(Triangolo_rettangolo([3,4,5]))

for figura in figure:
    print(f"Perimetro di {figura}: {figura.perimetro()}")
    print(f"Area di {figura}: {figura.area()}")
```

9 Strutture dati avanzate

9.1 Pile / Stack

9.1.1 Frase al contrario con pila

Scrivi un programma che legga da input una frase e che utilizzando una pila stampi le lettere della frase al contrario

Soluzione 1

Soluzione usando una lista come pila

```
frase = "Ciao Ryan come stai?"

pila = []
for lettera in frase:
    pila.append(lettera)
invertita = ''
while len(pila) > 0:
    invertita += pila[-1]
    pila.pop()

print(invertita)
```

Soluzione 2

Soluzione che prevede l'utilizzo della classe pila.

Classe Pila:

```
class Pila:
    def __init__(self) -> None:
        self.dati = []

    def push(self, dato):
        self.dati.append(dato)

    def pop(self):
        self.dati.pop()

    def top(self):
        if len(self.dati) > 0:
            return self.dati[-1]
        else:
            return None

    def empty(self):
        return len(self.dati) == 0
```

main:

```
from pila import Pila

frase = "Ciao Ryan come stai?"

pila = Pila()
for lettera in frase:
    pila.push(lettera)
invertita = ''
while not pila.empty():
    invertita += pila.top()
```

```
    pila.pop()

print(invertita)
```

9.1.2 Controllo parentesi

Scrivi un programma che legga in input un'espressione matematica contenente parentesi e che sia in grado di dire se le parentesi contenute nell'espressione siano state aperte e chiuse correttamente. Puoi implementare due versioni:

1. Versione semplificata che usa solo parentesi tonde
2. Versione completa che usa anche parentesi quadre e se vuoi anche graffe

Soluzione

```
from pila import Pila

def controllaParentesi(esp):
    p = Pila()
    for sim in esp:
        if sim in "[({":
            if (sim in "{[" and p.top() == "(") or (sim in "{" and p.top() == "["):
                return False
            p.push(sim)
        if sim in "})]":
            if p.empty():
                return False
            if sim == "}" and p.top() != "{":
                return False
            if sim == "]" and p.top() != "[":
                return False
            if sim == ")" and p.top() != "(":
                return False
            p.pop()
    if p.empty():
        return True
    else:
        return False


esp = "16{2x+[3y-7]}+5"
print(esp, controllaParentesi(esp))
```

9.1.3 Conversione in binario

Scrivi un programma che dato in input un numero intero positivo qualsiasi sia in grado, usando una pila, di stampare la sua rappresentazione binaria. (vai a rivedere l'algoritmo per la conversione da decimale a binario)

Soluzione

```
import os
os.system('cls')

from pila import Pila
```

```
def convertiABinario(decimale):
    cifre = Pila()
    while decimale > 0:
        cifre.push(decimale%2)
        decimale //= 2

    ris = ''
    while not cifre.empty():
        ris += str(cifre.top())
        cifre.pop()

    return ris


print(convertiABinario(2))
```

9.1.4 Coda con due pile

Implementa una coda utilizzando due stack

Soluzione

```
from pila import Pila

class CodaConPile:
    def __init__(self) -> None:
        self.pila1 = Pila() # da qui posso prendere dalla cima l'elemento front della coda
        self.pila2 = Pila() # qui metto in cima l'elemento back della coda

    def push(self, dato):
        while not self.pila1.empty():
            self.pila2.push(self.pila1.top())
            self.pila1.pop()
        self.pila2.push(dato)

    def front(self):
        while not self.pila2.empty():
            self.pila1.push(self.pila2.top())
            self.pila2.pop()
        return self.pila1.top()

    def pop(self):
        while not self.pila2.empty():
            self.pila1.push(self.pila2.top())
            self.pila2.pop()
        self.pila1.pop()

    def empty(self):
        return self.pila1.empty() and self.pila2.empty()


coda = CodaConPile()
coda.push(1)
```

```
coda.pop()
coda.push(2)
coda.push(3)
coda.push(4)
print(coda.front())
coda.pop()
print(coda.front())
coda.pop()
print(coda.front())
coda.pop()
print(coda.front())
# coda.pop() # questo mi da errore perchè è vuoto (giustamente)
```

9.1.5 Espressioni con frazioni

Dopo aver implementato la classe 'frazione' con tutte le quattro operazioni, scrivi una funzione che utilizzando anche uno stack riesce a risolvere un'espressione matematica contenente frazioni, operazioni aritmetiche (quelle che hai implementato per le frazioni) e parentesi.

9.1.6 Altri esercizi online

https://www.cs.princeton.edu/courses/archive/spr01/cs126/exercises/adt.html

https://www.geeksforgeeks.org/c-programs-gq/stack-queue-cc-programs-gq/

9.2 Code / Queue

9.2.1 Coda con output dato

Definisci opportunamente una classe coda, poi riempila in modo che quando poi vengono letti i numeri in essa contenuti venga stampato il seguente output: 1 2 3 4 5 6

9.2.2 Coda con output dato 2

Definisci opportunamente una classe coda, poi istanzia una coda e riempila inserendo i numeri in ordine da 1 a 10, infine usa opportunamente i metodi della classe per stampare il seguente output: 1 4 5 7 9

9.2.3 Classe Coda fatta di pile

Definisci la classe Coda utilizzando solo pile (già definite altrove come classe) invece che una lista

9.2.4 Classe Pila fatta di code

Definisci la classe Pila utilizzando solo code (già definite altrove come classe) invece che una lista

9.2.5 Classe Deque

Definisci la classe Deque che è una particolare coda in cui si possono aggiungere e togliere elementi sia in testa che in coda.

9.3 Grafi

9.3.1 Grafo da file

Leggi da un file di testo i dati riguardanti un grafo e istanzia un grafo da te definito.

Contenuto del file in cui la prima riga contiene i valori di ogni nodo e le altre righe le liste di adiacenza di ogni nodo:

```
0,1,2,3,4,5
1
2 3
3 4
4
0 2
0 1
```

Soluzione

```
# 7.3.1 Leggi da un file di testo i dati riguardanti un grafo e istanzia un grafo da te definito.
# Contenuto del file in cui la prima riga contiene i valori di ogni nodo e le altre righe le
liste di adiacenza di ogni nodo:

class Nodo:
    def __init__(self, val = None, archi = []) -> None:
        self.val = val
        self.archi = archi

    def __repr__(self) -> str:
        return f'{self.val}'

class Grafo:
    def __init__(self, nodi = []) -> None:
        self.nodi = nodi

    def __repr__(self) -> str:
        ris = ''
        for i in range(len(self.nodi)):
            ris += f'{self.nodi[i].val}: {self.nodi[i].archi}\n'
        return ris

with open("7-3-1.txt", 'r', encoding='utf-8') as f:
    vals = list(map(int, f.readline().strip().split()))
    adiacenze = [list(map(int, riga.split())) for riga in f]

print(vals)
print(adiacenze)
g = Grafo()
for i in range(len(vals)):
    g.nodi.append(Nodo(vals[i], adiacenze[i]))

print(g)
```

9.3.2 Grafo complementare

Letto in input un grafo G = (N,A) non orientato, costruire il grafo H = (N,A') complementare, tale che l'arco (u, v) ∈ A se e solo se (u, v) ∉ A'. Testa il programma sul grafo:

```
0 1 2 3
0 1, 0 2, 2 3
```

Dove la prima riga rappresenta i valori dei nodi, la seconda riga gli archi.

```
class Grafo:
    def __init__(self, vals = None, archi = None) -> None:
        if vals == None:
            self.vals = []
        else:
            self.vals = vals
        if archi == None:
            self.archi = []
        else:
            self.archi = archi

    def __repr__(self) -> str:
        ris = f'Nodi: {self.vals}\n'
        ris += f'Archi: {self.archi}\n'
        return ris

    def complementare(self):
        h = Grafo(vals = self.vals)
        for i in range(len(self.vals)):
            for j in range(i+1, len(self.vals)):
                if (i, j) not in self.archi and (j, i) not in self.archi:
                    h.archi.append((i,j))
        return h



with open('9-3-2.txt', 'r', encoding='utf-8') as f:
    vals = list(map(int, f.readline().strip().split()))
    archi = f.readline().strip().split(',')
    archi = [tuple(map(int, arco.strip().split())) for arco in archi]

g = Grafo(vals, archi)
print(g)
print()
h = g.complementare()
print(h)
```

9.3.3 Grafo casuale

Letti in input due interi n e k (0 < k < n), costruire un grafo G = (N,A) in modo casuale che abbia n nodi e ogni vertice abbia al massimo k archi uscenti (non bidirezionali).

Soluzione

```
from random import randint
```

```
class Nodo:
    def __init__(self, val = None, archi = []) -> None:
        self.val = val
        self.archi = archi

    def __repr__(self) -> str:
        return f'{self.val}'

class Grafo:
    def __init__(self, nodi = []) -> None:
        self.nodi = nodi

    def __repr__(self) -> str:
        ris = ''
        for i in range(len(self.nodi)):
            ris += f'{i}. Val: {self.nodi[i].val}, Archi: {self.nodi[i].archi}\n'
        return ris


def creaGrafoCasuale(n, k):
    g = Grafo()
    for i in range(n):
        val = randint(0,9)
        archi = []
        for j in range(randint(0,k)):
            dest = randint(0, n-1)
            while dest == i or dest in archi:
                dest = randint(0, n-1)
            archi.append(dest)
        g.nodi.append(Nodo(val, archi))
    return g


n = 4
k = 3
g = creaGrafoCasuale(n, k)
print(g)
```

9.3.4 Verifica di cammini

Letto in input un grafo non orientato G = (N,A) con n nodi e una serie di liste di numeri interi compresi tra 0 e n-1, verificare per ogni lista se la lista rappresenta un cammino sul grafo.

I dati in input sono:

```
0 1, 0 4, 1 2, 2 3, 2 4, 3 4
1 2 4 3 2 0
4 0 1 2 4
0 1 3 4
2 1 0 4 4 2 1
```

La prima riga rappresenta gli archi del grafo (i valori dei nodi non sono riportati perché inutili, se vuoi per semplificare l'esercizio aggiungili tu nel file di testo, se no generali tu a partire dagli archi) e le altre righe sono i cammini da verificare

Per esercitarti genera l'intero grafo e lavora su di esso, in realtà è possibile lavorare solamente sulla lista degli archi (ed è anche più veloce).

```
class Grafo:
    def __init__(self, vals=None, archi=None):
        self.vals = vals
        if vals == None:
            self.vals = []
        self.archi = archi
        if archi == None:
            self.archi = []

    def __repr__(self) -> str:
        ris = "Valori: "
        for val in self.vals:
            ris = ris + f"{val}, "
        ris += "\nArchi: "
        for arco in self.archi:
            ris = ris + f"{arco}"
        return ris

archi = []
percorsi = []
with open("7-3-4.txt", "r", encoding="utf-8") as f:
    riga = f.readline()
    riga = riga.strip().split(",")
    riga = [el.strip().split() for el in riga]
    for arco in riga:
        arco = [int(el) for el in arco]
        archi.append(arco)
    for riga in f:
        riga = riga.strip().split()
        riga = [int(el) for el in riga]
        percorsi.append(riga)

print(archi)
# print(percorsi)

for percorso in percorsi:
    valido = True
    for i in range(len(percorso)-1):
        arco = [percorso[i], percorso[i+1]]
        if arco not in archi and arco[::-1] not in archi:
            valido = False
    print(percorso, end = '')
    if valido:
        print(" valido")
    else:
        print(" non valido")
```

9.3.5 Grafo da cammini

Leggere in input k liste di numeri interi e costruire un grafo non orientato G = (N,A) per cui le k liste rappresentino dei cammini validi.

Soluzione

```
from random import randint

class Grafo:
    def __init__(self, vals=None, archi=None):
        self.vals = vals
        if vals == None:
            self.vals = []
        self.archi = archi
        if archi == None:
            self.archi = []

    def __repr__(self) -> str:
        ris = "Valori: "
        for val in self.vals:
            ris = ris + f"{val}, "
        ris += "\nArchi: "
        for arco in self.archi:
            ris = ris + f"{arco}"
        return ris


# invece che leggere i cammini da file li creo random
k = 4
maxlung = 6
numnodi = 5
cammini = []
# for _ in range(k):
#     cammino = []
#     for _ in range(randint(1,maxlung)):
#         cammino.append(randint(0, numnodi-1))
#     cammini.append(cammino)
cammini = [[randint(0, numnodi-1) for _ in range(randint(1,maxlung))] for _ in range(k)]
for cammino in cammini:
    print(cammino)

archi = []
for cammino in cammini:
    for i in range(len(cammino)-1):
        arco = [cammino[i], cammino[i+1]]
        arco.sort()
        if arco not in archi:
            archi.append(arco)
archi.sort()
print(archi)
```

9.3.6 Predecessori di un vertice

Dato un grafo orientato G e dato in input un vertice v, stampare tutti i vertici di G a cui v è adiacente (che hanno archi che puntano a v)

9.3.7 Aggiunta di un vertice

Dato un grafo G e in input un nodo v con i suo archi uscenti, aggiungere v a G.

9.3.8 Eliminazione di un vertice

Dato un grafo G e un vertice v di G, eliminare v da G.

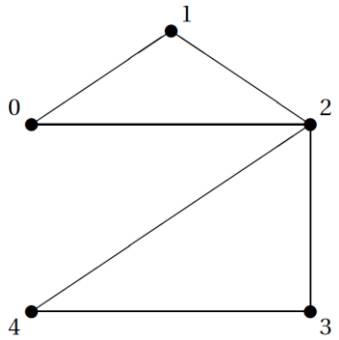
9.3.9 Grafo completo

Dato un grafo G, verifica se esso è completo cioè se tutti i nodi di G sono raggiungibili da ogni altro nodo di G con un solo passo.

9.3.10 Copertura di vertici

Dato un grafo non orientato G = (V,E), letto in input un sottoinsieme L di vertici di V, verificare se L è una copertura di vertici di G, ossia se per ogni (u, v) ∈ E risulta u ∈ L o v ∈ L (o entrambi).

Esempio: consideriamo il grafo G = (V,E) rappresentato in figura, in cui l'insieme dei vertici è V = {0,1,2, 3,4} e l'insieme degli spigoli è E = {(0,1),(0, 2), (1, 2), (2, 3), (2, 4), (3,4)}:



La lista L = {1,2,3} è una copertura di G, mentre L' = {0,1,3} non è una copertura, infatti gli estremi dello spigolo (4,2) non appartengono a L'.

Soluzione

```
class Grafo:
    def __init__(self, vals=[], archi=[]) -> None:
        self.vals = vals
        self.archi = archi

    def __repr__(self) -> str:
        ris = f'Nodi: {self.vals}\n'
        ris += f'Archi: {self.archi}\n'
        return ris

    def copertura(self, nodi):
        # prima magari controllo se i nodi esistono
        for nodo in nodi:
            if nodo < 0 or nodo >= len(self.vals):
```

```
                return False

        # poi controllo se ho almeno un arco che non tocca nessuno dei nodi passati
        for arco in self.archi:
            if arco[0] not in nodi and arco[1] not in nodi:
                return False
        # se non ne ho trovati allora restituisco true
        return True


g = Grafo([0, 1, 2, 3, 4], [(0, 1), (0, 2), (1, 2), (2, 3), (2, 4), (3, 4)])
print(g)
print(g.copertura([1,2,3]))
print(g.copertura([0,1,3]))
```

9.3.11 Orientamento per grado

Letto in input un grafo G = (V,E) non orientato, costruire e stampare un grafo G' = (V,E') orientato, tale che (u, v) ∈ E' se e solo se g(u) > g(v), dove con g(v) si è indicato il grado del vertice v (il numero di spigoli entranti o uscenti: in un grafo non orientato non c'è distinzione).

Soluzione

```
# 9.3.11 - Letto in input un grafo G = (V,E) non orientato,
# costruire e stampare un grafo G' = (V,E') orientato, tale che (u, v) ∈
# E' se e solo se g(u) > g(v), dove con g(v) si è indicato il grado del
# vertice v (il numero di spigoli entranti o uscenti: in un grafo non orientato
# non c'è distinzione).

class Nodo:
    def __init__(self, val, archi = None) -> None:
        self.val = val
        if archi == None:
            self.archi = []
        else:
            self.archi = archi

    def grado(self):
        return len(self.archi)


class Grafo:
    def __init__(self, nodi = None) -> None:
        if nodi == None:
            self.nodi = []
        else:
            self.nodi = nodi

    def __repr__(self) -> str:
        ris = ''
        for i, nodo in enumerate(self.nodi):
            ris += f'{i}. {nodo.val} : '
```

```
            for arco in nodo.archi:
                ris += f'{arco} '
            ris += '\n'
        return ris


def creaGrafoOrientato(g):
    ris = Grafo()
    for i, nodo in enumerate(g.nodi):
        archi = []
        for dest in nodo.archi:
            if nodo.grado() > g.nodi[dest].grado():
                archi.append(dest)
        ris.nodi.append(Nodo(nodo.val, archi))
    return ris

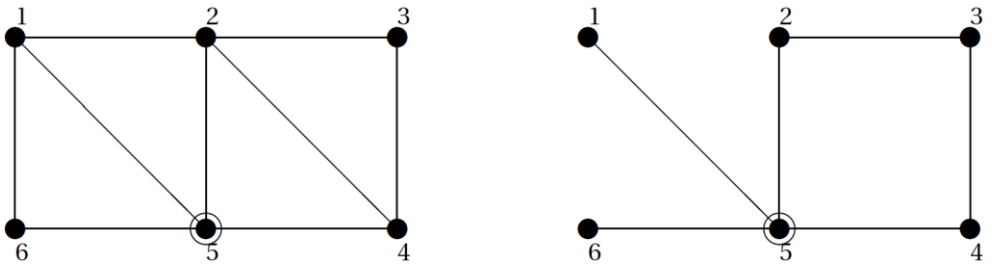
g = Grafo()
g.nodi.append(Nodo(0, [1,2,3]))
g.nodi.append(Nodo(1, [0,3]))
g.nodi.append(Nodo(2, [0,4]))
g.nodi.append(Nodo(3, [0,1,4]))
g.nodi.append(Nodo(4, [3,2]))

g2 = creaGrafoOrientato(g)
print(g2)
```

9.3.12 Eliminazione archi tra adiacenti

Leggere in input un grafo G = (V,E) non orientato e memorizzarlo mediante liste di adiacenza. Scelto arbitrariamente uno dei vertici v ∈ V di grado massimo, eliminare dal grafo tutti gli spigoli (u,w) ∈ E per ogni u e w adiacenti a v. Stampare le liste di adiacenza del grafo così modificato.

Esempio Sia G = (V,E) il grafo letto in input rappresentato in figura (a sinistra), con V = {1, 2, 3, 4, 5, 6} ed E = {(1,2),(2, 3), (3, 4), (4, 5), (5, 6), (6,1),(1,5),(2,5),(2, 4)}. I vertici di grado massimo sono 2 e 5 (entrambi di grado 4). Scegliendo il vertice 5, devono essere eliminati gli spigoli (1,2) (perché 1,2 ∈ N(5)), (1,6) (perché 1,6 ∈ N(5)) e (2,4) (perché 4,2 ∈ N(5)). si ottiene così il grafo rappresentato a destra nella figura.



Soluzione

```
# 9.3.12 - Leggere in input un grafo G = (V,E) non orientato e memorizzarlo
# mediante liste di adiacenza. Scelto arbitrariamente uno dei vertici v ∈
# V di grado massimo, eliminare dal grafo tutti gli spigoli (u,w) ∈
# E per ogni u e w adiacenti a v. Stampare le liste di adiacenza del grafo così
# modificato.
```

```python
class Nodo:
    def __init__(self, val, archi = None) -> None:
        self.val = val
        if archi == None:
            self.archi = []
        else:
            self.archi = archi

    def grado(self):
        return len(self.archi)


class Grafo:
    def __init__(self, nodi = None) -> None:
        if nodi == None:
            self.nodi = []
        else:
            self.nodi = nodi

    def __repr__(self) -> str:
        ris = ''
        for i, nodo in enumerate(self.nodi):
            ris += f'{i}. {nodo.val} : '
            for arco in nodo.archi:
                ris += f'{arco} '
            ris += '\n'
        return ris

    def scegliVGradoMassimo(self):
        gradi = [(i, nodo.grado()) for i, nodo in enumerate(self.nodi)]
        gradi.sort(key = lambda x: -x[1])
        # print(gradi)
        return gradi[0][0]

    def modificaGrafo(self, posNodo):
        for nodo in self.nodi:
            if posNodo in nodo.archi: # quindi adiacente
                eliminare = []
                for posAltro in nodo.archi:
                    if posNodo in self.nodi[posAltro].archi: # quindi anche questo è adiacente
                        eliminare.append(posAltro)
                for el in eliminare:
                    nodo.archi.remove(el)



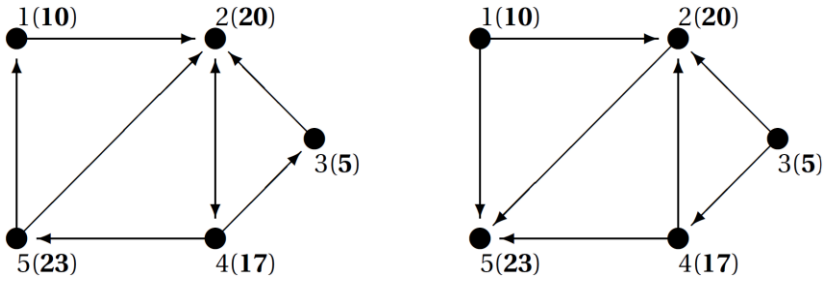
g = Grafo()
g.nodi.append(Nodo(0, [1,2,3]))
g.nodi.append(Nodo(1, [0,3]))
g.nodi.append(Nodo(2, [0,4,3]))
g.nodi.append(Nodo(3, [0,1,2,4]))
g.nodi.append(Nodo(4, [3,2]))
```

```
g.modificaGrafo(g.scegliVGradoMassimo())
print(g.scegliVGradoMassimo())
print(g)
```

9.3.13 Orientamento per pesi

Leggere in input un grafo orientato G = (V,E) e rappresentarlo mediante liste di adiacenza. Leggere in input un insieme di pesi (interi) associati ai vertici del grafo: $\{w_1, ...,w_n\}$. Modificando le liste di adiacenza con cui è stato rappresentato il grafo G, variare l'orientamento degli spigoli in modo tale che per ogni spigolo (u, v) risulti $w_u \leq w_v$.

Esempio Sia G = (V,E) il grafo orientato letto in input rappresentato in figura, con V = {1,2,3, 4,5} ed E = {(1,2),(2, 4), (3, 2), (4, 2), (4, 3), (4,5),(5,1),(5,2)}. Sia W l'insieme dei pesi associati ai vertici del grafo: W = {10, 30, 5, 17, 23}. Sulla sinistra è rappresentato il grafo letto in input e sulla destra il grafo prodotto dalla rielaborazione richiesta dall'esercizio.



Le dieci città principali del regno di Banania sono così collegate da strade:

```
Strada - Città collegate - Lunghezza (km)
SS1 Acerorosso - Bananopoli 50
SS2 Bananopoli - Lamponeggia 80
SS3 Bananopoli - Cedrosa 30
SS4 Lamponeggia - Cedrosa 50
SS5 Acerorosso - Lamponeggia 25
SS6 Acerorosso - Dolcemela 60
SS7 Lamponeggia - Dolcemela 20
SS8 Cedrosa - Erbavoglio 35
SS9 Dolcemela - Erbavoglio 40
SS10 Acerorosso - Finoporto 45
SS11 Finoporto - Giuggiolo 60
SS12 Dolcemela - Giuggiolo 80
SS13 Dolcemela - Indacovia 90
SS14 Erbavoglio - Hoppavilla 70
SS15 Hoppavilla - Indacovia 45
SS16 Giuggiolo - Indacovia 55
```

Aggiungi i dati ad un file di testo e poi scrivi un programma che legga tali dati e istanzi un grafo che rappresenti le strade.

Il programma deve poi essere in grado di:

1. Scrivere il percorso più breve (in km) fra Bananopoli e Hoppavilla
2. Scrivere il percorso più breve (in km) fra Giuggiolo e Cedrosa
3. Scrivere la strada più diretta (meno città intermedie) fra Bananopoli e Hoppavilla
4. Scrivere la strada più diretta (meno città intermedie) fra Giuggiolo e Cedrosa

Soluzione 1

Qui ho usato solo un dizionario per rappresentare il grafo

```
def distanza(nodi, orig, dest = None):
```

```
    if orig not in nodi:
        return None
    if dest != None and dest not in nodi:
        return None

    # lista delle distanze di ogni nodo da orig
    # all'inizio sono tutte sconosciute (None)
    dist = dict([(nodo, None) for nodo in nodi])
    # tranne l'origine che ha distanza 0 da se stessa
    dist[orig] = 0

    # lista in cui segno per ogni nodo da quale altro nodo
    # ci arrivo nel percorso ottimo (all'inizio è tutto sconosciuto)
    prec = dict([(nodo, None) for nodo in nodi])

    # nodi da controllare
    controllare = [orig]
    controllati = []

    # # nodo da cui partire
    # corrente = orig

    while len(controllare) > 0:
        # scelgo il prossimo nodo da controllare (corrente)
        # trovo il nodo con distanza minima in controllare
        corrente = controllare[0]
        for nodo in controllare:
            if dist[nodo] < dist[corrente]:
                corrente = nodo

        # controllo tutti i nodi raggiungibili da corrente con i suoi archi
        # print(nodi[corrente])
        for nodo, peso in nodi[corrente]:
            # ogni nodo che raggiungo da corrente con il minor costo visto finora
            # per quel nodo va aggiornato (distanza ottima e nodo precedente nel
            # percorso ottimo) e aggiunto a controllare
            if dist[nodo] == None or dist[corrente] + peso < dist[nodo]:
                dist[nodo] = dist[corrente] + peso
                prec[nodo] = corrente
                if nodo not in controllare and nodo not in controllati:
                    controllare.append(nodo)

        # rimuovo da controllare il nodo corrente, qui non ci potrò mai più
        # tornare perchè è impossibile che possa tornarci con distanza minore
        # di ora perchè quando scelgo il nodo da controllare prendo sempre
        # quello con distanza minima
        controllare.remove(corrente)
        controllati.append(corrente)

    # caso in cui non specifico una destinazione
    # restituisco tutte le distanze e i nodi precedenti per ogni nodo
    if dest == None:
```

```
        return (dist, prec)
    # caso in cui specifico un nodo destinazione
    # restituisco la singola distanza e tutto il percorso da orig a dest
    else:

        # se il nodo dest non è raggiungibile
        if dist[dest] == None:
            percorso = []
        # altrimenti ricostruisco il percorso a ritroso da dest a orig
        else:
            percorso = [dest]
            pos = dest
            while prec[pos] != None:
                pos = prec[pos]
                percorso.append(pos)
            percorso = percorso[::-1]

        return (dist[dest], percorso)


with open('9-3-14.txt', 'r', encoding='utf-8') as f:
    nodi = {}
    for riga in f:
        riga = riga.strip().split()
        n1 = riga[1]
        n2 = riga[3]
        peso = int(riga[4])
        if n1 in nodi:
            nodi[n1].append((n2, peso))
        else:
            nodi[n1] = [(n2, peso)]
        if n2 in nodi:
            nodi[n2].append((n1, peso))
        else:
            nodi[n2] = [(n1, peso)]

for nodo in nodi:
    print(nodo, nodi[nodo])

# 1.    Scrivere il percorso più breve (in km) fra Bananopoli e Hoppavilla
print(distanza(nodi, 'Bananopoli', 'Hoppavilla'))
# 2.    Scrivere il percorso più breve (in km) fra Giuggiolo e Cedrosa
print(distanza(nodi, 'Giuggiolo', 'Cedrosa'))
# 3.    Scrivere la strada più diretta (meno città intermedie) fra Bananopoli e Hoppavilla
# 4.    Scrivere la strada più diretta (meno città intermedie) fra Giuggiolo e Cedrosa
```

Soluzione 2

Qui ho usato le classi nodo e grafo. In grafo ho usato un dizionario per contenere i nodi e accedervi tramite nome della città

```
class Nodo:
    def __init__(self, val = None, archi = None) -> None:
```

```
        self.val = val
        if archi == None:
            self.archi = []
        else:
            self.archi = archi

class Grafo:
    def __init__(self, nodi = None) -> None:
        if nodi == None:
            self.nodi = {}
        else:
            self.nodi = nodi

    def __repr__(self) -> str:
        ris = ''
        for key, nodo in self.nodi.items():
            ris += f'{key}: '
            for arco in nodo.archi:
                ris += f'{arco} '
            ris += '\n'
        return ris


    def distanza(self, orig, dest = None):
        if orig not in self.nodi:
            return None
        if dest != None and dest not in self.nodi:
            return None

        # lista delle distanze di ogni nodo da orig
        # all'inizio sono tutte sconosciute (None)
        dist = dict([(nodo, None) for nodo in self.nodi])
        # tranne l'origine che ha distanza 0 da se stessa
        dist[orig] = 0

        # lista in cui segno per ogni nodo da quale altro nodo
        # ci arrivo nel percorso ottimo (all'inizio è tutto sconosciuto)
        prec = dict([(nodo, None) for nodo in self.nodi])

        # nodi da controllare
        controllare = [orig]
        controllati = []

        # # nodo da cui partire
        # corrente = orig

        while len(controllare) > 0:
            # scelgo il prossimo nodo da controllare (corrente)
            # trovo il nodo con distanza minima in controllare
            corrente = controllare[0]
            for nodo in controllare:
                if dist[nodo] < dist[corrente]:
                    corrente = nodo
```

```
        # controllo tutti i nodi raggiungibili da corrente con i suoi archi
        # print(nodi[corrente])
        for nodo, peso in self.nodi[corrente].archi:
            # ogni nodo che raggiungo da corrente con il minor costo visto finora
            # per quel nodo va aggiornato (distanza ottima e nodo precedente nel
            # percorso ottimo) e aggiunto a controllare
            if dist[nodo] == None or dist[corrente] + peso < dist[nodo]:
                dist[nodo] = dist[corrente] + peso
                prec[nodo] = corrente
                if nodo not in controllare and nodo not in controllati:
                    controllare.append(nodo)

        # rimuovo da controllare il nodo corrente, qui non ci potrò mai più
        # tornare perchè è impossibile che possa tornarci con distanza minore
        # di ora perchè quando scelgo il nodo da controllare prendo sempre
        # quello con distanza minima
        controllare.remove(corrente)
        controllati.append(corrente)

    # caso in cui non specifico una destinazione
    # restituisco tutte le distanze e i nodi precedenti per ogni nodo
    if dest == None:
        return (dist, prec)
    # caso in cui specifico un nodo destinazione
    # restituisco la singola distanza e tutto il percorso da orig a dest
    else:

        # se il nodo dest non è raggiungibile
        if dist[dest] == None:
            percorso = []
        # altrimenti ricostruisco il percorso a ritroso da dest a orig
        else:
            percorso = [dest]
            pos = dest
            while prec[pos] != None:
                pos = prec[pos]
                percorso.append(pos)
            percorso = percorso[::-1]

        return (dist[dest], percorso)

def passi(self, orig, dest = None):
    if orig not in self.nodi:
        return None
    if dest != None and dest not in self.nodi:
        return None

    # lista delle distanze di ogni nodo da orig
    # all'inizio sono tutte sconosciute (None)
    dist = dict([(nodo, None) for nodo in self.nodi])
    # tranne l'origine che ha distanza 0 da se stessa
```

```
        dist[orig] = 0

        # lista in cui segno per ogni nodo da quale altro nodo
        # ci arrivo nel percorso ottimo (all'inizio è tutto sconosciuto)
        prec = dict([(nodo, None) for nodo in self.nodi])

        # nodi da controllare
        controllare = [orig]
        controllati = []

        # # nodo da cui partire
        # corrente = orig

        while len(controllare) > 0:
            # scelgo il prossimo nodo da controllare (corrente)
            # trovo il nodo con distanza minima in controllare
            corrente = controllare[0]
            for nodo in controllare:
                if dist[nodo] < dist[corrente]:
                    corrente = nodo

            # controllo tutti i nodi raggiungibili da corrente con i suoi archi
            # print(nodi[corrente])
            for nodo, peso in self.nodi[corrente].archi:
                # ogni nodo che raggiungo da corrente con il minor costo visto finora
                # per quel nodo va aggiornato (distanza ottima e nodo precedente nel
                # percorso ottimo) e aggiunto a controllare
                if dist[nodo] == None or dist[corrente] + 1 < dist[nodo]:
                    dist[nodo] = dist[corrente] + 1
                    prec[nodo] = corrente
                    if nodo not in controllare and nodo not in controllati:
                        controllare.append(nodo)

            # rimuovo da controllare il nodo corrente, qui non ci potrò mai più
            # tornare perchè è impossibile che possa tornarci con distanza minore
            # di ora perchè quando scelgo il nodo da controllare prendo sempre
            # quello con distanza minima
            controllare.remove(corrente)
            controllati.append(corrente)

        # caso in cui non specifico una destinazione
        # restituisco tutte le distanze e i nodi precedenti per ogni nodo
        if dest == None:
            return (dist, prec)
        # caso in cui specifico un nodo destinazione
        # restituisco la singola distanza e tutto il percorso da orig a dest
        else:

            # se il nodo dest non è raggiungibile
            if dist[dest] == None:
                percorso = []
            # altrimenti ricostruisco il percorso a ritroso da dest a orig
```

```
            else:
                percorso = [dest]
                pos = dest
                while prec[pos] != None:
                    pos = prec[pos]
                    percorso.append(pos)
                percorso = percorso[::-1]

            return (dist[dest], percorso)


with open('9-3-14.txt', 'r', encoding='utf-8') as f:
    g = Grafo()
    for riga in f:
        riga = riga.strip().split()
        n1 = riga[1]
        n2 = riga[3]
        peso = int(riga[4])
        if n1 in g.nodi:
            g.nodi[n1].archi.append((n2, peso))
        else:
            g.nodi[n1] = Nodo(n1, [(n2, peso)])
        if n2 in g.nodi:
            g.nodi[n2].archi.append((n1, peso))
        else:
            g.nodi[n2] = Nodo(n2, [(n1, peso)])

print(g)

# 1.    Scrivere il percorso più breve (in km) fra Bananopoli e Hoppavilla
print(g.distanza('Bananopoli', 'Hoppavilla'))
# 2.    Scrivere il percorso più breve (in km) fra Giuggiolo e Cedrosa
print(g.distanza('Giuggiolo', 'Cedrosa'))
# 3.    Scrivere la strada più diretta (meno città intermedie) fra Bananopoli e Hoppavilla
print(g.passi('Bananopoli', 'Hoppavilla'))
# 4.    Scrivere la strada più diretta (meno città intermedie) fra Giuggiolo e Cedrosa
print(g.passi('Giuggiolo', 'Cedrosa'))
```

Soluzione 3

Qui ho adattato l'input per avere un grafo classico in cui i nodi sono indicati dalla posizione nella lista dei nodi

```
class Nodo:
    def __init__(self, val = None, archi = None) -> None:
        self.val = val
        if archi == None:
            self.archi = []
        else:
            self.archi = archi

    def __repr__(self) -> str:
        return f'{self.val}'

class Grafo:
```

```python
    def __init__(self, nodi = []) -> None:
        self.nodi = nodi

    def __repr__(self) -> str:
        ris = ''
        for i in range(len(self.nodi)):
            ris += f'{i}. Val: {self.nodi[i].val}, Archi: {self.nodi[i].archi}\n'
        return ris


    def distanza(self, orig, dest = None):
        if orig < 0 or orig >= len(self.nodi):
            return None
        if dest != None and (dest < 0 or dest >= len(self.nodi)):
            return None

        # lista delle distanze di ogni nodo da orig
        # all'inizio sono tutte sconosciute (None)
        dist = [None for _ in range(len(self.nodi))]
        # tranne l'origine che ha distanza 0 da se stessa
        dist[orig] = 0

        # lista in cui segno per ogni nodo da quale altro nodo
        # ci arrivo nel percorso ottimo (all'inizio è tutto sconosciuto)
        prec = [None for _ in range(len(self.nodi))]

        # nodi da controllare
        controllare = [orig]
        controllati = []

        # # nodo da cui partire
        # corrente = orig

        while len(controllare) > 0:
            # scelgo il prossimo nodo da controllare (corrente)
            # trovo il nodo con distanza minima in controllare
            corrente = controllare[0]
            for nodo in controllare:
                if dist[nodo] < dist[corrente]:
                    corrente = nodo

            # controllo tutti i nodi raggiungibili da corrente con i suoi archi
            for pos, peso in self.nodi[corrente].archi:
                # ogni nodo che raggiungo da corrente con il minor costo visto finora
                # per quel nodo va aggiornato (distanza ottima e nodo precedente nel
                # percorso ottimo) e aggiunto a controllare
                if dist[pos] == None or dist[corrente] + peso < dist[pos]:
                    dist[pos] = dist[corrente] + peso
                    prec[pos] = corrente
                    if pos not in controllare and pos not in controllati:
                        controllare.append(pos)

            # rimuovo da controllare il nodo corrente, qui non ci potrò mai più
```

```
            # tornare perchè è impossibile che possa tornarci con distanza minore
            # di ora perchè quando scelgo il nodo da controllare prendo sempre
            # quello con distanza minima
            controllare.remove(corrente)
            controllati.append(corrente)

        # caso in cui non specifico una destinazione
        # restituisco tutte le distanze e i nodi precedenti per ogni nodo
        if dest == None:
            return (dist, prec)
        # caso in cui specifico un nodo destinazione
        # restituisco la singola distanza e tutto il percorso da orig a dest
        else:

            # se il nodo dest non è raggiungibile
            if dist[dest] == None:
                percorso = []
            # altrimenti ricostruisco il percorso a ritroso da dest a orig
            else:
                percorso = [dest]
                pos = dest
                while prec[pos] != None:
                    pos = prec[pos]
                    percorso.append(pos)
                percorso = percorso[::-1]

            return (dist[dest], percorso)


    def passi(self, orig, dest = None):
        if orig < 0 or orig >= len(self.nodi):
            return None
        if dest != None and (dest < 0 or dest >= len(self.nodi)):
            return None

        # lista delle distanze di ogni nodo da orig
        # all'inizio sono tutte sconosciute (None)
        dist = [None for _ in range(len(self.nodi))]
        # tranne l'origine che ha distanza 0 da se stessa
        dist[orig] = 0

        # lista in cui segno per ogni nodo da quale altro nodo
        # ci arrivo nel percorso ottimo (all'inizio è tutto sconosciuto)
        prec = [None for _ in range(len(self.nodi))]

        # nodi da controllare
        controllare = [orig]
        controllati = []

        # # nodo da cui partire
        # corrente = orig
```

```
        while len(controllare) > 0:
            # scelgo il prossimo nodo da controllare (corrente)
            # trovo il nodo con distanza minima in controllare
            corrente = controllare[0]
            for nodo in controllare:
                if dist[nodo] < dist[corrente]:
                    corrente = nodo

            # controllo tutti i nodi raggiungibili da corrente con i suoi archi
            for pos, peso in self.nodi[corrente].archi:
                # ogni nodo che raggiungo da corrente con il minor costo visto finora
                # per quel nodo va aggiornato (distanza ottima e nodo precedente nel
                # percorso ottimo) e aggiunto a controllare
                if dist[pos] == None or dist[corrente] + 1 < dist[pos]:
                    dist[pos] = dist[corrente] + 1
                    prec[pos] = corrente
                    if pos not in controllare and pos not in controllati:
                        controllare.append(pos)

            # rimuovo da controllare il nodo corrente, qui non ci potrò mai più
            # tornare perchè è impossibile che possa tornarci con distanza minore
            # di ora perchè quando scelgo il nodo da controllare prendo sempre
            # quello con distanza minima
            controllare.remove(corrente)
            controllati.append(corrente)

        # caso in cui non specifico una destinazione
        # restituisco tutte le distanze e i nodi precedenti per ogni nodo
        if dest == None:
            return (dist, prec)
        # caso in cui specifico un nodo destinazione
        # restituisco la singola distanza e tutto il percorso da orig a dest
        else:

            # se il nodo dest non è raggiungibile
            if dist[dest] == None:
                percorso = []
            # altrimenti ricostruisco il percorso a ritroso da dest a orig
            else:
                percorso = [dest]
                pos = dest
                while prec[pos] != None:
                    pos = prec[pos]
                    percorso.append(pos)
                percorso = percorso[::-1]

            return (dist[dest], percorso)


with open('9-3-14.txt', 'r', encoding='utf-8') as f:
    posnodi = {}
    val = 0
    g = Grafo()
```

```
    for riga in f:
        riga = riga.strip().split()
        n1 = riga[1]
        n2 = riga[3]
        peso = int(riga[4])
        if n1 not in posnodi:
            posnodi[n1] = val
            val += 1
            g.nodi.append(Nodo(n1))
        if n2 not in posnodi:
            posnodi[n2] = val
            val += 1
            g.nodi.append(Nodo(n2))
        g.nodi[posnodi[n1]].archi.append((posnodi[n2], peso))
        g.nodi[posnodi[n2]].archi.append((posnodi[n1], peso))
        # print(posnodi)
        # print(g)

print(g)

# 1.    Scrivere il percorso più breve (in km) fra Bananopoli e Hoppavilla
print(g.distanza(posnodi['Bananopoli'], posnodi['Hoppavilla']))
# 2.    Scrivere il percorso più breve (in km) fra Giuggiolo e Cedrosa
print(g.distanza(posnodi['Giuggiolo'], posnodi['Cedrosa']))
# 3.    Scrivere la strada più diretta (meno città intermedie) fra Bananopoli e Hoppavilla
print(g.passi(posnodi['Bananopoli'], posnodi['Hoppavilla']))
# 4.    Scrivere la strada più diretta (meno città intermedie) fra Giuggiolo e Cedrosa
print(g.passi(posnodi['Giuggiolo'], posnodi['Cedrosa']))
```

Nuova soluzione

```
import heapq

class Nodo:
    def __init__(self, dato):
        self.dato = dato
        self.archi = []

class Grafo:
    def __init__(self):
        # notare che self.nodi è un dizionario e le chiavi saranno i nomi delle citta
        self.nodi = {}

    def aggiungi_nodo(self, dato):
        if dato not in self.nodi:
            self.nodi[dato] = Nodo(dato)

    def aggiungi_arco(self, src, dest, peso):
        self.aggiungi_nodo(src)
        self.aggiungi_nodo(dest)
        self.nodi[src].archi.append((dest, peso))
        self.nodi[dest].archi.append((src, peso))  # Grafo non orientato
```

```
    def dijkstra(self, start, end):
        distanze = {nodo: float('inf') for nodo in self.nodi}
        distanze[start] = 0
        prec = {nodo: None for nodo in self.nodi}
        pq = []
        heapq.heappush(pq, (0, start))

        while pq:
            dist, nodo = heapq.heappop(pq)
            if nodo == end:
                break
            for vicino, peso in self.nodi[nodo].archi:
                nuova_distanza = dist + peso
                if nuova_distanza < distanze[vicino]:
                    distanze[vicino] = nuova_distanza
                    prec[vicino] = nodo
                    heapq.heappush(pq, (nuova_distanza, vicino))

        cammino = []
        pos = end
        while pos is not None:
            cammino.insert(0, pos)
            pos = prec[pos]

        return cammino, distanze[end]

    def bfs(self, start, end):
        visitati = {nodo: False for nodo in self.nodi}
        prec = {nodo: None for nodo in self.nodi}
        fronte = [start]
        visitati[start] = True

        while fronte:
            nodo = fronte.pop(0)
            if nodo == end:
                break
            for vicino, _ in self.nodi[nodo].archi:
                if not visitati[vicino]:
                    fronte.append(vicino)
                    visitati[vicino] = True
                    prec[vicino] = nodo

        cammino = []
        pos = end
        while pos is not None:
            cammino.insert(0, pos)
            pos = prec[pos]

        return cammino

def leggi_dati(file_path):
    grafo = Grafo()
    with open(file_path, 'r') as file:
        file.readline()  # Salta la prima riga
        for linea in file:
            ss, citta1, trattino, citta2, distanza = linea.strip().split()
            distanza = int(distanza)
```

```
            grafo.aggiungi_arco(citta1, citta2, distanza)
    return grafo

# Leggi i dati dal file e crea il grafo
grafo = leggi_dati('3-14.txt')

# 1. Percorso più breve (in km) fra Bananopoli e Hoppavilla
cammino, distanza = grafo.dijkstra('Bananopoli', 'Hoppavilla')
print(f"Percorso più breve fra Bananopoli e Hoppavilla: {cammino} ({distanza} km)")

# 2. Percorso più breve (in km) fra Giuggiolo e Cedrosa
cammino, distanza = grafo.dijkstra('Giuggiolo', 'Cedrosa')
print(f"Percorso più breve fra Giuggiolo e Cedrosa: {cammino} ({distanza} km)")

# 3. Strada più diretta (meno città intermedie) fra Bananopoli e Hoppavilla
cammino = grafo.bfs('Bananopoli', 'Hoppavilla')
print(f"Strada più diretta fra Bananopoli e Hoppavilla: {cammino}")

# 4. Strada più diretta (meno città intermedie) fra Giuggiolo e Cedrosa
cammino = grafo.bfs('Giuggiolo', 'Cedrosa')
print(f"Strada più diretta fra Giuggiolo e Cedrosa: {cammino}")
```

Valerio, Teresa, Sergio, Nadia, Marco, Luca, Francesca e Beatrice sono compagni di classe. Capita che uno vada a casa dell'altro con il pullman; la seguente tabella riporta quanto tempo ci vuole in media perché uno arrivi dall'altro, in minuti (nelle colonne per brevità ciascuno è indicato dall'iniziale del suo nome). Quando non c'è scritto niente è perché quello spostamento non avviene mai: ad esempio Beatrice non va mai a casa di Teresa.

Una piccola difficoltà in più: non è detto che se uno va a casa dell'altro valga anche il viceversa… anzi, in molti casi questo non succede: Marco va da Francesca, ma Francesca non va da Marco.

Quando però c'è uno scambio (ad esempio Luca va da Teresa, e anche Teresa va da Luca) il tempo che ci mettono potrebbe essere diverso.

Fino a casa di → / **Da** casa di ↓	B	F	L	M	N	S	T	V
Beatrice			25	16	18	20		
Francesca					15			
Luca							32	
Marco	13	22						
Nadia			16					
Sergio			28		23			
Teresa			28		22			
Valerio	35			48		42		

Crea un grafo che rispetti i dati contenuti nella tabella e poi rispondi alle seguenti domande:

1. Stampa i ragazzi in ordine secondo il numero di persone che vanno a trovarle
2. Stampa i ragazzi in ordine secondo il numero di persone cha vanno a trovare
3. Qual è il ragazzo che vive più lontano rispetto a tutti gli altri?

Soluzione

In questa soluzione ho inserito manualmente i dati via codice (non leggendo da file)

```
class Nodo:
    def __init__(self, dato, archi = None):
        self.dato = dato
        if archi == None:
            self.archi = []
        self.archi = archi

class Grafo:
    def __init__(self):
        self.nodi = {}

    def aggiungiNodo(self, nodo):
        self.nodi[nodo.dato] = nodo

G = Grafo()
G.aggiungiNodo(Nodo('B', [('L', 25), ('M', 16), ('N', 18), ('S', 20)]))
G.aggiungiNodo(Nodo('F', [('N', 15)]))
G.aggiungiNodo(Nodo('L', [('T', 32)]))
G.aggiungiNodo(Nodo('M', [('B', 13), ('F', 22)]))
G.aggiungiNodo(Nodo('N', [('L', 16)]))
G.aggiungiNodo(Nodo('S', [('L', 28), ('N', 23)]))
G.aggiungiNodo(Nodo('T', [('L', 28), ('N', 22)]))
G.aggiungiNodo(Nodo('V', [('B', 35), ('M', 48), ('S', 42)]))

# 2.    Stampa i ragazzi in ordine secondo il numero di persone cha vanno a trovare
lista1 = [(nome, len(G.nodi[nome].archi)) for nome in G.nodi]
lista1.sort(key=lambda x: (x[1], x[0]))
print("2.   Stampa i ragazzi in ordine secondo il numero di persone cha vanno a trovare")
for nome, num in lista1:
    print(nome, num)

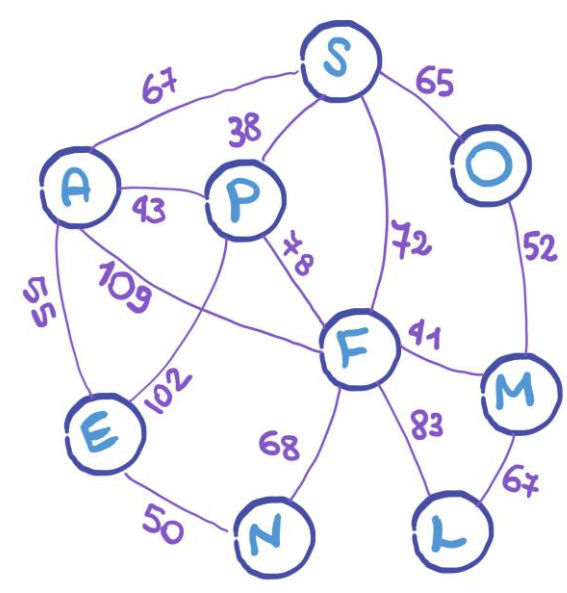
# 1.    Stampa i ragazzi in ordine secondo il numero di persone che vanno a trovarle
d = {}
for nodo in G.nodi:
    for arco in G.nodi[nodo].archi:
        if arco[0] in d:
            d[arco[0]] += 1
        else:
            d[arco[0]] = 1
lista2 = [(nome, d[nome]) for nome in d]
lista2 = list(d.items())
lista2.sort(key=lambda x: (x[1], x[0]))
print("1.   Stampa i ragazzi in ordine secondo il numero di persone che vanno a trovarle")
for nome, num in lista2:
```

```
    print(nome, num)


# 3.    Qual è il ragazzo che vive più lontano rispetto a tutti gli altri?
# questa domanda è ambigua, ma suppongo che si intenda il ragazzo che ha il tempo massimo di arrivo
# da parte di tutti gli altri, in pratica cerco l'arco più lungo di tutti.
max = None
chi = None
for nodo in G.nodi:
    for arco in G.nodi[nodo].archi:
        if max == None or arco[1] > max:
            max = arco[1]
            chi = arco[0]
if max != None:
    print("3.   Qual è il ragazzo che vive più lontano rispetto a tutti gli altri?")
    print(chi, "con distanza", max)
```

9.3.14 Distanze a Micimao

La Repubblica di Micimao (confinante col regno di Banania) ha nove città principali: Angoraville, East Meowia, Felinopolis, Lynxia, Manxester, Newcat, Ocelocity, Purrington e St. Cougar. Il seguente grafo rappresenta la rete stradale dello stato di Micimao (ogni città è rappresentata dalla sua iniziale):



Scrvi un programma che sia in grado di stampare una tabella come quella seguente che indica per ogni coppia di città la distanza minima da percorrere per andare da una all'altra:

km	A	E	F	L	M	N	O	P	S
Angoraville		55	109	192	150	105	132	43	67
East Meowia	55		118	126	159	50	187	98	122
Felinopolis	109	118		83	41	68	93	78	72
Lynxia	192	126	83		67	151	119	161	155
Manxester	150	159	41	67		109	52	119	113
Newcat	105	50	68	151	109		161	146	140
Ocelocity	132	187	93	119	52	161		103	65
Purrington	43	98	78	161	119	146	103		38
St. Cougar	67	122	72	155	113	140	65	38	

9.3.15 Strade da asfaltare

La seguente tabella mostra le case di una città fangosa (prendono il nome dalle famiglie che ci vivono) e quanto costerebbe (in migliaia di euro) asfaltare il percorso da una all'altra. Quando non c'è scritto niente è perché non è possibile asfaltare quel percorso. Inoltre, il percorso da A a B e quello da B ad A prevedono lo stesso costo, per cui la tabella riporta solo uno dei due.

	A	B	C	D	E	F	G	I	L	M
Alberti		12	6						9	
Brambilla			3	6	9			6	5	7
Coletti				4						
Donato					7					11
Erdani						6				5
Fumicola							9			8
Gianone								5		3
Illirio									11	10
Luzzati										8
Mura										

Scrivi un programma che sia in grado di dire:

1. data una coppia di case quali strade sia meglio asfaltare (minimizza il percorso) per collegare le due case con sole strade asfaltate.
2. Quali strade asfaltare per collegare tutte le case, asfaltando il meno possibile (algoritmo di kruskal)

9.3.16 Strade da asfaltare 2

Fai la stessa cosa che hai fatto per l'esercizio precedente ma con questo grafo:

	M	N	O	P	R	S	T	U	V	Z
Mariani		13	10	11	12					
Nedda			10	6	11					
Ortani				13	9					
Pozzo					10		8			
Rugato						5				
Santini							16	6	8	8
Terna								9	5	7
Uscari									10	6
Viola										5
Zante										

9.3.17 Città raggiungibili

Leggi dal seguente file di testo i dati riguardanti un grafo che indica un insieme di città collegate da strade. Scrivi ed utilizza poi una funzione che sia in grado di dire se da ogni città è possibile raggiungere qualsiasi altra città percorrendo meno di 10km.

Nel file la prima riga indica il numero di nodi, la seconda le strade (archi) che li collegano alle altre città

Mancano evidentemente i dati completi

```
5
1 25, 2 30, 4 60, 2 20, 3 70, 5 10
```

Soluzione

```python
class Nodo:
    def __init__(self, val = None, archi = None) -> None:
        self.val = val
        if archi == None:
            self.archi = []
        else:
            self.archi = archi

    def __repr__(self) -> str:
        return f'{self.val}'

class Grafo:
    def __init__(self, nodi) -> None:
        self.nodi = nodi
```

```python
    def __repr__(self) -> str:
        ris = ''
        for i in range(len(self.nodi)):
            ris += f'{i}. Val: {self.nodi[i].val}, Archi: {self.nodi[i].archi}\n'
        return ris


    def distanza(self, orig, dest = None):
        if orig < 0 or orig >= len(self.nodi):
            return None
        if dest != None and (dest < 0 or dest >= len(self.nodi)):
            return None

        # lista delle distanze di ogni nodo da orig
        # all'inizio sono tutte sconosciute (None)
        dist = [None for _ in range(len(self.nodi))]
        # tranne l'origine che ha distanza 0 da se stessa
        dist[orig] = 0

        # lista in cui segno per ogni nodo da quale altro nodo
        # ci arrivo nel percorso ottimo (all'inizio è tutto sconosciuto)
        prec = [None for _ in range(len(self.nodi))]

        # nodi da controllare
        controllare = [orig]
        controllati = []

        # # nodo da cui partire
        # corrente = orig

        while len(controllare) > 0:
            # scelgo il prossimo nodo da controllare (corrente)
            # trovo il nodo con distanza minima in controllare
            corrente = controllare[0]
            for nodo in controllare:
                if dist[nodo] < dist[corrente]:
                    corrente = nodo

            # controllo tutti i nodi raggiungibili da corrente con i suoi archi
            for pos, peso in self.nodi[corrente].archi:
                # ogni nodo che raggiungo da corrente con il minor costo visto finora
                # per quel nodo va aggiornato (distanza ottima e nodo precedente nel
                # percorso ottimo) e aggiunto a controllare
                if dist[pos] == None or dist[corrente] + peso < dist[pos]:
                    dist[pos] = dist[corrente] + peso
                    prec[pos] = corrente
                    if pos not in controllare and pos not in controllati:
                        controllare.append(pos)

            # rimuovo da controllare il nodo corrente, qui non ci potrò mai più
            # tornare perchè è impossibile che possa tornarci con distanza minore
            # di ora perchè quando scelgo il nodo da controllare prendo sempre
            # quello con distanza minima
```

```
            controllare.remove(corrente)
            controllati.append(corrente)

        # caso in cui non specifico una destinazione
        # restituisco tutte le distanze e i nodi precedenti per ogni nodo
        if dest == None:
            return (dist, prec)
        # caso in cui specifico un nodo destinazione
        # restituisco la singola distanza e tutto il percorso da orig a dest
        else:

            # se il nodo dest non è raggiungibile
            if dist[dest] == None:
                percorso = []
            # altrimenti ricostruisco il percorso a ritroso da dest a orig
            else:
                percorso = [dest]
                pos = dest
                while prec[pos] != None:
                    pos = prec[pos]
                    percorso.append(pos)
                percorso = percorso[::-1]

            return (dist[dest], percorso)

def funz(g, x):
    for nodo in range(len(g.nodi)):
        distanze = g.distanza(nodo)[0]
        # print(distanze)
        for distanza in distanze:
            if distanza == None or distanza > x:
                return False
    return True


with open('9-3-19.txt', 'r', encoding='utf-8') as f:
    nnodi = int(f.readline().strip())
    archi = f.readline().strip().split(',')
    archi = [tuple(map(int, arco.split())) for arco in archi]
    g = Grafo([Nodo() for i in range(nnodi)])
    for arco in archi:
        g.nodi[arco[0]].archi.append((arco[1], arco[2]))
        g.nodi[arco[1]].archi.append((arco[0], arco[2]))

print(g)
print(funz(g, 100))
print(funz(g, 200))
```

9.3.18 Instradamento più veloce

Leggi dal seguente file di testo i dati riguardanti un grafo che rappresentano le informazioni note riguardo al traffico in una rete. Scrivi e utilizza poi una funzione che dica al nodo 0 a quale nodo conviene spedire i dati perché arrivino nel più breve tempo possibile al nodo 3

Nel file la prima riga indica il numero dei nodi, le righe seguenti i tempi noti di trasmissione dei dati tra un nodo e un altro (in ms). Notare che siccome i canali di comunicazione possono essere gestiti in maniera non simmetrica o monodirezionale il grafo risulta orientato.

```
5
1 2, 2 10, 4 8
2 3
3 7, 4 2
4 3
0 3, 3 2
```

Soluzione

```python
class Nodo:
    def __init__(self, archi):
        self.archi = archi

    def __repr__(self):
        return str(self.archi)

class Grafo:
    def __init__(self, nodi = None):
        self.nodi = nodi
        if nodi == None:
            self.nodi = []

    def __repr__(self):
        ris = ''
        for i, nodo in enumerate(self.nodi):
            ris += f"{i}. {nodo}\n"
        return ris

    def nextCammino(self, start, end):
        if start < 0 or start >= len(self.nodi) or end < 0 or end >= len(self.nodi):
            return None
        if start == end:
            return None
        cammino = self.cammino(start, end)[0] # [0] perchè la funzione mi dà anche la lunghezza
        return cammino[1]




    def cammino(self, start, end):
        import heapq

        distanze = [float('inf') for nodo in self.nodi]
        prec = [None for nodo in self.nodi]
```

```
        distanze[start] = 0

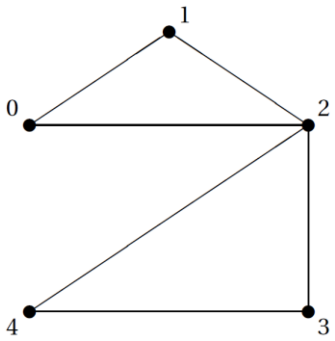
        pq = []
        heapq.heappush(pq, (0, start))
        while pq:
            dist, nodo = heapq.heappop(pq)
            for arco in self.nodi[nodo].archi:
                dest = arco[0]
                peso = arco[1]
                if distanze[dest] > distanze[nodo] + peso:
                    distanze[dest] = distanze[nodo] + peso
                    heapq.heappush(pq, (distanze[dest], dest))
                    prec[dest] = nodo
        cammino = []
        lunghezza = distanze[end]
        pos = end
        while pos is not None:
            cammino.insert(0, pos)
            pos = prec[pos]
        return [cammino, lunghezza]

g = Grafo()
with open('3-20.txt', 'r', encoding='utf-8') as f:
    f.readline()
    lista = []
    for riga in f:
        archi = riga.strip().split(',')
        archi = [arco.split() for arco in archi]
        archi = [[int(num) for num in arco] for arco in archi]
        g.nodi.append(Nodo(archi))

print(g)
print(g.nextCammino(0, 3))
```

9.3.19 Somma nodi e connessione

Istanzia il seguente grafo:



Scrivi:

1. una funzione che somma i valori di tutti i nodi del grafo.
2. Una funzione che determina se il grafo è connesso.

Soluzione 1

con grafo che contiene direttamente l'elenco di archi (creazione grafo più semplice, funzione un po più complessa)

```
# 9.3.21    Istanzia il seguente grafo:
# Scrivi:
# 1.    una funzione che somma i valori di tutti i nodi del grafo.
# 2.    Una funzione che determina se il grafo è connesso.

class Nodo:
    def __init__(self, val) -> None:
        self.val = val

    def __repr__(self) -> str:
        return f'{self.val}'


class Grafo:
    def __init__(self, nodi=None, archi=None) -> None:
        if nodi == None:
            self.nodi = []
        else:
            self.nodi = nodi
        if archi == None:
            self.archi = []
        else:
            self.archi = archi

    def sommaValori(self):
        somma = 0
        for nodo in self.nodi:
            somma += nodo.val
        return somma

    def connesso(self):
        if len(self.nodi) == 0:
            return True

        rimasti = [i for i in range(len(self.nodi))]
        rimasti.pop(0)
        print(self.nodi)
        coda = [self.nodi[0].val]
        while len(coda) > 0:
            corr = coda[0]
            coda.pop(0)
            for arco in self.archi:
                if arco[0] == corr:
                    if arco[1] in rimasti:
                        coda.append(arco[1])
                        rimasti.remove(arco[1])
                if arco[1] == corr:
                    if arco[0] in rimasti:
                        coda.append(arco[0])
                        rimasti.remove(arco[0])
```

```
        return len(rimasti) == 0


g = Grafo([Nodo(0), Nodo(1), Nodo(2), Nodo(3), Nodo(4)], [
          (0, 1), (0, 2), (1, 2), (2, 4), (2, 3), (3, 4)])
print(g.connesso())

g = Grafo([Nodo(0), Nodo(1), Nodo(2), Nodo(3), Nodo(4), Nodo(5)], [
          (0, 1), (0, 2), (1, 2), (2, 4), (2, 3), (3, 4)])
print(g.connesso())
```

Soluzione 2

con nodi che contengono la lista di adiacenza (funzione più semplice)

```
class Nodo:
    def __init__(self, val, archi = None) -> None:
        self.val = val
        if archi == None:
            self.archi = []
        else:
            self.archi = archi

    def __repr__(self) -> str:
        return f'{self.val}'


class Grafo:
    def __init__(self, nodi=None) -> None:
        if nodi == None:
            self.nodi = []
        else:
            self.nodi = nodi


    def sommaValori(self):
        somma = 0
        for nodo in self.nodi:
            somma += nodo.val
        return somma

    def connesso(self):
        if len(self.nodi) == 0:
            return True

        rimasti = [i for i in range(len(self.nodi))]
        rimasti.pop(0)
        print(self.nodi)
        coda = [self.nodi[0].val]
```

```
        while len(coda) > 0:
            corr = coda[0]
            coda.pop(0)
            for dest in self.nodi[corr].archi:
                if dest in rimasti:
                    coda.append(dest)
                    rimasti.remove(dest)


        return len(rimasti) == 0



g = Grafo()
g.nodi.append(Nodo(0, [1, 2]))
g.nodi.append(Nodo(1, [0, 2]))
g.nodi.append(Nodo(2, [0, 1, 3, 4]))
g.nodi.append(Nodo(3, [2, 4]))
g.nodi.append(Nodo(4, [2, 3]))
print(g.connesso())

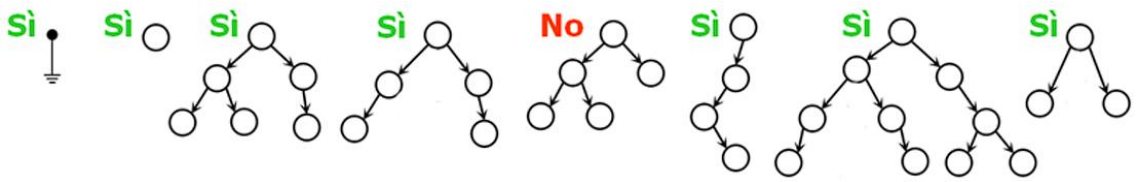
g = Grafo()
g.nodi.append(Nodo(0, [1, 2]))
g.nodi.append(Nodo(1, [0, 2]))
g.nodi.append(Nodo(2, [0, 1, 3, 4]))
g.nodi.append(Nodo(3, [2, 4]))
g.nodi.append(Nodo(4, [2, 3]))
g.nodi.append(Nodo(5))
print(g.connesso())
```

9.4 Alberi

9.4.1 Albero isobato

Un albero binario si dice isobato se tutti i cammini dalla radice alle foglie hanno la stessa lunghezza. Dopo aver definito opportunamente la struttura dati, scrivi una funzione che determina se un albero è isobato.

Esempi:



Variante: Usa un albero generico e non uno binario.

Soluzione

```
# Un albero binario si dice isobato se tutti i cammini
# dalla radice alle foglie hanno la stessa lunghezza. Dopo aver definito
# opportunamente la struttura dati, scrivi una funzione che determina se un
```

```python
# albero è isobato.
# Vedi gli esempi nell'immagine allegata

class Nodo:
    def __init__(self, val, sx = None, dx = None) -> None:
        self.val = val
        self.sx = sx
        self.dx = dx

    def __repr__(self) -> str:
        return f'{self.val}'

    def isobato(self, prof = [], curr = 0):
        if self.sx == None and self.dx == None:
            prof.append(curr)
        if self.sx != None:
            self.sx.isobato(prof, curr+1)
        if self.dx != None:
            self.dx.isobato(prof, curr+1)
        # if prof == []:
        #     return True
        p = prof[0]
        for num in prof:
            if num != p:
                return False
        return True


radice = Nodo(1, Nodo(2, Nodo(10)), Nodo(9, Nodo(36), Nodo(44)))
print(radice.isobato())
radice = Nodo(1, Nodo(2), Nodo(9, Nodo(36), Nodo(44)))
print(radice.isobato())
```

Soluzione con albero generico

```python
class Nodo:
    def __init__(self, val, figli=None) -> None:
        self.val = val
        if figli == None:
            self.figli = []
        else:
            self.figli = figli

    def __repr__(self) -> str:
        return f'{self.val}'

    def isobato(self, prof=[], curr=0):
        if self.figli == []:
            prof.append(curr)
        for figlio in self.figli:
            figlio.isobato(prof, curr+1)
        if prof == []:
```

```
            return True
        p = prof[0]
        for num in prof:
            if num != p:
                return False
        return True


radice = Nodo(1, [Nodo(2, [Nodo(10)]), Nodo(4, [Nodo(19)]), Nodo(9, [Nodo(36), Nodo(44)])])
print(radice.isobato())
radice = Nodo(1, [Nodo(2), Nodo(4, [Nodo(19)]), Nodo(9, [Nodo(36), Nodo(44)])])
print(radice.isobato())
```

Soluzione alternativa

```
from mytree import Tree

def f(nodo):
    if nodo.figli == []:
        return 0
    else:
        profFigli = [f(figlio) for figlio in nodo.figli]
        prof = profFigli[0]
        if prof == -1:
            return -1
        for profFiglio in profFigli:
            if profFiglio != prof:
                return -1
        return prof + 1

def isobato(nodo):
    if f(nodo) == -1:
        return False
    else:
        return True

t1 = Tree(5, [
        Tree(3, [
            Tree(1, [])
        ]),
        Tree(7, [
            Tree(5, [
                Tree(4, [])
            ]),
            Tree(9, [])
        ])
    ])

t1.stampa()
print()
print(isobato(t1))
```

```
t2 = Tree(5, [
        Tree(3, [
            Tree(1, [])
        ]),
        Tree(7, [
            Tree(5, []),
            Tree(9, [])
        ])
    ])

t2.stampa()
print()
print(isobato(t2))
```

9.4.2 Numero di nodi

Scrivi una funzione in grado di restituire il numero di nodi di un albero. L'albero può essere binario o no, in base a ciò la funzione cambia.

Soluzione 1

implementando la funzione come metodo della classe

```
# 9.4.2 - Scrivi una funzione in grado di restituire il numero di
# nodi di un albero. L'albero può essere binario o no, in base a
# ciò la funzione cambia.

class Nodo:
    def __init__(self, val, figli = None) -> None:
        self.val = val
        if figli == None:
            self.figli = []
        else:
            self.figli = figli

    def __repr__(self) -> str:
        return f'{self.val}'

    def numNodi(self):
        ris = 1
        for figlio in self.figli:
            ris += figlio.numNodi()
        return ris

radice = Nodo(1, [Nodo(2, [Nodo(10)]), Nodo(4, [Nodo(19)]), Nodo(9, [Nodo(36), Nodo(44)])])
print(radice.numNodi())
radice = Nodo(1, [Nodo(2), Nodo(4, [Nodo(19)]), Nodo(9, [Nodo(36), Nodo(44)])])
print(radice.numNodi())
```

Soluzione 2

fatta importando la classe e implementando la funzione come funzione esterna alla classe

```
from mytree import Tree

def contaNodi(nodo):
    ris = 1
    for figlio in nodo.figli:
        ris += contaNodi(figlio)

    return ris

t = Tree(5, [
        Tree(3, [
            Tree(1, [])
        ]),
        Tree(2, []),
        Tree(7, [
            Tree(1, []),
            Tree(5, [
                Tree(4, [])
            ]),
            Tree(9, [])
        ])
    ])

t.stampa()

print()
print(contaNodi(t))
```

9.4.3 Numero di foglie

Scrivi una funzione in grado di contare le foglie di un albero.

Soluzione

```
# 9.4.3 Scrivi una funzione in grado di
# contare le foglie di un albero.

class Nodo:
    def __init__(self, val, figli = None) -> None:
        self.val = val
        if figli == None:
            self.figli = []
        else:
            self.figli = figli

    def __repr__(self) -> str:
        return f'{self.val}'

    def numFoglie(self):
        if self.figli == []:
            return 1
        else:
            ris = 0
```

```
            for figlio in self.figli:
                ris += figlio.numFoglie()
            return ris


radice = Nodo(1, [Nodo(2, [Nodo(10)]), Nodo(4, [Nodo(19)]), Nodo(9, [Nodo(36), Nodo(44)])])
print(radice.numFoglie())
radice = Nodo(1, [Nodo(2), Nodo(4, [Nodo(19)]), Nodo(9, [Nodo(36), Nodo(44)])])
print(radice.numFoglie())
```

Soluzione con albero binario

```
class Nodo:
    def __init__(self, val, sx=None, dx=None) -> None:
        self.val = val
        self.sx = sx
        self.dx = dx

    def __repr__(self) -> str:
        return f'{self.val}'

    def numNodi(self):
        if self.sx == None and self.dx == None:
            return 1
        else:
            ris = 0
            if self.sx != None:
                ris += self.sx.numNodi()
            if self.dx != None:
                ris += self.dx.numNodi()
            return ris


radice = Nodo(1, [Nodo(2, [Nodo(10)]), Nodo(4, [Nodo(19)]), Nodo(9, [Nodo(36), Nodo(44)])])
print(radice.numNodi())
radice = Nodo(1, [Nodo(2), Nodo(4, [Nodo(19)]), Nodo(9, [Nodo(36), Nodo(44)])])
print(radice.numNodi())
```

9.4.4 Profondità massima

Scrivi una funzione in grado di verificare la profondità massima di un albero

Soluzione

```
# 9.4.4 - Scrivi una funzione in grado di verificare la
# profondità massima di un albero

class Nodo:
    def __init__(self, val, figli = None) -> None:
        self.val = val
        if figli == None:
            self.figli = []
        else:
```

```
            self.figli = figli

    def __repr__(self) -> str:
        return f'{self.val}'

    def profMax(self):
        if self.figli == []:
            return 0
        else:
            risFigli = [figlio.profMax() for figlio in self.figli]
            return max(risFigli) + 1

    def profMax2(self, prof = 0):
        if self.figli == []:
            return prof
        else:
            return max([figlio.profMax2(prof+1) for figlio in self.figli])



radice = Nodo(1, [Nodo(2, [Nodo(10)]), Nodo(4, [Nodo(19)]), Nodo(9, [Nodo(36), Nodo(44)])])
print(radice.profMax())
print(radice.profMax2())
radice = Nodo(1, [Nodo(2, [Nodo(10, [Nodo(15)]), Nodo(3)]), Nodo(4, [Nodo(19)]), Nodo(9)])
print(radice.profMax())
print(radice.profMax2())
```

9.4.5 Ricerca di un valore

Scrivi una funzione in grado di dire se un certo valore è contenuto nell'albero

Variante: viene restituita la profondità in cui si trova il nodo

Soluzione

```
from alberi import Nodo

def contiene(nodo, val):
    if nodo.dato == val:
        return True

    for figlio in nodo.figli:
        if contiene(figlio, val):
            return True

    return False

# Variante: viene restituita la profondità in cui si trova il nodo
def contiene2(nodo, val):
    if nodo.dato == val:
        return [True, 0]

    for figlio in nodo.figli:
```

```
        ris = contiene2(figlio, val)
        if ris[0] == True:
            return [True, ris[1]+1]

    return [False, None]

a = Nodo(5, [
        Nodo(1, [
            Nodo(10, []),
            Nodo(4, [])
        ]),
        Nodo(2, []),
        Nodo(3, [
            Nodo(7, [])
        ])
    ])

print(contiene(a, 22))
print(contiene(a, 3))
print(contiene2(a, 22))
print(contiene2(a, 3))
```

9.4.6 Albero artussiano

Definiamo un albero come "artussiano" se è composto solo da:

1. nodi foglia
2. nodi con un solo figlio
3. nodi con due figli aventi lo stesso numero di discendenti

Codificare una funzione che riceve in input un albero e determina se l'albero è "artussiano".

9.4.7 Punteggio massimo

Supponiamo che percorrendo un cammino dalla radice alle foglie si totalizzi un punteggio pari alla somma dei valori contenuti nei nodi percorsi.

Scrivere una funzione maxPunti che calcola il punteggio massimo che possiamo totalizzare percorrendo un cammino dalla radice alle foglie.

Soluzione

```
from alberi import Nodo

def maxPunti(nodo):
    if nodo.figli == []:
        return nodo.dato

    risfigli = [maxPunti(figlio) for figlio in nodo.figli]
    return max(risfigli)+nodo.dato


a = Nodo(5, [
```

```
        Nodo(1, [
            Nodo(10, []),
            Nodo(4, [])
        ]),
        Nodo(2, []),
        Nodo(3, [
            Nodo(7, [])
        ])
    ])

print(maxPunti(a))
```

9.4.8 Cammino con punteggio k

Supponiamo che percorrendo un cammino dalla radice alle foglie si totalizzi un punteggio pari alla somma dei valori contenuti nei nodi percorsi.

Vogliamo percorrere l'albero dalla radice ad una foglia totalizzando esattamente un certo punteggio: né far meno, né sforare. Scrivere una funzione esisteCammino che dati un albero ed un intero k, dice se esiste un cammino dalla radice ad una foglia che permette di totalizzare esattamente k punti. (difficile)

Soluzione

```
from alberi import Nodo

def esisteCammino(nodo, x):
    if nodo.figli == []:
        if nodo.dato == x:
            return True
        else:
            return False

    for figlio in nodo.figli:
        if esisteCammino(figlio, x-nodo.dato):
            return True

    return False


a = Nodo(5, [
        Nodo(1, [
            Nodo(10, []),
            Nodo(4, [])
        ]),
        Nodo(2, []),
        Nodo(3, [
            Nodo(7, [])
        ])
    ])

print(esisteCammino(a, 7))
```

9.4.9 Numero di cammini

Scrivi e usa una funzione che conta quanti sono i possibili cammini dalla radice alle foglie di un albero.

Soluzione

```
class Nodo:
    def __init__(self, val, figli = None) -> None:
        self.val = val
        if figli == None:
            self.figli = []
        else:
            self.figli = figli

    def __repr__(self) -> str:
        return f'{self.val}'

    def numCammini(self):
        if self.figli == []:
            return 1
        else:
            risFigli = [figlio.numCammini() for figlio in self.figli]
            return sum(risFigli)
            # ris = 0
            # for figlio in self.figli:
            #     ris += figlio.numCammini()
            return ris

def numCammini(radice):
    ris = 0
    coda = [radice]
    while len(coda) > 0:
        curr = coda[0]
        if curr.figli == []:
            ris += 1
        else:
            for figlio in curr.figli:
                coda.append(figlio)
        coda.pop(0)
    return ris

def numCammini2(radice):
    ris = 0
    pila = [radice]
    while len(pila) > 0:
        curr = pila[-1]
        if curr.figli == []:
            ris += 1
        else:
            for figlio in curr.figli:
                pila.append(figlio)
        pila.pop()
```

```
    return ris

# radice1 = Nodo(1, [Nodo(2, [Nodo(10)]), Nodo(4, [Nodo(19)]), Nodo(9, [Nodo(36), Nodo(44)])])
radice = Nodo(1, [Nodo(2, [Nodo(10, [Nodo(15)]), Nodo(3)]), Nodo(4, [Nodo(19), Nodo(5)]),
Nodo(9)])

print(radice.numCammini())
print(numCammini(radice))
```

9.4.10 Somma per livello

Scrivi e usa una funzione che calcola la somma dei valori contenuti ad ogni diversa profondità di un albero.

Soluzione

```
from albero import Nodo

albero = Nodo(2, [
            Nodo(7, [
                Nodo(2),
                Nodo(6, [
                    Nodo(5),
                    Nodo(11)
                ])
            ]),
            Nodo(5, [
                Nodo(9, [
                    Nodo(4)
                ])
            ])
        ])

def somme_profondita(nodo, profondita=0, ris = None):
    if ris == None:
        ris = [0]
    if len(ris) <= profondita:
        ris.append(0)
    ris[profondita] += nodo.dato
    for figlio in nodo.figli:
        somme_profondita(figlio, profondita+1, ris)
    return ris

# da rivedere insieme in classe
def somme_profondita2(nodo):
    if nodo.figli == []:
        return [nodo.dato]
    else:
        mioris = [nodo.dato]
        for figlio in nodo.figli:
            risfiglio = somme_profondita2(figlio)
            risfiglio = [0] + risfiglio
            for i, ris in enumerate(risfiglio):
                if i >= len(mioris):
```

```
                    mioris.append(0)
                mioris[i] += ris
        return mioris


print(somme_profondita(albero))
print(somme_profondita2(albero))
```

9.4.11 Stampa di un albero

Scrivi una funzione in grado di stampare nel modo migliore possibile il contenuto di un albero.

Soluzione per alberi generici

```
class Nodo:
    def __init__(self, dato = None, figli = None) -> None:
        self.dato = dato
        self.figli = figli
        if figli == None:
            self.figli = []


    def __repr__(self, prof=0) -> str:
        ris = ''
        for i in range(prof):
                ris += '    '
        ris += f"{self.dato}\n"
        for i, figlio in enumerate(self.figli):
                ris += f"{figlio.__repr__(prof+1)}"
        return ris

a = Nodo(5, [
        Nodo(1, [
            Nodo(10, []),
            Nodo(4, [])
        ]),
        Nodo(2, []),
        Nodo(3, [
            Nodo(7, [])
        ])
    ])

print(a)
```

Soluzione per alberi binari

```
class Nodo:
    def __init__(self, dato = None, figli = None) -> None:
        self.dato = dato
        self.figli = figli
        if figli == None:
            self.figli = []


    def __repr__(self, prof=0) -> str:
```

```
        ris = ''
        for i in range(prof):
                ris += '    '
        ris += f"{self.dato}\n"
        for i, figlio in enumerate(self.figli):
                ris += f"{figlio.__repr__(prof+1)}"
        return ris

a = Nodo(5, [
        Nodo(1, [
            Nodo(10, []),
            Nodo(4, [])
        ]),
        Nodo(2, []),
        Nodo(3, [
            Nodo(7, [])
        ])
    ])

print(a)
```

9.4.12 Larghezza di un albero

La larghezza di un albero è il numero massimo di nodi che stanno tutti al medesimo livello. Scrivi e usa una funzione che calcola la larghezza di un albero.

9.4.13 Tipi di visita

"Visitare" un albero significa esaminare sequenzialmente tutti i suoi nodi. Ci sono tre tipi principali di visite:

1. Nella visita in ordine simmetrico si visita il sottoalbero sinistro, quindi si esamina la radice e infine si visita il sottoalbero destro.
2. Nella visita in ordine anticipato si esamina prima la radice e quindi si visitano il sottoalbero sinistro e quello destro.
3. Nella visita in ordine posticipato si visitano prima il sottoalbero sinistro poi quello destro e infine si esamina la radice.

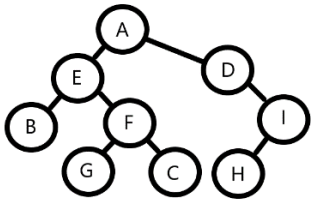
9.4.14 Albero dalle visite

Gli ordini di visita di un albero binario T di 9 nodi sono i seguenti:

1. A, E, B, F, G, C, D, I, H (anticipato)
2. B, G, C, F, E, H, I, D, A (posticipato)
3. B, E, G, F, C, A, D, H, I (simmetrico).

Istanzia l'albero binario T.

Soluzione



9.4.15 Visite di un albero binario

Scrivi le funzioni per effettuare le visite (e le stampe) anticipata, differita e simmetrica di un albero binario

Variante: invece che fare la stampa, restituisci la lista dei valori.

Soluzione

```
class Nodo:
    def __init__(self, dato = None, sx = None, dx = None) -> None:
        self.dato = dato
        self.sx = sx
        self.dx = dx

    def __repr__(self, prof=0, lato = '') -> str:
        ris = ''
        for i in range(prof):
                ris += '    '
        ris += f"{lato}{self.dato}\n"
        if self.sx != None:
            ris += f"{self.sx.__repr__(prof+1, 'sx: ')}"
        if self.dx != None:
            ris += f"{self.dx.__repr__(prof+1, 'dx: ')}"
        return ris

    def anticipata(self):
        ris = [self.dato]
        if self.sx != None:
            ris += self.sx.anticipata()
        if self.dx != None:
            ris += self.dx.anticipata()
        return ris

    def differita(self):
        ris = []
        if self.sx != None:
            ris += self.sx.differita()
        if self.dx != None:
            ris += self.dx.differita()
        ris.append(self.dato)
        return ris

    def simmetrica(self):
        ris = []
        if self.sx != None:
            ris += self.sx.simmetrica()
```

```
        ris.append(self.dato)
        if self.dx != None:
            ris += self.dx.simmetrica()
        return ris


a = Nodo(1,
        Nodo(2,
            Nodo(3),
            Nodo(4,
                Nodo(5),
                Nodo(6)
            )
        ),
        Nodo(7,
            dx = Nodo(8,
                Nodo(9)
            )
        )
    )

print(a)
print("Anticipata:", a.anticipata())
print("Differita:", a.differita())
print("Simmetrica:", a.simmetrica())
```

9.4.16 Albero binario di ricerca: Spiegazione e esercizio

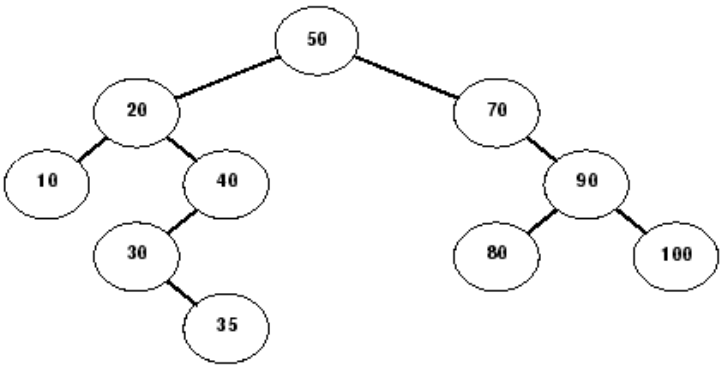
Un albero binario di ricerca (ABR) è un albero binario tale che:

sui valori delle chiavi dei suoi nodi è definito un ordinamento totale;

soddisfa la seguente proprietà: per ogni nodo n dell'albero:

1. tutte le chiavi dei nodi contenuti nel sottoalbero sinistro di n hanno valore strettamente minore della chiave contenuta in n,
2. tutte le chiavi dei nodi contenuti nel sottoalbero destro di n hanno valore strettamente maggiore della chiave contenuta in n.

Un esempio, in cui le chiavi sono interi con l'ovvio ordinamento...



Si noti che in un ABR non ci possono essere due nodi con la stessa chiave.

Un'importante proprietà di un ABR è la seguente: la visita in ordine simmetrico di un albero binario di ricerca genera una sequenza crescente di tutte le sue chiavi.

Data una sequenza di numeri interi casuali, costruisci il corrispettivo albero binario di ricerca.

Soluzione

```
from random import randint

class Nodo:
    def __init__(self, dato = None, sx = None, dx = None) -> None:
        self.dato = dato
        self.sx = sx
        self.dx = dx

    def __repr__(self, prof=0, lato = '') -> str:
        ris = ''
        for i in range(prof):
                ris += '    '
        ris += f"{lato}{self.dato}\n"
        if self.sx != None:
            ris += f"{self.sx.__repr__(prof+1, 'sx: ')}"
        if self.dx != None:
            ris += f"{self.dx.__repr__(prof+1, 'dx: ')}"
        return ris

    def aggiungi(self, dato):
        if dato < self.dato:
            if self.sx == None:
                self.sx = Nodo(dato)
            else:
                self.sx.aggiungi(dato)
        if dato > self.dato:
            if self.dx == None:
                self.dx = Nodo(dato)
            else:
                self.dx.aggiungi(dato)

a = None
numeri = [randint(0, 20) for _ in range(20)]
print(f"Numeri da inserire: {numeri}")

a = Nodo(numeri[0])
print(f"\ninserisco {numeri[0]}")
print("L'albero diventa:")
print(a)

for numero in numeri[1:]:
    print(f"\ninserisco {numero}")
    a.aggiungi(numero)
    print("L'albero diventa:")
    print(a)
```

```
    print()
```

9.4.17 Albero inverso

Dato un albero binario, i cui nodi contengono elementi interi, si scriva una funzione per ottenere l'albero inverso, ovvero un albero in cui il figlio destro (con relativo sottoalbero) e scambiato con il figlio sinistro (con relativo sottoalbero).

Soluzione

```
from albero_binario import Nodo

albero = Nodo(2,
            Nodo(7,
                Nodo(2),
                Nodo(6,
                    Nodo(5),
                    Nodo(11)
                )
            ),
            Nodo(5,
                None,
                Nodo(9,
                    Nodo(4)
                )
            )
        )

def albero_inverso(nodo):
    if nodo == None:
        return None
    if nodo.sx == None and nodo.dx == None:
        return nodo
    if nodo.sx != None:
        nodo.sx = albero_inverso(nodo.sx)
    if nodo.dx != None:
        nodo.dx = albero_inverso(nodo.dx)
    nodo.sx, nodo.dx = nodo.dx, nodo.sx
    return nodo

print(albero)
print(albero_inverso(albero))
```

9.4.18 Foglie con somma del cammino

Scrivi una funzione che sia in grado di ricevere un albero binario e aggiungere ad ogni foglia dell'albero un figlio contenente la somma dei valori che appaiono nel cammino dalla radice a tale foglia.

Soluzione

```
class Nodo:
    def __init__(self, val, sx = None, dx = None) -> None:
        self.val = val
        self.sx = sx
```

```
        self.dx = dx

    def __repr__(self, prof=0, lato = '') -> str:
        ris = ''
        for i in range(prof):
                ris += '    '
        ris += f"{lato}{self.val}\n"
        if self.sx != None:
            ris += f"{self.sx.__repr__(prof+1, 'sx: ')}"
        if self.dx != None:
            ris += f"{self.dx.__repr__(prof+1, 'dx: ')}"
        return ris


    def f(self, somma = 0):
        somma += self.val
        if self.sx == None and self.dx == None:
            self.sx = Nodo(somma)
        else:
            if self.sx != None:
                self.sx.f(somma)
            if self.dx != None:
                self.dx.f(somma)


a = Nodo(1,
        Nodo(2,
            Nodo(3),
            Nodo(4,
                Nodo(5),
                Nodo(6)
            )
        ),
        Nodo(7,
            dx = Nodo(8,
                Nodo(9)
            )
        )
    )

print(a)
a.f()
print()
print(a)
```

9.4.19 Albero binario completo

Un albero binario completo di altezza k è un albero binario in cui tutti i nodi, tranne le foglie, hanno esattamente due figli, e tutte le foglie si trovano al livello k. Scrivi una funzione che verifica se l'albero è completo.

Soluzione

```python
class Nodo:
    def __init__(self, val, sx=None, dx=None) -> None:
        self.val = val
        self.sx = sx
        self.dx = dx

    def __repr__(self, prof=0, lato="") -> str:
        ris = ""
        for i in range(prof):
            ris += "    "
        ris += f"{lato}{self.val}\n"
        if self.sx != None:
            ris += f"{self.sx.__repr__(prof+1, 'sx: ')}"
        if self.dx != None:
            ris += f"{self.dx.__repr__(prof+1, 'dx: ')}"
        return ris

    def completo(self, prof = 0, lista = None):
        if lista == None:
            lista = []
        if prof >= len(lista):
            lista.append(1)
        else:
            lista[prof] += 1
        if self.sx != None:
            self.sx.completo(prof+1, lista)
        if self.dx != None:
            self.dx.completo(prof+1, lista)

        print(self.val, prof, lista)

        for i in range(len(lista)-1):
            if lista[i] * 2 != lista[i+1]:
                return False
        return True



a = Nodo(1,
        Nodo(2,
            Nodo(3),
            Nodo(4,
                Nodo(5),
                Nodo(6)
            )
        ),
        Nodo(7,
            dx = Nodo(8,
                Nodo(9)
            )
        )
    )
print(a)
```

```
print(a.completo())

a = Nodo(1,
        Nodo(2,
            Nodo(3),
            Nodo(4)
        ),
        Nodo(7,
            Nodo(8),
            Nodo(9)
        )
    )
print(a)
print(a.completo())
```

9.4.20 Altezza minimale

Dato un albero binario T, definiamo altezza minimale di un nodo v la minima distanza di v da una delle foglie del suo sottoalbero. Scrivi e usa una funzione che riceve in input un nodo e restituisce la sua altezza minimale.

Soluzione

```
from albero_binario import Nodo

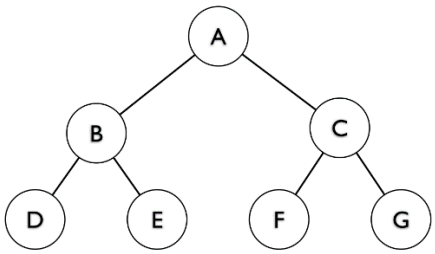
albero = Nodo(2,
            Nodo(7,
                Nodo(2),
                Nodo(6,
                    Nodo(5),
                    Nodo(11)
                )
            ),
            Nodo(5,
                None,
                Nodo(9,
                    Nodo(4)
                )
            )
        )

def altezza_minimale(nodo):
    if nodo == None:
        return -1
    if nodo.sx == None and nodo.dx == None:
        return 0
    return max([altezza_minimale(nodo.sx), altezza_minimale(nodo.dx)]) + 1


print(albero)
print(altezza_minimale(albero))
```

9.4.21 Lunghezza di cammino

In un albero binario, definiamo lunghezza di cammino come la somma delle distanze dei nodi dalla radice. Ad esempio, la lunghezza del cammino dell'albero nella seguente figura è pari a 10



Scrivi e usa una funzione che riceve un albero e restituisce la lunghezza di cammino dell'albero

Soluzione

```
from albero_binario import Nodo

albero = Nodo(2,
            Nodo(7,
                Nodo(2),
                Nodo(6,
                    Nodo(5),
                    Nodo(11)
                )
            ),
            Nodo(5,
                None,
                Nodo(9,
                    Nodo(4)
                )
            )
        )

def lunghezza_di_cammino(nodo, prof=0):
    if nodo == None:
        return 0
    if nodo.sx == None and nodo.dx == None:
        return prof
    return prof + lunghezza_di_cammino(nodo.sx, prof+1) + lunghezza_di_cammino(nodo.dx, prof+1)



print(albero)
print(lunghezza_di_cammino(albero))
```

10 Pygame

Di seguito è riportato l'elenco dei comandi più usati con una breve descrizione. Trovi la documentazione completa di pygame su https://www.pygame.org/docs/ .

screen = pygame.display.set_mode(size)	Restituisce la Surface dell'intera finestra
pygame.display.set_caption(caption)	Setta la scritta sulla barra superiore della finestra
clock = pygame.time.Clock()	Costruttore della classe Clock
clock.tick(fps)	Aspetta che passi il prossimo tick cioè intervallo di tempo che misura 1/fps secondi
pygame.event.get()	Restituisce la lista degli eventi avvenuti
event.type	Restituisce il tipo di evento (se la variabile event è un evento)
QUIT, MOUSEBUTTONDOWN, MOUSEBUTTONUP	Codici dei tipi di evento importati da pygame.locals
pygame.init()	Inizializza la libreria pygame
pygame.quit()	Disattiva la libreria pygame, da eseguire prima di chiudere l'applicazione
sys.exit()	Termina l'applicazione, serve importare sys
pygame.mouse.get_pressed()	Restituisce la tupla contenente una lista di valori true o false associata ad ogni tasto del mouse. Posizione 0 tasto sinistro, 1 tasto centrale, 2 tasto destro
pygame.mouse.get_pos()	Restituisce la tupla (x, y) delle coordinate del cursore del mouse
pygame.key.get_pressed()	Restituisce un oggetto che puoi usare come una tupla che in ogni posizione contiene un valore true o false che indica se un determinato tasto è premuto o no. Per le posizioni si usano i codici importati da pygame.locals. Ad esempio: K_w è la posizione per il tasto 'w', K_UP è la freccia in alto…
pygame.display.flip()	funzione che aggiorna lo schermo
pygame.display.update()	Funzione analoga a flip ma che con l'utilizzo di parametri aggiuntivi permette di aggiornare porzioni di schermo. Senza parametri è identica a flip
pygame.Surface((larghezza, altezza))	Costruttore della classe Surface che di fatto è un immagine
pygame.Rect(posx, posy, larghezza, altezza)	Costruttore della classe Rect, che indica posizione e ingombro di qualcosa, in pratica una hitbox
rect = image.get_rect()	Funzione che a partire da una Surface restituisce un Rect con le stesse dimensioni. È necessario poi assegnare la posizione al Rect

rect.bottom, .bottomleft, .bottomright, .center, .centerx, .centery, .top, .topright…	Modi per accedere (leggere o modificare) alla posizione del rect. Modificata una si modificano tutte. Può riferirsi ad una coppia x, y (come center o topleft) o solo x o y (come top o centerx). la posizione indicata nel costruttore equivale a topleft
rect.colliderect(altrorect)	Funzione della classe Rect che controlla se il rect collide con un altro rect ricevuto come parametro
rect.collidepoint(coordinate)	Funzione della classe Rect che controlla se un punto ricevuto come parametro collide col rect
colore = (255,255,255)	I colori si indicano col formato rgb
image.fill(colore)	Riempie una Surface o immagine con un colore
pygame.draw.rect(image, colore, (x, y, larg, alt), spessore) oppure pygame.draw.rect(image, colore, rect, spessore)	Funzione per disegnare un rettangolo, riceve come parametri una Surface o immagine su cui disegnare il rettangolo, un colore, una tupla con i dati del rettangolo o un Rect da cui estrae i valori e opzionalmente uno spessore. Se lo spessore non è indicato disegna un rettangolo pieno se no disegna il bordo del rettangolo con quello spessore.
pygame.draw.circle(image, colore, centro, raggio, spessore)	Analogo a draw.rect ma riceve due parametri centro e raggio invece che la tupla del rettangolo
screen.blit(image, rect) oppure screen.blit(image, pos)	Disegna su screen (una Surface) l'immagine 'image' nella posizione indicata dal rect. Posso indicare la posizione con una tupla (x, y) invece che con un Rect
image = pygame.image.load("cartella/nomefile.estensione)	Funzione che carica un'immagine con un certo path (composto da percorso cartella più nome) e restituisce una Surface. ./ cartella corrente (inutile scriverlo) ../ cartella superiore immagini/ entra nella cartella immagini ../immagini esce dalla cartella corrente ed entra in immagini
image = pygame.transform.scale(image, size)	Funzione che modifica la dimensione di un'immagine (Surface). Riceve una Surface e una size (tupla largh altezza) e restituisce la nuova Surface ridimensionata.
image = pygame.transform.rotate(image, gradi)	Funzione che riceve un'immagine e restituisce l'immagine ruotata di tot gradi
font = pygame.font.Font(None, 50) oppure font = pygame.font.Font("arial", 50)	Costruttore dell'oggetto Font, serve per poter poi costruire un'immagine con una scritta. Il primo parametro è il nome del font, se indico None prende il font di default. Il secondo parametro è l'altezza del font in pixel.
imgtext = font.render(testo, True, colore)	Crea un'immagine (Surface) che contiene la scritta indicata dal parametro "testo" del colore indicato. È necessario passare il secondo parametro che indica se l'antialiasing deve essere attivo o no. Posso scrivere True o 1 oppure False o 0.

11 Altro

11.1 Sistema

Questo tipo di programmi non rientra nel normale percorso di studio ma siccome è utile e interessante lo aggiungo per gli interessati. Parte degli esercizi con le relative soluzioni sono presi da https://www.programmareinpython.it/esercizi-python/

11.1.1 Info sistema operativo

Scrivi una funzione che fornisca in output il nome del Sistema Operativo utilizzato con eventuali relative informazioni sulla release corrente.

Soluzione

```
import platform


def sys_info():
    print("Il Sistema attualmente in uso è: " + platform.system())
    print("Info Release: " + platform.release())
```

11.1.2 Dimensione dei file

Scrivi una funzione che calcoli la somma (espressa in MB) delle dimensioni dei file presenti nella cartella di lavoro.

Soluzione

```
import os


def file_size():
    totale = 0
    cartella = os.getcwd()
    for file in os.listdir(cartella):
        totale += os.path.getsize(os.path.join(cartella, file))
    print(f"La somma delle dimensioni dei file presenti nella cartella '{cartella}' è:
{(totale/1048576):.2f}MB")
```

La cosa più utile di questo programma è la funzione os.path.join

11.1.3 Postino Gmail

Scrivi una funzione "postino" che sia in grado di spedire delle eMail tramite Gmail! (aiuto: puoi usare il modulo smtplib)

Soluzione

```
# soluzione da ricontrollare e forse correggere


import smtplib
```

```
def postino():
    print("""
    Questa è la funzione Postino: spedisce eMail utilizzando Gmail!
    Server: smtp.gmail.com
    Porta: 587
    Si richiedono: Username, Password, Destinatario, Oggetto e Messaggio da inviare.
    """)

    username = input("Inserisci il tuo username: ")
    password = input("Inserisci la tua password: ")
    destinatario = input("Inserisci l'email del destinatario: ")
    oggetto = input("Inserisci l'oggetto della mail: ")
    messaggio = input("Ora puoi inserire il messaggio che vuoi inviare: ")
    contenuto = f"Subject: {oggetto}\n\n{messaggio}"
    print("Sto effettuando la connessione col Server...")
    email = smtplib.SMTP("smtp.gmail.com",587)
    email.ehlo()
    email.starttls()
    email.login(username,password)
    print("Sto inviando...")
    email.sendmail(username, destinatario, contenuto)
    email.quit()
    print("Messaggio Inviato!")
```

11.1.4 Ricerca di file pdf

Scrivi una funzione "cercatrice" che scansioni un dato percorso di sistema alla ricerca di file di tipo pdf.

La funzione dovrà avere le seguenti caratteristiche:

1. Il percorso fornito dovrà essere anzitutto validato, in quanto deve portare a una cartella esistente
2. La funzione dovrà fornire un elenco dei file pdf (con/relativo/percorso) man mano che questi vengono trovati
3. In fine la funzione dovrà fornire in output il totale dei file .pdf che sono stati trovati durante la scansione.

Soluzione

```
import os


def cercatrice(percorso):
    if not os.path.isdir(percorso):
        print(f"Il percorso inserito '{percorso}' risulta non essere un percorso idoneo. Verifica
e riprova, grazie.\n")
        return None
```

```
    contatore = 0
    print(f"Sto effettuando la scansione di '{percorso}' alla ricerca di file .pdf\n")
    for cartella, sottocartelle, files in os.walk(percorso):
        for file in files:
            if file.endswith(".pdf"):
                pdf = os.path.join(cartella,file)
                print(f"Trovato file pdf: {pdf}\n")
                contatore += 1
    print(f"\nScansione Ultimata! Ho trovato {contatore} files con estensione pdf.")
```

11.1.5 Backup di file

Scrivi una funzione "file_backup" che sia in grado di effettuare copie di backup di determinati tipi di file, con le seguenti caratteristiche:

1. Percorso da scansionare, di backup e tipologia di file da copiare dovranno essere passati dall'utente tramite input
2. Il programma dovrà verificare la presenza o meno di una cartella di backup al percorso fornito, e qualora questa non fosse presente dovrà crearla
3. La funzione dovrà anche gestire l'eventuale scelta da parte dell'utente, di un percorso da scansionare che non esiste

Soluzione

```
import os
import shutil


def file_backup():
    print("Ciao! Questo script effettua copie di backup per file di un'estensione passata come input.\n")
    percorso = input("Inserisci il percorso da scansionare: ")
    if not os.path.isdir(percorso):
        print(f"Il percorso inserito '{percorso}' risulta non essere un percorso idoneo. Verifica e riprova, grazie.\n")
        return file_backup()
    estensione = input("Che tipologia di file desideri salvare? [esempio: .jpg .pdf .epub]: ")
    backup_folder = input("Inserisci la cartella dove desideri salvare i tuoi file: ")
    if not os.path.isdir(backup_folder):
          os.makedirs(backup_folder)
    contatore = 0
    print(f"Sto effettuando la scansione di '{percorso}' alla ricerca di file '{estensione}'\n")
    for cartella, sottocartelle, files in os.walk(percorso):
        for file in files:
            if file.endswith(estensione):
                match = os.path.join(cartella,file)
                print(f"Trovato file {estensione}: {match}")
                shutil.copy(match,backup_folder)
```

```
                contatore += 1
    print("Copia Terminata")
    print(f"Ho trovato '{contatore}' files con estensione {estensione}, ora disponibili anche in
'{backup_folder}'")
```